Important Information

Thank you for choosing Freenove products!

Getting Started

If you have not yet downloaded the zip file, associated with this kit, please do so now and unzip it.
https://github.com/Freenove/Freenove_ESP32_S3_Display

Get Support and Offer Input

Freenove provides free and responsive product and technical support, including but not limited to:

- Product quality issues
- Product use and build issues
- Questions regarding the technology employed in our products for learning and education
- Your input and opinions are always welcome
- We also encourage your ideas and suggestions for new products and product improvements

For any of the above, you may send us an email to:

support@freenove.com

Safety and Precautions

Please follow the following safety precautions when using or storing this product:

- Keep this product out of the reach of children under 6 years old.
- This product should be **used only when there is adult supervision present** as young children lack necessary judgment regarding safety and the consequences of product misuse.
- This product contains small parts and parts, which are sharp. This product contains electrically conductive parts. **Use caution with electrically conductive parts near or around power supplies, batteries and powered (live) circuits.**
- When the product is turned ON, activated or tested, some parts will move or rotate. **To avoid injuries to hands and fingers keep them away from any moving parts!**
- It is possible that an improperly connected or shorted circuit may cause overheating. **Should this happen, immediately disconnect the power supply or remove the batteries and do not touch anything until it cools down!** When everything is safe and cool, review the product tutorial to identify the cause.
- Only operate the product in accordance with the instructions and guidelines of this tutorial, otherwise parts may be damaged or you could be injured.
- Store the product in a cool dry place and avoid exposing the product to direct sunlight.
- After use, always turn the power OFF and remove or unplug the batteries before storing.

About Freenove

Freenove provides open source electronic products and services worldwide.

Freenove is committed to assist customers in their education of robotics, programming and electronic circuits so that they may transform their creative ideas into prototypes and new and innovative products. To this end, our services include but are not limited to:

- Educational and Entertaining Project Kits for Robots, Smart Cars and Drones
- Educational Kits to Learn Robotic Software Systems for Arduino, Raspberry Pi and micro: bit
- Electronic Component Assortments, Electronic Modules and Specialized Tools
- **Product Development and Customization Services**

You can find more about Freenove and get our latest news and updates through our website:

http://www.freenove.com

sale@freenove.com

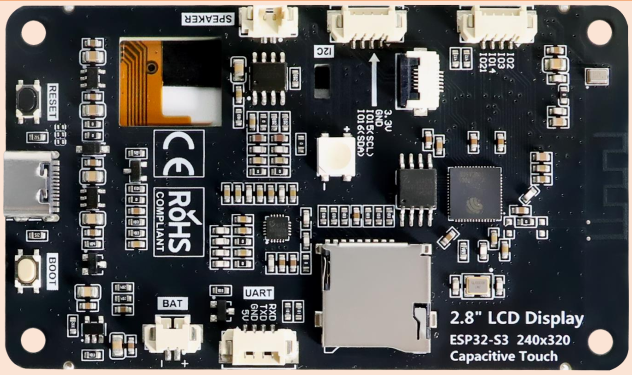
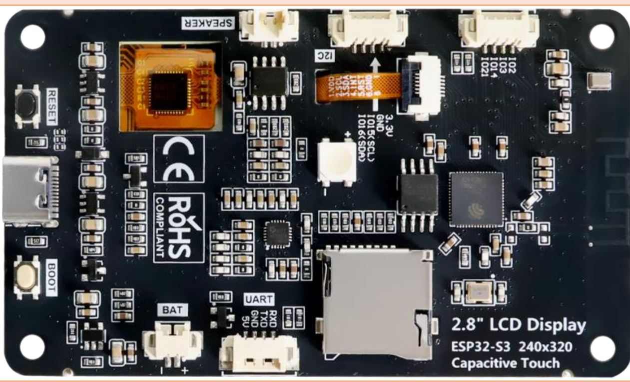
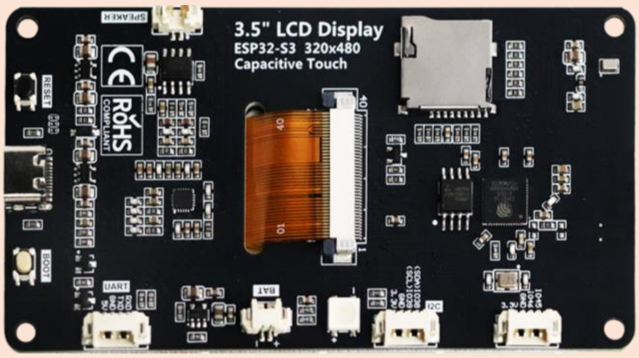
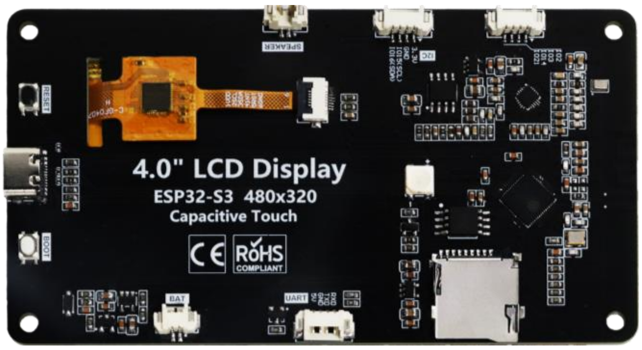
Copyright

FREENOVE

Contents

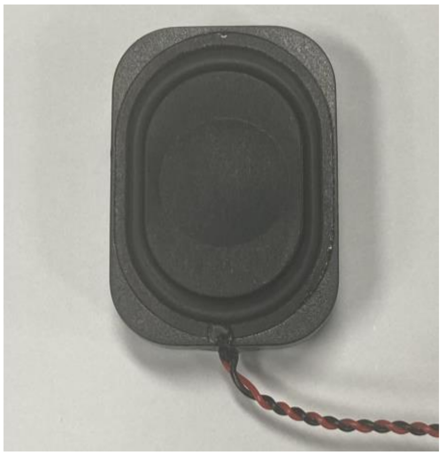
Freenove ESP32S3 Display

As of the time this tutorial was written, the Freenove ESP32-S3 Display is available in four different screen sizes. Due to differences in hardware configuration, the tutorial content varies between the two models. To ensure you receive accurate guidance, please confirm your device model before use and select the tutorial documentation that matches it.

Models	Specifications		Images
FNK0104A (NonTouch)	Size	2.8 inch	
	Resolution	240x320	
	Driver	ILI9341	
FNK0104B	Size	2.8 inch	
	Resolution	240x320	
	Driver	ILI9341	
FNK0104N	Size	3.5 inch	
	Resolution	320x480	
	Driver	ST77922	
FNK0104S	Size	4.0 inch	
	Resolution	320x480	
	Driver	ST7796	

List

If you have any concerns, please feel free to contact us via support@freenove.com

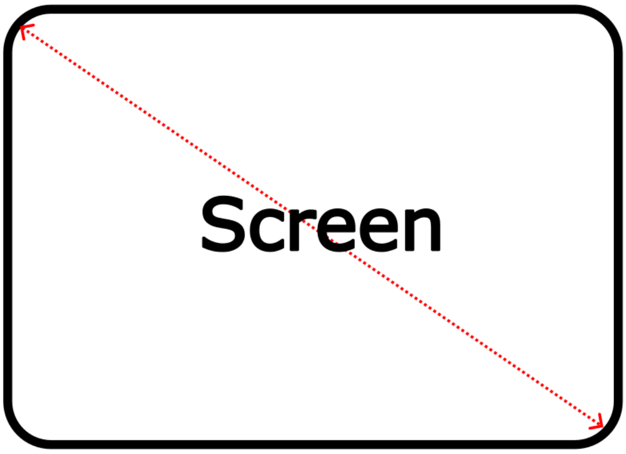
Freenove ESP32 Display x 1 	USB-C Data Cable x 1 
Battery adapter cable x1 	Speaker x1 
Jumper Wire x1 	

Related Knowledge

Screen Size

An internationally recognized unit of length, the **inch** (equivalent to 2.54 centimeters) has been the standard measurement for screen sizes in the display industry with a long history. This convention originated in the early television era when British manufacturers first adopted inches to measure cathode-ray tube (CRT) dimensions. The practice quickly became the global norm and remains the industry standard today.

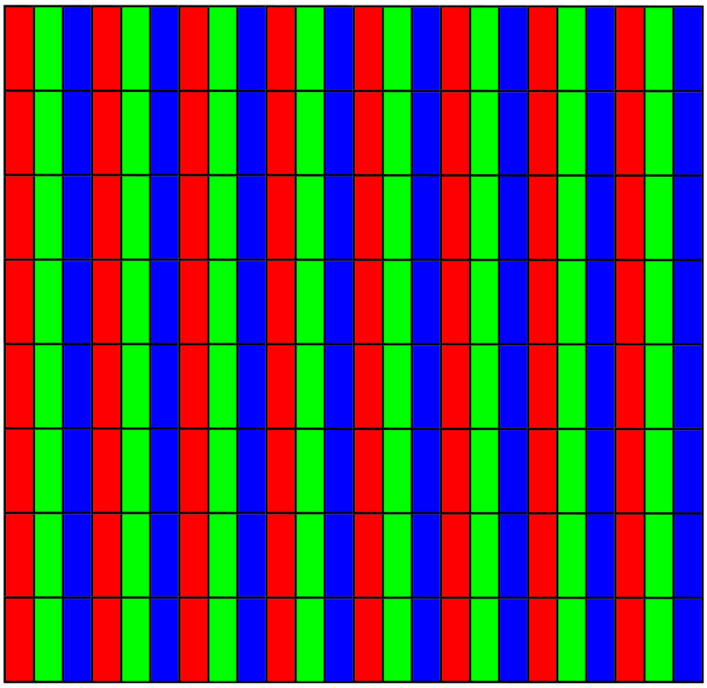
A critical clarification: screen size always refers to **the diagonal measurement of the display** panel. For instance, a 4-inch display features a 4-inch (10.16 cm) diagonal, ensuring standardized and comparable sizing across all devices.



Resolution

Screen resolution quantifies a display's pixel density along its horizontal and vertical axes, expressed as **width × height** (e.g., 320 × 480). This specification applies universally across displays, such as smartphones, monitors, and television screens.

- **Pixel**: The smallest unit of a display, composed of red (R), green (G), and blue (B) subpixels. Through precise brightness variation, these subpixels generate the complete color spectrum.
- **Resolution vs. Clarity**: Higher resolution means more pixels per unit area, resulting in sharper and more detailed imagery.



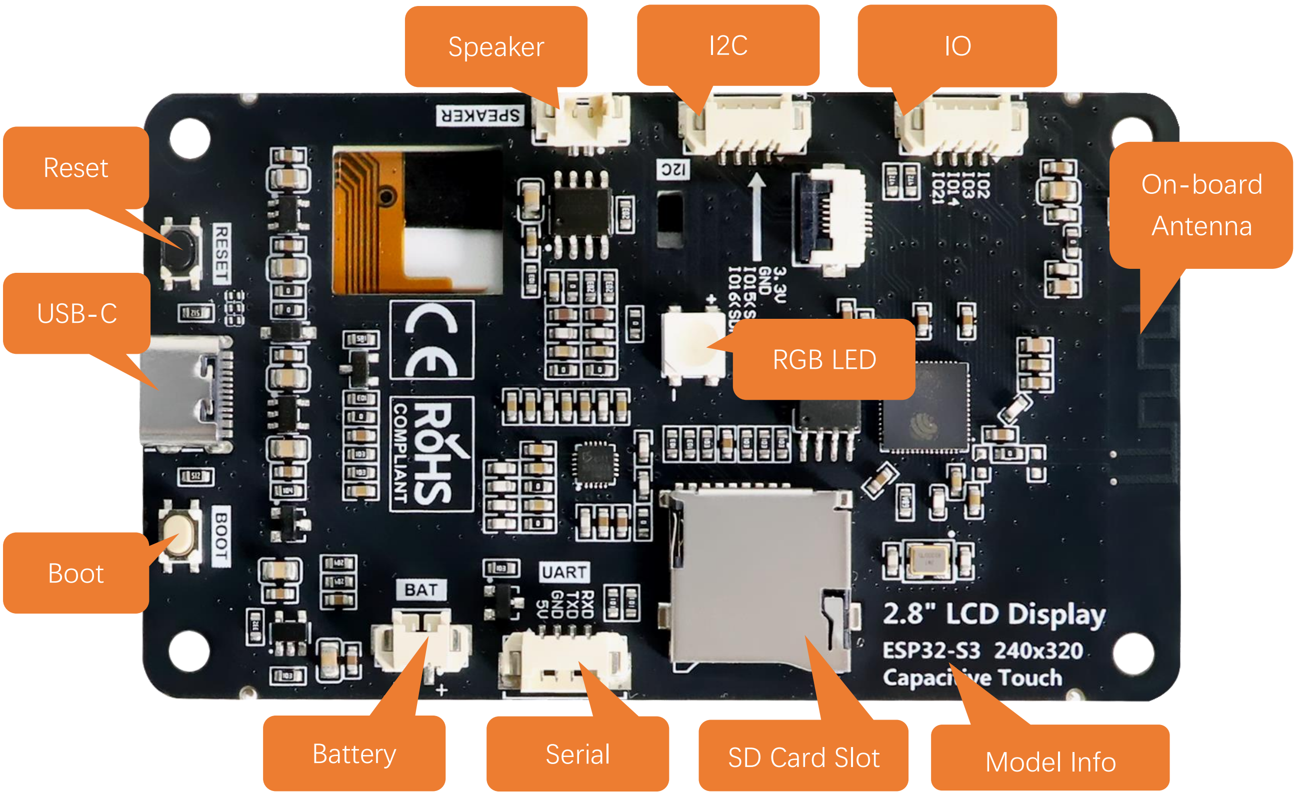
Display Driver

The display driver IC serves as the core component of display panels, processing signals from the main controller and controlling pixel illumination. Below is an overview of three widely used display driver ICs.

Display Driver IC	Resolution	Advantages
ILI9341	240x320	widely used with abundant resources available
ST77922	320x480 or 480x480	Suitable for learning and development
ST7796	320x480	High-definition display with richer colors

Preface

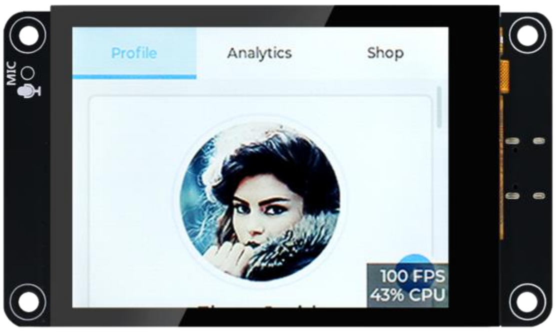
Hardware Interfaces



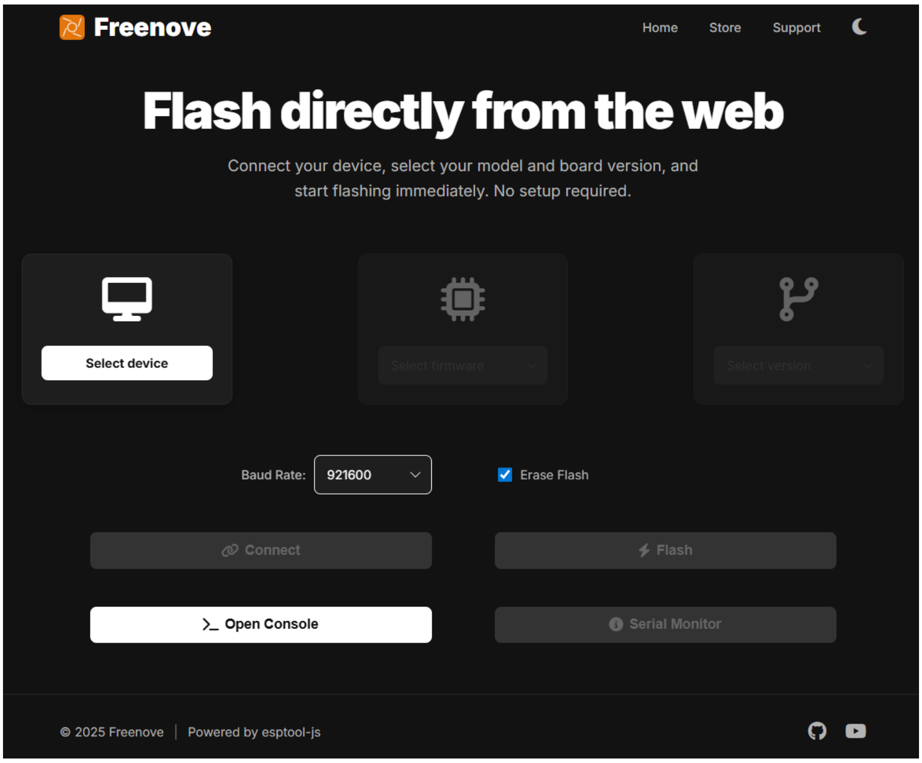
Hardware Testing (Important)

When you first power on the Freenove ESP32 S3 Display, you will see an LVGL Demo. This is expected behavior. You can learn how to use the screen by following the tutorials in the chapters ahead.

Please rest assured! This is not "malicious software" or a "used device."
This is a classic **benchmark/demo program** built on LVGL, a popular open-source embedded graphics library. It serves as an industry-standard tool to evaluate and demonstrate the development board's performance in rendering smooth graphics, handling animations, and supporting responsive touch interactions. You can easily overwrite it by **uploading a new sketch**. When you follow the tutorial to upload your first program, this demo will be automatically erased and replaced.



We provide a convenient online flashing tool, eliminating the need to download any complex software. Simply select the firmware that matches your device model, and you can easily reproduce the effects shown in the tutorial within a few minutes to determine whether there is a hardware failure.



If you have any concerns, please feel free to contact us via support@freenove.com

Battery (Optional)

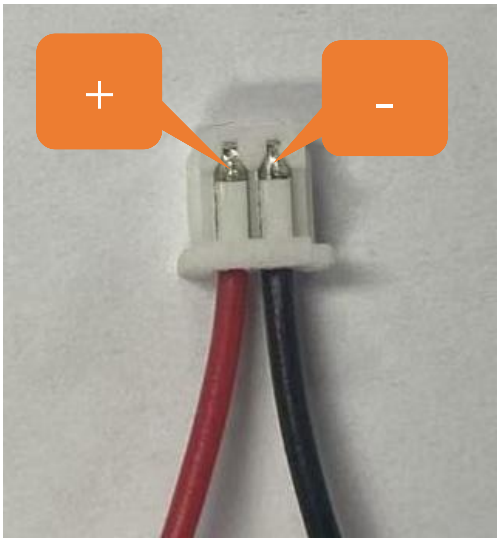
Please note that this product does not come with lithium batteries; please purchase them yourself.

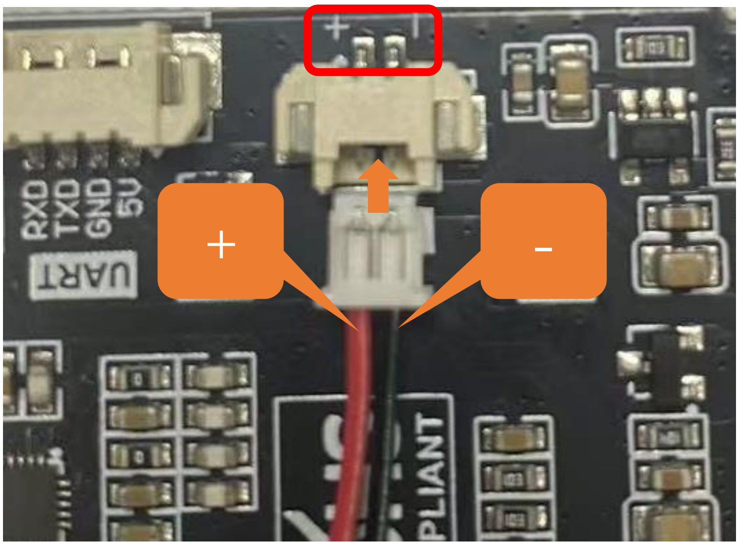
This device supports both **USB-powered and lithium battery-powered operation**. For optimal safety, USB power is recommended. Due to the **hazardous nature of lithium batteries**, we advise against their use unless absolutely necessary.

This device features an **MX1.25mm** connector and supports lithium batteries of various capacities. Note: The input voltage must be maintained within **3.7-4.2V** range.

Market-available batteries may feature **two distinct wiring configurations where the positive (+) and negative (–) terminals are reversed between models**. Please verify the battery's wiring matches the product requirements (refer to the diagram below) to prevent equipment failure or safety risks due to improper connection.

The red cable is the positive terminal while the black one is negative.

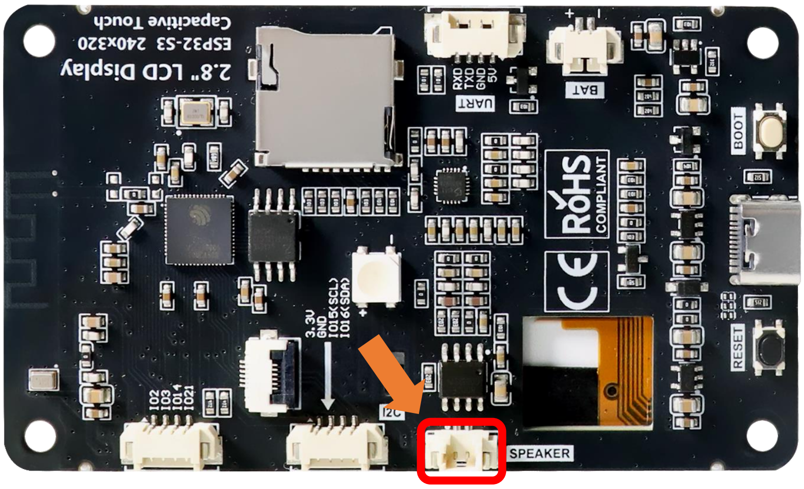




We recommend using a charger specially designed for lithium batteries. Due to various specifications and quality of lithium batteries, using a proper charger helps ensure peak performance, safety, and battery longevity.
While our product also supports USB charging as a backup option, please note that this method does not support fast charging and is limited to standard slow charging.

Speaker

There is a speaker connector (PH1.25mm) on the Freenove ESP32-S3 Display.



SD Card

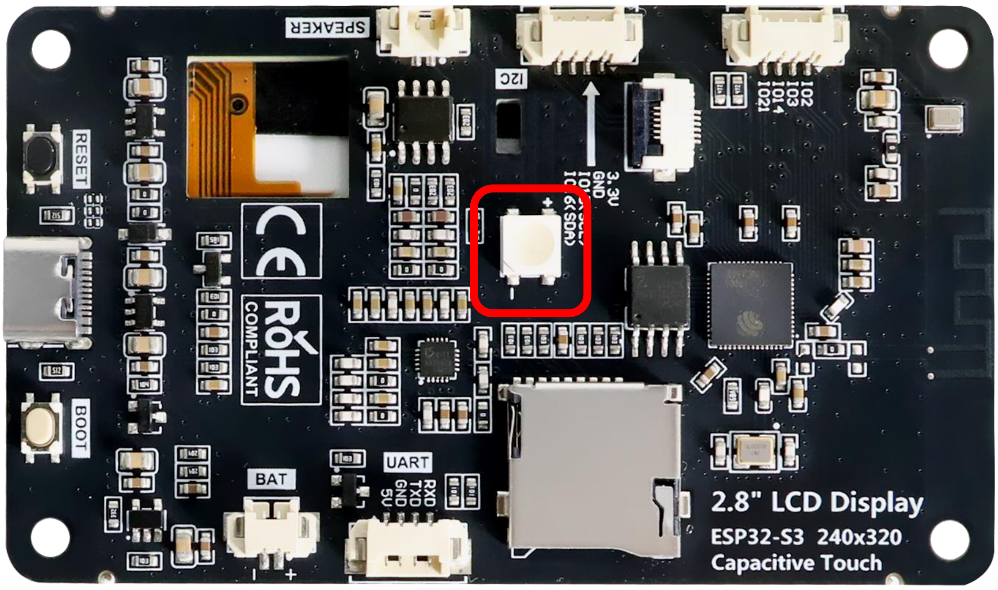
This connector circuit employs the 4 - bit communication mode of SD MMC to establish data communication with the SD card and provides support for high - speed Micro SD card storage.

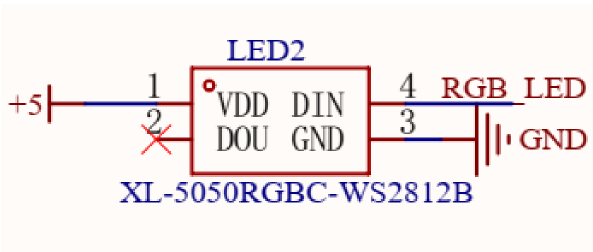
Item	Definition
SD Card	SD_CLK
	SD_CMD
	SD_D0
	SD_D1
	SD_D2
	SD_D3

Note: This product does not include SD cards or SD card readers. Please buy them yourself

RGB LED

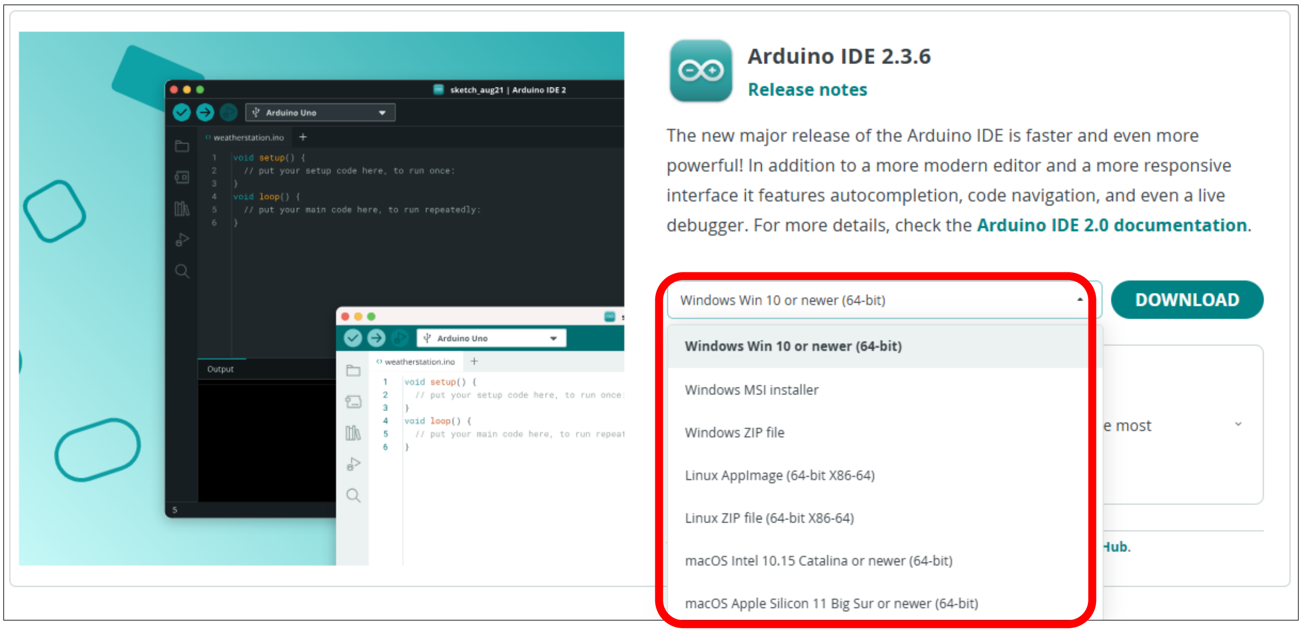
The Freenove ESP32-S3 Display includes an RGB LED (red, green, blue) that can blend colors to create various lighting effects.





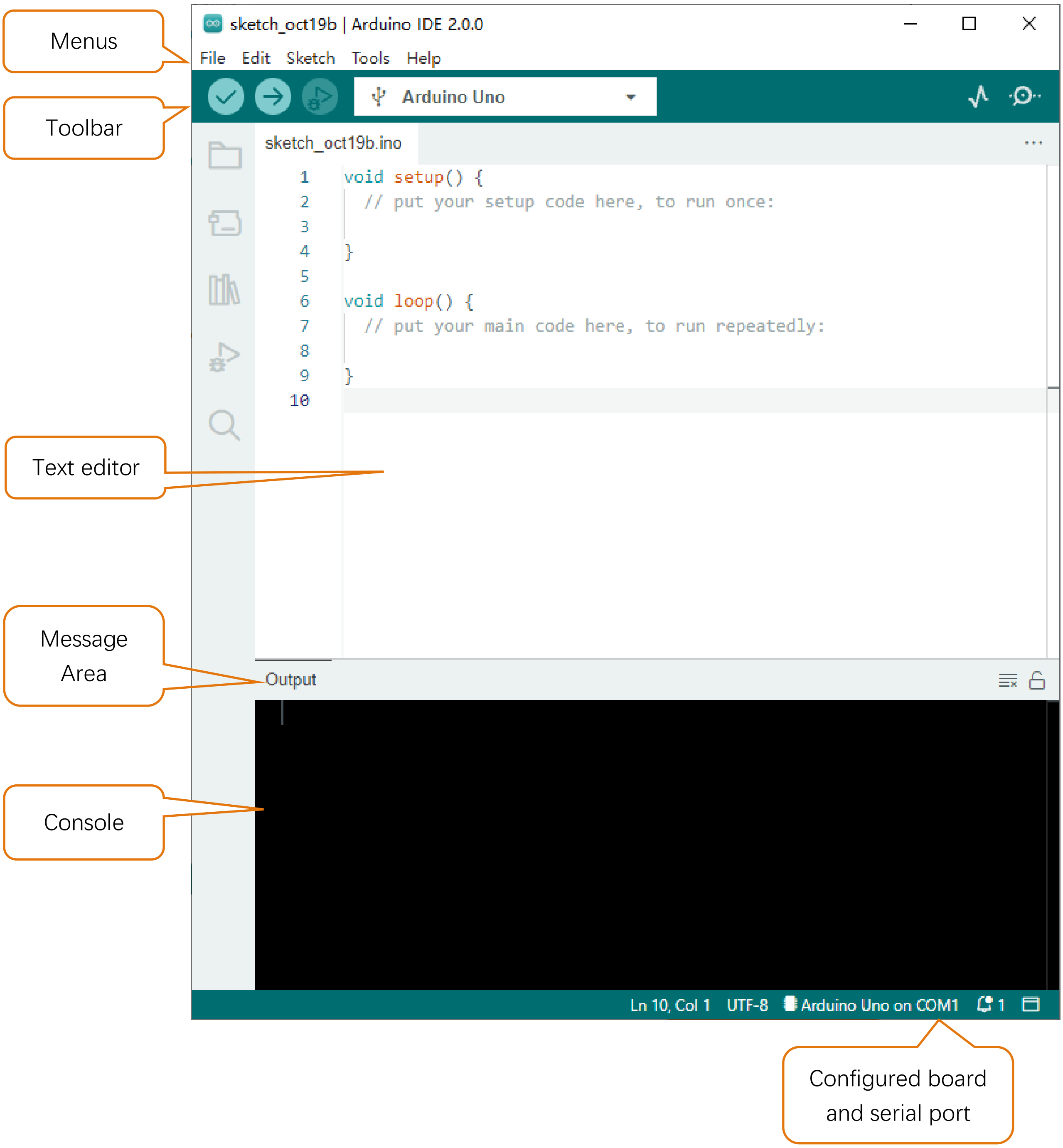
Programming Software

We use the Arduino Software (IDE) to write and upload the code for this product.
First, install Arduino Software (IDE): visit https://www.arduino.cc/en/software/, Select and download corresponding installer according to your operating system. If you are a Windows user, please select the "Windows" to download and install it correctly.

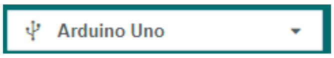


After the download completes, run the installer. For Windows users, there may pop up an installation dialog box of driver during the installation process. When it popes up, please allow the installation.
After installation completes, an Arduino Software shortcut will be generated in the desktop. Run the Arduino Software.

The interface of Arduino Software is as follows:



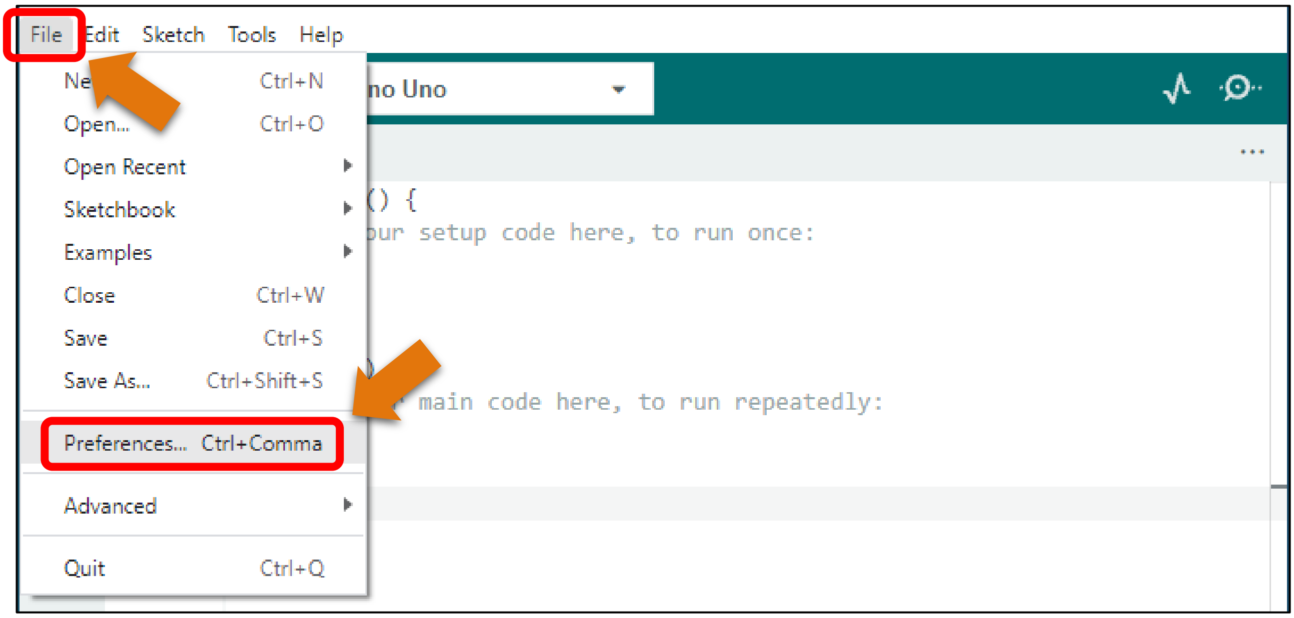
Programs written with Arduino Software (IDE) are called **sketches**. These sketches are written in the text editor and saved with the file extension**.ino**. The editor features text cutting/pasting and searching/replacing. The message area gives feedback while saving and exporting and also displays errors. The console displays text output by the Arduino Software (IDE), including complete error messages and other information. The bottom right-hand corner of the window displays the configured board and serial port. The toolbar buttons allow you to verify and upload programs, create, open, and save sketches, and open the serial monitor.

	Verify Check your code for compile errors.
	Upload Compile your code and upload them to the configured board.
	Debug Debug code running on the board. (Some development boards do not support this function)
	Development board selection Configure the support package and upload port of the development board.
	Serial Plotter Receive serial port data and plot it in a discounted graph.
	Serial Monitor Open the serial monitor.

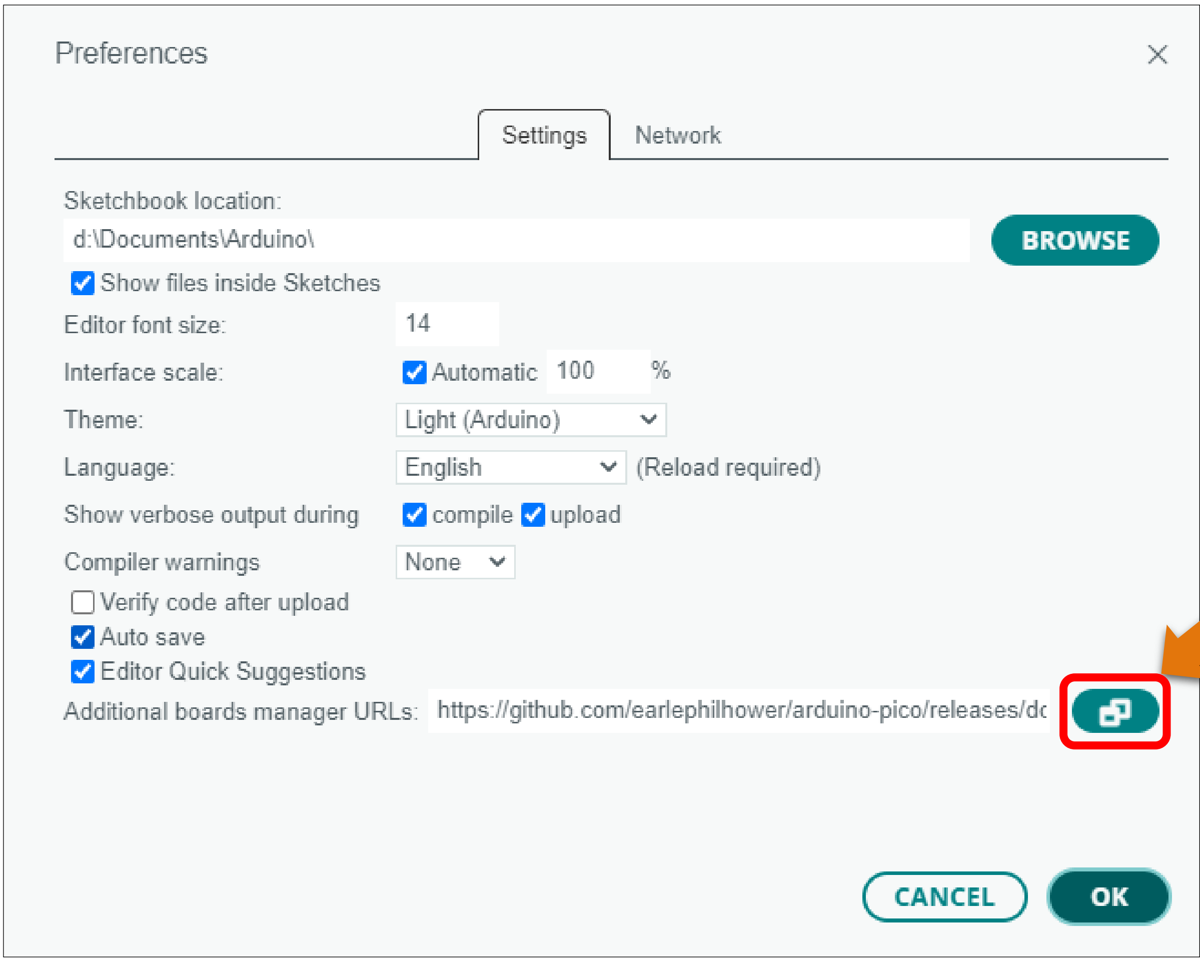
Additional commands are found within the five menus: File, Edit, Sketch, Tools, Help. The menus are context sensitive, which means only those items relevant to the work currently being carried out are available.

Environment Configuration

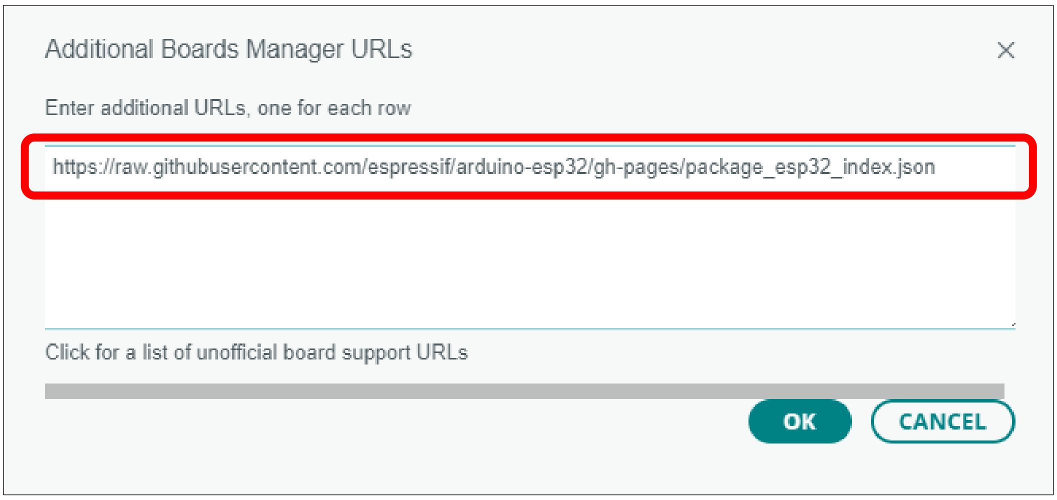
First, open the software platform Arduino, and then click File in Menus and select Preferences.



Second, click on the symbol behind "Additional Boards Manager URLs"

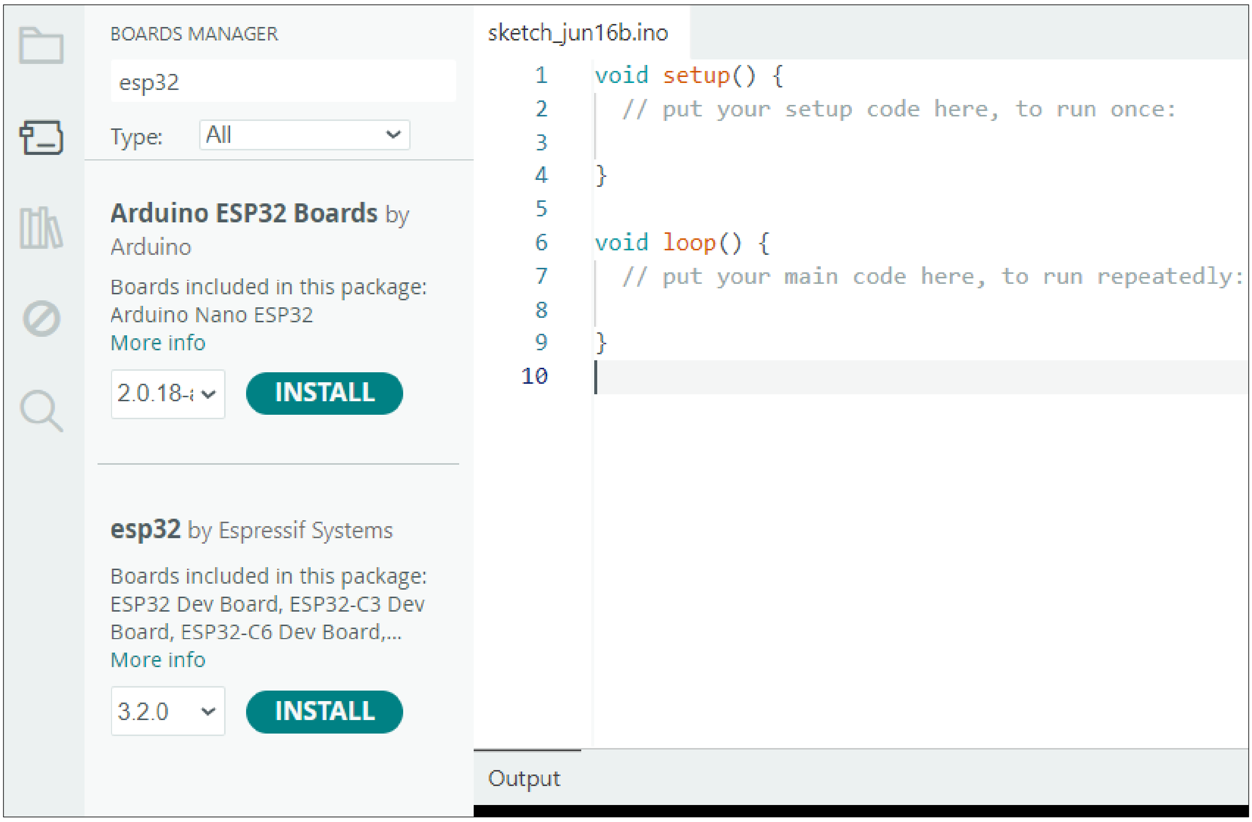


Third, fill in https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json in the new window, click OK, and click OK on the Preferences window again.

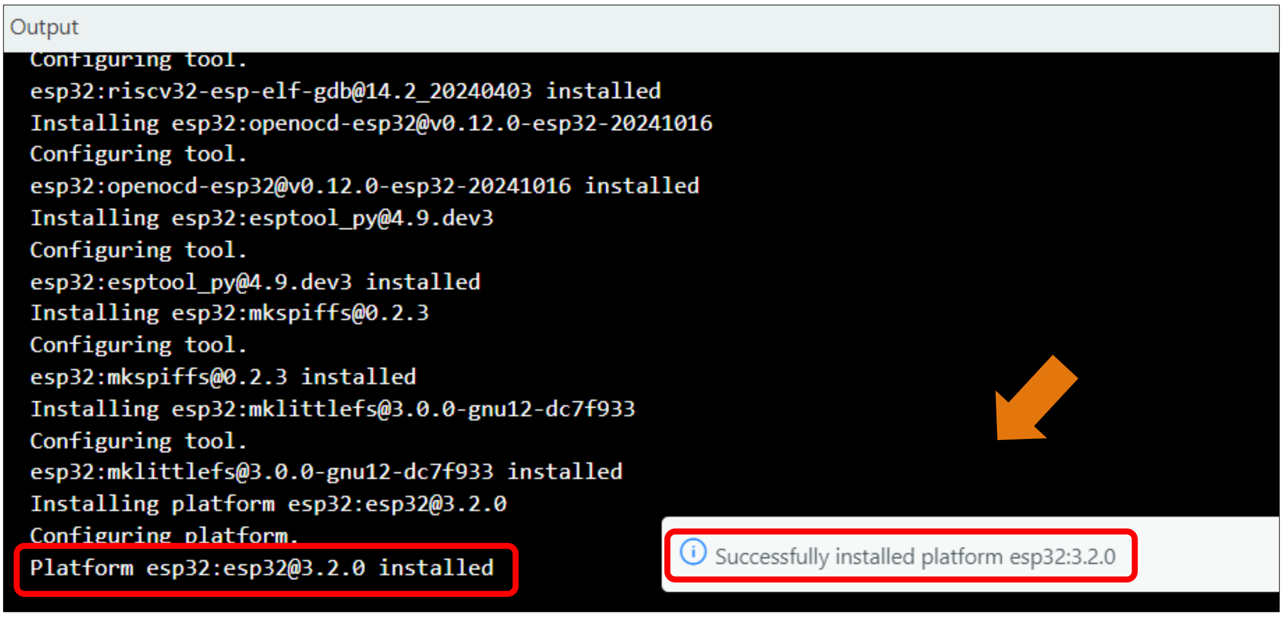


Note: if you copy and paste the URL directly, you may lose the "-". Please check carefully to make sure the link is correct.

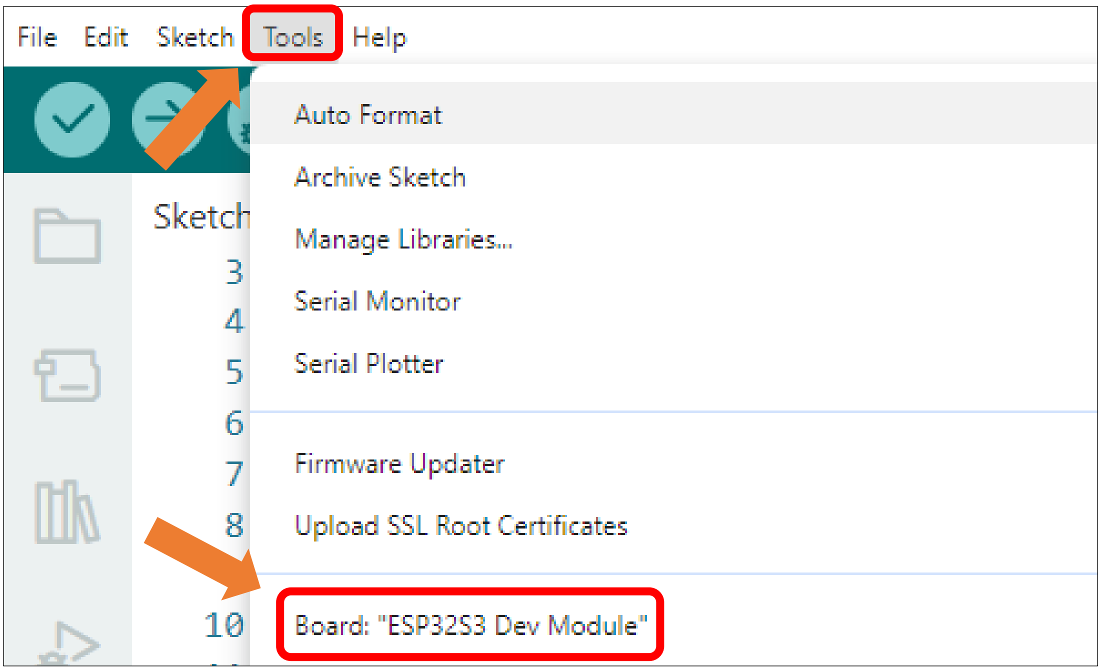
Fourth, click "Boards Manager". Enter **"esp32"** in Boards manager, select **3.2.0**, and click "INSTALL".



Arduino will download these files automatically. Wait for the installation to complete.



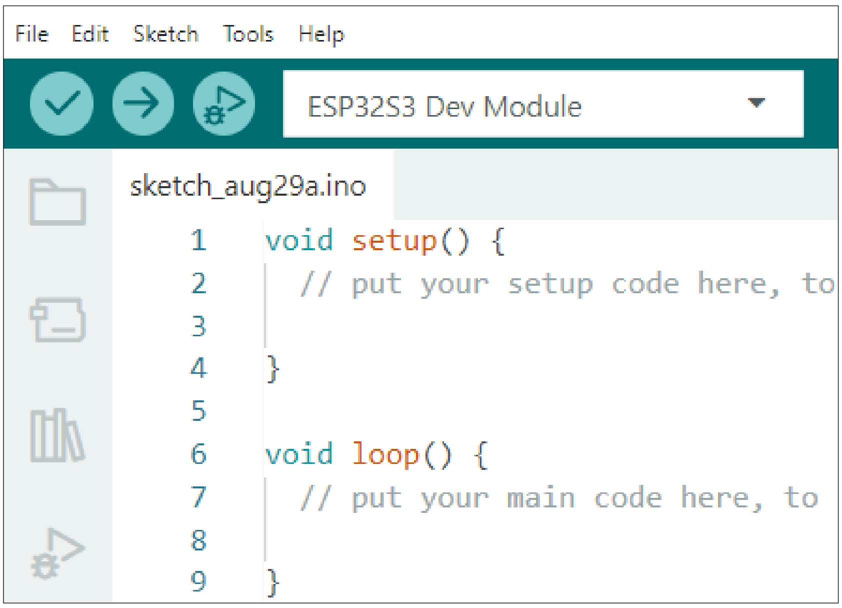
When finishing installation, click Tools in the Menus again and select Board: "ESP32S3 Dev Module", and then you can see information of ESP32S3.



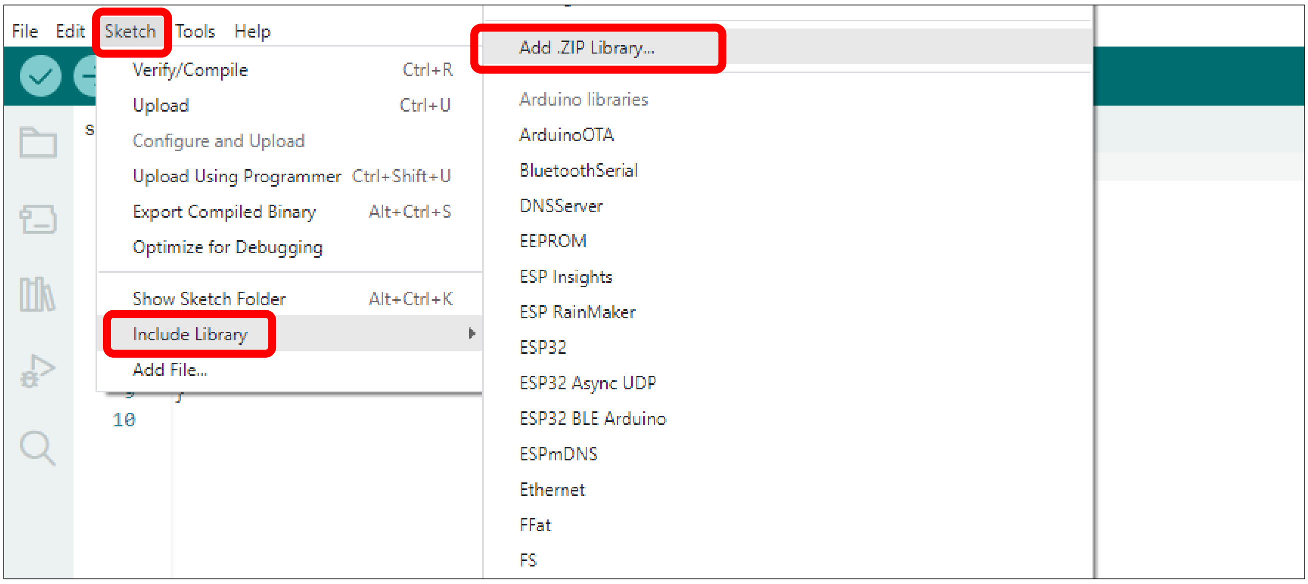
Library Installation

Before starting the learning process, it is necessary to install some libraries in advance to enable the code to be compiled properly. For convenience, we have already packaged these libraries and placed them in the **Freenove_ESP32_S3_Display/Libraries** folder. Please refer to the following steps to install these libraries into the Arduino IDE.

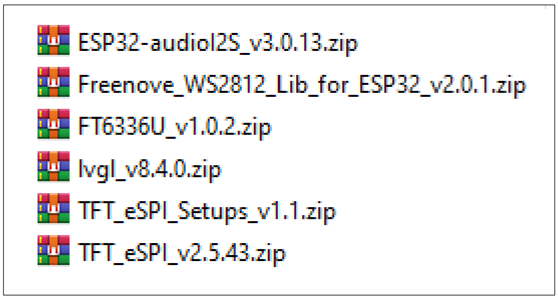
1. Open Arduino IDE.



2. Select Sketch->Include Library->Add .ZIP library….

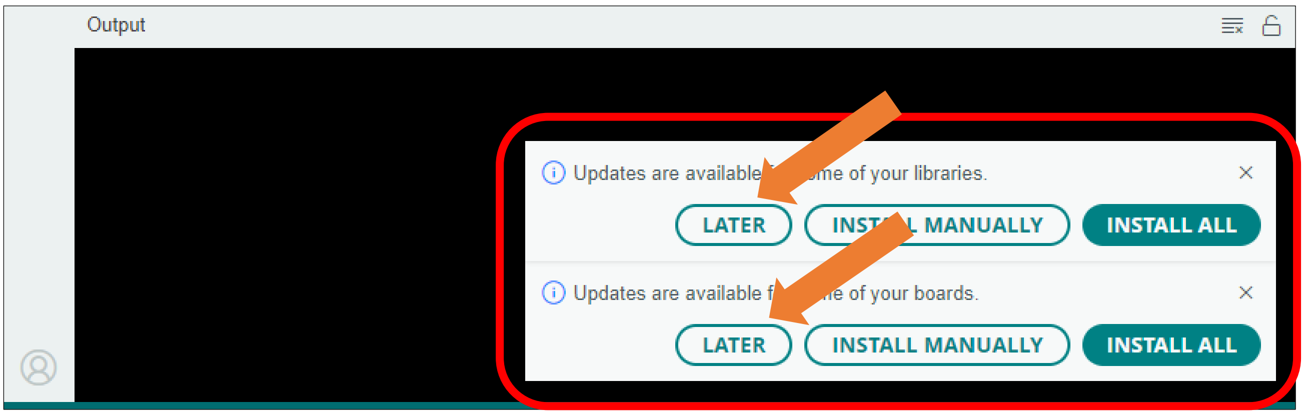


3. On the newly pop-up window, select the files from the **Freenove_ESP32_S3_Display/Libraries**. Click Open to install the library.



4. **Repeat the above steps until all the six libraries are installed to Arduino.** So far, all libraries have been installed.

Note: Some libraries are not the latest version. Please do not update them even if it prompts every time you open the IDE. Just click LATER. Otherwise, it may lead to compilation failure.



Chapter 1 Serial

Project 1.1 USB_Serial

The Freenove ESP32-S3 Display is equipped with an onboard USB2.0 port, which can be used to upload code to the ESP32-S3.

Meanwhile, it can also abstract the physical USB channel into a virtual COM port based on the USB CDC protocol, enabling the computer host to communicate with the ESP32-S3 through standard serial port tools without requiring additional hardware.

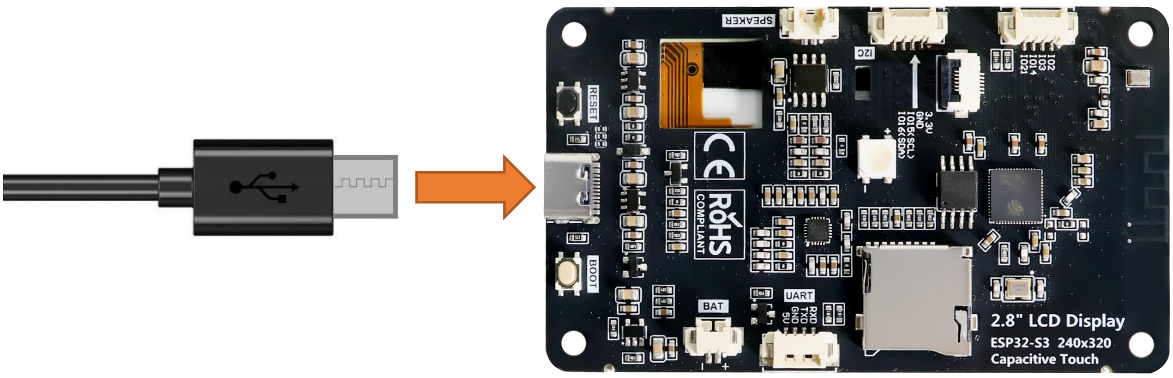
In this section, we will upload code via the USB interface and virtualize it as a serial port (COM) to enable data communication with the computer.

Component List

Freenove ESP32-S3 Display x 1	USB cable x1
	

Circuit

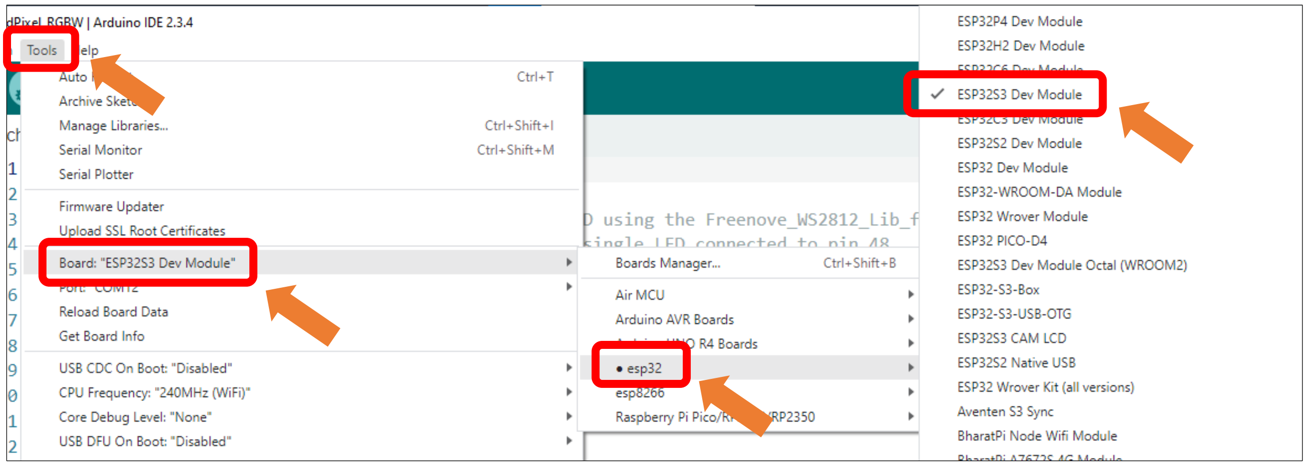
Connect Freenove ESP32-S3 Display to the computer with USB cable.



Configuration

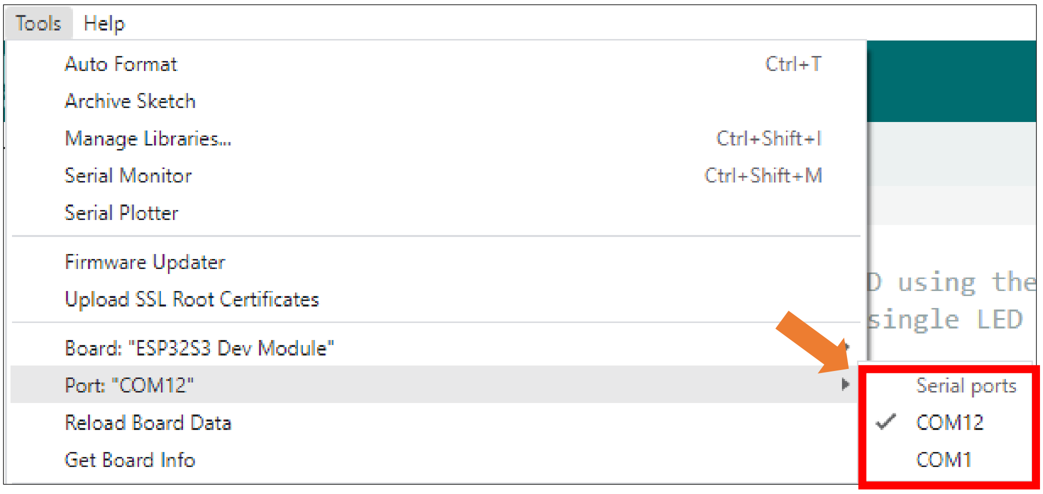
If you have not installed ESP32 SDK in Arduino IDE, please refer to Environment Configuration.

If you have installed it, please continue to proceed.
Select **Tools** –> **Board** -> **esp32** -> **ESP32S3 Dev Module**.

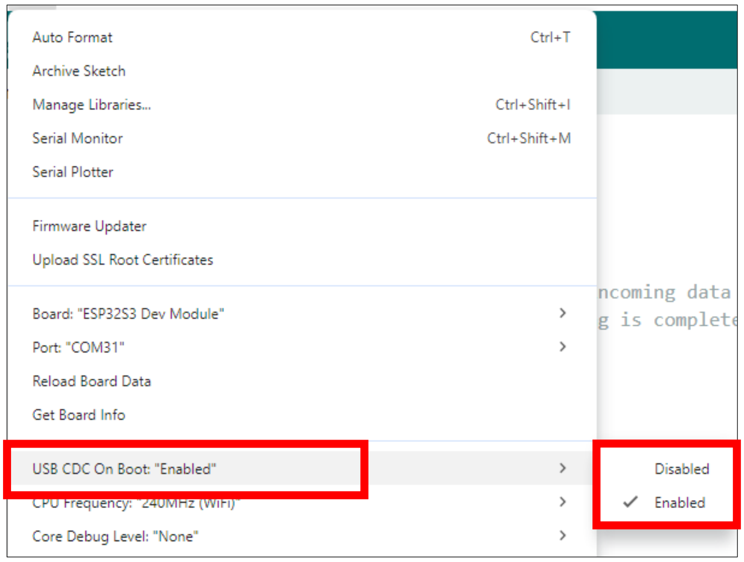


After connecting the Freenove ESP32-S3 Display, the system will assign a serial communication port named in the format 'COMx' (**where 'x' is a numeric ID that may vary across computers**). You must select the correct port under Tools → Port.

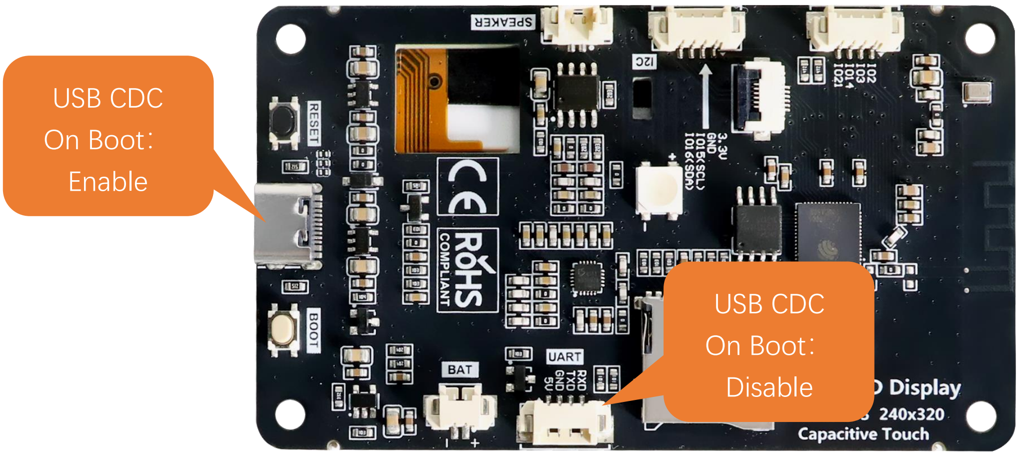
Note: COM1 is typically NOT the port of the Freenove ESP32-S3 Display.



Enable the "USB CDC On Boot" feature.



Please note that when "USB CDC On Boot" is set to "Enable", the ESP32-S3 will virtualize the onboard USB as a serial port after code upload, enabling serial communication with external devices via the data cable.



If “USB CDC On Boot” is set to “Disable”, the onboard USB interface can only be used to upload code.

Sketch

Open “**Sketch_01.1_SerialRW**” folder under **“Freenove_ESP32_S3_Display\Sketches”** and double-click “**Sketch_01.1_SerialRW.ino**”.

Sketch_01.1_SerialRW

```
String inputString = "";      //a String to hold incoming data
bool stringComplete = false;  // whether the string is complete

void setup() {
  Serial.begin(115200);
  Serial.println(String("\nESP32-S3 initialization completed!\n")
                 + String("Please input some characters,\n")
```

```
                  + String("select \"Newline\" below and click send button. \n"));
}

void loop() {
  if (Serial.available()) {         // judge whether data has been received
    char inChar = Serial.read();         // read one character
    inputString += inChar;
    if (inChar == '\n') {
      stringComplete = true;
    }
  }
  if (stringComplete) {
    Serial.printf("InputString: %s", inputString);
    inputString = "";
    stringComplete = false;
  }
}
```

Code Explanation

Set the baud rate to 115200.

```
Serial.begin(115200);
```

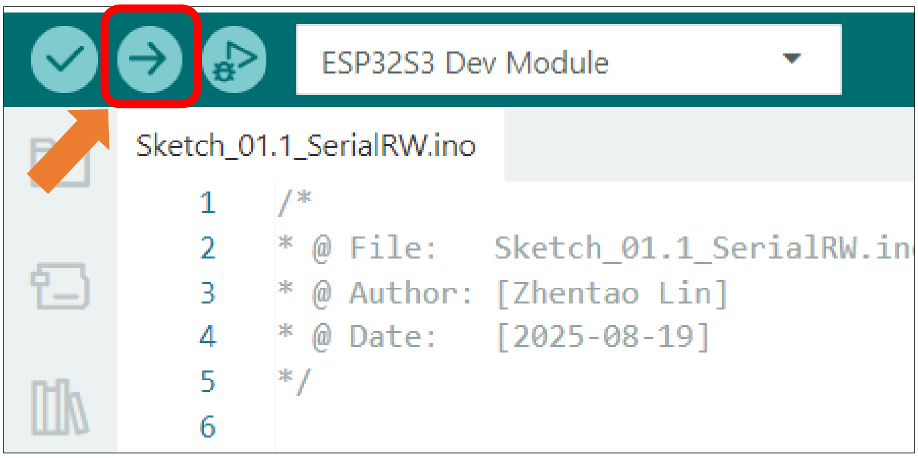
Determine whether there is data in the serial port buffer.

```
if (Serial.available())
```

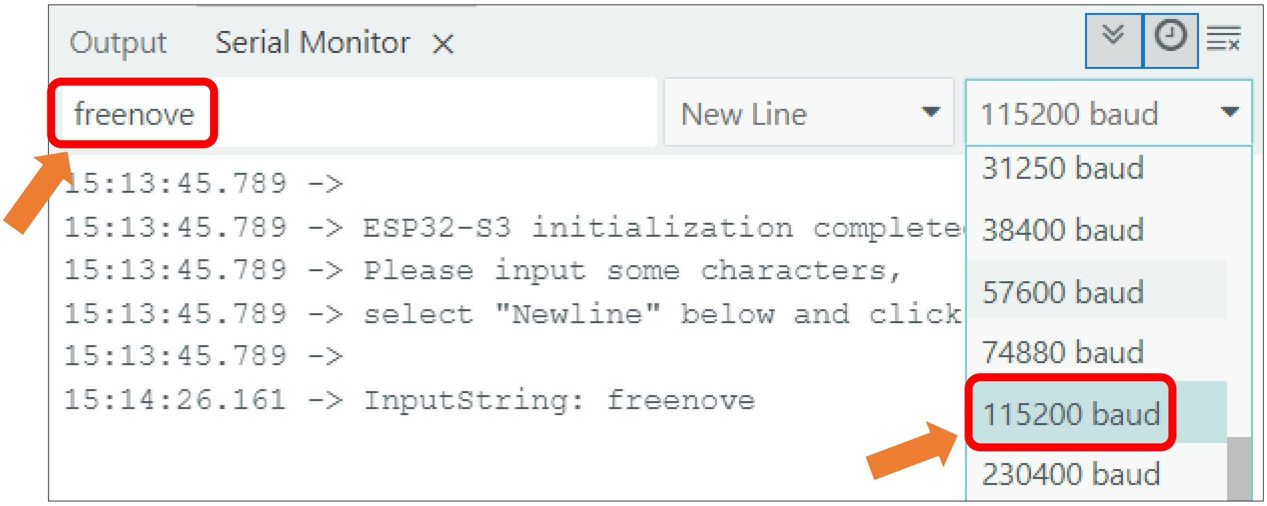
Receive serial port data and save it in the inputString string.

```
char inChar = Serial.read();         // read one character
inputString += inChar;
```

The purpose of this code is to display data on the serial monitor. Click “**Upload**” to upload the code to Freenove ESP32-S3 Display.



After downloading the code, open the serial port monitor, and set the baud rate to **115200**, input any data in the messages bard and press Enter key, Freenove ESP32-S3 Display will print the received data.



If your serial monitor remains unresponsive, please verify that "USB CDC On Boot" is set to "Enable", then recompile and upload the code.

Important notes:

1. When "USB CDC On Boot" is enabled, the USB port serves both for code uploading and as a Serial interface, while the hardware UART port operates as Serial1.

2. When "USB CDC On Boot" is disabled, the USB port is used exclusively for code uploading and cannot function as a Serial interface. In this case, the UART port is Serial.

3. All examples in this tutorial require "USB CDC On Boot" to be enabled by default. To communicate with external devices via the onboard UART, use Serial1 instead, which shares identical usage methods with Serial.

Reference

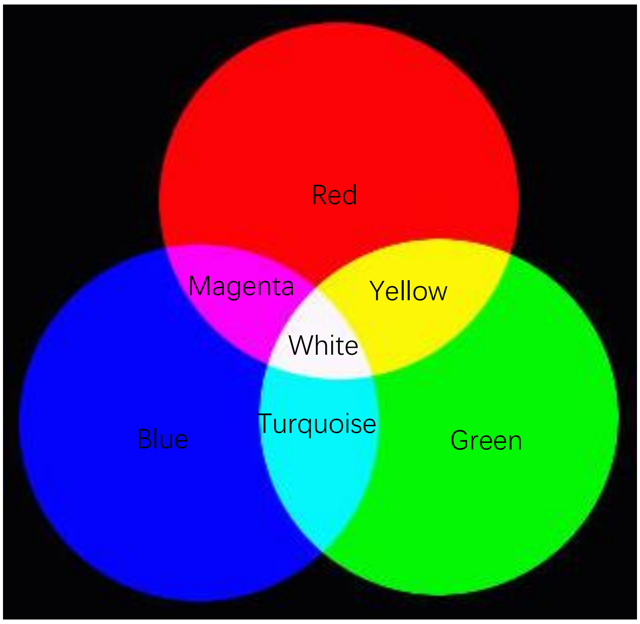
`int available()`
Serial.available() checks the number of bytes currently available to read in the Serial receive buffer. It returns the number of bytes available (int type), or 0 if the buffer is empty.
`int read ()`
Serial.read() reads one byte of data from the Serial receive buffer and returns it as an int. If no data is available to read, it returns -1.

Chapter 2 RGB

Project 2.1 RGB

Related Knowledge

Red, green, and blue are called the three primary colors. When you combine these three primary colors of different brightness, it can produce almost all kinds of visible light.



RGB

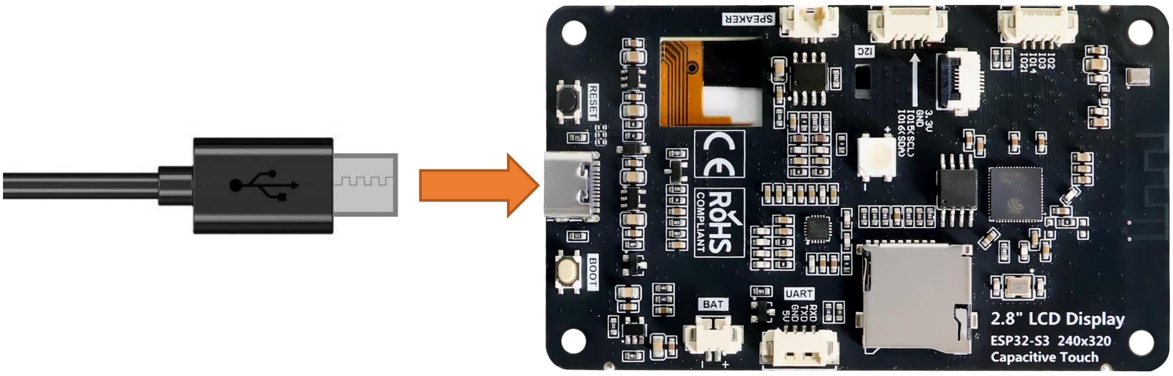
The onboard RGB LED can emit three basic colors of red, green and blue, and supports 256-level brightness adjustment, which means that it can emit 2^24=16,777,216 different colors.

Component List

Freenove ESP32-S3 Display x 1	USB cable x1
	

Circuit

Connect Freenove ESP32-S3 Display to the computer with USB cable.



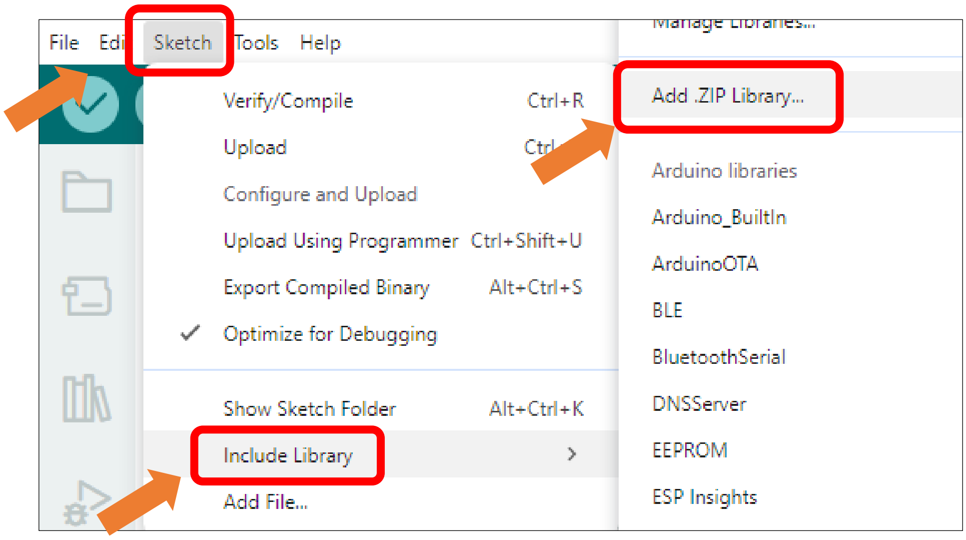
Sketch

Open "**Sketch_02.1_LedPixel**" folder under **"Freenove_ESP32_S3_Display\Sketches"** and double-click "**Sketch_02.1_LedPixel.ino**".

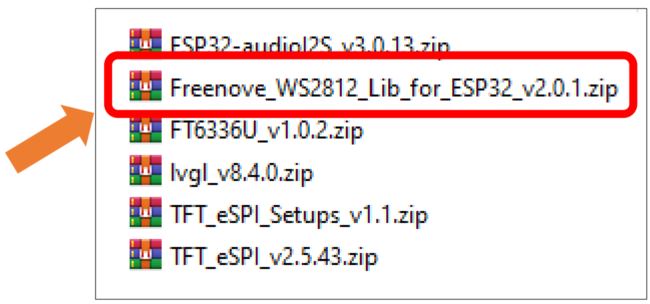
Install the needed libraries.

If you have not installed all libaries as per the previous section, please follow the following steps to install the ws2812 library.

Click **Sketch** -> **Include Library** -> **Add .ZIP Library...**



Select **Freenove_WS2812_Lib_for_ESP32_v2.0.1.zip**



Sketch_02.1_LedPixel

The following is the program code:

```
#include "Freenove_WS2812_Lib_for_ESP32.h"

#define FNK0104AB_2P8_240x320_ILI9341
//#define FNK0104N_3P5_320x480_ST77922
//#define FNK0104S_4P0_320x480_ST7796

#ifdef FNK0104N_3P5_320x480_ST77922
 #define LEDS_PIN   40
#else
 #define LEDS_PIN   42
#endif
#define LEDS_COUNT 1
#define CHANNEL    0

Freenove_ESP32_WS2812 strip = Freenove_ESP32_WS2812(LEDS_COUNT, LEDS_PIN, CHANNEL, TYPE_GRB);

uint8_t m_color[5][3] = { { 255, 0, 0 }, { 0, 255, 0 }, { 0, 0, 255 }, { 255, 255, 255 }, { 0,
0, 0 } };
int delayval = 100;

void setup() {
  strip.begin();
  strip.setBrightness(10);
}
void loop() {
  for (int j = 0; j < 5; j++) {
    for (int i = 0; i < LEDS_COUNT; i++) {
      strip.setLedColorData(i, m_color[j][0], m_color[j][1], m_color[j][2]);
      strip.show();
      delay(delayval);
    }
    delay(500);
  }
}
```

Code Explanation

Define the pins for the RGB LED.

```
#ifdef FNK0104N_3P5_320x480_ST77922
 #define LEDS_PIN   40
#else
 #define LEDS_PIN   42
#endif
#define LEDS_COUNT 1
```

```
#define CHANNEL    0
```

Initialize the LED, set the brightness to 10

```
strip.begin();
strip.setBrightness(10);
```

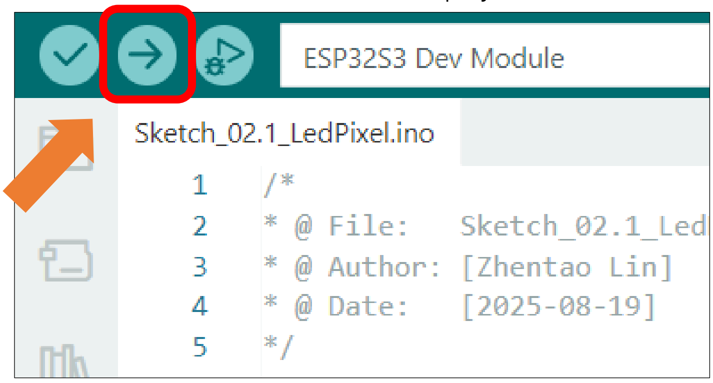
Cycle through five different colors.

```
for (int j = 0; j < 5; j++) {
  for (int i = 0; i < LEDS_COUNT; i++) {
    strip.setLedColorData(i, m_color[j][0], m_color[j][1], m_color[j][2]);
    strip.show();
    delay(delayval);
  }
  delay(500);
}
```

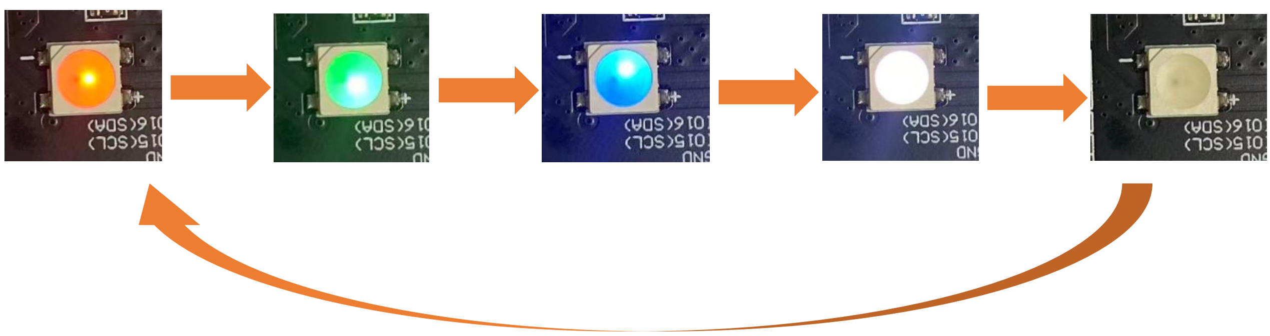
Before compiling and uploading the code, please be sure to confirm the hardware model you are using. Based on your device, please modify the macro definition (#define) at the top of the code: remove the comment symbols (//) in front of the corresponding model, and ensure that the other models remain commented out. The default setting is the 2.8-inch model.

```
#define FNK0104AB_2P8_240x320_ILI9341
//#define FNK0104N_3P5_320x480_ST77922
//#define FNK0104S_4P0_320x480_ST7796
```

Click "**Upload**" to upload the code to Freenove_ESP32_S3_Display.



The RGB LEDs will cycle through Red -> Green -> Blue –> White -> Off every 500ms after code upload.



Reference

Freenove_ESP32_WS2812(u16 n = 8, u8 pin_gpio = 2, u8 chn = 0, LED_TYPE t = TYPE_GRB)
A constructor to create a ws2812 object. Before each use of the constructor, please add "**#include "Freenove_WS2812_Lib_for_ESP32.h"**" **Parameters**

n: The number of led.

pin_gpio: The pin connected to the LED.

Chn: RMT channel, which has eight channels, 0-7, and uses channel 0 by default. This means that you can use eight ws2812 modules for the display at the same time, and these modules do not interfere with each other.

t: Types of LED.

TYPE_RGB: The sequence of ws2812 module loading color is red, green and blue.

TYPE_RBG: The sequence of ws2812 module loading color is red, blue and green.

TYPE_GRB: The sequence of ws2812 module loading color is green, red and blue.

TYPE_GBR: The sequence of ws2812 module loading color is green, blue and red.

TYPE_BRG: The sequence of ws2812 module loading color is blue, red and green.

TYPE_BGR: The sequence of ws2812 module loading color is blue, green and red.

Project 2.2 Rainbow

In the previous project, we have mastered the use of LEDPixel. This project will realize a slightly complicated rainbow light. The component list and the circuit are exactly the same as the project fashionable light.

Sketch

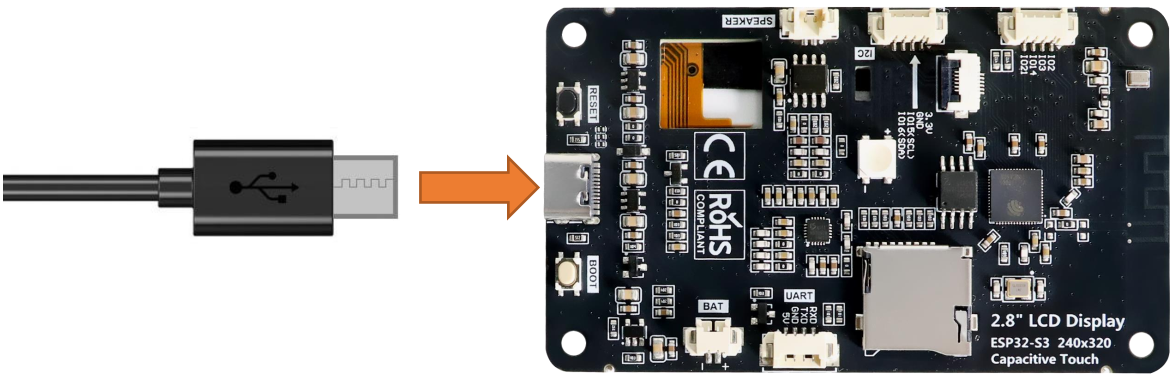
Continue to use the following color model to equalize the color distribution of the LED and gradually change.

Component List

Freenove ESP32-S3 Display x 1	USB cable x1
	

Circuit

Connect Freenove ESP32-S3 Display to the computer with USB cable.



Sketch

Open "**Sketch_02.2_Rainbow**" folder under **"Freenove_ESP32_S3_Display\Sketches"** and double-click "**Sketch_02.2_Rainbow**".

Sketch_02.2_Rainbow

The following is the program code:

```
#include "Freenove_WS2812_Lib_for_ESP32.h"

#define FNK0104AB_2P8_240x320_ILI9341
//#define FNK0104N_3P5_320x480_ST77922
//#define FNK0104S_4P0_320x480_ST7796

#ifdef FNK0104N_3P5_320x480_ST77922
 #define LEDS_PIN   40
#else
 #define LEDS_PIN   42
#endif
#define LEDS_COUNT 1
#define CHANNEL    0

Freenove_ESP32_WS2812 strip = Freenove_ESP32_WS2812(LEDS_COUNT, LEDS_PIN, CHANNEL, TYPE_GRB);

void setup() {
  strip.begin();
  strip.setBrightness(20);
}

void loop() {
  for (int j = 0; j < 255; j += 1) {
    for (int i = 0; i < LEDS_COUNT; i++) {
      strip.setLedColorData(i, strip.Wheel((i * 256 / LEDS_COUNT + j) & 255));
    }
    strip.show();
    delay(15);
  }
}
```

Code Explanation

In the loop(), two "for" loops are used, the internal "for" loop(for-j) is used to set the color of each LED, and the external "for" loop(for-i) is used to change the color, in which the self-increment value in i+=2 can be changed to change the color step distance. Changing the delay parameter changes the speed of the color change. `strip.Wheel((i * 256 / LEDS_COUNT + j) & 255)` will take color from the color model at equal intervals starting from i.

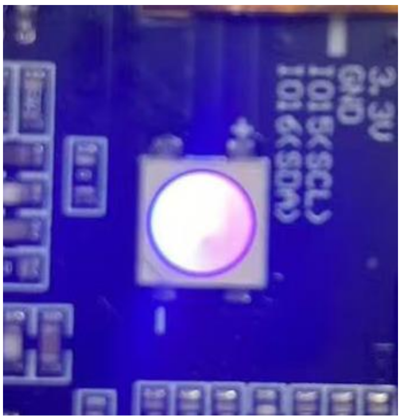
```
for (int j = 0; j < 255; j += 1) {
  for (int i = 0; i < LEDS_COUNT; i++) {
    strip.setLedColorData(i, strip.Wheel((i * 256 / LEDS_COUNT + j) & 255));
  }
  strip.show();
  delay(15);
}
```

Before compiling and uploading the code, please be sure to confirm the hardware model you are using. Based on your device, please modify the macro definition (#define) at the top of the code: remove the comment symbols (//) in front of the corresponding model, and ensure that the other models remain commented out. The default setting is the 2.8-inch model.

```
#define FNK0104AB_2P8_240x320_ILI9341
//#define FNK0104N_3P5_320x480_ST77922
//#define FNK0104S_4P0_320x480_ST7796
```

This code achieves the function of printing data on serial monitor. Click "**Upload**" to upload the code to Freenove_ESP32_S3_Display.

After uploading the code, the LED lights will display a smooth rainbow gradient effect.



Reference

`bool ledcAttachChannel(uint8_t pin, uint32_t freq, uint8_t resolution, uint8_t channel);`

This function binds the specified PWM channel to a GPIO pin and configures the frequency and resolution of the PWM signal.

Parameters:

pin: GPIO pin number to bind

freq: PWM signal frequency in Hz

resolution: Bit depth for PWM duty cycle resolution (range: 1-16 bits).
For example, 8 represents 2^8 (0~255) levels.
channel: PWM channel number to assign (integer)

```
void ledcWrite(uint8_t channel, uint32_t duty);
```

This function sets the duty cycle for a specified PWM channel to control output signal intensity.

Parameters:

channel: PWM channel number
duty: Duty cycle value (range determined by resolution bit depth)

Chapter 3 Button

The Boot button on the Freenove ESP32-S3 Display can be configured as a regular input button after the program starts.

Project 3.1 Button RGB

In the previous chapter, we learned about RGB LEDs. In this section, we will further explore how to integrate RGB LEDs with buttons.

Related Knowledge

Button

In embedded systems, buttons are one of the most common human-machine input devices. When a button is not pressed, its opposing contacts are connected while adjacent contacts remain disconnected. Conversely, when the button is pressed, adjacent contacts connect while opposing contacts disconnect.

Usage of the Boot Button on the Freenove ESP32-S3 Display:

Manual Entry into Download Mode:

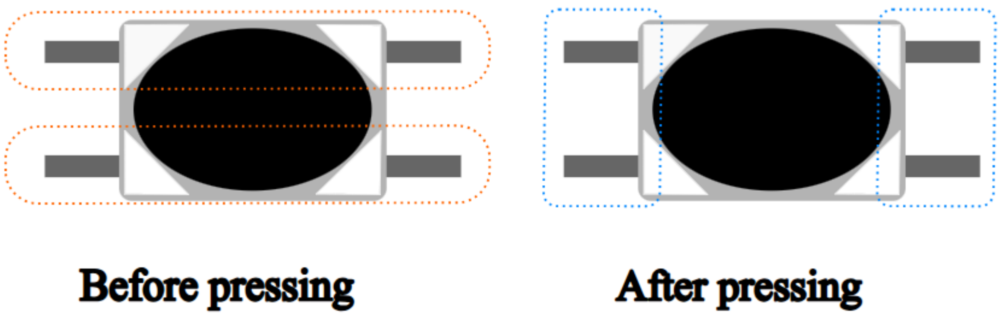
When the board is powered on or reset, pressing and holding the Boot button forces the board into download mode.

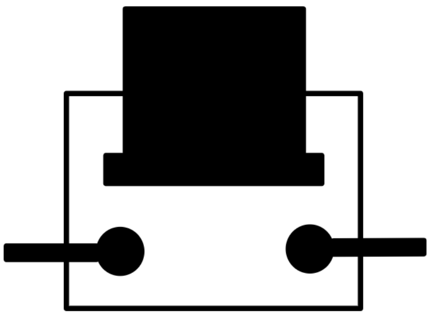
In this mode, users can upload programs to the board via the serial port.

Use as a Regular Input Button:

If the Boot button is not pressed during power-on or reset, the board enters normal operation mode.

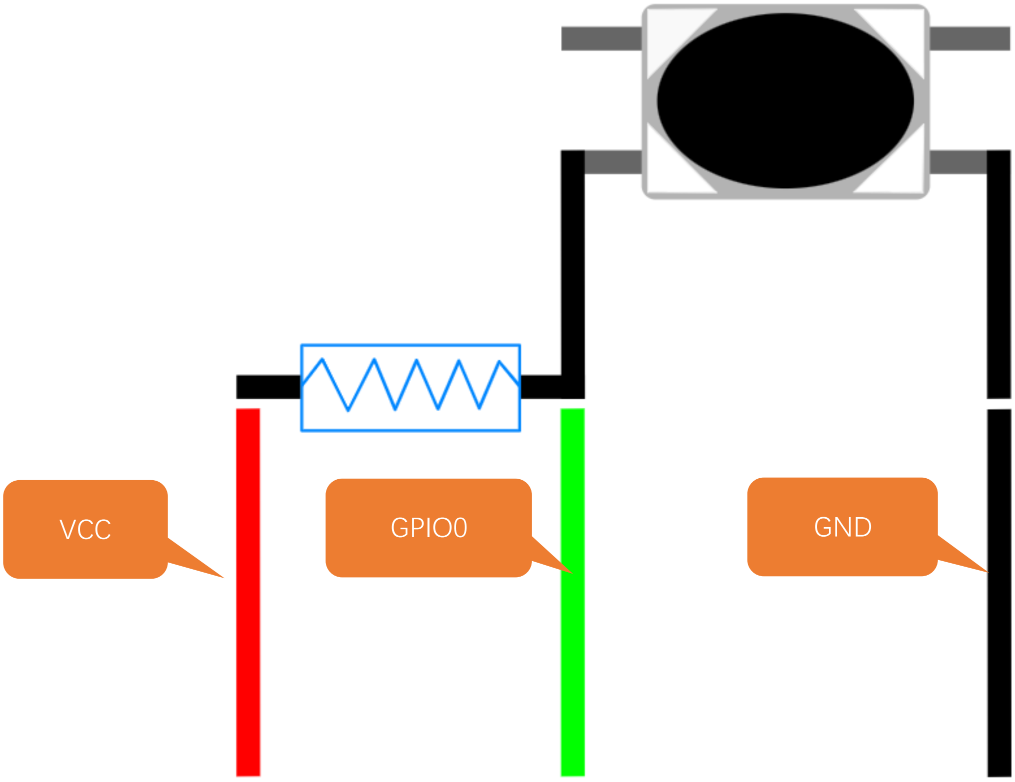
Once in operation mode, the Boot button can function as GPIO0, typically configured as a standard input button for user interaction.





Pull-up Resistor

The primary function of a pull-up resistor is to ensure that the button maintains a **stable high-level state** when not pressed, preventing false triggering caused by floating pins or signal noise. The pull-up resistor is connected between the GPIO pin and VCC.



When the button is not pressed, Vcc is connected to GPIO0 through the resistor, resulting in a high-level signal at this pin.

When the button is pressed, GPIO0 is connected to GND, resulting in a low-level signal.

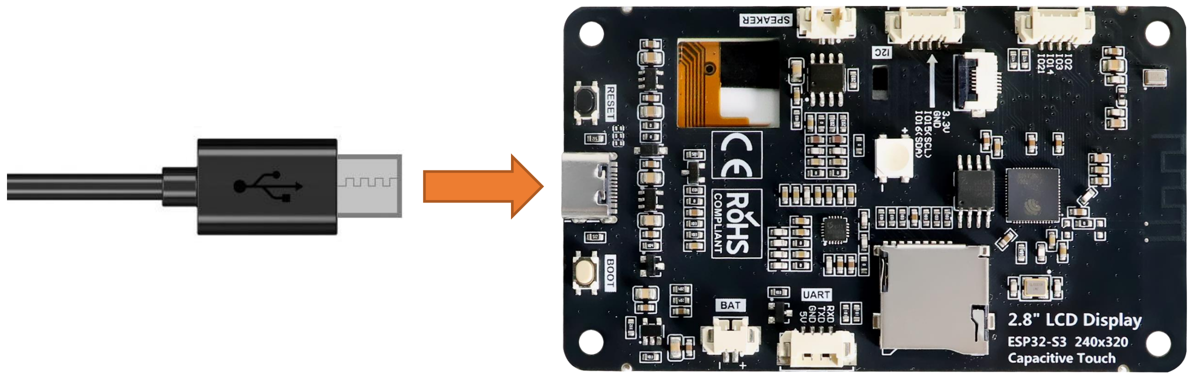
Therefore, by reading the GPIO0 pin state, we can determine whether the button is pressed or not.

Component List

Freenove ESP32-S3 Display x 1	USB cable x1
	

Circuit

Connect Freenove ESP32-S3 Display to the computer with USB cable.



Sketch

Open “**Sketch_03.1_Button_RGB**” folder under **“Freenove_ESP32_S3_Display\Sketches”** and double-click “**Sketch_03.1_Button_RGB.ino**”.

Sketch_03.1_Button_RGB

The following is the program code:

```
#include <Arduino.h>
#include "Freenove_WS2812_Lib_for_ESP32.h"

#define FNK0104AB_2P8_240x320_ILI9341
//#define FNK0104N_3P5_320x480_ST77922
//#define FNK0104S_4P0_320x480_ST7796

#ifdef FNK0104N_3P5_320x480_ST77922
 #define LEDS_PIN    40
#else
 #define LEDS_PIN    42
#endif
#define LEDS_COUNT 1
#define CHANNEL     0
#define KEY_PIN 0

Freenove_ESP32_WS2812 strip = Freenove_ESP32_WS2812(LEDS_COUNT, LEDS_PIN, CHANNEL, TYPE_GRB);

uint8_t m_color[5][3] = { { 255, 0, 0 }, { 0, 255, 0 }, { 0, 0, 255 }, { 255, 255, 255 }, { 0,
0, 0 } };

static unsigned int keyCount = 0;
static unsigned int lastKeyCount = 1;
```

```
void buttonInit(void) {
  pinMode(KEY_PIN, INPUT_PULLUP);
}

bool readButton(void) {
  return digitalRead(KEY_PIN);
}

void switchRGB(int value) {
  int colorIndex = value % 4;
  switch (colorIndex) {
    case 1:
      strip.setLedColorData(0, m_color[0][0], m_color[0][1], m_color[0][2]);
      strip.show();
      break;
    case 2:
      strip.setLedColorData(0, m_color[1][0], m_color[1][1], m_color[1][2]);
      strip.show();
      break;
    case 3:
      strip.setLedColorData(0, m_color[2][0], m_color[2][1], m_color[2][2]);
      strip.show();
      break;
    default:
      strip.setLedColorData(0, m_color[4][0], m_color[4][1], m_color[4][2]);
      strip.show();
      break;
  }
}

void setup() {
  strip.begin();
  strip.setBrightness(10);
  buttonInit();
  Serial.println("ESP32-S3 initialization completed!");
}

void loop() {
  if (readButton() == 0) {
    delay(20);
    if (readButton() == 0) {
      keyCount++;
      while (!readButton());
```

```
    }
  }
  if (keyCount != lastKeyCount) {
    switchRGB(keyCount);
    lastKeyCount = keyCount;
  }
}
```

Code Explanation

Define the pins for the button and the RGB LED.

```
#ifdef FNK0104N_3P5_320x480_ST77922
 #define LEDS_PIN    40
#else
 #define LEDS_PIN    42
#endif
#define LEDS_COUNT 1
#define CHANNEL     0
#define KEY_PIN 0
```

Initialize the RGB LED and the button.

```
strip.begin();
strip.setBrightness(10);
buttonInit();
```

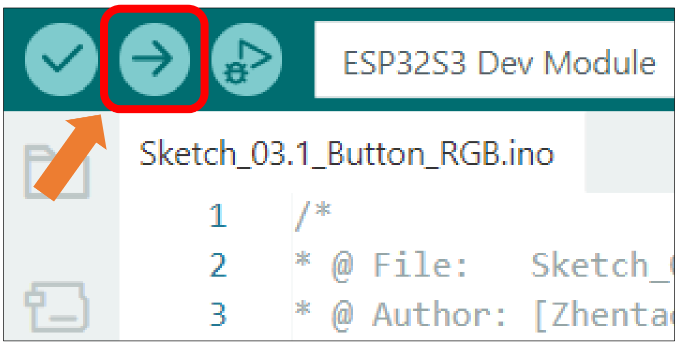
Control the color of the RGB LED through the button.

```
if (readButton() == 0) {
  delay(20);
  if (readButton() == 0) {
    keyCount++;
    while (!readButton());
  }
}
if (keyCount != lastKeyCount) {
  switchRGB(keyCount);
  lastKeyCount = keyCount;
}
```

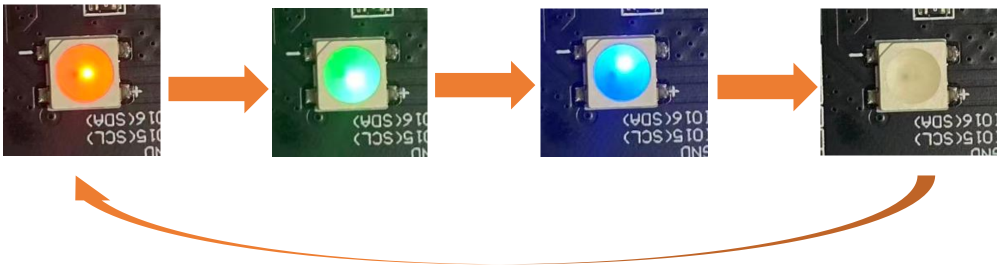
Before compiling and uploading the code, please be sure to confirm the hardware model you are using. Based on your device, please modify the macro definition (#define) at the top of the code: remove the comment symbols (//) in front of the corresponding model, and ensure that the other models remain commented out. The default setting is the 2.8-inch model.

```
#define FNK0104AB_2P8_240x320_ILI9341
//#define FNK0104N_3P5_320x480_ST77922
//#define FNK0104S_4P0_320x480_ST7796
```

Click “**Upload**” to upload the code to Freenove ESP32-S3 Display.



The RGB LEDs will cycle through Red -> Green -> Blue -> OFF every 500ms after code upload.



Chapter 4 Button Interrupt

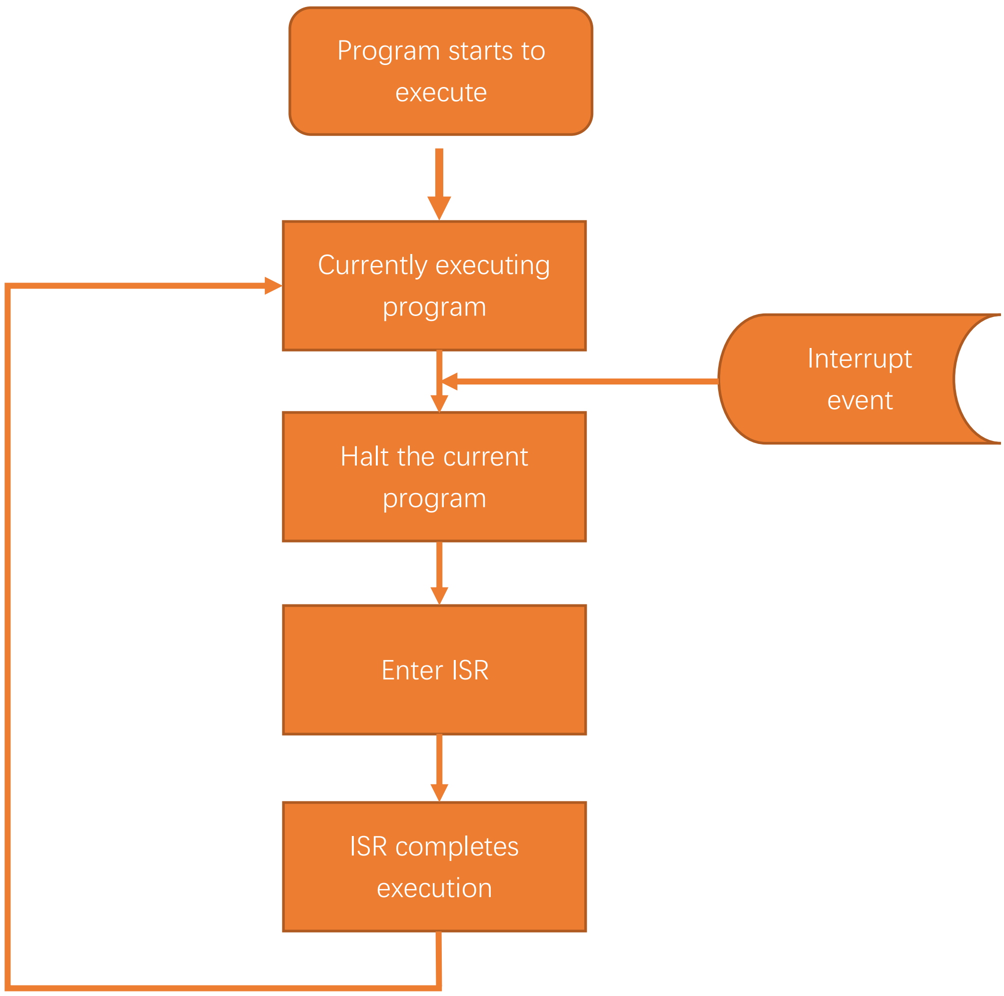
In this chapter will learn to use interrupt to detect the button state.

Project 4.1 Button Interrupt UART

Related Knowledge

How Interrupts Work

When an interrupt event is triggered, the Freenove ESP32-S3 Display receives an interrupt signal. At this moment, the Freenove ESP32-S3 Display pauses the currently executing program and instead executes the Interrupt Service Routine (ISR). The ISR contains the code that needs to be processed after the interrupt event occurs. Once the ISR execution is complete, the Freenove ESP32-S3 Display will return to the state it was in before the interrupt occurred and continue executing the paused program, as shown in the diagram below.

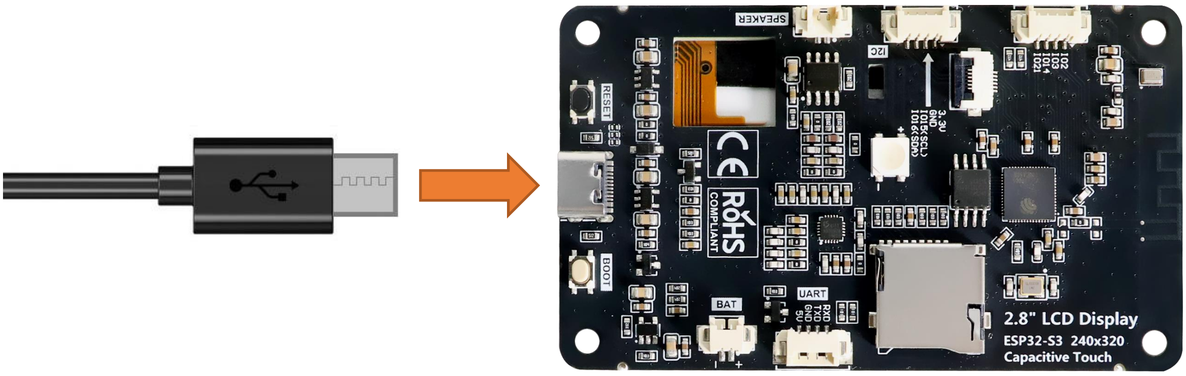


Component List

Freenove ESP32-S3 Display x 1	USB cable x1
	

Circuit

Connect Freenove ESP32-S3 Display to the computer with USB cable.



Sketch

Open "**Sketch_04.1_Button_Interrupt_UART**" folder under **"Freenove_ESP32_S3_Display\Sketches"** and double-click "**Sketch_04.1_Button_Interrupt_UART.ino**".

Sketch_04.1_Button_Interrupt_UART

The following is the program code:

```
#include <Arduino.h>

#define KEY_PIN 0
volatile int interruptCounter = 0;
int lastValue = 0;
portMUX_TYPE mux = portMUX_INITIALIZER_UNLOCKED;

void buttonInit(void) {
   pinMode(KEY_PIN, INPUT_PULLUP);
   attachInterrupt(digitalPinToInterrupt(KEY_PIN), handleInterrupt, FALLING);
}
```

```
void delayDebounce(unsigned int delayTime) {
   unsigned int i = delayTime;
   while (--i > 0) {
     if (digitalRead(KEY_PIN) == HIGH) {
       return;
     }
   }
}

void handleInterrupt(void) {
   portENTER_CRITICAL_ISR(&mux);
   interruptCounter++;
   delayDebounce(20);
   portEXIT_CRITICAL_ISR(&mux);
}

void setup() {
   Serial.begin(115200);
   buttonInit();
   Serial.println("ESP32-S3 initialization completed!");
}

void loop() {
   if (interruptCounter != lastValue) {
     Serial.printf("interruptCounter: %d\r\n", interruptCounter);
     lastValue = interruptCounter;
   }
}
```

Code Explanation

Define the pin of the button.

```
#define KEY_PIN 0
```

Configure the button pin hardware interrupt to trigger on the falling edge signal.

```
attachInterrupt(digitalPinToInterrupt(KEY_PIN), handleInterrupt, FALLING);
```

The interrupt processing function, counting how many times the button is pressed.

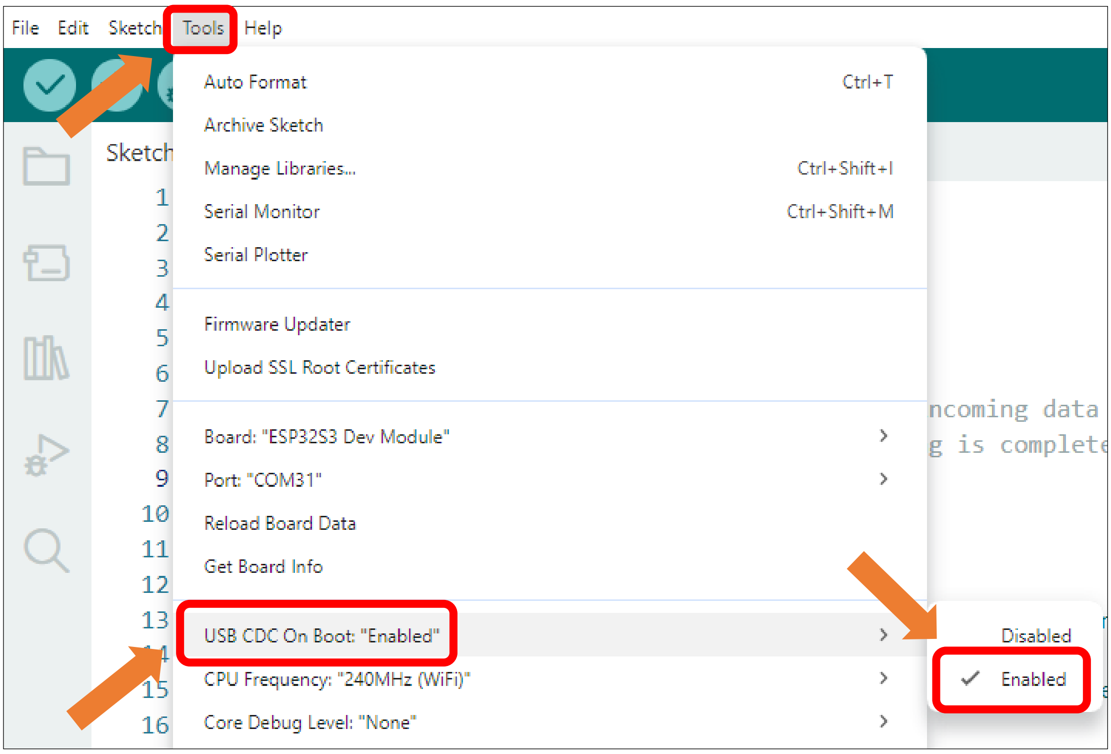
```
void handleInterrupt(void) {
   portENTER_CRITICAL_ISR(&mux);
   interruptCounter++;
   delayDebounce(20);
   portEXIT_CRITICAL_ISR(&mux);
}
```

When the number of button presses changes, print new data in the serial monitor.

```
void loop() {
   if (interruptCounter != lastValue) {
```

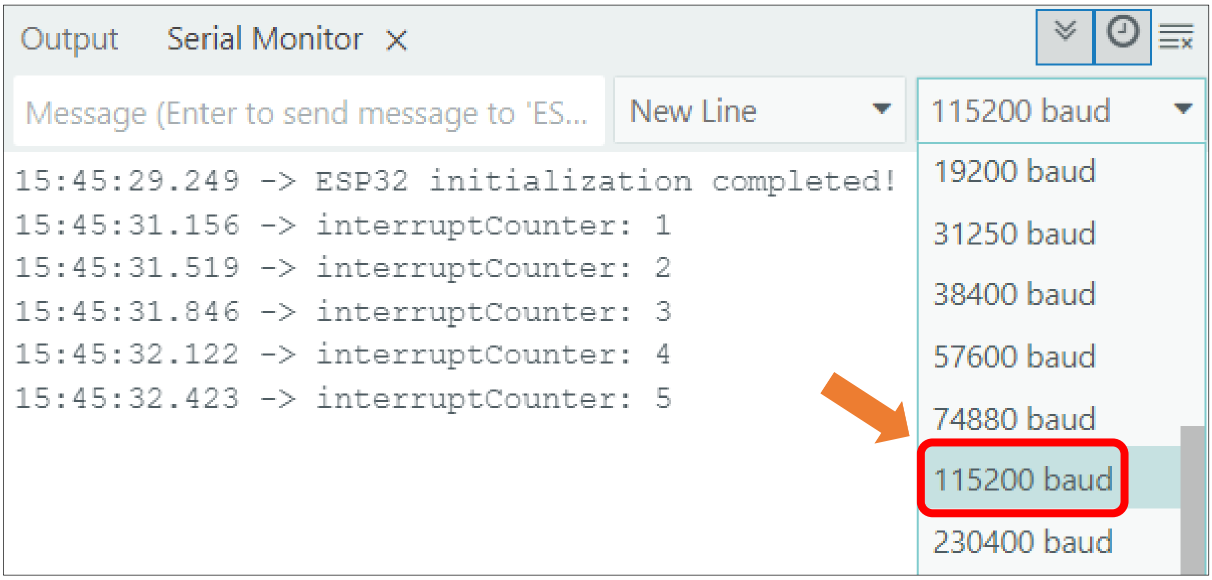
```
            Serial.printf("interruptCounter: %d\r\n", interruptCounter);
            lastValue = interruptCounter;
        }
    }
```

Enable the "USB CDC On Boot" feature.



Click "**Upload**" to upload the code to Freenove ESP32-S3 Display.

When the code finishes uploading, open the serial monitor and set the baud rate to 115200. The serial monitor will print the number of button presses.



Reference

```
void attachInterrupt(uint8_t pin, void (*)(void), int mode);
```

external interrupts

Parameters:

pin: The digital pin number for the interrupt

void: The name of the interrupt function

mode:

- LOW (trigger on low level)
- CHANGE (trigger on change)
- RISING (trigger on rising edge)
- FALLING (trigger on falling edge)

Chapter 5 Battery Voltage

Project 5.1 Battery Voltage

Related Knowledge

ADC

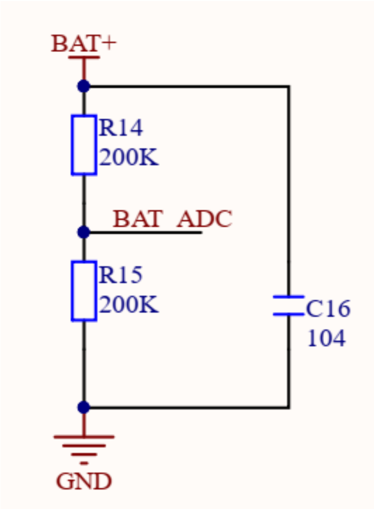
ADC is an electronic integrated circuit used to convert analog signals such as voltages to digital or binary form consisting of 1s and 0s. The range of our ADC on Freenove ESP32-S3 Display is 12 bits, which means the resolution is 2^12=4095, and it represents a range (at 3.3V) will be divided equally to 4095 parts. The rage of analog values corresponds to ADC values. So the more bits the ADC has, the denser the partition of analog will be and the greater the precision of the resulting conversion.

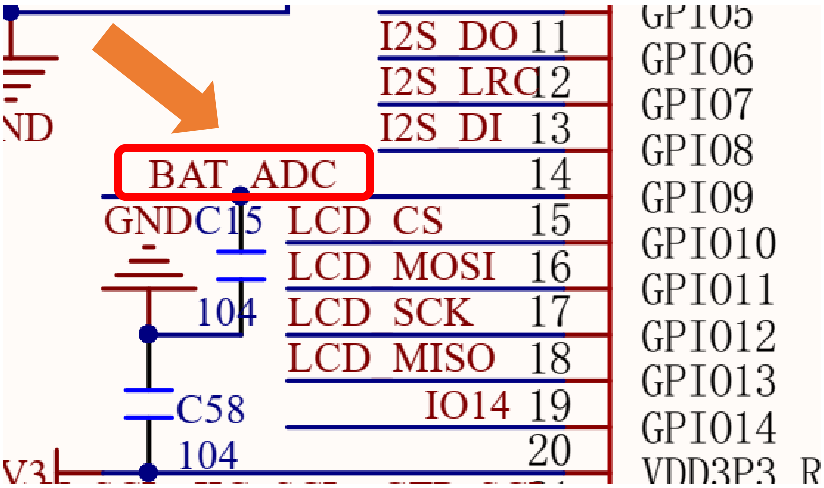
Subsection 1: the analog in rang of 0V---3.3/4095 V corresponds to digital 0;
Subsection 2: the analog in rang of 3.3/4095 V---2*3.3 /4095V corresponds to digital 1;
The following analog will be divided accordingly.
The conversion formula is as follows:

$$ADC\ Value = \frac{\text{Analog Voltage}}{3.3} * 4095$$

Battery Voltage

From the schematic diagram, we can know that Freenove ESP32-S3 Display reads the battery voltage through the ADC pin (GPIO9).





The ADC pin on this board has a voltage sampling range of 0~3.3V. However, when powered by a 3.7V lithium battery, the output voltage in a fully charged state can reach 4.2V, exceeding the ADC's rated input range.

To ensure accurate voltage measurement and ADC pin protection, the circuit employs a resistive voltage divider that scales the battery voltage down by a factor of 0.5. At a full charge (4.2V), the divided voltage at the ADC input is 2.1V, well within the safe operating range. This design enables reliable battery monitoring while preventing overvoltage damage to the ADC.

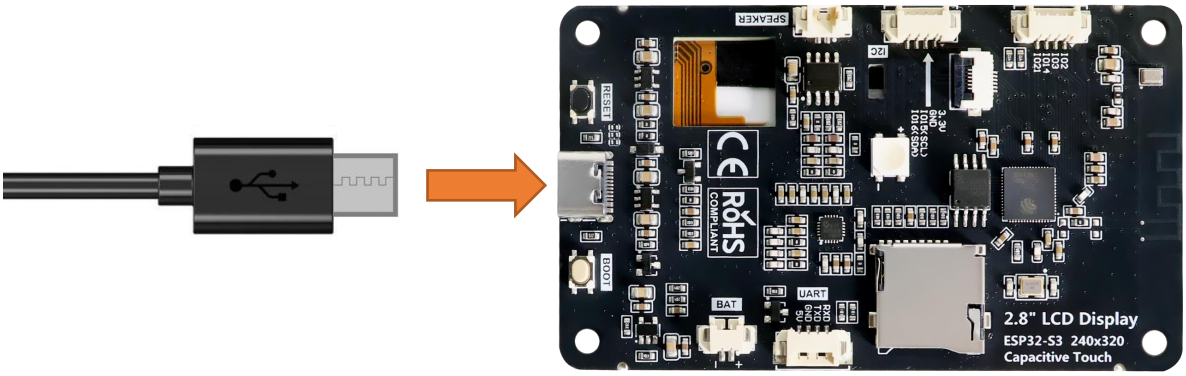
Please note that this kit does not include a lithium battery, please buy one yourself. For more information about battery, please refer to Battery.

Component List

Freenove ESP32-S3 Display x 1	USB cable x1
	

Circuit

Connect Freenove ESP32-S3 Display to the computer with USB cable.



Sketch

Next, we download the code to Freenove ESP32-S3 Display to test Serial. Open "**Sketch_05.1_Battery_Voltage.ino**" folder under **"Freenove_ESP32_S3_Display\Sketches"** and double-click "**Sketch_05.1_Battery_Voltage.ino**".

Sketch_05.1_Battery_Voltage

The following is the program code:

```
/*
* @ File:   Sketch_05.1_Battery_Voltage.ino
* @ Author: [Eason Shen]
* @ Date:   [2026-05-15]
*/

#define FNK0104AB_2P8_240x320_ILI9341
//#define FNK0104N_3P5_320x480_ST77922
//#define FNK0104S_4P0_320x480_ST7796

#ifdef FNK0104N_3P5_320x480_ST77922
 #define BAT_ADC_PIN  8
#else
 #define BAT_ADC_PIN  9
#endif

void setup(){
  Serial.begin(115200);
  pinMode(BAT_ADC_PIN, INPUT);
}

void loop() {
  int adcValue= analogRead(BAT_ADC_PIN);
```

```
  float batteryVoltage = analogReadMilliVolts(BAT_ADC_PIN) * 2.0 / 1000;
  Serial.printf("Reading on Pin(%d), adcValue=%d, batteryVoltage2=%fV\n", BAT_ADC_PIN,
adcValue, batteryVoltage);
  delay(300);
}
```

Code Explanation

Define the pins for the button and RGB LED.

```
#ifdef FNK0104N_3P5_320x480_ST77922
 #define BAT_ADC_PIN  8
#else
 #define BAT_ADC_PIN  9
#endif
```

Set the baud rate to 115200.

```
Serial.begin(115200);
```

Set the battery voltage detecting pin to input mode.

```
pinMode(BAT_ADC_PIN, INPUT);
```

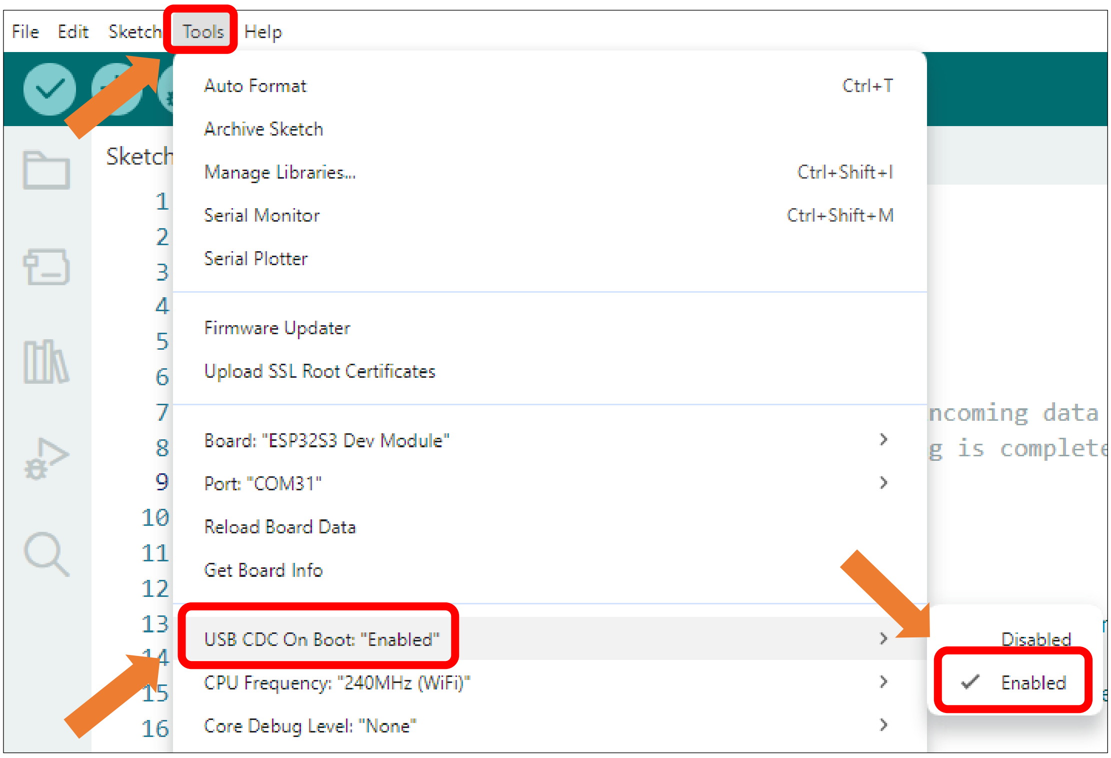
Measure the battery voltage every 300ms and outputs the reading to the serial monitor.

```
void loop() {
  int adcValue= analogRead(BAT_ADC_PIN);
  float batteryVoltage = analogReadMilliVolts(BAT_ADC_PIN) * 2.0 / 1000;
  Serial.printf("Reading on Pin(%d), adcValue=%d, batteryVoltage2=%fV\n", BAT_ADC_PIN,
adcValue, batteryVoltage);
  delay(300);
}
```

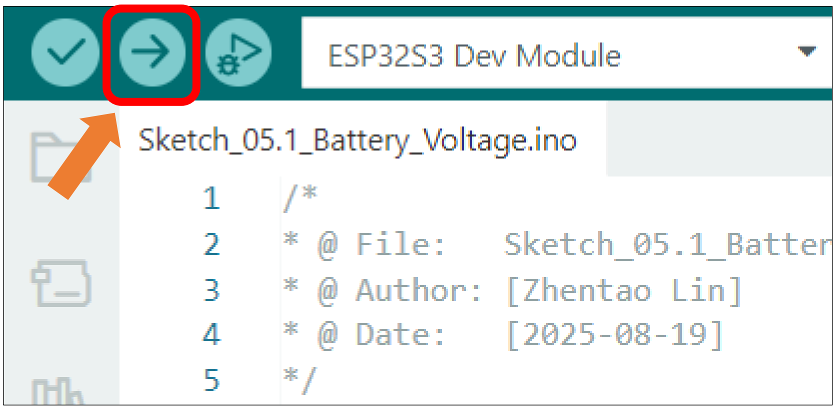
Before compiling and uploading the code, please be sure to confirm the hardware model you are using. Based on your device, please modify the macro definition (#define) at the top of the code: remove the comment symbols (//) in front of the corresponding model, and ensure that the other models remain commented out. The default setting is the 2.8-inch model.

```
#define FNK0104AB_2P8_240x320_ILI9341
//#define FNK0104N_3P5_320x480_ST77922
//#define FNK0104S_4P0_320x480_ST7796
```

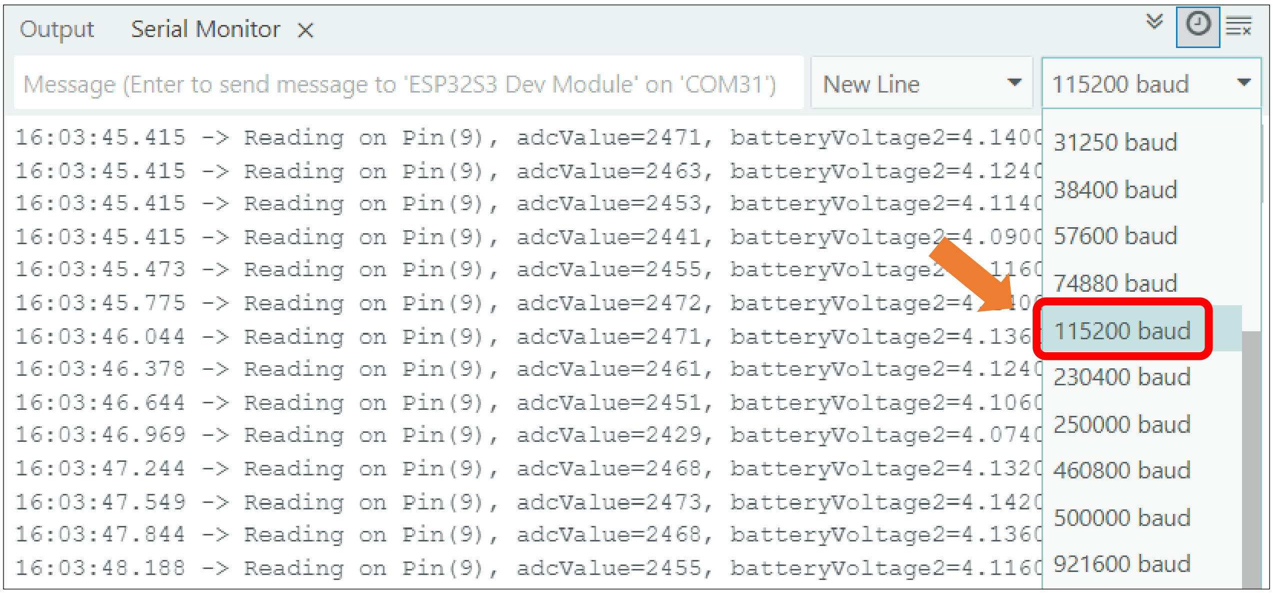
Enable the "USB CDC On Boot" feature.



Click “**Upload**” to upload the code to Freenove ESP32-S3 Display.



Once the program starts, open the serial monitor to view the battery voltage readings, updated every 300ms.



Reference

`uint32_t analogReadMilliVolts(uint8_t pin)`
This function reads the voltage directly from the analog pin and returns the value in millivolts (mV), with a maximum return value of 3300mV (3.3V)

Chapter 6 SD Card

Project 6.1 SD Test

Component List

Freenove ESP32-S3 Display x 1	USB cable x1
	

Please note that this kit does not include SD card and card reader, please buy them by yourself.

Component Knowledge

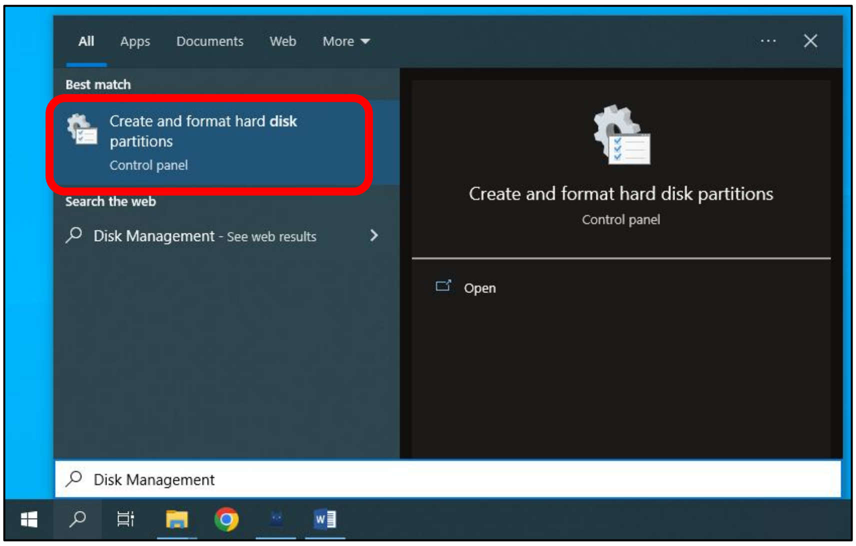
Format SD card

To initiate the project, you'll first need to prepare a blank SD card and a compatible card reader. The initial setup involves assigning a drive letter to the SD card and formatting it properly. Below you'll find system-specific instructions to complete these preparatory steps. Please follow the guide corresponding to your operating system.

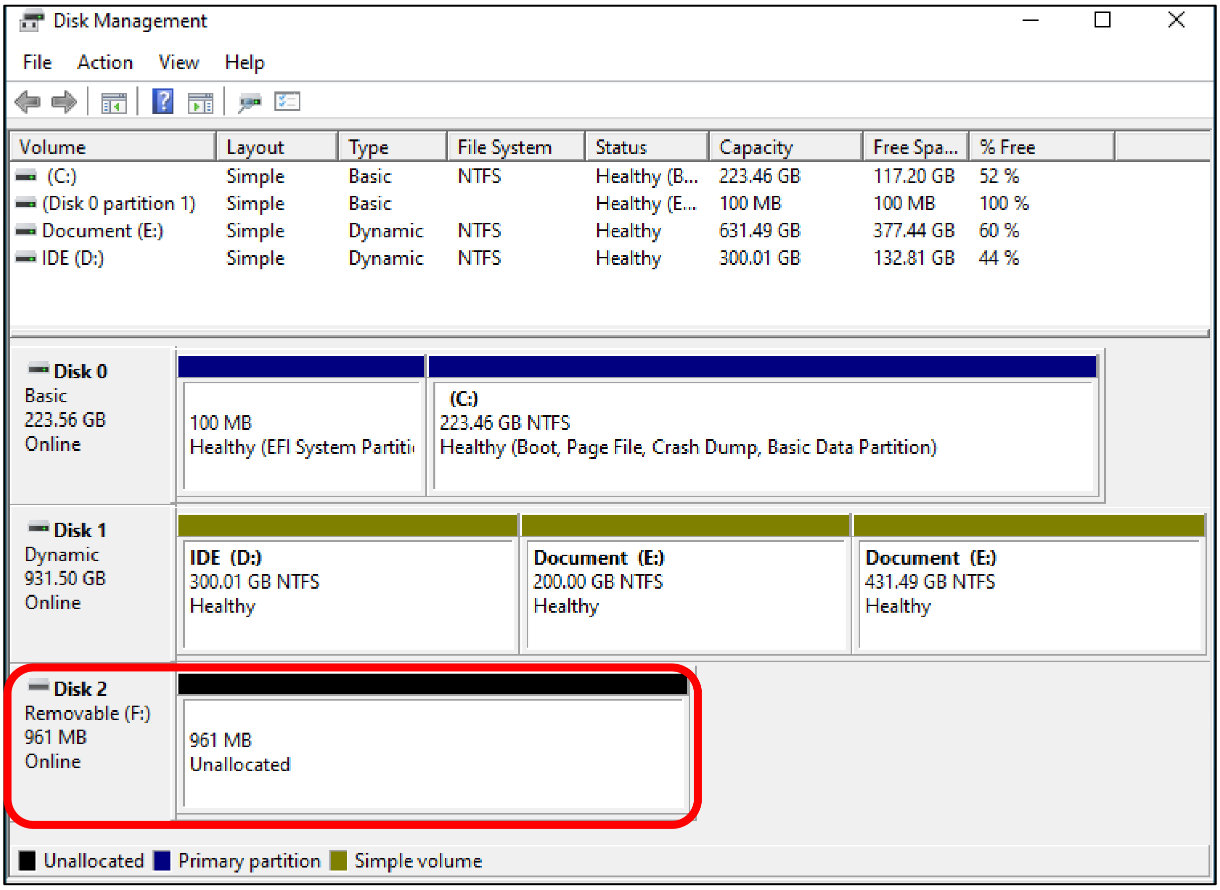
Windows

Insert the SD card into the card reader, and then insert the card reader into the computer.

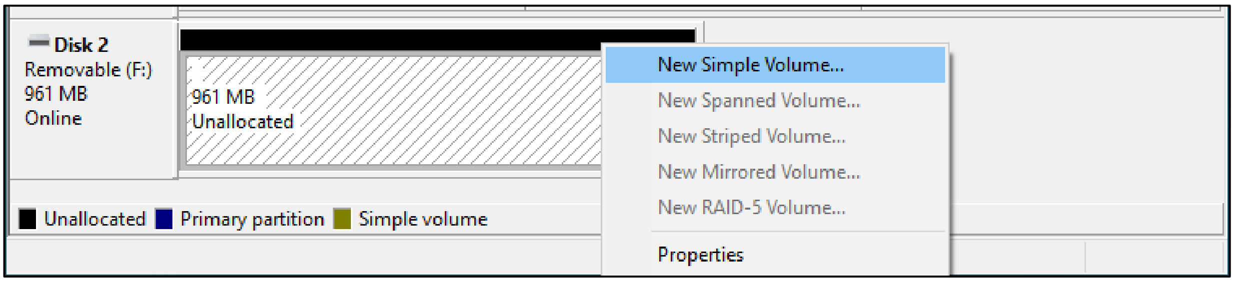
In the Windows search box, enter "Disk Management" and select "Create and format hard disk partitions".



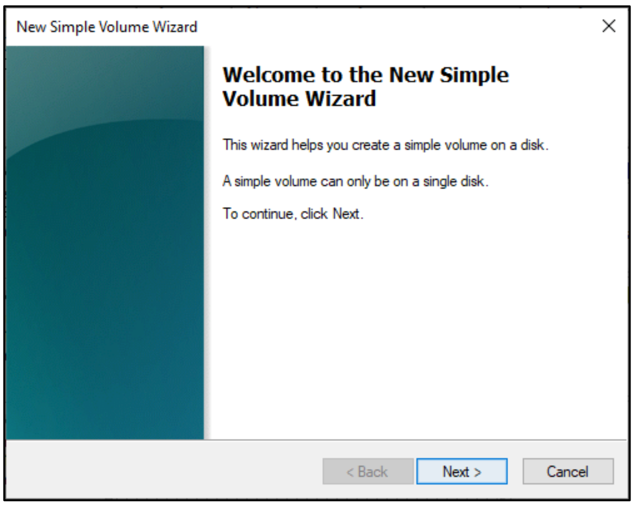
In the newly opened window, locate an unallocated volume with a size close to 1GB.

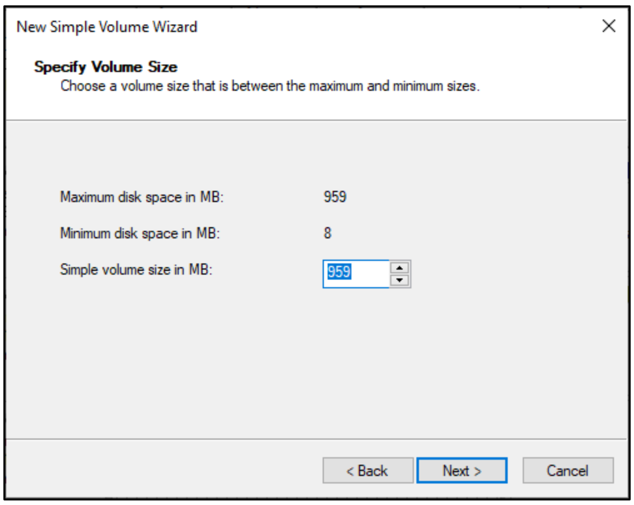


Click to select the volume, right-click and select "New Simple Volume".

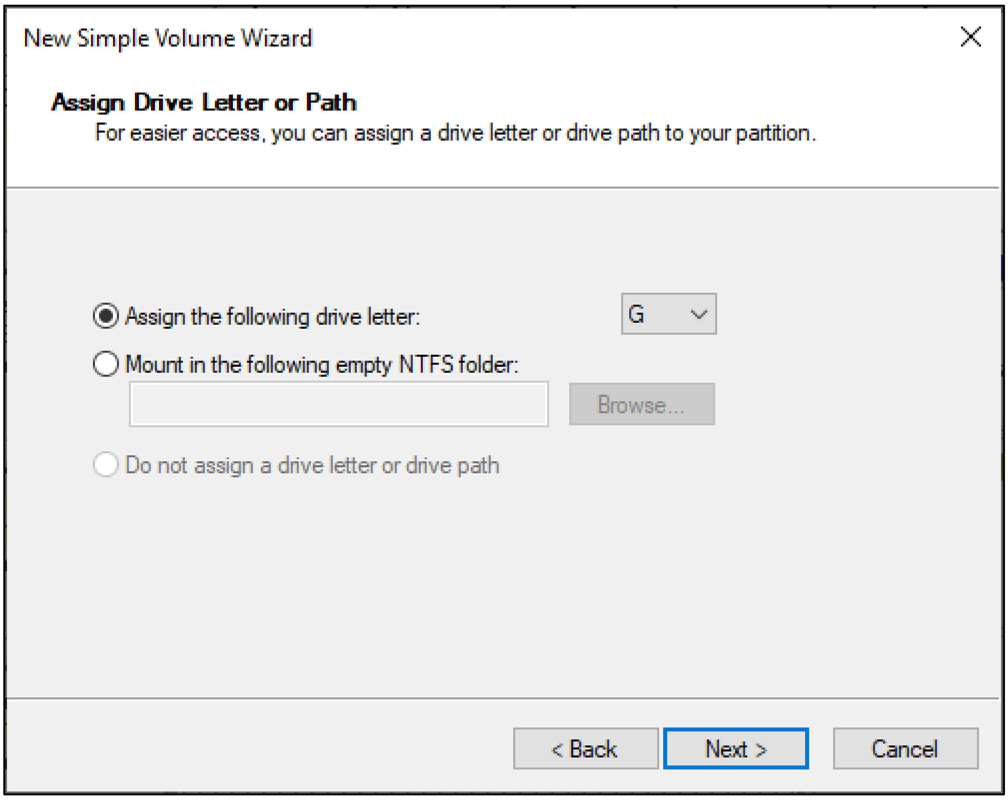


Click Next.

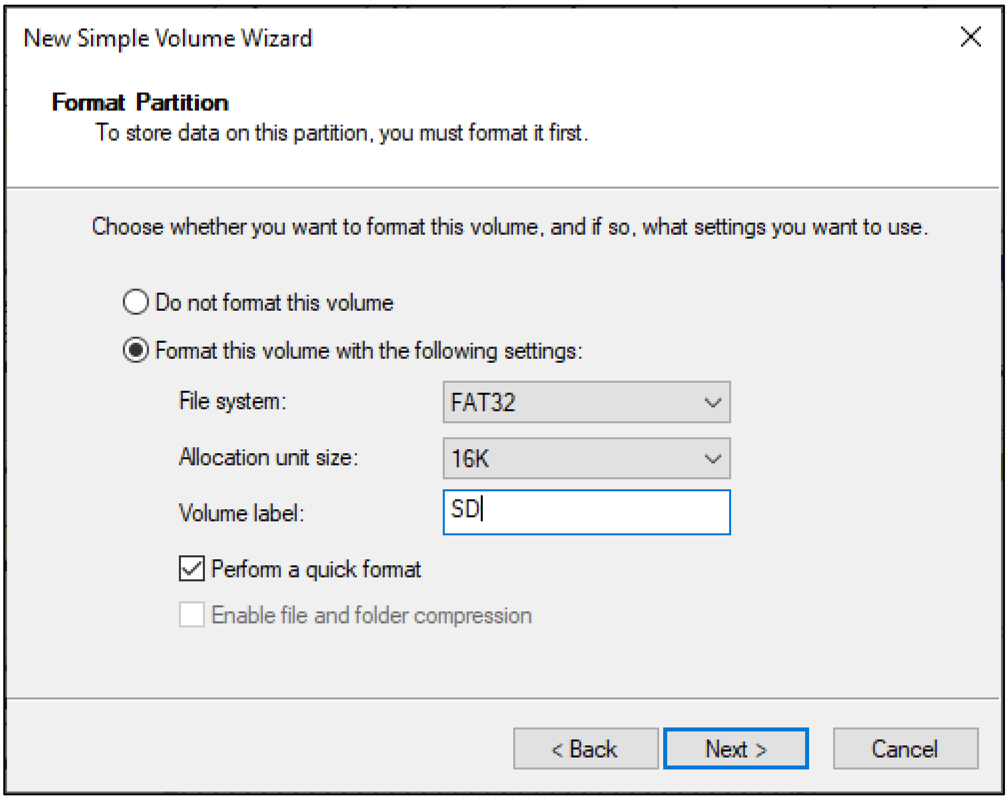




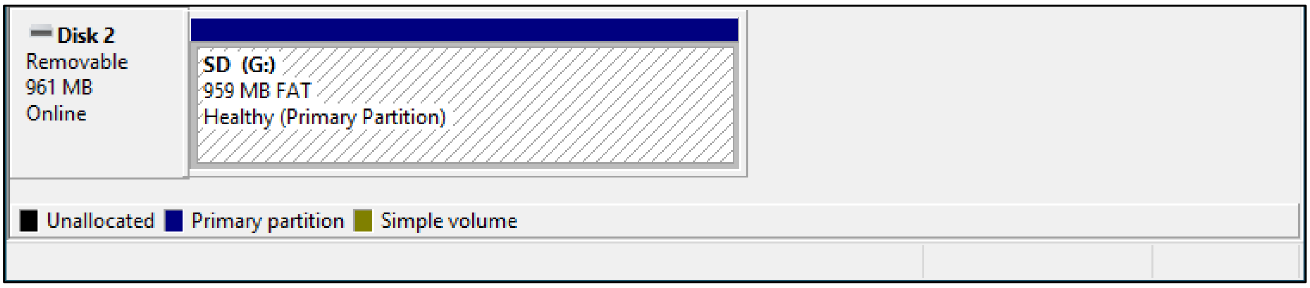
On the right-hand side, you can select a preferred drive letter for the SD card or simply proceed with the default setting by clicking on the "Next" button.



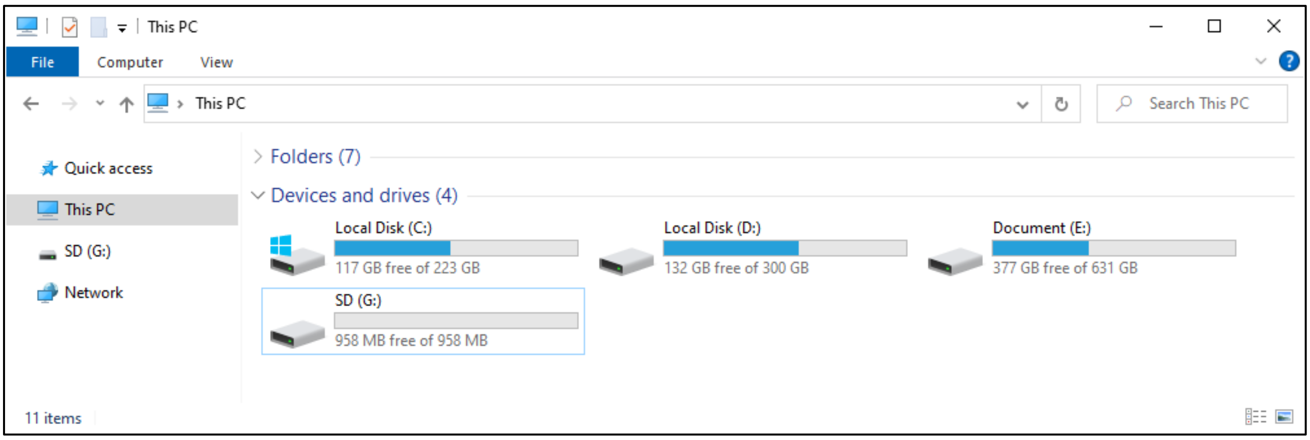
When formatting the SD card, select the file system as FAT (or FAT32) and set the allocation unit size to 16K. You can set the volume label to any name of your choice. Once you have made these selections, click on the "Next" button to proceed.



Click Finish. Wait for the SD card initialization to complete.

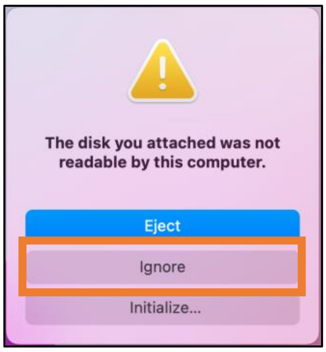


After completing the formatting process, you should be able to see the SD card in the "This PC" section of your computer.

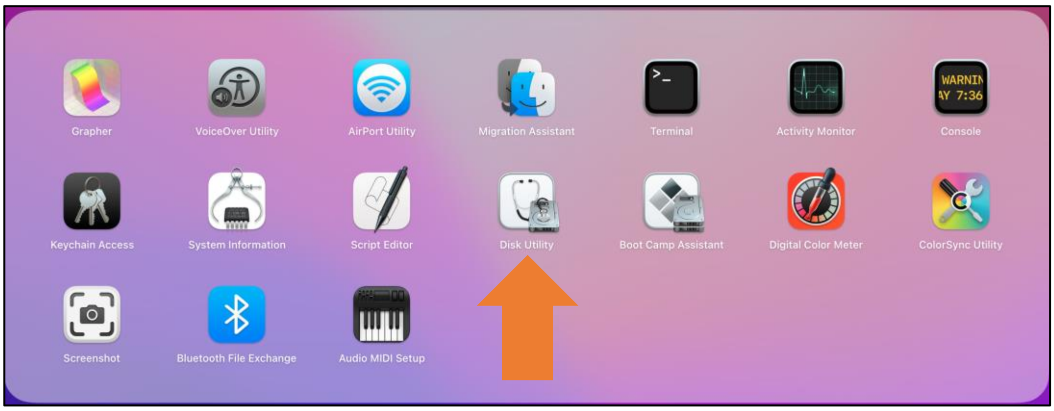


MAC

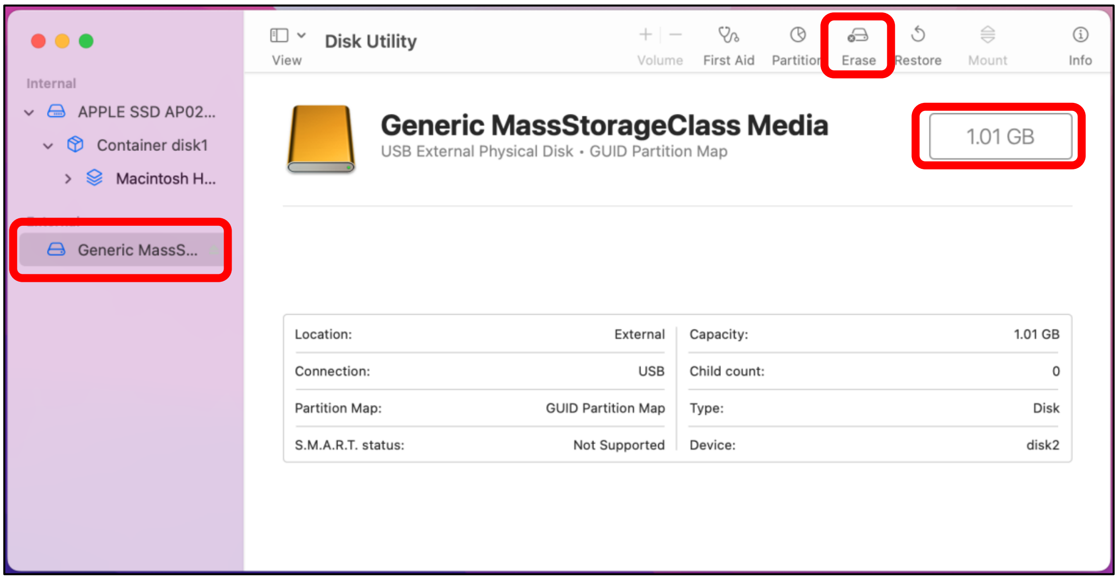
Insert the SD card into the card reader and then insert the card reader into your computer. Some computers may display a prompt with the following message. In this case, please click on the "Ignore" option to proceed.



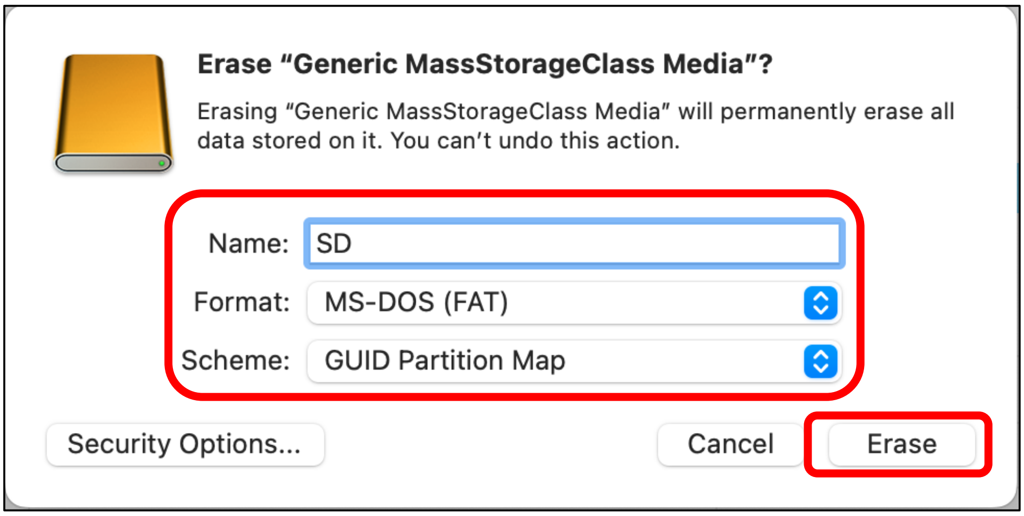
Find "Disk Utility" in the MAC system and click to open it.



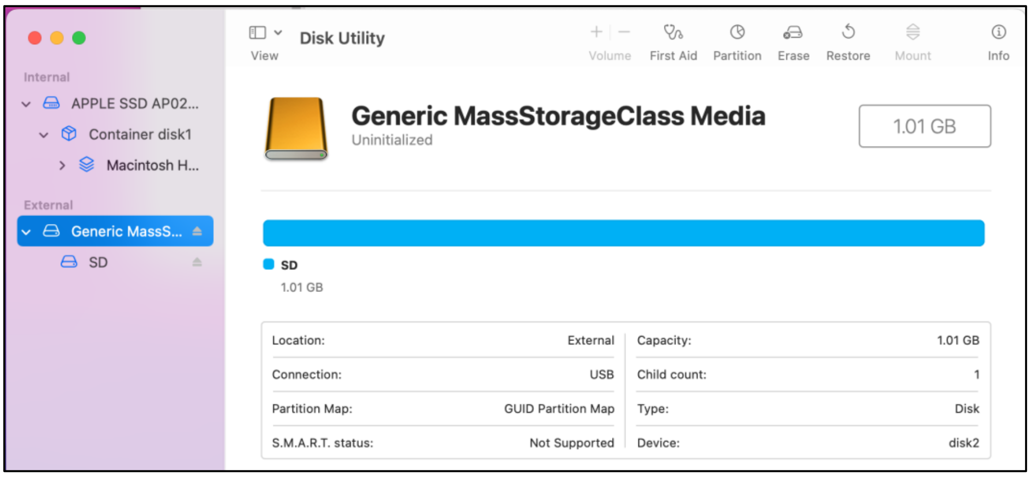
Select "Generic MassStorageClass Media", note that its size is about 1G. It is important to select the correct item to avoid accidentally erasing other device. Once you have verified that you have selected the correct device, click on the "Erase" button to proceed with erasing the SD card.



Select the configuration as shown in the figure below, and then click "Erase".



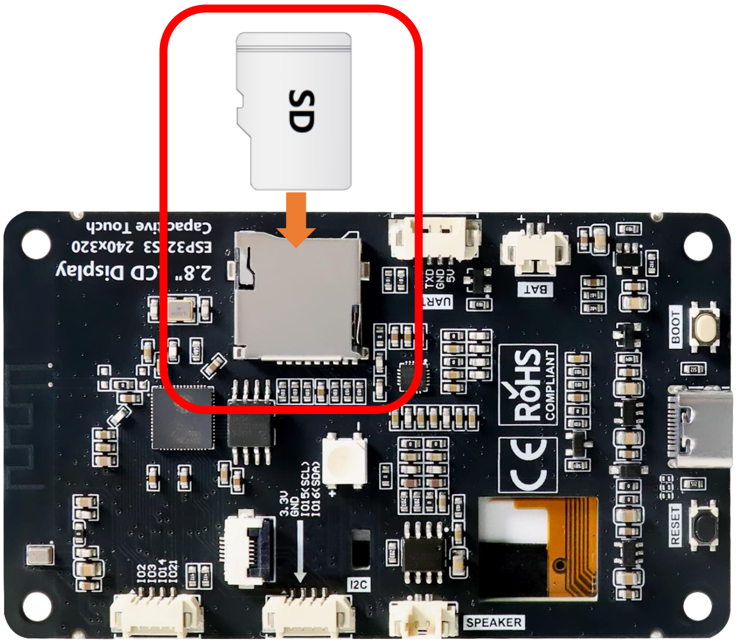
Please wait for the formatting process to complete. Once it is finished, the interface should resemble the image below. You should now be able to see a new disk named "SD" on your desktop.



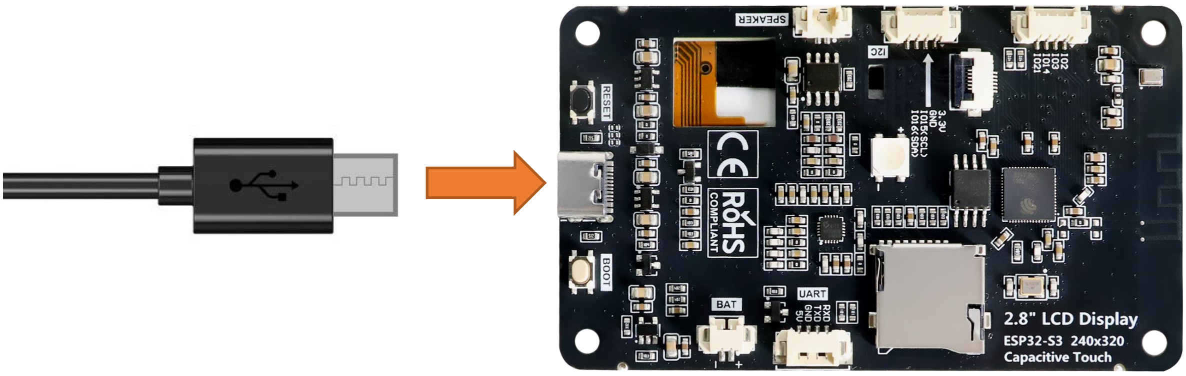
Circuit

Before connecting the USB cable, insert the SD card into the SD card slot on the back of the ESP32-S3.

Please note that this kit does not include SD card and card reader; please buy them yourself.



Connect Freenove ESP32-S3 to the computer using the USB cable.



If you have any concerns, please feel free to contact us via support@freenove.com

Sketch

Next, we download the code to Freenove_ESP32_S3_Display to test Serial. Open "**Sketch_06.1_SD_Test .ino**" folder under **"Freenove_ESP32_S3_Display\Sketches"** and double-click "**Sketch_06.1_SD_Test.ino**".

Sketch_06.1_SD_Test

The following is the program code:

```
#include "driver_sdmmc.h"

#define FNK0104AB_2P8_240x320_ILI9341
//#define FNK0104N_3P5_320x480_ST77922
//#define FNK0104S_4P0_320x480_ST7796

#ifdef FNK0104N_3P5_320x480_ST77922
 #define SD_MMC_CMD 4  // Please do not modify it.
 #define SD_MMC_CLK 5  // Please do not modify it.
 #define SD_MMC_D0  6  // Please do not modify it.
 #define SD_MMC_D1  7  // Please do not modify it.
 #define SD_MMC_D2  2  // Please do not modify it.
 #define SD_MMC_D3  3  // Please do not modify it.
#else
 #define SD_MMC_CMD 40  // Please do not modify it.
 #define SD_MMC_CLK 38  // Please do not modify it.
 #define SD_MMC_D0  39  // Please do not modify it.
 #define SD_MMC_D1  41  // Please do not modify it.
 #define SD_MMC_D2  48  // Please do not modify it.
 #define SD_MMC_D3  47  // Please do not modify it.
#endif
void setup() {
  // Initialize serial communication at 115200 bits per second
  Serial.begin(115200);
  delay(3000);

  // Initialize the SDMMC interface with the specified pins
  sdmmc_init(SD_MMC_CLK, SD_MMC_CMD, SD_MMC_D0, SD_MMC_D1, SD_MMC_D2, SD_MMC_D3);

  // List files in the root directory and print them
  std::list<File_Entry> fileList = list_dir("/", 0);
  print_file_list(fileList);

  // Create a new directory and list files again
  create_dir("/mydir");
  fileList = list_dir("/", 0);
  print_file_list(fileList);
```

```

  // Remove the directory and list files again
  remove_dir("/mydir");
  fileList = list_dir("/", 2);
  print_file_list(fileList);

  // Write text to a new file
  const char* text = "Hello ";
  write_file("/foo.txt", (const uint8_t*)text, strlen(text));

  // Append text to the existing file
  text = "World!\n";
  append_file("/foo.txt", (const uint8_t*)text, strlen(text));

  // Read the file content into a buffer and print it
  uint8_t buffer[100];
  read_file("/foo.txt", buffer, sizeof(buffer));
  Serial.write(buffer, strlen((char*)buffer));

  // Rename the file and read the new file content
  rename_file("/foo.txt", "/hello.txt");
  read_file("/hello.txt", buffer, sizeof(buffer));
  Serial.write(buffer, strlen((char*)buffer));

  // Print SD card information
  Serial.println("\r\nSD card info:");
  Serial.printf("Total space: %lluMB\r\n", SD_MMC.totalBytes() / (1024 * 1024));
  Serial.printf("Used space: %lluMB\r\n", SD_MMC.usedBytes() / (1024 * 1024));
}

void loop() {
  // Delay for 10 seconds before the next iteration
  delay(10000);
}
```

Code Explanation

Include necessary header files.

```
#include "driver_sdmmc.h"
```

Define SD card pins. In this example, we read and write the SD card via SPI.

```
#ifdef FNK0104N_3P5_320x480_ST77922
 #define SD_MMC_CMD 4  // Please do not modify it.
 #define SD_MMC_CLK 5  // Please do not modify it.
 #define SD_MMC_D0  6  // Please do not modify it.
 #define SD_MMC_D1  7  // Please do not modify it.
 #define SD_MMC_D2  2  // Please do not modify it.
```

```
 #define SD_MMC_D3  3  // Please do not modify it.
#else
 #define SD_MMC_CMD 40  // Please do not modify it.
 #define SD_MMC_CLK 38  // Please do not modify it.
 #define SD_MMC_D0  39  // Please do not modify it.
 #define SD_MMC_D1  41  // Please do not modify it.
 #define SD_MMC_D2  48  // Please do not modify it.
 #define SD_MMC_D3  47  // Please do not modify it.
#endif
```

Set the baud rate to 115200.

```
Serial.begin(115200);
```

Initialize the SD card

```
sdmmc_init(SD_MMC_CLK, SD_MMC_CMD, SD_MMC_D0, SD_MMC_D1, SD_MMC_D2, SD_MMC_D3);
```

List files in the root directory and print them

```
std::list<File_Entry> fileList = list_dir("/", 0);
print_file_list(fileList);
```

Create a directory named “mydir” and list all files under the root directory.

```
create_dir("/mydir");
fileList = list_dir("/", 0);
```

Delete the “mydir” directory and list all files under the root directory.

```
remove_dir("/mydir");
fileList = list_dir("/", 2);
```

Write "Hello" in the hello.txt file, append "World!" to it, then read and print the file contents.

```
// Write text to a new file
const char* text = "Hello ";
write_file("/foo.txt", (const uint8_t*)text, strlen(text));

// Append text to the existing file
text = "World!\n";
append_file("/foo.txt", (const uint8_t*)text, strlen(text));

// Read the file content into a buffer and print it
uint8_t buffer[100];
read_file("/foo.txt", buffer, sizeof(buffer));
Serial.write(buffer, strlen((char*)buffer));
```

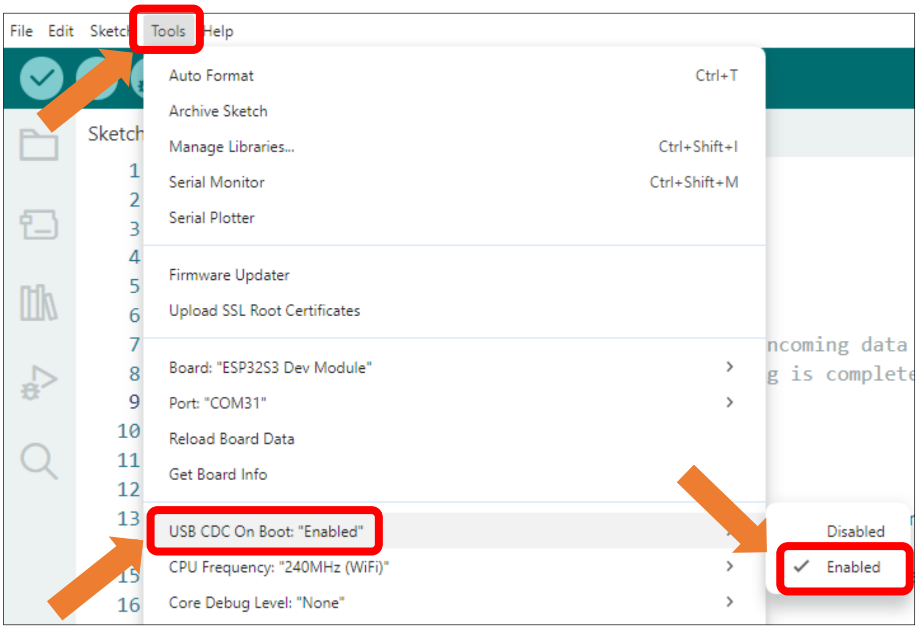
Rename foo.txt to hello.txt and read the file.

```
// Rename the file and read the new file content
rename_file("/foo.txt", "/hello.txt");
read_file("/hello.txt", buffer, sizeof(buffer));
Serial.write(buffer, strlen((char*)buffer));
```

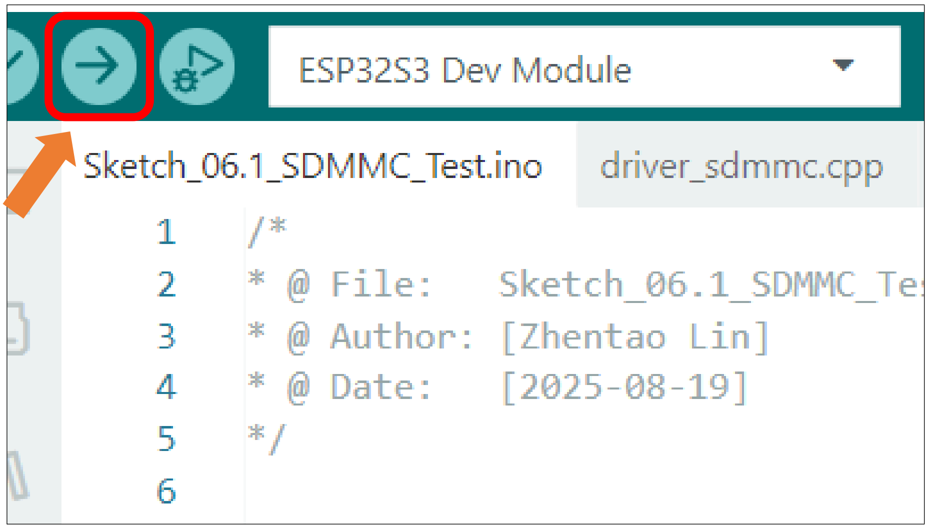
Before compiling and uploading the code, please be sure to confirm the hardware model you are using. Based on your device, please modify the macro definition (#define) at the top of the code: remove the comment symbols (//) in front of the corresponding model, and ensure that the other models remain commented out. The default setting is the 2.8-inch model.

```
#define FNK0104AB_2P8_240x320_ILI9341
//#define FNK0104N_3P5_320x480_ST77922
//#define FNK0104S_4P0_320x480_ST7796
```

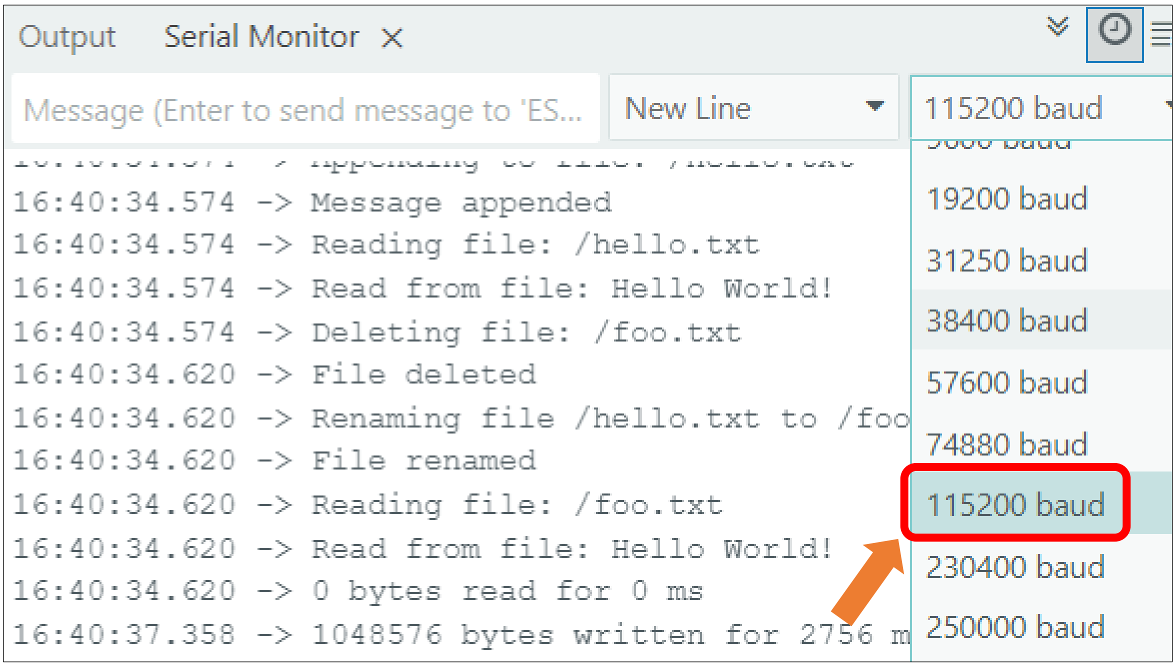
Enable the "USB CDC On Boot" feature.



Click "**Upload**" to upload the code to Freenove_ESP32_S3_Display.



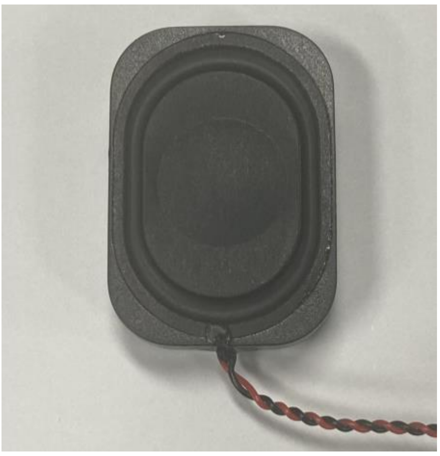
Upon the code runs, open the serial and set the baud rate to 115200.



Chapter 7 Music

Project 7.1 Music

Component List

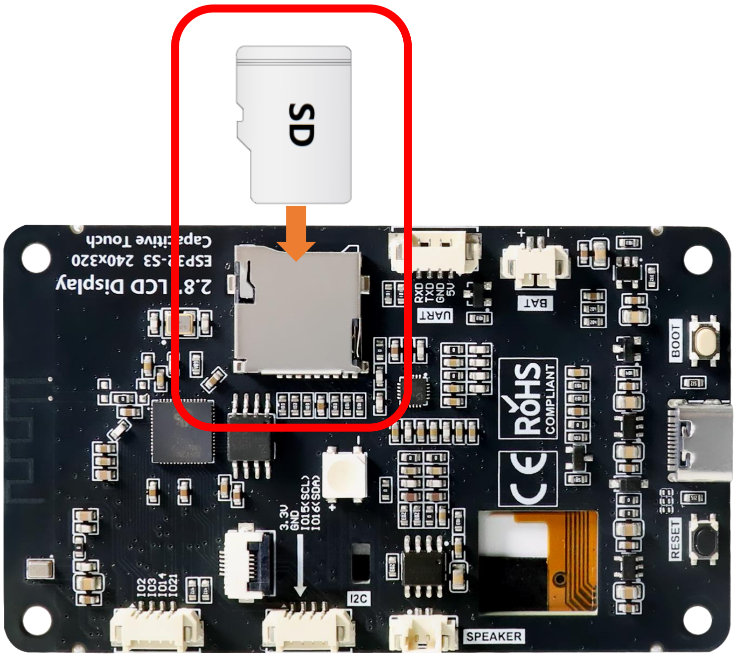
Freenove ESP32-S3 Display x 1	USB cable x1
	
Speaker x1 	

Please note that this kit does not include SD card and card reader, please buy them by yourself.

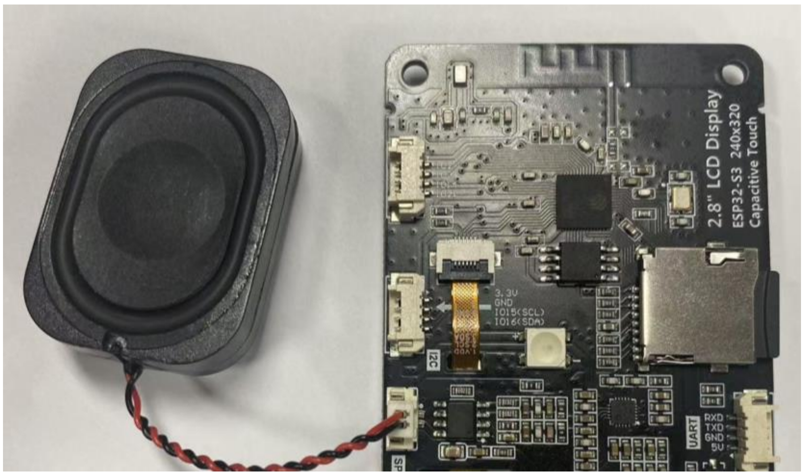
Circuit

Before connecting the USB cable, insert the SD card into the SD card slot on the back of the ESP32-S3.

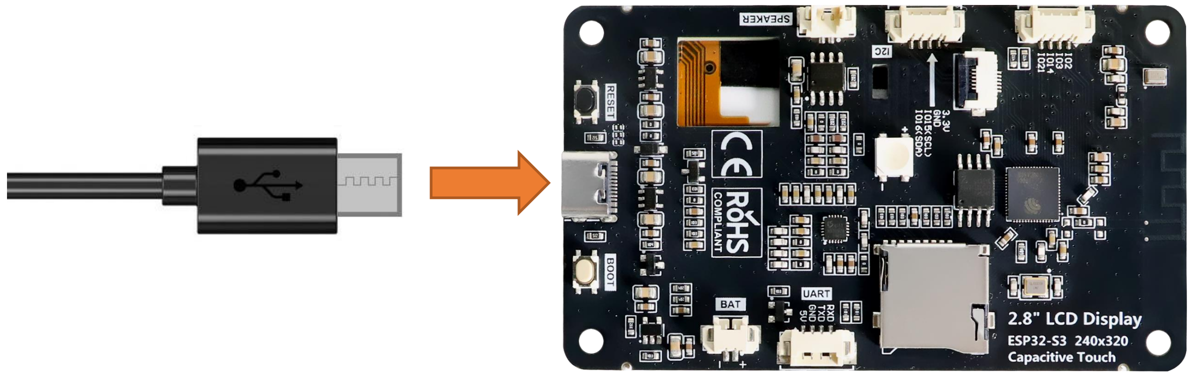
Please note that this kit does not include SD card and card reader; please buy them yourself.



Connect speaker



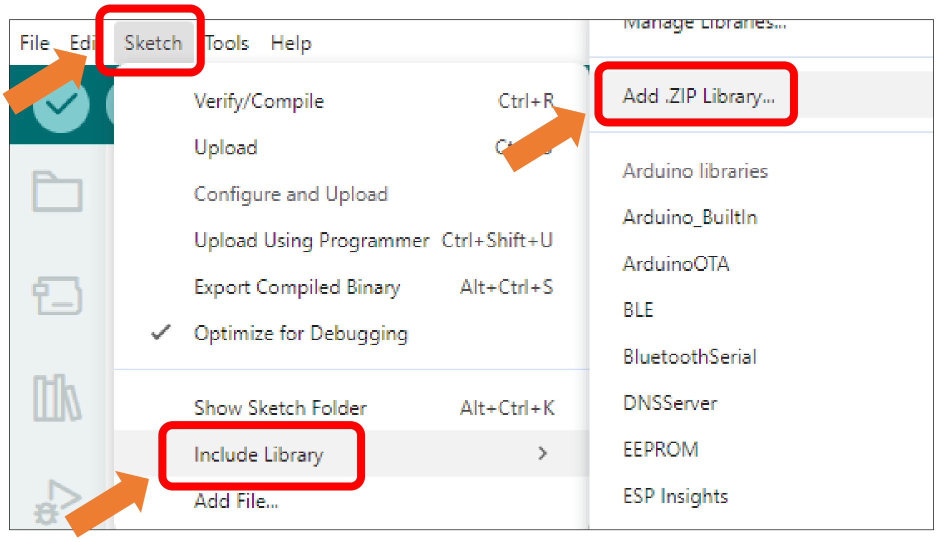
Connect Freenove ESP32-S3 to the computer using the USB cable.



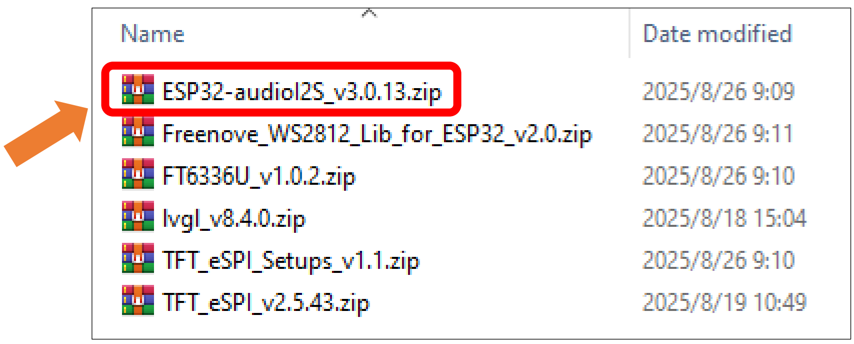
Sketch

Install the needed libraries.

Click **Sketch** -> **Include Library** -> **Add .ZIP Library...**



Select **ESP32-audioI2S_v3.0.13.zip**



Next, we download the code to Freenove_ESP32_S3_Display to test. Open"**Sketch_07.1_Music**" folder under **"Freenove_ESP32_S3_Display\Sketches"** and double-click "**Sketch_07.1_Music.ino**".

Sketch_07.1_Music

The following is the program code:

```
#include <Arduino.h>
#include "FS.h"
```

```
#include "SD_MMC.h"
#include "SPI.h"
#include "es8311.h"
#include "Audio.h"
#include "Wire.h"
#include "ESP_I2S.h"
#include "demo_music.h"

#define FNK0104AB_2P8_240x320_ILI9341
//#define FNK0104N_3P5_320x480_ST77922
//#define FNK0104S_4P0_320x480_ST7796

#ifdef FNK0104N_3P5_320x480_ST77922
 //ESP32-S3 IO Pin define
 #define SD_SCK 5
 #define SD_CMD 4
 #define SD_D0 6
 #define SD_D1 7
 #define SD_D2 2
 #define SD_D3 3

 //I2S IO Pin define
 #define I2S_MCK 17
 #define I2S_BCK 18
 #define I2S_DINT 16
 #define I2S_DOUT 15
 #define I2S_WS 21
 #define AP_ENABLE 1
 #define I2C_SCL 39        /*!< GPIO number used for I2C master clock */
 #define I2C_SDA 38        /*!< GPIO number used for I2C master data  */
 #define I2C_SPEED 400000  /*!< I2C master clock frequency */
#else
 //ESP32-S3 IO Pin define
 #define SD_SCK 38
 #define SD_CMD 40
 #define SD_D0 39
 #define SD_D1 41
 #define SD_D2 48
 #define SD_D3 47

 //I2S IO Pin define
 #define I2S_MCK 4
 #define I2S_BCK 5
 #define I2S_DINT 6
```

```
 #define I2S_DOUT 8
 #define I2S_WS 7
 #define AP_ENABLE 1
 #define I2C_SCL 15        /*!< GPIO number used for I2C master clock */
 #define I2C_SDA 16        /*!< GPIO number used for I2C master data  */
 #define I2C_SPEED 400000  /*!< I2C master clock frequency */
#endif

Audio audio;
I2SClass es8311_i2s;

void driver_es8311_init(void) {
  pinMode(AP_ENABLE, OUTPUT);
  digitalWrite(AP_ENABLE, LOW);

  Wire.begin(I2C_SDA, I2C_SCL, I2C_SPEED);

  es8311_i2s.setPins(I2S_BCK, I2S_WS, I2S_DOUT, I2S_DINT, I2S_MCK);
  if (!es8311_i2s.begin(I2S_MODE_STD, 44100, I2S_DATA_BIT_WIDTH_16BIT, I2S_SLOT_MODE_STEREO,
I2S_STD_SLOT_LEFT)) {
    Serial.println("Failed to initialize I2S bus!");
  }
}

void setup() {
  Serial.begin(115200);

  //SD card init
  if (!SD_MMC.setPins(SD_SCK, SD_CMD, SD_D0, SD_D1, SD_D2, SD_D3)) {
    Serial.println("Pin change failed!");
    return;
  }
  while (!SD_MMC.begin()) {
    Serial.println("SD card does not exist,please insert SD card.");
    delay(100);
  }

  driver_es8311_init();
  if (es8311_codec_init() != ESP_OK) {
    Serial.println("ES8311 init failed!");
    return;
  }
  //audio init
  audio.setPinout(I2S_BCK, I2S_WS, I2S_DOUT, I2S_MCK);
```

```
  audio.setVolume(10);
  if (!demo_music()) {
    return;
  }

  //play music
  demo_music_play(1);
}

void loop() {
  audio.loop();
}
```

Code Explanation

Include necessary header files.

```
#include <Arduino.h>
#include "FS.h"
#include "SD_MMC.h"
#include "SPI.h"
#include "es8311.h"
#include "Audio.h"
#include "Wire.h"
#include "ESP_I2S.h"
#include "demo_music.h"
```

Define the pins.

```
#ifdef FNK0104N_3P5_320x480_ST77922
 //ESP32-S3 IO Pin define
 #define SD_SCK 5
 #define SD_CMD 4
 #define SD_D0 6
 #define SD_D1 7
 #define SD_D2 2
 #define SD_D3 3

 //I2S IO Pin define
 #define I2S_MCK 17
 #define I2S_BCK 18
 #define I2S_DINT 16
 #define I2S_DOUT 15
 #define I2S_WS 21
 #define AP_ENABLE 1
 #define I2C_SCL 39        /*!< GPIO number used for I2C master clock */
 #define I2C_SDA 38        /*!< GPIO number used for I2C master data  */
 #define I2C_SPEED 400000  /*!< I2C master clock frequency */
#else
```

```
 //ESP32-S3 IO Pin define
 #define SD_SCK 38
 #define SD_CMD 40
 #define SD_D0 39
 #define SD_D1 41
 #define SD_D2 48
 #define SD_D3 47

 //I2S IO Pin define
 #define I2S_MCK 4
 #define I2S_BCK 5
 #define I2S_DINT 6
 #define I2S_DOUT 8
 #define I2S_WS 7
 #define AP_ENABLE 1
 #define I2C_SCL 15        /*!< GPIO number used for I2C master clock */
 #define I2C_SDA 16        /*!< GPIO number used for I2C master data  */
 #define I2C_SPEED 400000  /*!< I2C master clock frequency */
#endif
```

Declare an I2S object

```
Audio audio;
I2SClass es8311_i2s;
```

Set the baud rate to 115200

```
Serial.begin(115200);
```

SD card init

```
//SD card init
if (!SD_MMC.setPins(SD_SCK, SD_CMD, SD_D0, SD_D1, SD_D2, SD_D3)) {
  Serial.println("Pin change failed!");
  return;
}
while (!SD_MMC.begin()) {
  Serial.println("SD card does not exist,please insert SD card.");
  delay(100);
}
```

Read audio data and play it.

```
//audio init
audio.setPinout(I2S_BCK, I2S_WS, I2S_DOUT, I2S_MCK);
audio.setVolume(10);
if (!demo_music()) {
  return;
}
```

play music

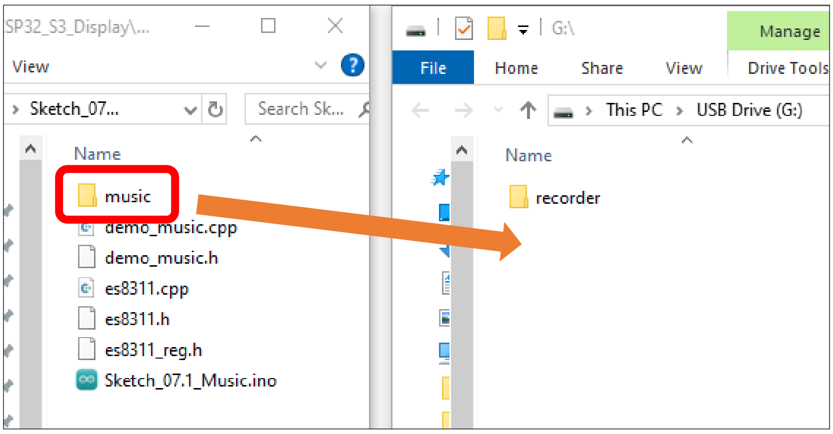
```
demo_music_play(1);
```

Before compiling and uploading the code, please be sure to confirm the hardware model you are using. Based on your device, please modify the macro definition (#define) at the top of the code: remove the comment symbols (//) in front of the corresponding m

```
#define FNK0104AB_2P8_240x320_ILI9341
//#define FNK0104N_3P5_320x480_ST77922
//#define FNK0104S_4P0_320x480_ST7796
```

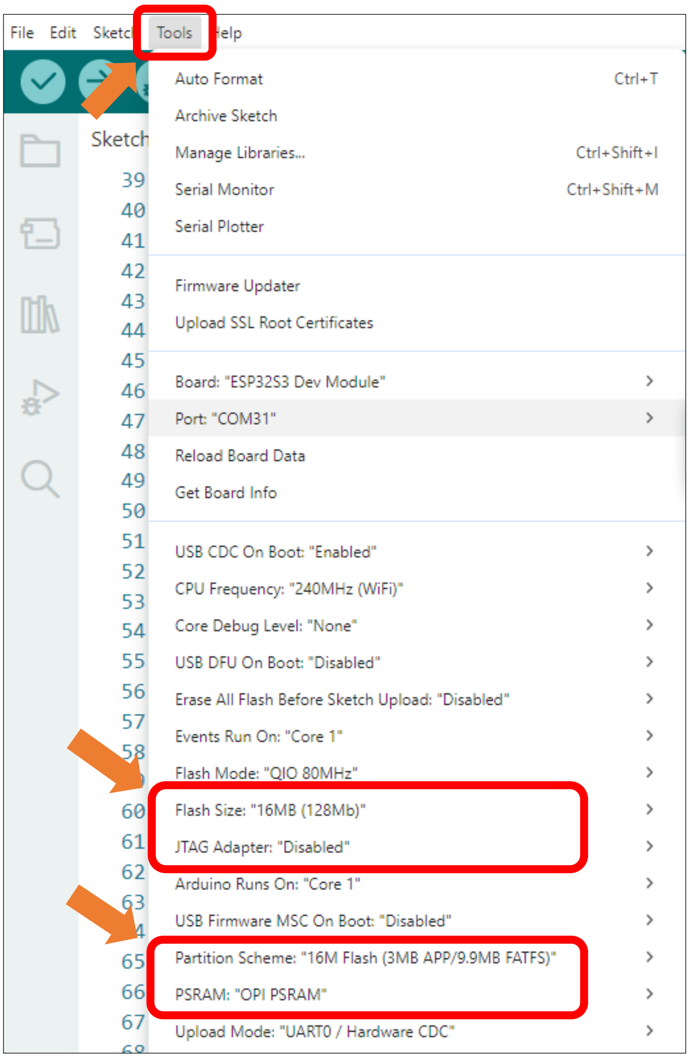
This product does not include SD card, and SD card reader, please buy them by yourself. For more information, please refer to SD card sections.

Before uploading the code, copy the music to the root directory of the SD card with the SD card reader.

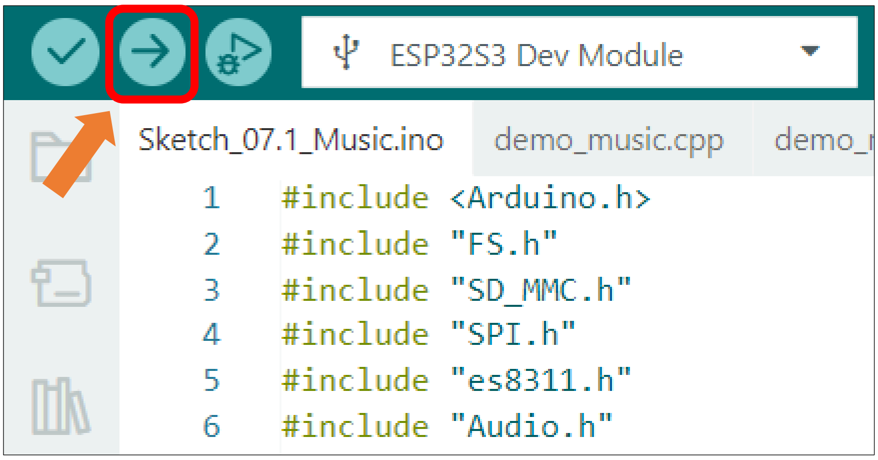


It is necessary to change the settings in Arduino IDE before clicking the Uploading button, as shown below.

Caution: Incorrect settings will result in compilation error or uploading failure. To achieve desired result, please configure exactly the same as below.



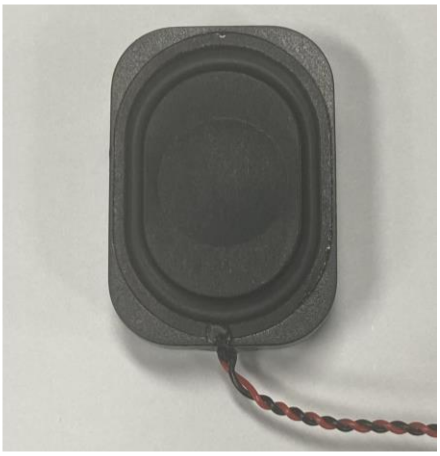
Click "**Upload**" to upload the code to Freenove ESP32-S3 Display.



The speaker plays the music in the SD card.

Project 7.2 Echo

Component List

Freenove ESP32-S3 Display x 1	USB cable x1
	
Speakerx1 	

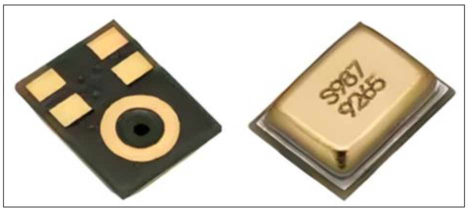
Please note that this kit does not include SD card and card reader, please buy them by yourself.

Component knowledge

MEMS-MIC

A MEMS Microphone (Micro-Electro-Mechanical Systems Microphone) is a miniature microphone manufactured using MEMS technology. It integrates mechanical sensing elements and electronic circuits on the same chip to achieve sound signal acquisition and conversion. Its working principle primarily involves a tiny vibrating diaphragm that detects sound pressure changes, then converts these mechanical vibrations into electrical signals, enabling sound capture and transmission.

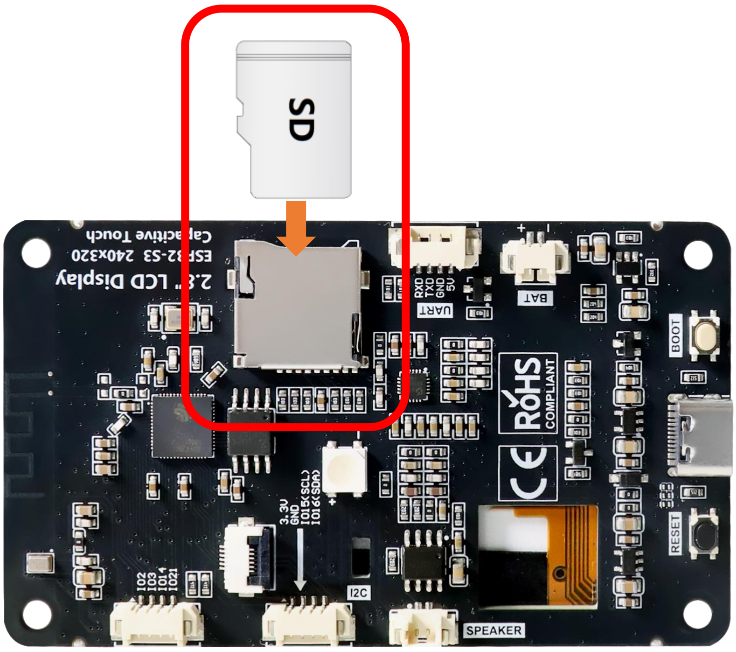
MEMS microphones are characterized by their compact size, high sensitivity, excellent stability, and ease of mass production. They are widely used in electronic devices such as smartphones, earphones, and smart speakers. Compared to traditional microphones, MEMS microphones better meet the dual demands of modern electronic products for both miniaturization and performance.



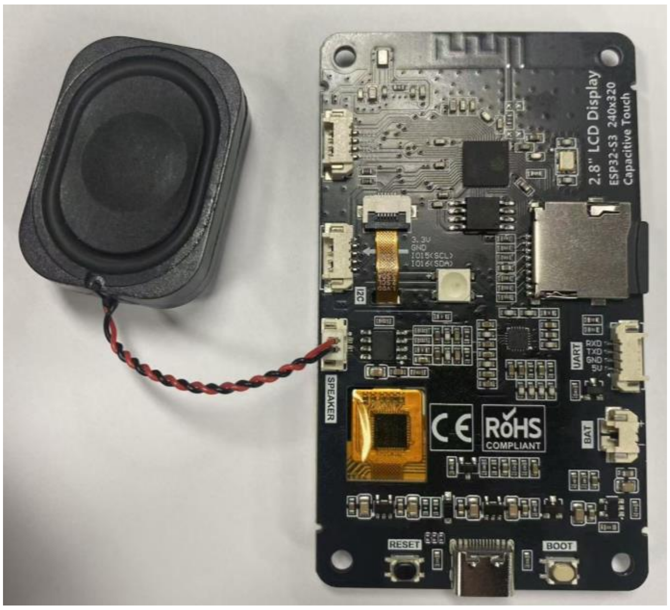
Circuit

Before connecting the USB cable, insert the SD card into the SD card slot on the back of the ESP32-S3.

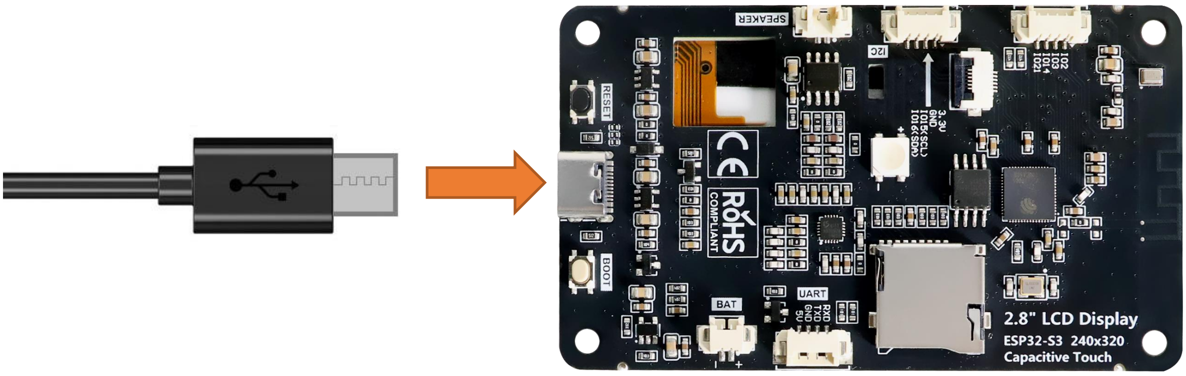
Please note that this kit does not include SD card and card reader; please buy them yourself.



Connect speaker



Connect Freenove ESP32-S3 Display to the computer using the USB cable.



Sketch

Sketch_07.2_Echo

The following is the program code:

```
#include <Arduino.h>
#include "FS.h"
#include "SD_MMC.h"
#include "SPI.h"
#include "es8311.h"
#include "Audio.h"
#include "Wire.h"
#include "ESP_I2S.h"

#define FNK0104AB_2P8_240x320_ILI9341
//#define FNK0104N_3P5_320x480_ST77922
//#define FNK0104S_4P0_320x480_ST7796

```

```
#ifdef FNK0104N_3P5_320x480_ST77922
 //ESP32-S3 IO Pin define
 #define SD_SCK 5
 #define SD_CMD 4
 #define SD_D0 6
 #define SD_D1 7
 #define SD_D2 2
 #define SD_D3 3

 //I2S IO Pin define
 #define I2S_MCK 17
 #define I2S_BCK 18
 #define I2S_DINT 16
 #define I2S_DOUT 15
 #define I2S_WS 21
 #define AP_ENABLE 1
 #define I2C_SCL 39        /*!< GPIO number used for I2C master clock */
 #define I2C_SDA 38        /*!< GPIO number used for I2C master data  */
 #define I2C_SPEED 400000  /*!< I2C master clock frequency */
#else
 //ESP32-S3 IO Pin define
 #define SD_SCK 38
 #define SD_CMD 40
 #define SD_D0 39
 #define SD_D1 41
 #define SD_D2 48
 #define SD_D3 47

 //I2S IO Pin define
 #define I2S_MCK 4
 #define I2S_BCK 5
 #define I2S_DINT 6
 #define I2S_DOUT 8
 #define I2S_WS 7
 #define AP_ENABLE 1
 #define I2C_SCL 15        /*!< GPIO number used for I2C master clock */
 #define I2C_SDA 16        /*!< GPIO number used for I2C master data  */
 #define I2C_SPEED 400000  /*!< I2C master clock frequency */
#endif

I2SClass es8311_i2s;

void driver_es8311_init(void) {
  pinMode(AP_ENABLE, OUTPUT);
```

```
  digitalWrite(AP_ENABLE, LOW);

  Wire.begin(I2C_SDA, I2C_SCL, I2C_SPEED);

  es8311_i2s.setPins(I2S_BCK, I2S_WS, I2S_DOUT, I2S_DINT, I2S_MCK);
  if (!es8311_i2s.begin(I2S_MODE_STD, 16000, I2S_DATA_BIT_WIDTH_16BIT, I2S_SLOT_MODE_MONO,
I2S_STD_SLOT_LEFT)) {
    Serial.println("Failed to initialize I2S bus!");
  }
}

void setup()
{
  Serial.begin(115200);
  while (!Serial) {
    delay(10);
  }

  Serial.println("Start initializing the audio device...");
  driver_es8311_init();
  if (es8311_codec_init() != ESP_OK) {
    Serial.println("ES8311 init failed!");
    return;
  }
  delay(3000);
  Serial.println("Initialization completed.");
}

void loop()
{
  uint8_t *wav_buffer;
  size_t wav_size;
  // Record 5 seconds of audio data
  Serial.println("Start recording for 5 seconds...");
  wav_buffer = es8311_i2s.recordWAV(5, &wav_size);
  Serial.println("Recording completed.");
  delay(1000);

  Serial.println("Start playing the recording...");
  es8311_i2s.playWAV(wav_buffer, wav_size);
  Serial.println("Playback has been completed.");
  free(wav_buffer);
  delay(1000);
}
```

Code Explanation

Include necessary header files.

```
#include <Arduino.h>
#include "FS.h"
#include "SD_MMC.h"
#include "SPI.h"
#include "es8311.h"
#include "Audio.h"
#include "Wire.h"
#include "ESP_I2S.h"
```

Define the pins.

```
#ifdef FNK0104N_3P5_320x480_ST77922
 //ESP32-S3 IO Pin define
 #define SD_SCK 5
 #define SD_CMD 4
 #define SD_D0 6
 #define SD_D1 7
 #define SD_D2 2
 #define SD_D3 3

 //I2S IO Pin define
 #define I2S_MCK 17
 #define I2S_BCK 18
 #define I2S_DINT 16
 #define I2S_DOUT 15
 #define I2S_WS 21
 #define AP_ENABLE 1
 #define I2C_SCL 39        /*!< GPIO number used for I2C master clock */
 #define I2C_SDA 38        /*!< GPIO number used for I2C master data  */
 #define I2C_SPEED 400000  /*!< I2C master clock frequency */
#else
 //ESP32-S3 IO Pin define
 #define SD_SCK 38
 #define SD_CMD 40
 #define SD_D0 39
 #define SD_D1 41
 #define SD_D2 48
 #define SD_D3 47

 //I2S IO Pin define
 #define I2S_MCK 4
 #define I2S_BCK 5
 #define I2S_DINT 6
 #define I2S_DOUT 8
```

```
 #define I2S_WS 7
 #define AP_ENABLE 1
 #define I2C_SCL 15        /*!< GPIO number used for I2C master clock */
 #define I2C_SDA 16        /*!< GPIO number used for I2C master data  */
 #define I2C_SPEED 400000  /*!< I2C master clock frequency */
#endif
```

Declare an I2S object

```
I2SClass es8311_i2s;
```

Set the baud rate to 115200

```
Serial.begin(115200);
```

Initialize the audio device.

```
Serial.println("Start initializing the audio device...");
driver_es8311_init();
if (es8311_codec_init() != ESP_OK) {
  Serial.println("ES8311 init failed!");
  return;
}
```

Implement audio recording and playback functionality

```
void loop()
{
  uint8_t *wav_buffer;
  size_t wav_size;
  // Record 5 seconds of audio data
  Serial.println("Start recording for 5 seconds...");
  wav_buffer = es8311_i2s.recordWAV(5, &wav_size);
  Serial.println("Recording completed.");
  delay(1000);

  Serial.println("Start playing the recording...");
  es8311_i2s.playWAV(wav_buffer, wav_size);
  Serial.println("Playback has been completed.");
  free(wav_buffer);
  delay(1000);
}
```

Before compiling and uploading the code, please be sure to confirm the hardware model you are using. Based on your device, please modify the macro definition (#define) at the top of the code: remove the comment symbols (//) in front of the corresponding m

```
#define FNK0104AB_2P8_240x320_ILI9341
//#define FNK0104N_3P5_320x480_ST77922
//#define FNK0104S_4P0_320x480_ST7796
```

Chapter 8 BLE

Project 8.1 BLE USART

Component List

Freenove ESP32-S3 Display x 1	USB cable x1
	

Component Knowledge

BLE（Bluetooth Low Energy）

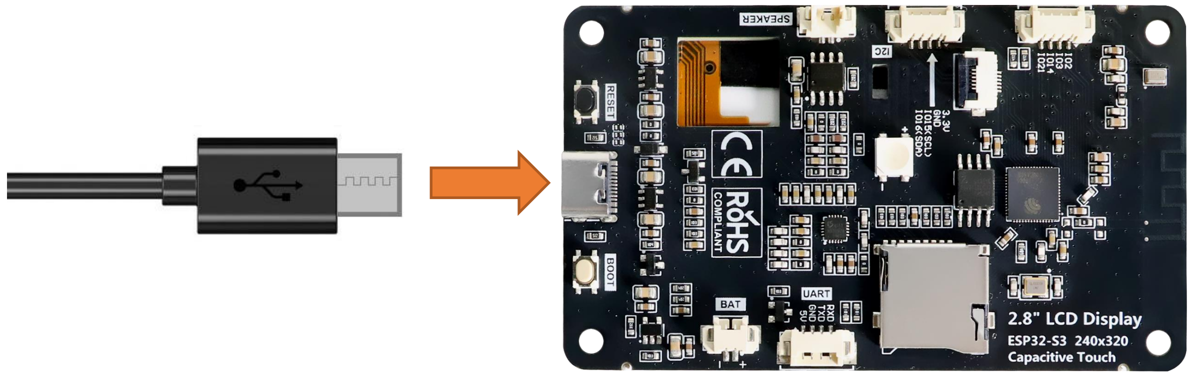
Low Energy Bluetooth (BLE) is a new feature introduced in the Bluetooth 4.0 specification, specifically designed for low-power devices and suitable for applications involving intermittent transmission of small amounts of data.

The BLE architecture follows a client-server model and consists of the following key components:
GATT (Generic Attribute Protocol): The foundational protocol for BLE communication, defining a hierarchical data structure of services and characteristics.
Service: A collection of data that performs a specific function or feature, containing one or more characteristics.
Characteristic: A specific data point within a service, consisting of a value and descriptors.
UUID: A 128-bit universally unique identifier used to distinguish services and characteristics.

BLE device can function simultaneously as a Peripheral (providing services) and a Central (connecting to peripherals). In this section, we will learn how to configure the Freenove_ESP32_S3_Display as a peripheral device to provide services.

Circuit

Connect Freenove ESP32-S3 to the computer using the USB cable.



Sketch

Next, we download the code to Freenove_ESP32_S3_Display to test. Open"**Sketch_08.1_BLE_USART**" folder under **"Freenove_ESP32_S3_Display\Sketches"** and double-click "**Sketch_08.1_BLE_USART.ino**".

Sketch_08.1_BLE_USART

The following is the program code:

```
#include "BLEDevice.h"
#include "BLEServer.h"
#include "BLEUtils.h"
#include "BLE2902.h"
#include "String.h"

BLECharacteristic *pCharacteristic;
bool deviceConnected = false;
uint8_t txValue = 0;
long lastMsg = 0;
String rxload="Test\n";

#define SERVICE_UUID           "6E400001-B5A3-F393-E0A9-E50E24DCCA9E"
#define CHARACTERISTIC_UUID_RX "6E400002-B5A3-F393-E0A9-E50E24DCCA9E"
#define CHARACTERISTIC_UUID_TX "6E400003-B5A3-F393-E0A9-E50E24DCCA9E"

class MyServerCallbacks: public BLEServerCallbacks {
     void onConnect(BLEServer* pServer) {
       deviceConnected = true;
     };
     void onDisconnect(BLEServer* pServer) {
       deviceConnected = false;
     }
```

```
};

class MyCallbacks: public BLECharacteristicCallbacks {
    void onWrite(BLECharacteristic *pCharacteristic) {
      String rxValue = pCharacteristic->getValue();
      if (rxValue.length() > 0) {
        rxload="";
        for (int i = 0; i < rxValue.length(); i++){
          rxload +=(char)rxValue[i];
        }
      }
    }
};

void setupBLE(String BLEName){
  const char *ble_name=BLEName.c_str();
  BLEDevice::init(ble_name);
  BLEServer *pServer = BLEDevice::createServer();
  pServer->setCallbacks(new MyServerCallbacks());
  BLEService *pService = pServer->createService(SERVICE_UUID);
  pCharacteristic=
pService->createCharacteristic(CHARACTERISTIC_UUID_TX,BLECharacteristic::PROPERTY_NOTIFY);
  pCharacteristic->addDescriptor(new BLE2902());
  BLECharacteristic *pCharacteristic =
pService->createCharacteristic(CHARACTERISTIC_UUID_RX,BLECharacteristic::PROPERTY_WRITE);
  pCharacteristic->setCallbacks(new MyCallbacks());
  pService->start();
  pServer->getAdvertising()->start();
  Serial.println("Waiting a client connection to notify...");
}

void setup() {
  Serial.begin(115200);
  setupBLE("ESP32S3_BLE");
}

void loop() {
  long now = millis();
  if (now - lastMsg > 100) {
    if (deviceConnected&&rxload.length()>0) {
        Serial.println(rxload);
        rxload="";
    }
    if(Serial.available()>0){
```

```
            String str=Serial.readString();
            const char *newValue=str.c_str();
            pCharacteristic->setValue(newValue);
            pCharacteristic->notify();
        }
        lastMsg = now;
    }
}
```

Code Explanation

Include the necessary header libraries.

```
#include "BLEDevice.h"
#include "BLEServer.h"
#include "BLEUtils.h"
#include "BLE2902.h"
#include "String.h"
```

Define service UUID and characteristic UUID.

```
#define SERVICE_UUID           "6E400001-B5A3-F393-E0A9-E50E24DCCA9E"
#define CHARACTERISTIC_UUID_RX "6E400002-B5A3-F393-E0A9-E50E24DCCA9E"
#define CHARACTERISTIC_UUID_TX "6E400003-B5A3-F393-E0A9-E50E24DCCA9E"
```

The MyServerCallbacks class handles device connection and disconnection events and updates the deviceConnected status.

```
class MyServerCallbacks: public BLEServerCallbacks {
    void onConnect(BLEServer* pServer) {
      deviceConnected = true;
    };
    void onDisconnect(BLEServer* pServer) {
      deviceConnected = false;
    }
};
```

The MyServerCallbacks class handles the received data and save them to the rxload string.

```
class MyCallbacks: public BLECharacteristicCallbacks {
    void onWrite(BLECharacteristic *pCharacteristic) {
      String rxValue = pCharacteristic->getValue();
      if (rxValue.length() > 0) {
        rxload="";
        for (int i = 0; i < rxValue.length(); i++){
          rxload +=(char)rxValue[i];
        }
      }
    }
};
```

Set the baud rate to 115200.

```
Serial.begin(115200);
```

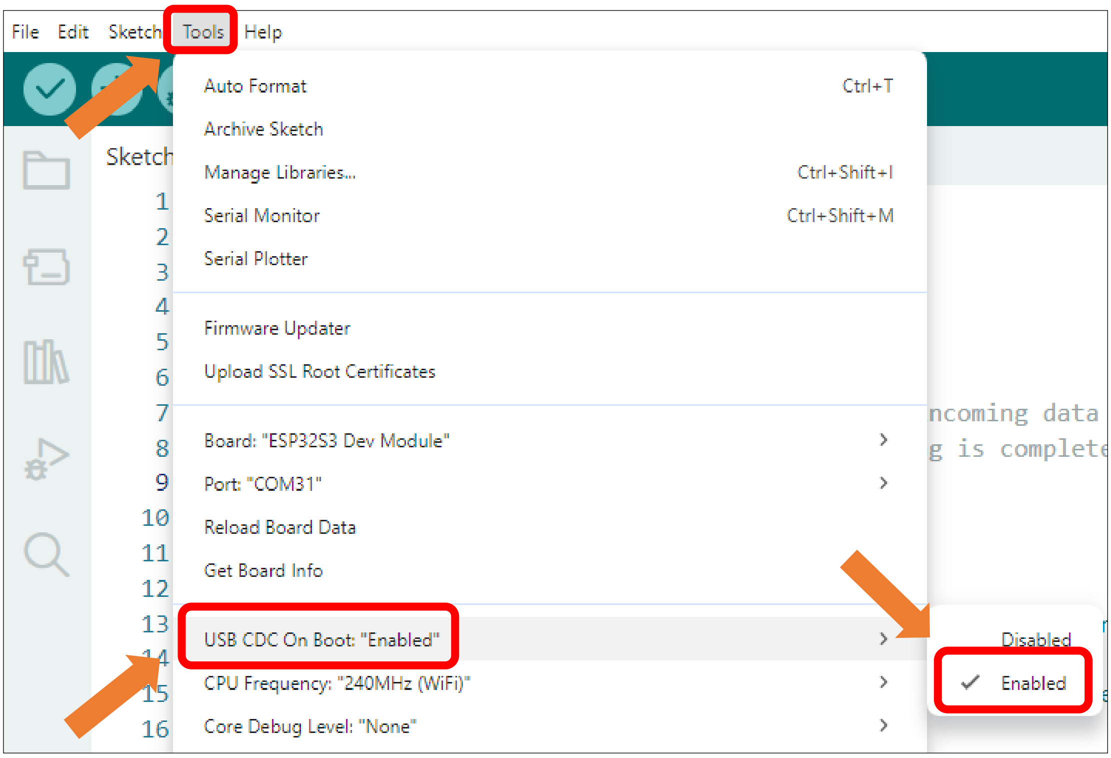
BLE Device Initialization, Service Creation, and Characteristic Setup

```
setupBLE("ESP32S3_BLE");
```

The Loop function check the connection status and serial data every 100 milliseconds.

```
void loop() {
   long now = millis();
   if (now - lastMsg > 100) {
     if (deviceConnected&&rxload.length()>0) {
         Serial.println(rxload);
         rxload="";
     }
     if(Serial.available()>0){
         String str=Serial.readString();
         const char *newValue=str.c_str();
         pCharacteristic->setValue(newValue);
         pCharacteristic->notify();
     }
     lastMsg = now;
   }
}
```

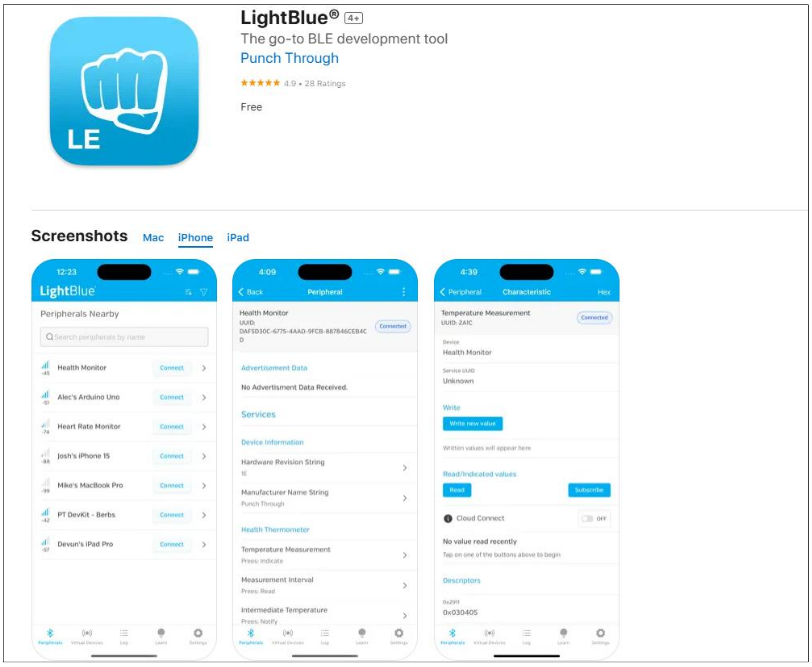
Enable the "USB CDC On Boot" feature.



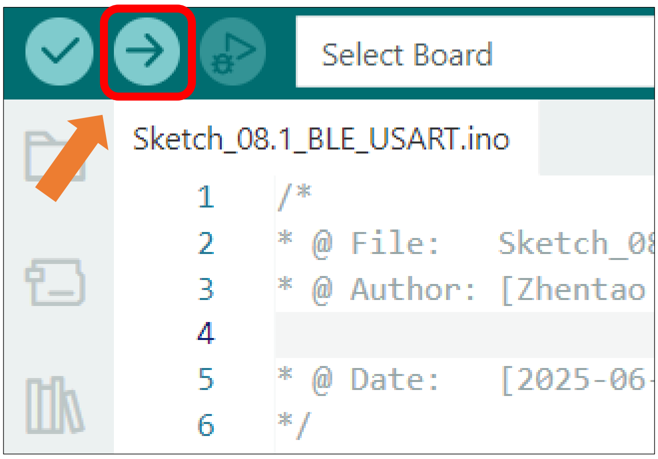
LightBlue

If you do not have this software installed on your phone, you can refer to this link:

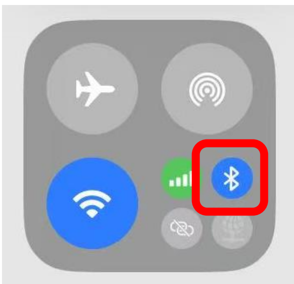
https://apps.apple.com/us/app/lightblue/id557428110



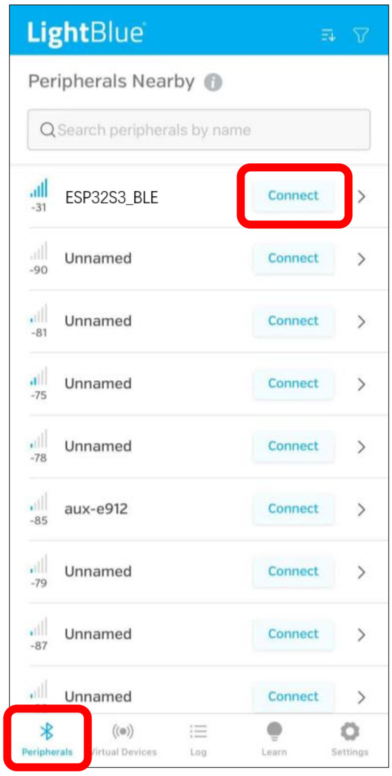
Click “**Upload**” to upload the code to Freenove ESP32-S3 Display.

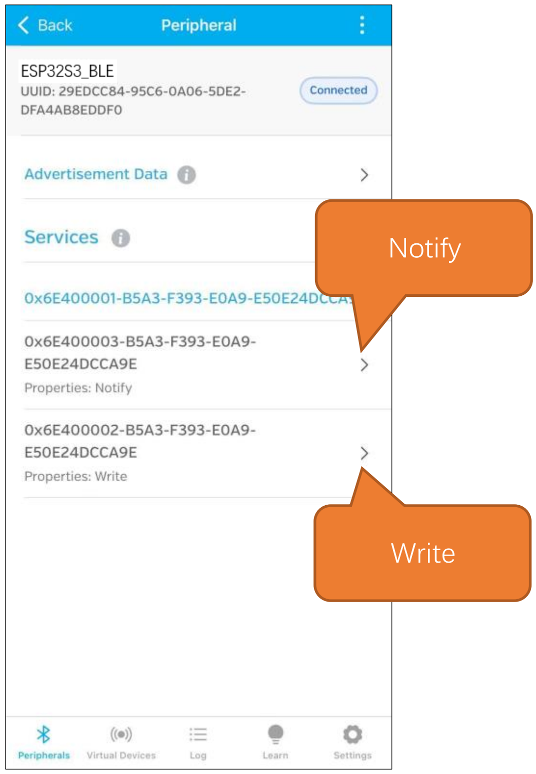


Turn ON Bluetooth on your phone, and open the LightBlue APP.



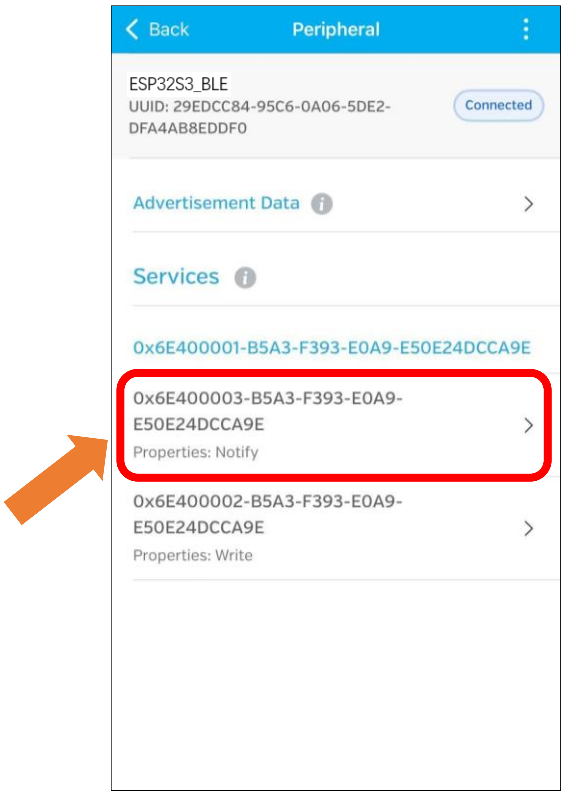
In the Scan page, swipe down to refresh the name of Bluetooth that the phone searches for. Click the Connection button of ESP32S3_BLE



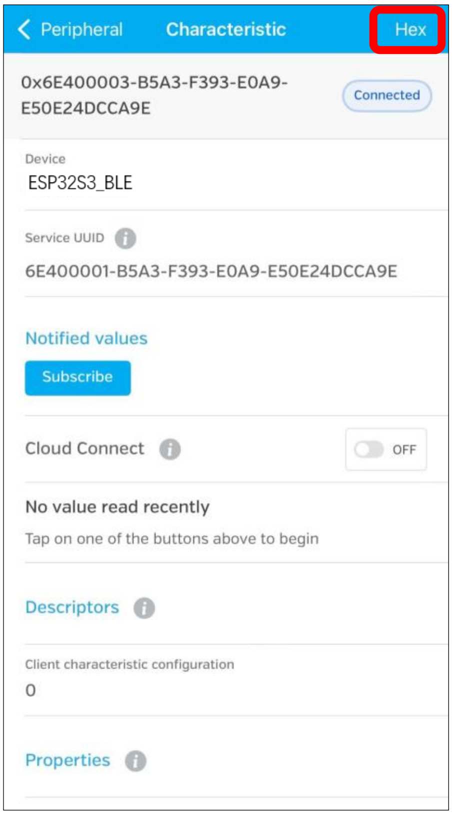


Receiving Data

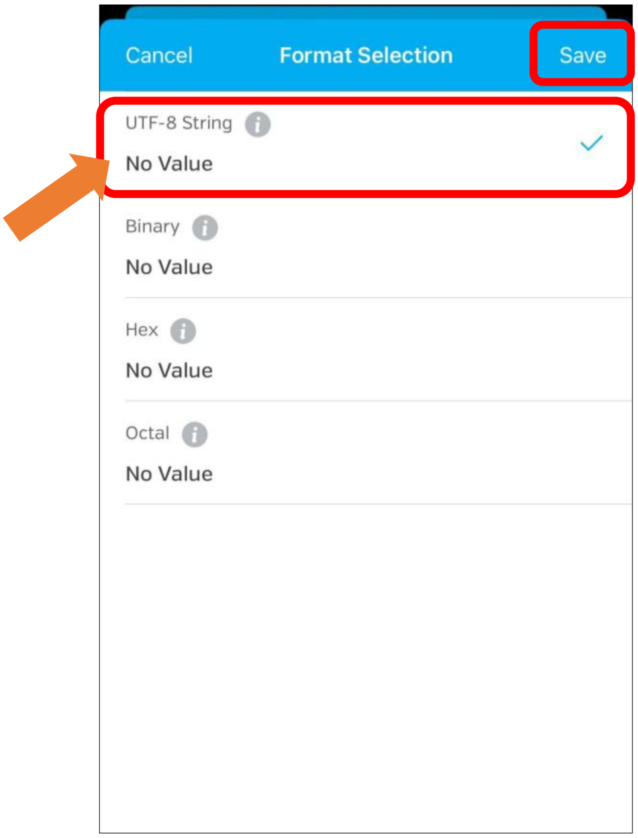
Click Notify



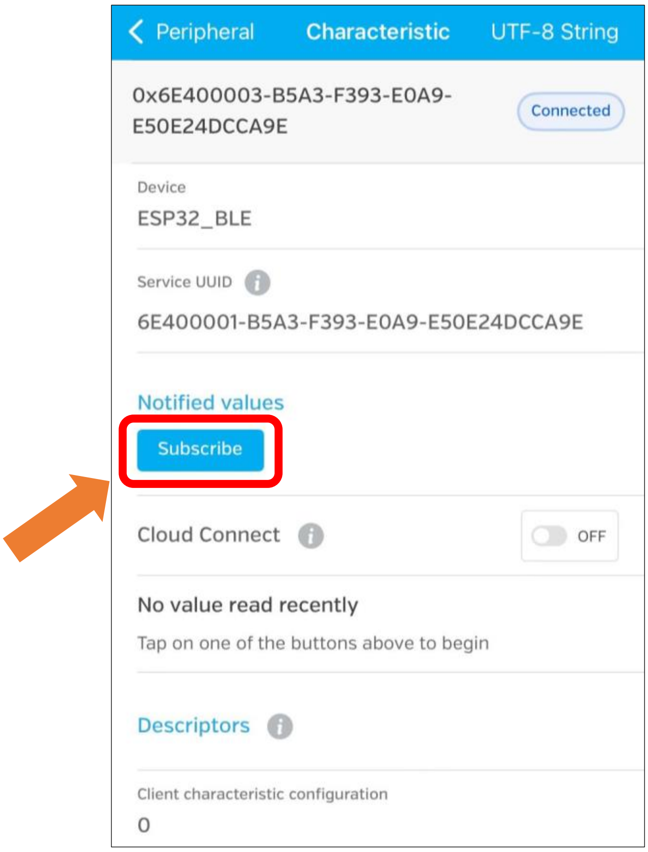
click the top right corner to change the string type.



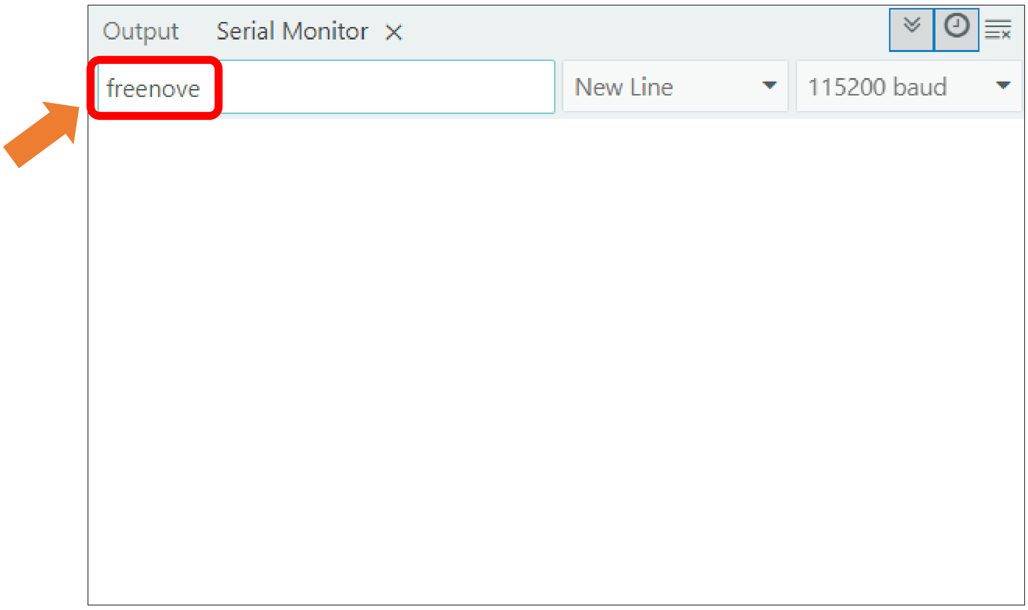
Select UTF-8 String, and click “Save”.



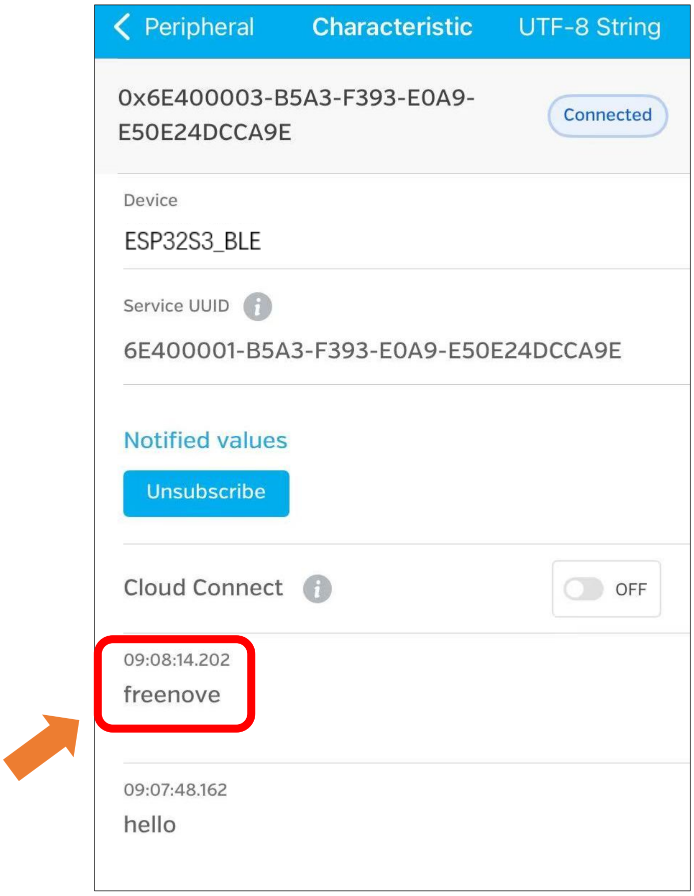
Click Subscribe



Send data on the serial monitor.

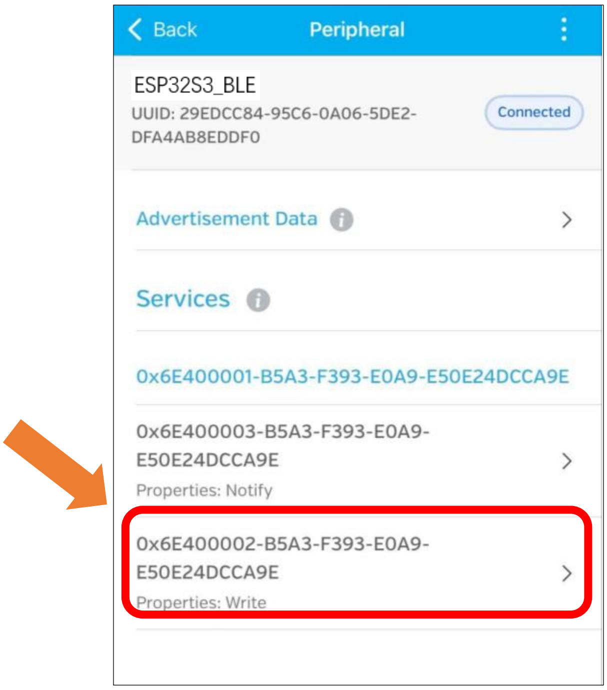


Data will be received on the LightBlue app.

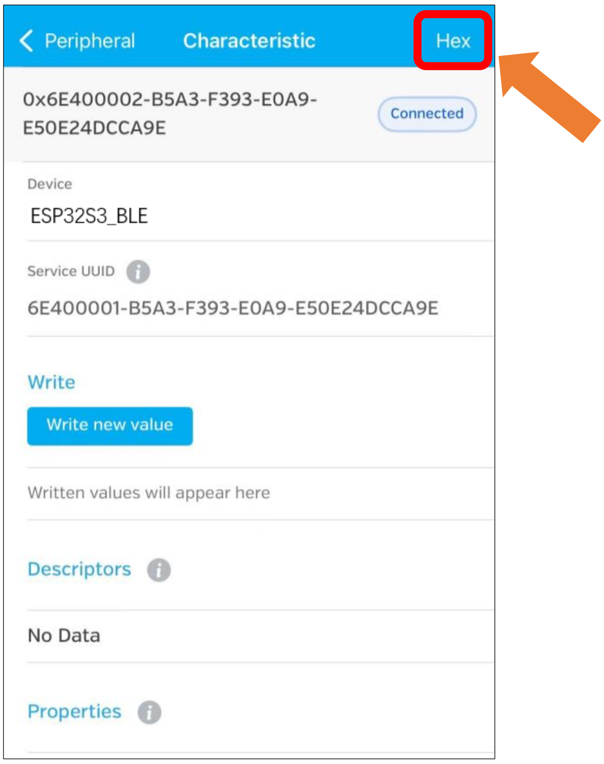


Sending Data

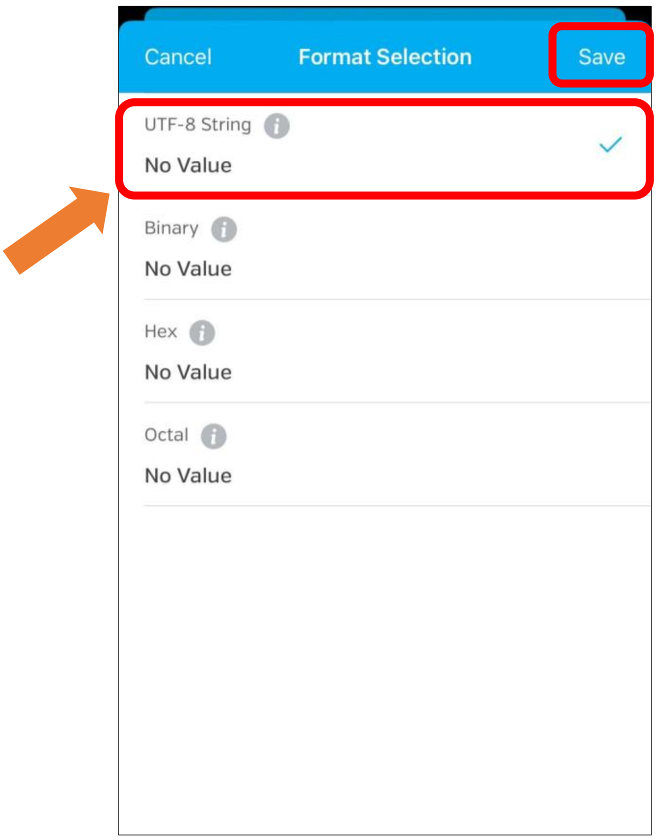
Click Write



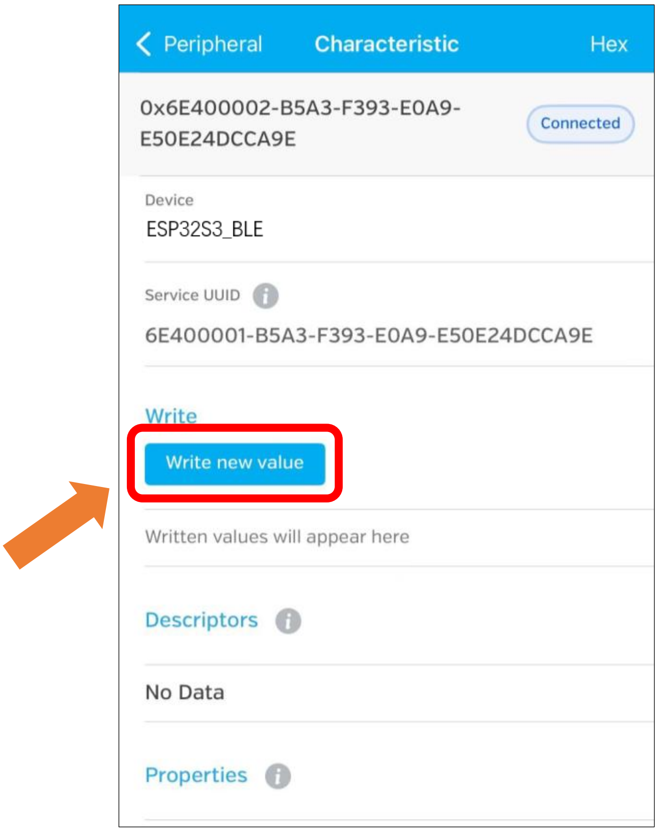
Click the top right corner to change the string type.



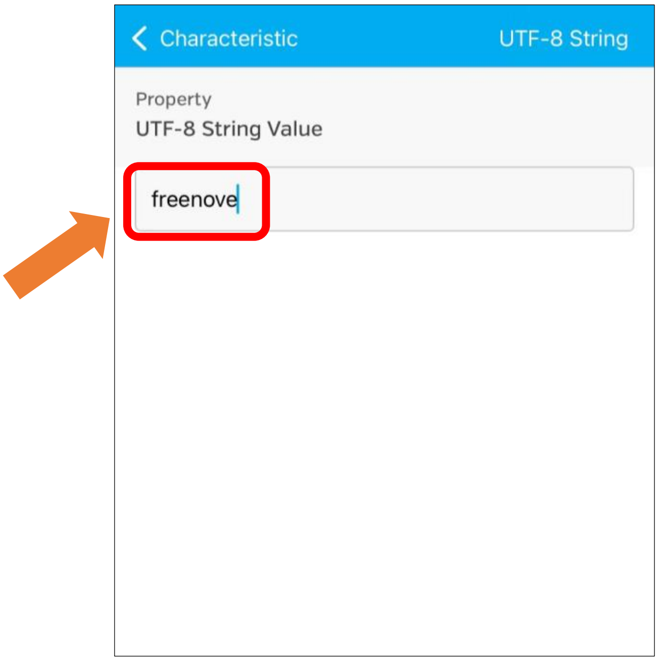
Select UTF-8 String and click "Save".



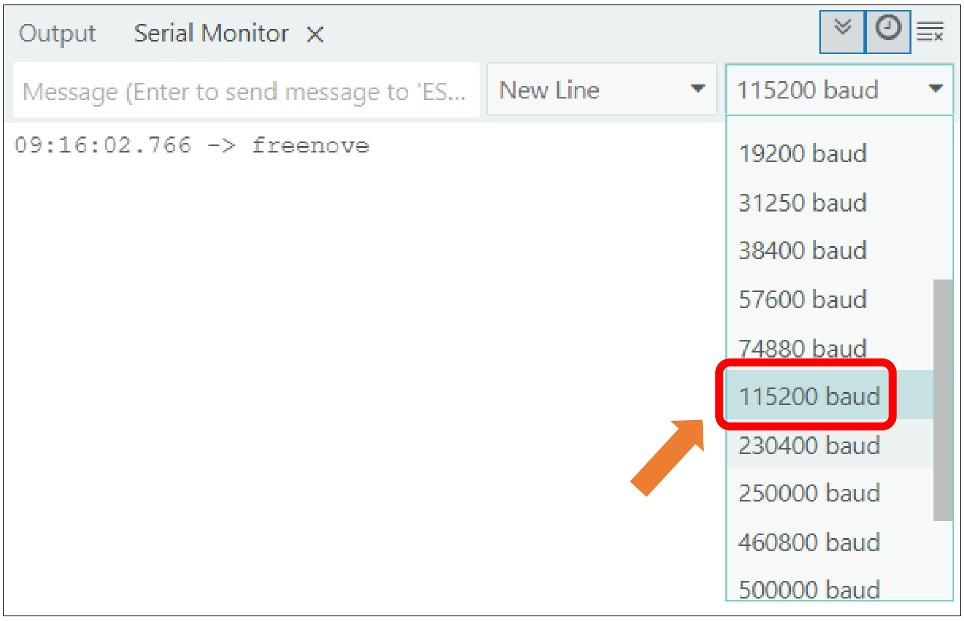
Click “Write new value”



Enter the messages to send.



Set the baud rate to 115200, and the serial monitor will print the data received.



Project 8.2 BLE RGB

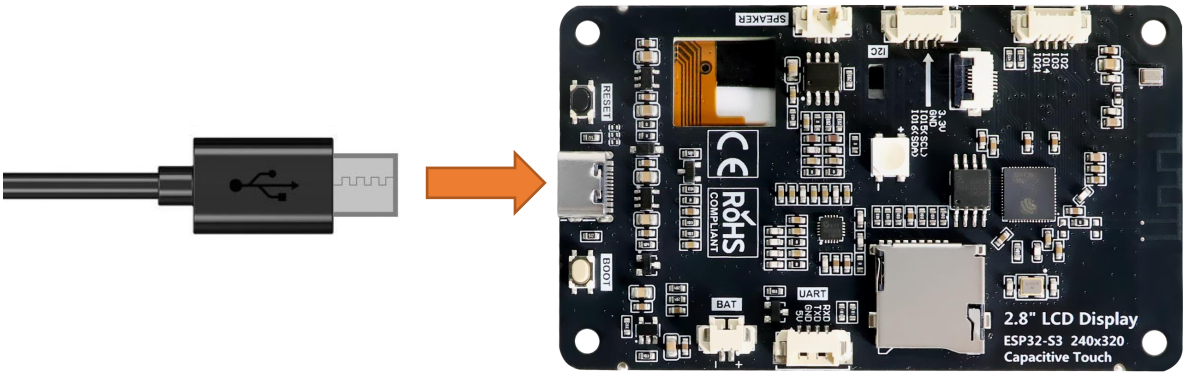
In this section we will control the RGB LED via BLE.

Component List

Freenove ESP32-S3 Display x 1	USB cable x1
	

Circuit

Connect Freenove ESP32-S3 to the computer using the USB cable.



Sketch

Next, we download the code to Freenove_ESP32_S3_Display to test. Open"**Sketch_08.2_BLE_RGB**" folder under **"Freenove_ESP32_S3_Display\Sketches"** and double-click "**Sketch_08.2_BLE_RGB.ino**".

Sketch_08.2_BLE_RGB

The following is the program code:

```
#include "BLEDevice.h"
#include "BLEServer.h"
#include "BLEUtils.h"
#include "BLE2902.h"
#include "String.h"
#include "Freenove_WS2812_Lib_for_ESP32.h"
```

```

BLECharacteristic *pCharacteristic;
bool deviceConnected = false;
uint8_t txValue = 0;
long lastMsg = 0;
String rxload = "Test\n";

#define SERVICE_UUID "6E400001-B5A3-F393-E0A9-E50E24DCCA9E"
#define CHARACTERISTIC_UUID_RX "6E400002-B5A3-F393-E0A9-E50E24DCCA9E"
#define CHARACTERISTIC_UUID_TX "6E400003-B5A3-F393-E0A9-E50E24DCCA9E"

#define KEY_PIN 0
#define LEDS_COUNT 1
#define LEDS_PIN   42
#define CHANNEL    0

Freenove_ESP32_WS2812 strip = Freenove_ESP32_WS2812(LEDS_COUNT, LEDS_PIN, CHANNEL, TYPE_GRB);

uint8_t m_color[5][3] = { { 255, 0, 0 }, { 0, 255, 0 }, { 0, 0, 255 }, { 255, 255, 255 }, { 0,
0, 0 } };

void stripInit(void){
   strip.begin();
   strip.setBrightness(10);
}

void switchRGB(int value) {
   int colorIndex = value % 4;
   switch (colorIndex) {
     case 1:
       strip.setLedColorData(0, m_color[0][0], m_color[0][1], m_color[0][2]);
       strip.show();
       break;
     case 2:
       strip.setLedColorData(0, m_color[1][0], m_color[1][1], m_color[1][2]);
       strip.show();
       break;
     case 3:
       strip.setLedColorData(0, m_color[2][0], m_color[2][1], m_color[2][2]);
       strip.show();
       break;
     default:
       strip.setLedColorData(0, m_color[4][0], m_color[4][1], m_color[4][2]);
       strip.show();
```

```
        break;
    }
}

class MyServerCallbacks : public BLEServerCallbacks {
   void onConnect(BLEServer *pServer) {
     deviceConnected = true;
   };
   void onDisconnect(BLEServer *pServer) {
     deviceConnected = false;
   }
};

class MyCallbacks : public BLECharacteristicCallbacks {
   void onWrite(BLECharacteristic *pCharacteristic) {
     String rxValue = pCharacteristic->getValue();
     if (rxValue.length() > 0) {
       rxload = "";
       for (int i = 0; i < rxValue.length(); i++) {
         rxload += (char)rxValue[i];
       }
     }
   }
};

void setupBLE(String BLEName) {
   const char *ble_name = BLEName.c_str();
   BLEDevice::init(ble_name);
   BLEServer *pServer = BLEDevice::createServer();
   pServer->setCallbacks(new MyServerCallbacks());
   BLEService *pService = pServer->createService(SERVICE_UUID);
   pCharacteristic = pService->createCharacteristic(CHARACTERISTIC_UUID_TX,
BLECharacteristic::PROPERTY_NOTIFY);
   pCharacteristic->addDescriptor(new BLE2902());
   BLECharacteristic *pCharacteristic = pService->createCharacteristic(CHARACTERISTIC_UUID_RX,
BLECharacteristic::PROPERTY_WRITE);
   pCharacteristic->setCallbacks(new MyCallbacks());
   pService->start();
   pServer->getAdvertising()->start();
   Serial.println("Waiting a client connection to notify...");
}

void setup() {
   Serial.begin(115200);
```

```
  stripInit();
  switchRGB(0);
  setupBLE("ESP32S3_BLE");
}

void loop() {
  long now = millis();
  if (now - lastMsg > 100) {
    if (deviceConnected && rxload.length() > 0) {
      Serial.println(rxload);
      if (strncmp(rxload.c_str(), "red_on", 6) == 0) {
        switchRGB(1);
      } else if (strncmp(rxload.c_str(), "red_off", 7) == 0) {
        switchRGB(0);
      } else if (strncmp(rxload.c_str(), "green_on", 8) == 0) {
        switchRGB(2);
      } else if (strncmp(rxload.c_str(), "green_off", 9) == 0) {
        switchRGB(0);
      } else if (strncmp(rxload.c_str(), "blue_on", 7) == 0) {
        switchRGB(3);
      } else if (strncmp(rxload.c_str(), "blue_off", 8) == 0) {
        switchRGB(0);
      }
      rxload = "";
    }
    if (Serial.available() > 0) {
      String str = Serial.readString();
      const char *newValue = str.c_str();
      pCharacteristic->setValue(newValue);
      pCharacteristic->notify();
    }
    lastMsg = now;
  }
}
```

Code Explanation

Include the necessary header files.

```
#include "BLEDevice.h"
#include "BLEServer.h"
#include "BLEUtils.h"
#include "BLE2902.h"
#include "String.h"
#include "Freenove_WS2812_Lib_for_ESP32.h"
```

Define Service UUID and Characteristic UUID.

```
#define SERVICE_UUID "6E400001-B5A3-F393-E0A9-E50E24DCCA9E"
```

```
#define CHARACTERISTIC_UUID_RX "6E400002-B5A3-F393-E0A9-E50E24DCCA9E"
#define CHARACTERISTIC_UUID_TX "6E400003-B5A3-F393-E0A9-E50E24DCCA9E"
```

Define pins for the RGB LED.

```
#define KEY_PIN 0
#define LEDS_COUNT 1
#define LEDS_PIN   42
#define CHANNEL    0
```

The MyServerCallbacks class handles device connection and disconnection events and updates the deviceConnected status.

```
class MyServerCallbacks: public BLEServerCallbacks {
    void onConnect(BLEServer* pServer) {
      deviceConnected = true;
    };
    void onDisconnect(BLEServer* pServer) {
      deviceConnected = false;
    }
};
```

The MyCallbacks class handles the receiving data and save them to the rxload string.

```
class MyCallbacks: public BLECharacteristicCallbacks {
    void onWrite(BLECharacteristic *pCharacteristic) {
      String rxValue = pCharacteristic->getValue();
      if (rxValue.length() > 0) {
        rxload="";
        for (int i = 0; i < rxValue.length(); i++){
          rxload +=(char)rxValue[i];
        }
      }
    }
};
```

Set the baud rate to 115200

```
Serial.begin(115200);
```

Initialize RGB LED setting, initialize the BLE device, create services, and set up characteristics.

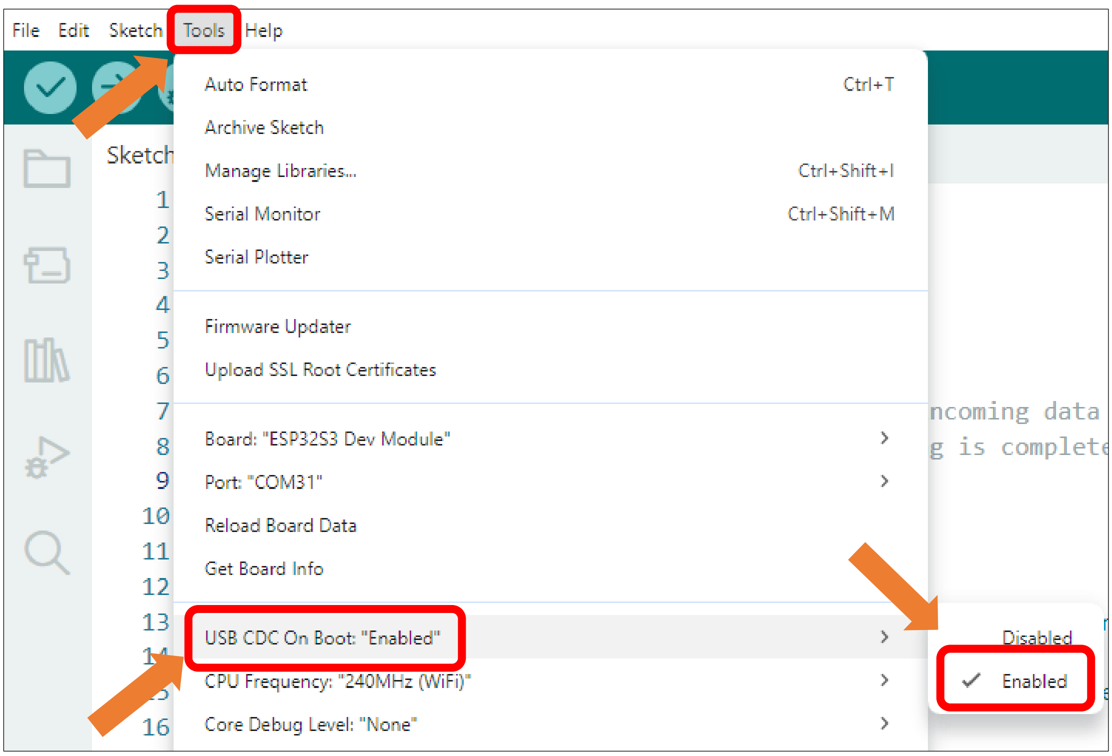
```
stripInit();
switchRGB(0);
setupBLE("ESP32S3_BLE");
```

The Loop function checks the sending command every 100 milliseconds.

```
void loop() {
   long now = millis();
   if (now - lastMsg > 100) {
     if (deviceConnected && rxload.length() > 0) {
       Serial.println(rxload);
       if (strncmp(rxload.c_str(), "red_on", 6) == 0) {
         setRGB(1, 0, 0);
       } else if (strncmp(rxload.c_str(), "red_off", 7) == 0) {
```

```
        setRGB(0, 0, 0);
      } else if (strncmp(rxload.c_str(), "green_on", 8) == 0) {
        setRGB(0, 1, 0);
      } else if (strncmp(rxload.c_str(), "green_off", 9) == 0) {
        setRGB(0, 0, 0);
      } else if (strncmp(rxload.c_str(), "blue_on", 7) == 0) {
        setRGB(0, 0, 1);
      } else if (strncmp(rxload.c_str(), "blue_off", 8) == 0) {
        setRGB(0, 0, 0);
      }
      rxload = "";
    }
    if (Serial.available() > 0) {
      String str = Serial.readString();
      const char *newValue = str.c_str();
      pCharacteristic->setValue(newValue);
      pCharacteristic->notify();
    }
    lastMsg = now;
  }
}
```

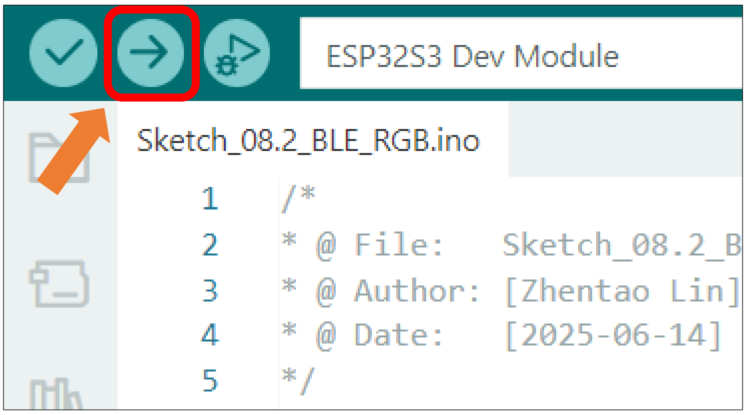
Enable the "USB CDC On Boot" feature.



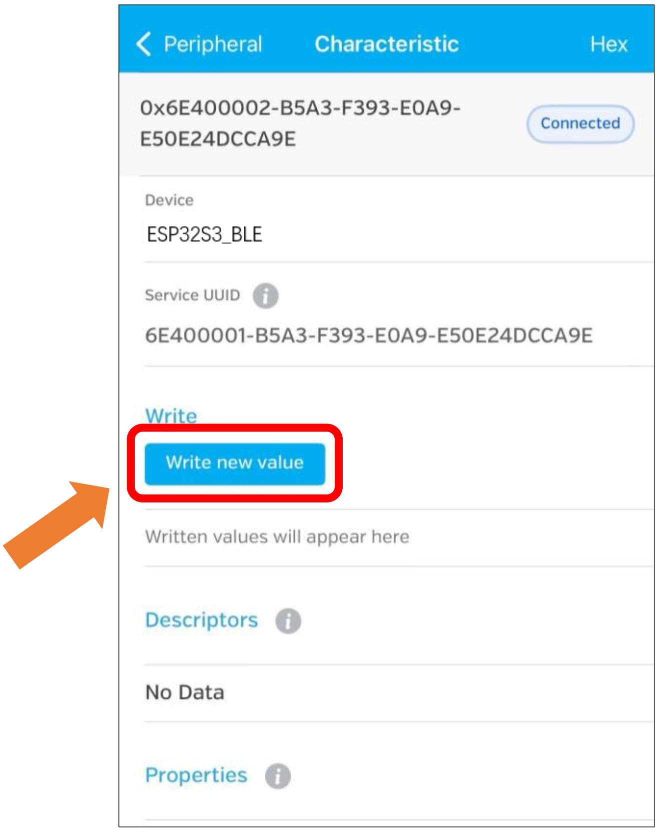
Before compiling and uploading the code, please be sure to confirm the hardware model you are using. Based on your device, please modify the macro definition (#define) at the top of the code: remove the comment symbols (//) in front of the corresponding m

```
#define FNK0104AB_2P8_240x320_ILI9341
//#define FNK0104N_3P5_320x480_ST77922
//#define FNK0104S_4P0_320x480_ST7796
```

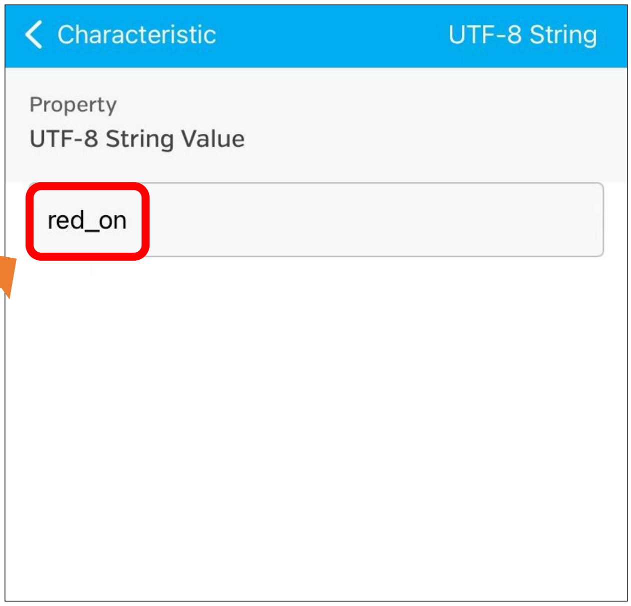
Click “**Upload**” to upload the code to Freenove ESP32-S3 Display.



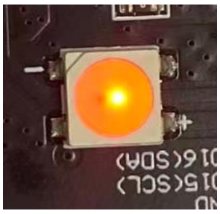
Click “Write new value”



Enter the messages to send. Here we take “red_on” as an example.



The RGB LED on the Freenove ESP32-S3 Display emits red light.



You can also use the following instructions to control the RGB LED:

red_off: red light off
green_on: green light on
green_off: green light off
blue_on: blue light on
blue_off: blue light off

Chapter 9 WIFI Web Server

Project 9.1 WIFI Web Servers LED

Component List

Freenove ESP32-S3 Display x 1	USB cable x1
	

Component Knowledge

Wi-Fi

Wi-Fi is a wireless LAN (WLAN) technology compliant with the IEEE 802.11 standard, enabling devices to transmit data or connect to the internet via radio waves within short-range coverage (typically up to tens of meters). In embedded systems, Wi-Fi modules provide wireless communication capabilities, allowing devices to connect to routers, access cloud services, or interact with other devices.

Its core features include:

- Wireless connectivity (eliminating the need for physical cabling)
- Moderate to high-speed data transfer (depending on protocol versions such as 802.11n/ac)
- Secure communication (ensured by encryption protocols like WPA2/WPA3)

In embedded development, Wi-Fi is one of the key technologies for enabling IoT (Internet of Things) device connectivity.

Web

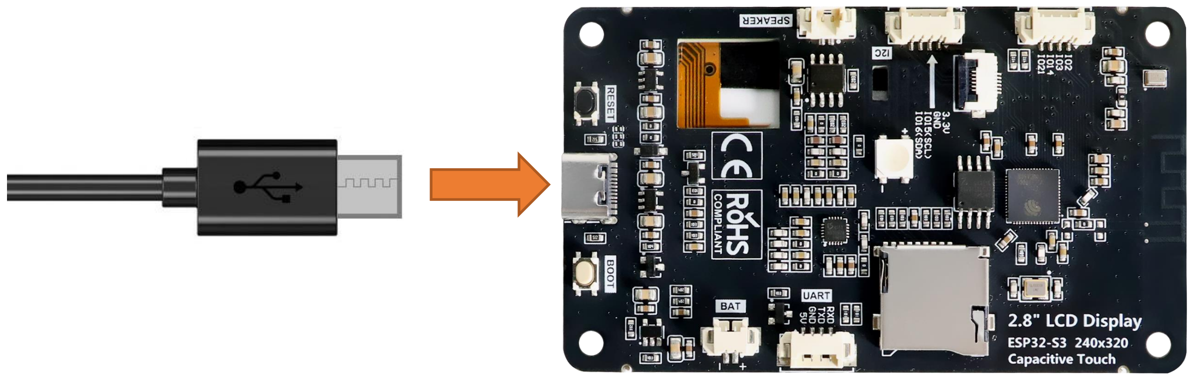
The Web (World Wide Web) is an internet-based information system that integrates global resources through hypertext links. In the embedded field, Web technology refers to devices equipped with lightweight built-in web servers, allowing users to remotely access the device interface via browsers (such as Chrome or Edge). By entering the device's IP address, users can open an interactive page to perform functions like status monitoring, parameter configuration, or firmware upgrades. Its core value lies in cross-platform compatibility (no need for dedicated software installation) and standardized protocols (HTTP/HTTPS), making it a mainstream solution for remote management of embedded devices.

HTML & CSS & JavaScript

- **HTML (HyperText Markup Language)** is the standard language for structuring web pages. It uses tags to define page elements, forming the foundational framework. In embedded web interfaces, HTML describes the layout of components such as device status displays and configuration forms.
- **CSS (Cascading Style Sheets)** controls the visual presentation of web pages, including colors, fonts, spacing, and responsive layouts. Through CSS rules, developers style HTML elements into intuitive interactive interfaces. The combination of HTML and CSS enables the creation of lightweight device control pages without complex graphics libraries, reducing resource overhead in embedded systems.
- **JavaScript (a scripting language)** adds dynamic behavior and interaction logic to web pages. In embedded web interfaces, JavaScript plays a crucial role: it responds to user actions and enables asynchronous communication with the device backend through technologies like **AJAX** or **WebSocket**. This allows the webpage to fetch real-time device status data, update displays without refreshing, and send control commands or configuration parameters—delivering a smooth and efficient remote management experience.

Circuit

Connect Freenove ESP32 -S3 to the computer using the USB cable.



Sketch

Next, we download the code to Freenove_ESP32_S3_Display to test. Open "**Sketch_09.1_WiFi_Web_Servers_LED**" folder under **"Freenove_ESP32_S3_Display\Sketches"** and double-click "**Sketch_09.1_WiFi_Web_Servers_LED.ino**".

Important Note: Before upload the code, please ensure that the Wi-Fi SSID and Password are correctly configured in the code, and that the Wi-Fi network to connect is 2.4GHz.

```
const char* ssid     = "**************";
const char* password = "**************";
```

Sketch_09.1_WiFi_Web_Servers_LED

The following is the program code:

```
#include <WiFi.h>
#include "Freenove_WS2812_Lib_for_ESP32.h"

// WiFi credentials
```

```
const char* ssid     = "**********";
const char* password = "**********";

// Web server on port 80
WiFiServer server(80);

// HTTP request buffer
String request;

#define FNK0104AB_2P8_240x320_ILI9341
//#define FNK0104N_3P5_320x480_ST77922
//#define FNK0104S_4P0_320x480_ST7796

#ifdef FNK0104N_3P5_320x480_ST77922
 #define LEDS_PIN    40
#else
 #define LEDS_PIN    42
#endif
#define LEDS_COUNT 1
#define CHANNEL     0

Freenove_ESP32_WS2812 strip = Freenove_ESP32_WS2812(LEDS_COUNT, LEDS_PIN, CHANNEL, TYPE_GRB);

// LED state tracking (0=Red, 1=Green, 2=Blue)
bool ledStates[3] = {false, false, false}; // All OFF initially
String redLedState = "OFF";
String greenLedState = "OFF";
String blueLedState = "OFF";

// Timeout settings
unsigned long currentTime = millis();
unsigned long previousTime = 0;
const long timeoutTime = 2000;  // 2 seconds

void setupStrip(void) {
  strip.begin();
  strip.setBrightness(10);
  strip.setLedColorData(0, 0, 0, 0); // Turn off all LEDs
  strip.show();
}

void updateLedStrip(void) {
  uint8_t r = ledStates[0] ? 255 : 0;
  uint8_t g = ledStates[1] ? 255 : 0;
```

```
  uint8_t b = ledStates[2] ? 255 : 0;

  strip.setLedColorData(0, r, g, b);
  strip.show();
}

void setup() {
  Serial.begin(115200);

  // Initialize LED strip
  setupStrip();

  // Connect to WiFi
  Serial.print("Connecting to ");
  Serial.println(ssid);
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }

  Serial.println("");
  Serial.println("WiFi connected.");
  Serial.println("IP address: ");
  Serial.println(WiFi.localIP());

  server.begin();
}

void loop() {
  WiFiClient client = server.available();

  if (client) {
    currentTime = millis();
    previousTime = currentTime;
    Serial.println("New Client.");
    String currentLine = "";

    while (client.connected() && currentTime - previousTime <= timeoutTime) {
      currentTime = millis();
      if (client.available()) {
        char c = client.read();
        Serial.write(c);
        request += c;
```

```

        if (c == '\n') {
          if (currentLine.length() == 0) {
            // HTTP response headers
            // Handle incoming commands first. These are AJAX requests.
            if (request.indexOf("GET /red/ON") >= 0) {
              Serial.println("Red LED ON");
              redLedState = "ON";
              ledStates[0] = true;
            } else if (request.indexOf("GET /red/OFF") >= 0) {
              Serial.println("Red LED OFF");
              redLedState = "OFF";
              ledStates[0] = false;
            } else if (request.indexOf("GET /green/ON") >= 0) {
              Serial.println("Green LED ON");
              greenLedState = "ON";
              ledStates[1] = true;
            } else if (request.indexOf("GET /green/OFF") >= 0) {
              Serial.println("Green LED OFF");
              greenLedState = "OFF";
              ledStates[1] = false;
            } else if (request.indexOf("GET /blue/ON") >= 0) {
              Serial.println("Blue LED ON");
              blueLedState = "ON";
              ledStates[2] = true;
            } else if (request.indexOf("GET /blue/OFF") >= 0) {
              Serial.println("Blue LED OFF");
              blueLedState = "OFF";
              ledStates[2] = false;
            }

            // Update LED strip based on states
            updateLedStrip();

            // Send HTML page for non-AJAX requests
            if (request.indexOf("/red/") < 0 && request.indexOf("/green/") < 0 &&
request.indexOf("/blue/") < 0) {
              // Send the Complete HTML page
              client.println("HTTP/1.1 200 OK");
              client.println("Content-type:text/html");
              client.println("Connection: close");
              client.println();

              // Generate HTML response
```

```
            client.println("<!DOCTYPE html><html>");
            client.println("<head><meta charset=\"UTF-8\">");
            client.println("<meta name=\"viewport\" content=\"width=device-width, initial-scale=1.0\">");
            client.println("<title>ESP32 Web Server LED</title>");
            client.println("<style>");
            client.println("html{font-family:'Segoe UI',Roboto,sans-serif;background:#f5f5f5;height:100vh;display:flex;justify-content:center;align-items:center;}");
            client.println("body{margin:0;padding:20px;background:white;border-radius:15px;box-shadow:0 10px 30px rgba(0,0,0,0.1);max-width:800px;width:90vw;}");
            client.println(".container{display:grid;grid-template-columns:repeat(3, 1fr);gap:20px;margin-top:20px;}");
            client.println(".card{border-radius:15px;padding:25px;text-align:center;min-width:180px;transition:all 0.3s;cursor:pointer;border:2px solid;position:relative;overflow:hidden;}");
            client.println(".card:hover{transform:translateY(-5px);box-shadow:0 15px 35px rgba(0,0,0,0.15);}");
            client.println(".card:active{transform:translateY(0) scale(0.98);}");
            client.println("#red-card{background:linear-gradient(145deg,#ffcdd2,#ef9a9a);border-color:#f44336;}");
            client.println("#green-card{background:linear-gradient(145deg,#c8e6c9,#a5d6a7);border-color:#4caf50;}");
            client.println("#blue-card{background:linear-gradient(145deg,#bbdefb,#90caf9);border-color:#2196f3;}");
            client.println(".card.off-state{filter:grayscale(70%) brightness(0.85);}");
            client.println("h1{color:#333;text-align:center;font-size:2.4rem;margin-bottom:10px;border-bottom:2px solid #eee;padding-bottom:15px;}");
            client.println(".status{font-size:1.4rem;font-weight:600;margin:25px 0;color:#333;position:relative;z-index:2;}");
            client.println(".device-info{text-align:center;margin-top:25px;color:#777;font-size:0.9rem;padding-top:15px;border-top:1px solid #eee;}");
            client.println("@media (max-width:768px){.container{grid-template-columns:1fr;} body{padding:15px;} h1{font-size:2rem;}}");
            client.println("</style></head>");
            client.println("<body>");
            client.println("<h1>ESP32 Web Server LED</h1>");
            client.println("<div class=\"container\">");

            // Red LED control
            client.println("<div class=\"card " + String(redLedState == "OFF" ? "off-state" : "") + "\" id=\"red-card\" onclick=\"toggleLED('red', this)\">");
            client.println("<h2>Red LED</h2>");
            client.println("<p class=\"status\" id=\"status-red\">" + redLedState + "</p>");
```

```
              client.println("</div>");

              // Green LED control
              client.println("<div class=\"card " + String(greenLedState == "OFF" ? "off-state" : "") + "\" id=\"green-card\" onclick=\"toggleLED('green', this)\">");
              client.println("<h2>Green LED</h2>");
              client.println("<p class=\"status\" id=\"status-green\">" + greenLedState + "</p>");
              client.println("</div>");

              // Blue LED control
              client.println("<div class=\"card " + String(blueLedState == "OFF" ? "off-state" : "") + "\" id=\"blue-card\" onclick=\"toggleLED('blue', this)\">");
              client.println("<h2>Blue LED</h2>");
              client.println("<p class=\"status\" id=\"status-blue\">" + blueLedState + "</p>");
              client.println("</div>");

              client.println("</div>");
              client.println("<div class=\"device-info\">Device IP: " + WiFi.localIP().toString() + " | ESP32 Web Server LED</div>");
              client.println("<script>");
              client.println("function toggleLED(color, element) {");
              client.println("  const statusElement = document.getElementById('status-' + color);");
              client.println("  const currentState = statusElement.innerText;");
              client.println("  const newState = currentState === 'ON' ? 'OFF' : 'ON';");
              client.println("  fetch('/' + color + '/' + newState);");
              client.println("  statusElement.innerText = newState;");
              client.println("  if (newState === 'ON') { element.classList.remove('off-state'); } else { element.classList.add('off-state'); }");
              client.println("}");
              client.println("</script>");
              client.println("</body></html>");
            }
            break;
          } else {
            currentLine = "";
          }
        } else if (c != '\r') {
          currentLine += c;
        }
      }
    }
```

```
    request = "";
    client.stop();
    Serial.println("Client disconnected.");
    Serial.println("");
  }
}
```

Code Explanation

Include necessary header files.

```
#include <WiFi.h>
```

Configure WiFi SSID and password.

```
const char* ssid     = "**********";
const char* password = "**********";
```

Define pins for the RGB LED.

```
#ifdef FNK0104N_3P5_320x480_ST77922
 #define LEDS_PIN   40
#else
 #define LEDS_PIN   42
#endif
#define LEDS_COUNT 1
#define CHANNEL    0
```

Initialize the configuration related to RGB LED.

```
strip.begin();
strip.setBrightness(10);
strip.setLedColorData(0, 0, 0, 0); // Turn off all LEDs
strip.show();
```

Connect to WIFI.

```
// Connect to WiFi
Serial.print("Connecting to ");
Serial.println(ssid);
WiFi.begin(ssid, password);
while (WiFi.status() != WL_CONNECTED) {
  delay(500);
  Serial.print(".");
}

Serial.println("");
Serial.println("WiFi connected.");
Serial.println("IP address: ");
Serial.println(WiFi.localIP());

server.begin();
```

Check if there are any new clients connected to the server.

```
WiFiClient client = server.available();
```

Parse the URL command and control the corresponding LED on and off.

```
// Handle incoming commands first. These are AJAX requests.
if (request.indexOf("GET /red/ON") >= 0) {
  Serial.println("Red LED ON");
  redLedState = "ON";
  ledStates[0] = true;
} else if (request.indexOf("GET /red/OFF") >= 0) {
  Serial.println("Red LED OFF");
  redLedState = "OFF";
  ledStates[0] = false;
} else if (request.indexOf("GET /green/ON") >= 0) {
  Serial.println("Green LED ON");
  greenLedState = "ON";
  ledStates[1] = true;
} else if (request.indexOf("GET /green/OFF") >= 0) {
  Serial.println("Green LED OFF");
  greenLedState = "OFF";
  ledStates[1] = false;
} else if (request.indexOf("GET /blue/ON") >= 0) {
  Serial.println("Blue LED ON");
  blueLedState = "ON";
  ledStates[2] = true;
} else if (request.indexOf("GET /blue/OFF") >= 0) {
  Serial.println("Blue LED OFF");
  blueLedState = "OFF";
  ledStates[2] = false;
}
```

Use CSS styles to enhance the visual appeal of web interfaces.

```
client.println("html{font-family:'Segoe UI',Roboto,sans-serif;background:#f5f5f5;height:100vh;display:flex;justify-content:center;align-items:center;}");
client.println("body{margin:0;padding:20px;background:white;border-radius:15px;box-shadow:0 10px 30px rgba(0,0,0,0.1);max-width:800px;width:90vw;}");
client.println(".container{display:grid;grid-template-columns:repeat(3, 1fr);gap:20px;margin-top:20px;}");
client.println(".card{border-radius:15px;padding:25px;text-align:center;min-width:180px;transition:all 0.3s;cursor:pointer;border:2px solid;position:relative;overflow:hidden;}");
client.println(".card:hover{transform:translateY(-5px);box-shadow:0 15px 35px rgba(0,0,0,0.15);}");
client.println(".card:active{transform:translateY(0) scale(0.98);}");
client.println("#red-card{background:linear-gradient(145deg,#ffcdd2,#ef9a9a);border-color:#f44336;}");

```

```
client.println("#green-card{background:linear-gradient(145deg,#c8e6c9,#a5d6a7);border-color:#4caf50;}");
client.println("#blue-card{background:linear-gradient(145deg,#bbdefb,#90caf9);border-color:#2196f3;}");
client.println(".card.off-state{filter:grayscale(70%) brightness(0.85);}");
client.println("h1{color:#333;text-align:center;font-size:2.4rem;margin-bottom:10px;border-bottom:2px solid #eee;padding-bottom:15px;}");
client.println(".status{font-size:1.4rem;font-weight:600;margin:25px 0;color:#333;position:relative;z-index:2;}");
client.println(".device-info{text-align:center;margin-top:25px;color:#777;font-size:0.9rem;padding-top:15px;border-top:1px solid #eee;}");
client.println("@media (max-width:768px){.container{grid-template-columns:1fr;} body{padding:15px;} h1{font-size:2rem;}}");
```

Generate the HTML contents to control the LED.

```
// Red LED control
client.println("<div class=\"card " + String(redLedState == "OFF" ? "off-state" : "") + "\" id=\"red-card\" onclick=\"toggleLED('red', this)\">");
client.println("<h2>Red LED</h2>");
client.println("<p class=\"status\" id=\"status-red\">" + redLedState + "</p>");
client.println("</div>");

// Green LED control
client.println("<div class=\"card " + String(greenLedState == "OFF" ? "off-state" : "") + "\" id=\"green-card\" onclick=\"toggleLED('green', this)\">");
client.println("<h2>Green LED</h2>");
client.println("<p class=\"status\" id=\"status-green\">" + greenLedState + "</p>");
client.println("</div>");

// Blue LED control
client.println("<div class=\"card " + String(blueLedState == "OFF" ? "off-state" : "") + "\" id=\"blue-card\" onclick=\"toggleLED('blue', this)\">");
client.println("<h2>Blue LED</h2>");
client.println("<p class=\"status\" id=\"status-blue\">" + blueLedState + "</p>");
client.println("</div>");
```

Disconnect from the client and print the disconnection information in the serial monitor

```
client.stop();
Serial.println("Client disconnected.");
Serial.println("");
```

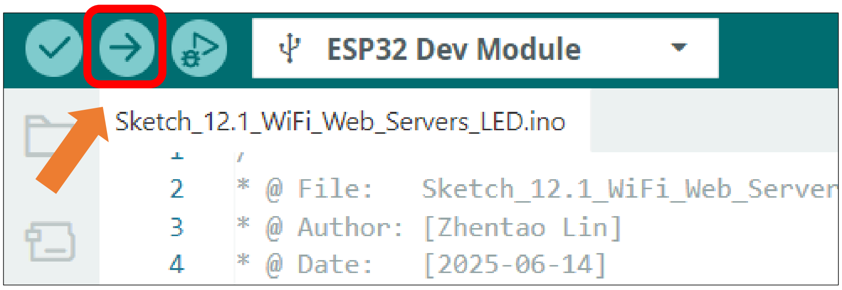
Before compiling and uploading the code, please be sure to confirm the hardware model you are using. Based on your device, please modify the macro definition (#define) at the top of the code: remove the comment symbols (//) in front of the corresponding m

```
#define FNK0104AB_2P8_240x320_ILI9341
//#define FNK0104N_3P5_320x480_ST77922
//#define FNK0104S_4P0_320x480_ST7796
```

Input correct WiFi(2.4GHz) SSID and password.

```
// WiFi credentials
const char* ssid     = "**************";
const char* password = "**************";
```

Click “**Upload**” to upload the code to Freenove ESP32 Display.



When the serial monitor prints the following information, it means the network connection is successful. Use a mobile phone or computer browser to open the IP address printed by the serial monitor.

Please note: If it keeps printing dots, please confirm whether the WiFi is in the 2.4G band.

```
14:31:24.553 -> Connecting to FYI_2.4G
14:31:25.115 -> .....
14:31:27.088 -> WiFi connected.
14:31:27.088 -> IP address:
14:31:27.122 -> 192.168.1.29
```

The following screen will be displayed in the computer's browser, where you can control the color of the RGB light by clicking on the three cards.

On the phone web browser, you'll see the following contents. The color of the RGB light can be controlled by clicking on the three cards.

Chapter 10 TFT Display

Project 10.1 TFT_Rainbow

Component List

Freenove ESP32-S3 Display x 1	USB cable x1
	

Component Knowledge

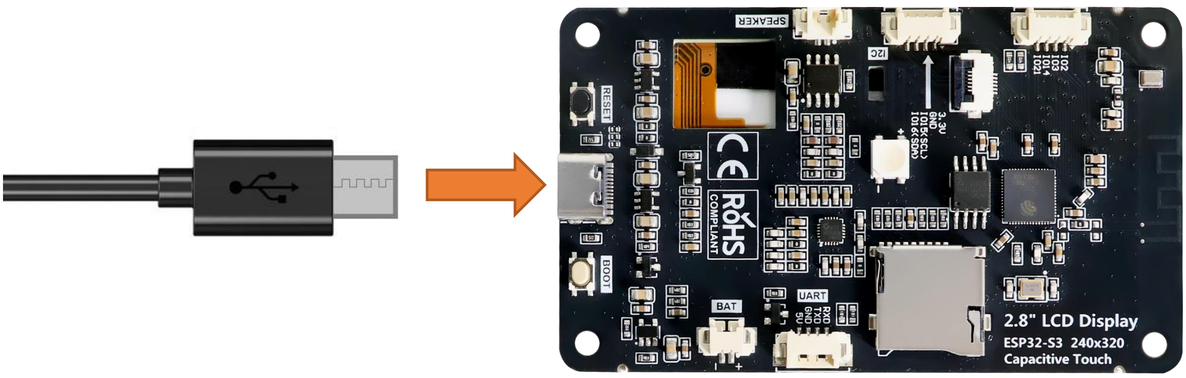
TFT Display

TFT (Thin Film Transistor) is an electronic component that serves as the foundation for TFT displays, the mainstream display technology in modern laptops and desktop computers. In these displays, each individual liquid crystal pixel is controlled by its own dedicated thin-film transistor embedded directly behind it. This architecture classifies TFT screens as a form of active-matrix LCD (AMLCD) technology.

As one of the finest LCD color displays available, TFT screens offer superior performance characteristics including rapid response times, exceptional brightness levels, and outstanding contrast ratios.

Circuit

Connect Freenove ESP32 -S3 to the computer using the USB cable.

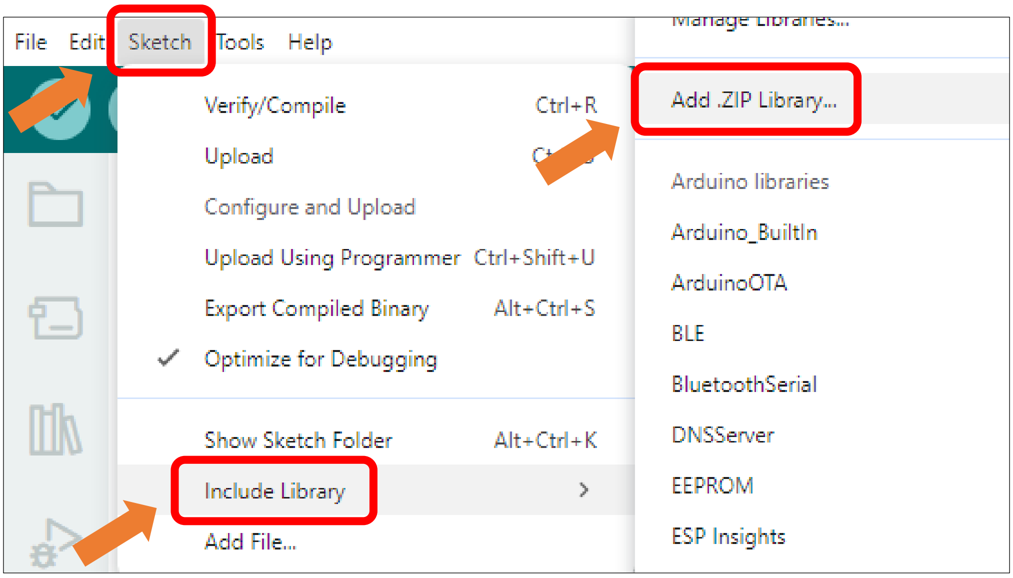


Sketch

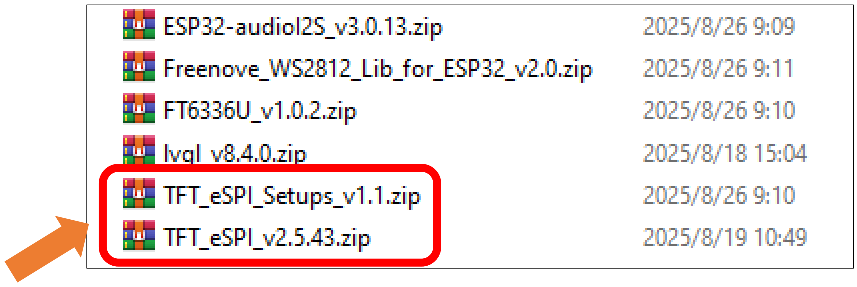
Open " **Sketch_10.1_TFT_Rainbow.ino** " folder under **"Freenove_ESP32_S3_Display\Sketches"** and double-click "**Sketch_10.1_TFT_Rainbow.ino**".

Install Libraries

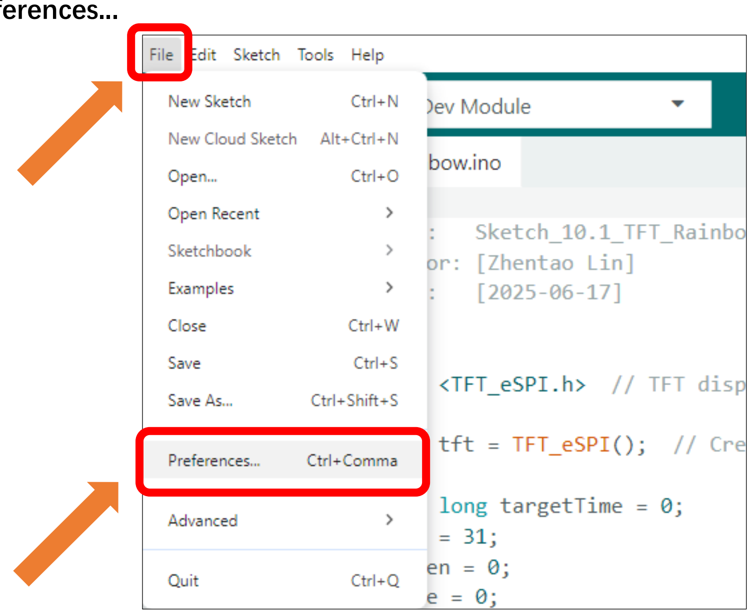
Click **Sketch** -> **Include Library** -> **Add .ZIP Library...**



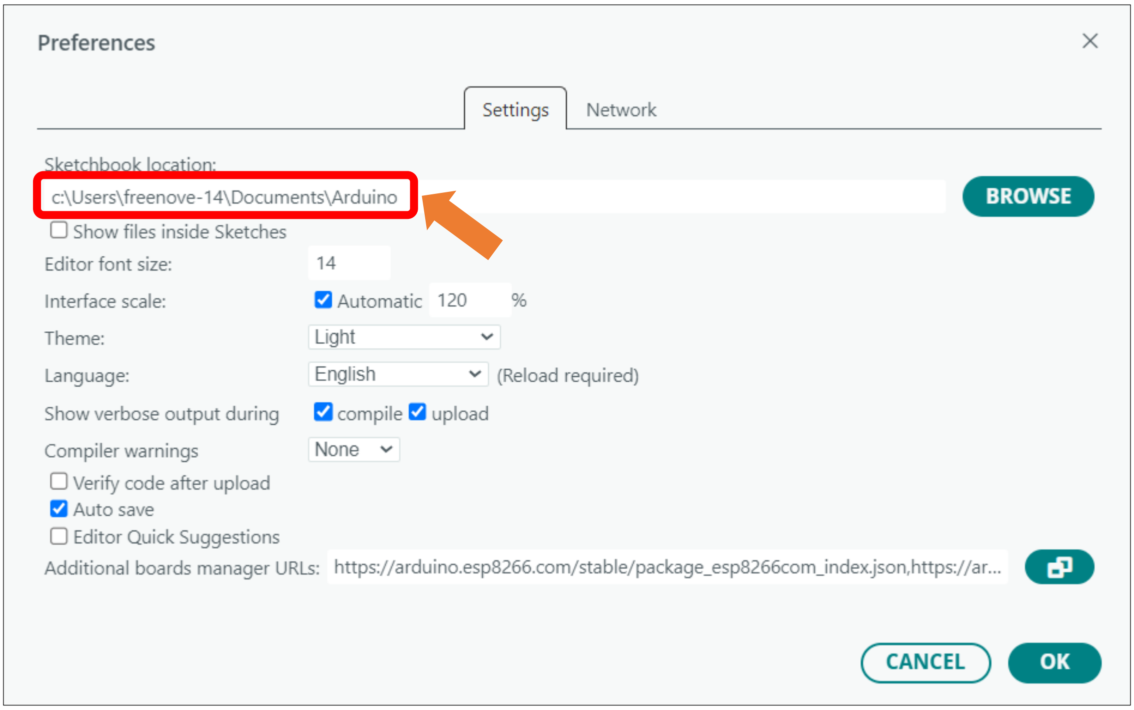
Install **TFT_eSPI_v2.5.43.zip** and **TFT_eSPI_Setups_v1.1.zip**



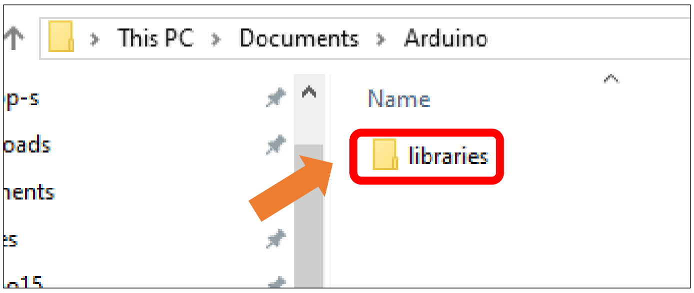
Click **File** -> **Preferences...**



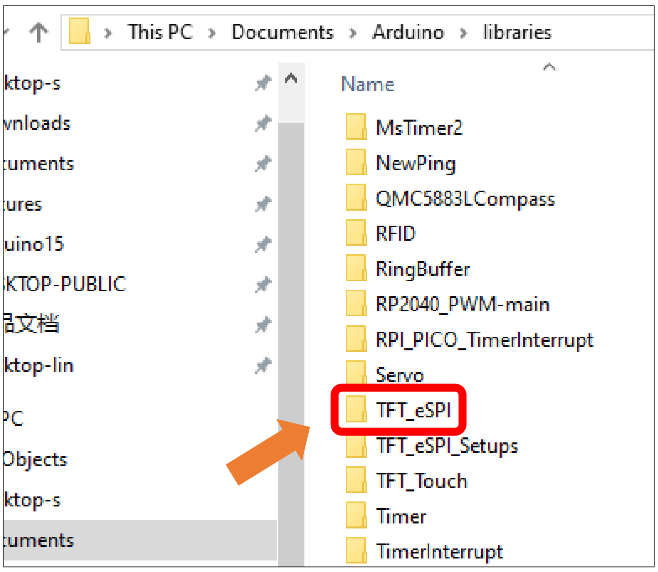
Linux and Mac users can use the **cd** command on Terminal to enter the sketchbook location, and Windows users can copy and paste it in the file explorer.



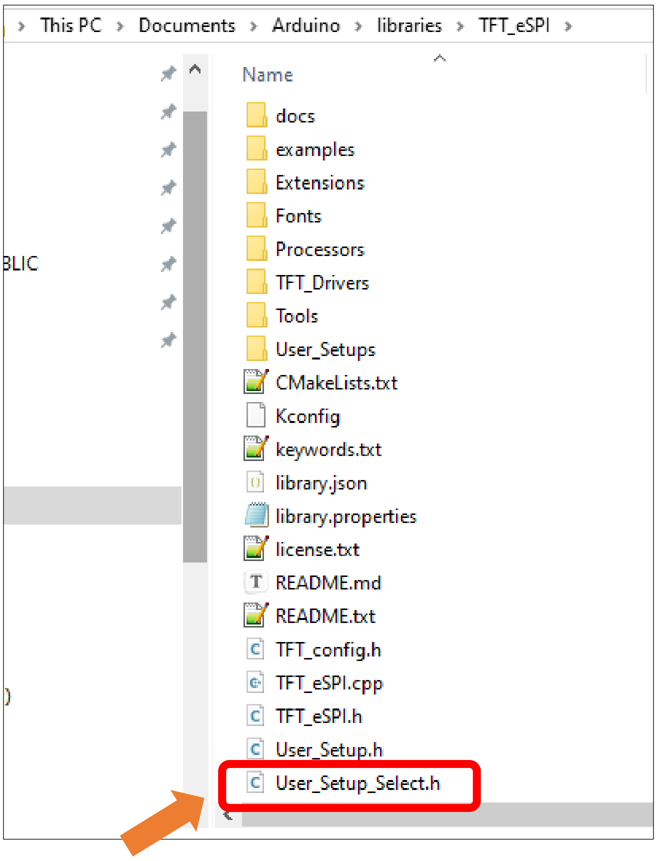
Double click **libraries**



Double click **TFT_eSPI**



Open the **User_Setup_Select.h** file



Remove the "//" comment markers from the macro definition line **corresponding to your Freenove ESP32 Display model**.

```
//#define FNK0086A_2P8_240x320_ST7789
//#define FNK0102A_1P14_135x240_ST7789
//#define FNK0102H_3P5_320x480_ST7796
//#define FNK0103B_2P8_240x320_ST7789
//#define FNK0103F_2P8_240x320_ILI9341
//#define FNK0103L_3P2_240x320_ST7789
//#define FNK0103N_3P5_320x480_ST7796
//#define FNK0103S_4P0_320x480_ST7796
#define FNK0104AB_2P8_240x320_ILI9341
//#define FNK0104N_3P5_320x480_ST77922
//#define FNK0104S_4P0_320x480_ST7796
//#define FNK0115A_4P3_320x480_ST7796
```

Here we take the 2.8-inch display as an example, modifying it as shown below:

```
//#define FNK0086A_2P8_240x320_ST7789
//#define FNK0102A_1P14_135x240_ST7789
//#define FNK0102H_3P5_320x480_ST7796
//#define FNK0103B_2P8_240x320_ST7789
//#define FNK0103F_2P8_240x320_ILI9341
//#define FNK0103L_3P2_240x320_ST7789
//#define FNK0103N_3P5_320x480_ST7796
//#define FNK0103S_4P0_320x480_ST7796
#define FNK0104AB_2P8_240x320_ILI9341
//#define FNK0104N_3P5_320x480_ST77922
//#define FNK0104S_4P0_320x480_ST7796
//#define FNK0115A_4P3_320x480_ST7796
```

Important Note: Only one macro definition should be uncommented.

Sketch_10.1_TFT_Rainbow

The following is the program code:

```
#include <TFT_eSPI.h>
#include <SPI.h>

#ifdef FNK0104N_3P5_320x480_ST77922
  #define SCREEN_WIDTH 480
  #define SCREEN_HEIGHT 320
  TFT_eSPI tft_qspi = TFT_eSPI();
  TFT_eSprite tft = TFT_eSprite(&tft_qspi);
  ST77922 tft_st77922 = ST77922();
#elif defined FNK0104AB_2P8_240x320_ILI9341
  #define SCREEN_WIDTH 320
  #define SCREEN_HEIGHT 240
  TFT_eSPI tft = TFT_eSPI();  // Create TFT object instance
#elif defined FNK0104H_4P0_320x480_ST7796
  #define SCREEN_WIDTH 480
  #define SCREEN_HEIGHT 320
  TFT_eSPI tft = TFT_eSPI();  // Create TFT object instance
#endif

unsigned long targetTime = 0;     // Timing variable to control animation intervals
byte red = 31;                    // Start with full red
byte green = 0;                   // No green
byte blue = 0;                    // No blue
byte state = 0;                   // State machine variable
unsigned int colour = red << 11;  // Initial color value: Red only

```

```
void setup(void) {
  #ifdef FNK0104N_3P5_320x480_ST77922
    tft_st77922.Init();
    tft_st77922.Set_Rotation(1);
    tft.createSprite(tft_st77922.Get_Width(), tft_st77922.Get_Height());
    tft.setSwapBytes(true);
  #else
    tft.init();                   // Initialize the TFT screen
    tft.setRotation(1);           // Set screen rotation (landscape mode)
  #endif
  targetTime = millis() + 100;  // Set initial target time for rainbow effect
  #ifdef FNK0104N_3P5_320x480_ST77922
    tft.fillSprite(TFT_RED);  // Fill screen with solid red
    tft_st77922.Fill_Colors(0, 0, tft_st77922.Get_Width(), tft_st77922.Get_Height(), (uint16_t
*)tft.getPointer());
  #else
    tft.fillScreen(TFT_RED);  // Fill screen with solid red
  #endif
  delay(1000);
  #ifdef FNK0104N_3P5_320x480_ST77922
    tft.fillSprite(TFT_GREEN);  // Fill screen with solid green
    tft_st77922.Fill_Colors(0, 0, tft_st77922.Get_Width(), tft_st77922.Get_Height(), (uint16_t
*)tft.getPointer());
  #else
    tft.fillScreen(TFT_GREEN);
  #endif
  delay(1000);
  #ifdef FNK0104N_3P5_320x480_ST77922
    tft.fillSprite(TFT_BLUE);  // Fill screen with solid blue
    tft_st77922.Fill_Colors(0, 0, tft_st77922.Get_Width(), tft_st77922.Get_Height(), (uint16_t
*)tft.getPointer());
  #else
    tft.fillScreen(TFT_BLUE);
  #endif
  delay(1000);
  #ifdef FNK0104N_3P5_320x480_ST77922
    tft.fillSprite(TFT_BLACK);  // Fill screen with solid black
    tft_st77922.Fill_Colors(0, 0, tft_st77922.Get_Width(), tft_st77922.Get_Height(), (uint16_t
*)tft.getPointer());
  #else
    tft.fillScreen(TFT_BLACK);
  #endif
  delay(1000);
  #ifdef FNK0104N_3P5_320x480_ST77922
```

```
    tft.fillSprite(TFT_WHITE);  // Fill screen with solid white
    tft_st77922.Fill_Colors(0, 0, tft_st77922.Get_Width(), tft_st77922.Get_Height(), (uint16_t
*)tft.getPointer());
  #else
    tft.fillScreen(TFT_WHITE);
  #endif
  delay(1000);
}

void loop() {
  if (targetTime < millis()) {      // Check if it's time to start the rainbow animation
    targetTime = millis() + 10000;  // Set next trigger after 10 seconds

    for (int i = 0; i < SCREEN_WIDTH; i++) {// Generate horizontal rainbow using vertical
lines
      tft.drawFastVLine(i, 0, SCREEN_HEIGHT, colour);
      switch (state) {
        case 0:
          green += 2;  // Transition from red to yellow (increase green)
          if (green == 64) {
            green = 63;  // Cap at max green value
            state = 1;
          }
          break;
        case 1:
          red--;  // Transition from yellow to green (decrease red)
          if (red == 255) {
            red = 0;
            state = 2;
          }
          break;
        case 2:
          blue++;  // Transition from green to cyan (increase blue)
          if (blue == 32) {
            blue = 31;  // Cap at max blue value
            state = 3;
          }
          break;
        case 3:
          green -= 2;  // Transition from cyan to blue (decrease green)
          if (green == 255) {
            green = 0;
            state = 4;
          }
```

```
          break;
        case 4:
          red++;  // Transition from blue to magenta (increase red)
          if (red == 32) {
            red = 31;  // Cap at max red value
            state = 5;
          }
          break;
        case 5:
          blue--;  // Transition from magenta to red (decrease blue)
          if (blue == 255) {
            blue = 0;
            state = 0;
          }
          break;
      }
      colour = red << 11 | green << 5 | blue;  // Combine RGB values into 16-bit color
    }
    tft.setTextColor(TFT_BLACK);           // Set text color to black
    tft.setCursor(12, 5);                  // Set cursor position
    tft.print("Original ADAfruit font!");  // Print simple string
    tft.setTextColor(TFT_BLACK, TFT_BLACK);  // Transparent background
    tft.drawCentreString("Font size 2", 80, 14, 2);             // Font 2
    tft.drawCentreString("Font size 4", 80, 30, 4);             // Font 4
    tft.drawCentreString("12.34", 80, 54, 6);                   // Font 6
    tft.drawCentreString("12.34 is in font size 6", 80, 92, 2);  // Font 2
    float pi = 3.14159;                                       // Value to print
    int precision = 3;                                        // Number of decimal digits
    int xpos = 50;                                            // X position
    int ypos = 110;                                           // Y position
    int font = 2;                                             // Font type
    xpos += tft.drawFloat(pi, precision, xpos, ypos, font);  // Draw float and update x-
position
    tft.drawString(" is pi", xpos, ypos, font);              // Continue drawing string from
updated x position
    #ifdef FNK0104N_3P5_320x480_ST77922
      tft_st77922.Fill_Colors(0, 0, tft_st77922.Get_Width(), tft_st77922.Get_Height(),
(uint16_t *)tft.getPointer());
    #endif
    delay(6000);  // Wait before next cycle
  }
}
```

Code Explanation

Include the necessary header file.

```
#include <TFT_eSPI.h>
#include <SPI.h>
```

Implement the rainbow animation effect.

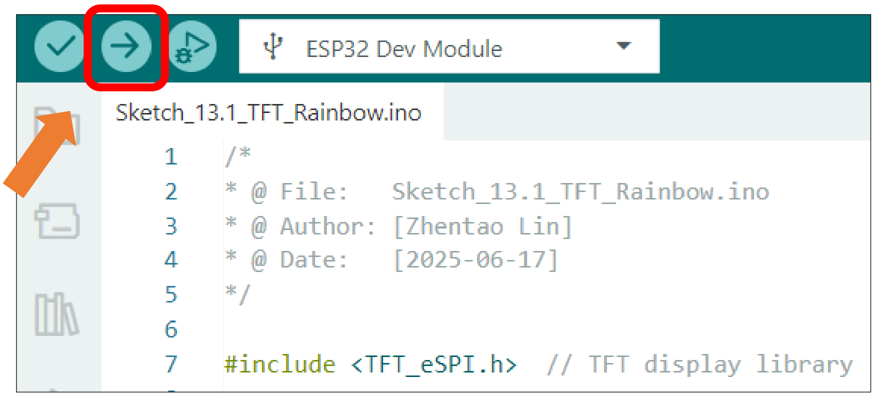
```
for (int i = 0; i < SCREEN_WIDTH; i++) {// Generate horizontal rainbow using vertical lines
  tft.drawFastVLine(i, 0, SCREEN_HEIGHT, colour);
  switch (state) {
    case 0:
      green += 2;  // Transition from red to yellow (increase green)
      if (green == 64) {
        green = 63;  // Cap at max green value
        state = 1;
      }
      break;
    case 1:
      red--;  // Transition from yellow to green (decrease red)
      if (red == 255) {
        red = 0;
        state = 2;
      }
      break;
    case 2:
      blue++;  // Transition from green to cyan (increase blue)
      if (blue == 32) {
        blue = 31;  // Cap at max blue value
        state = 3;
      }
      break;
    case 3:
      green -= 2;  // Transition from cyan to blue (decrease green)
      if (green == 255) {
        green = 0;
        state = 4;
      }
      break;
    case 4:
      red++;  // Transition from blue to magenta (increase red)
      if (red == 32) {
        red = 31;  // Cap at max red value
        state = 5;
      }
      break;
    case 5:
```

```
        blue--;  // Transition from magenta to red (decrease blue)
        if (blue == 255) {
          blue = 0;
          state = 0;
        }
        break;
    }
    colour = red << 11 | green << 5 | blue;  // Combine RGB values into 16-bit color
  }
```

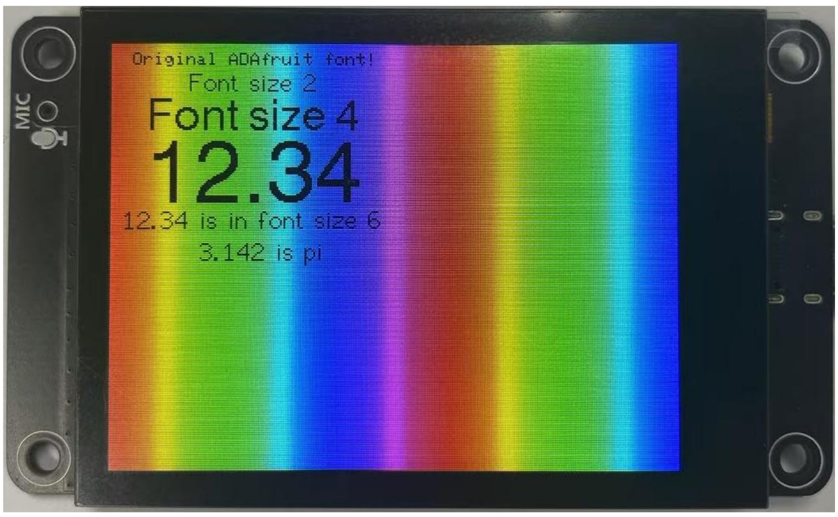
Text display.

```
  tft.setTextColor(TFT_BLACK);            // Set text color to black
  tft.setCursor(12, 5);                   // Set cursor position
  tft.print("Original ADAfruit font!");  // Print simple string
  tft.setTextColor(TFT_BLACK, TFT_BLACK);  // Transparent background
  tft.drawCentreString("Font size 2", 80, 14, 2);             // Font 2
  tft.drawCentreString("Font size 4", 80, 30, 4);             // Font 4
  tft.drawCentreString("12.34", 80, 54, 6);                   // Font 6
  tft.drawCentreString("12.34 is in font size 6", 80, 92, 2);  // Font 2
```

Click "**Upload**" to upload the code to Freenove_ESP32_S3_Display.



The TFT screen will change colors in the order of red -> green -> blue -> black -> white before displaying text and rainbow effects.



Project 10.2 Flash JPG DMA

Component List

Freenove ESP32-S3 Display x 1	USB cable x1
	

Component Knowledge

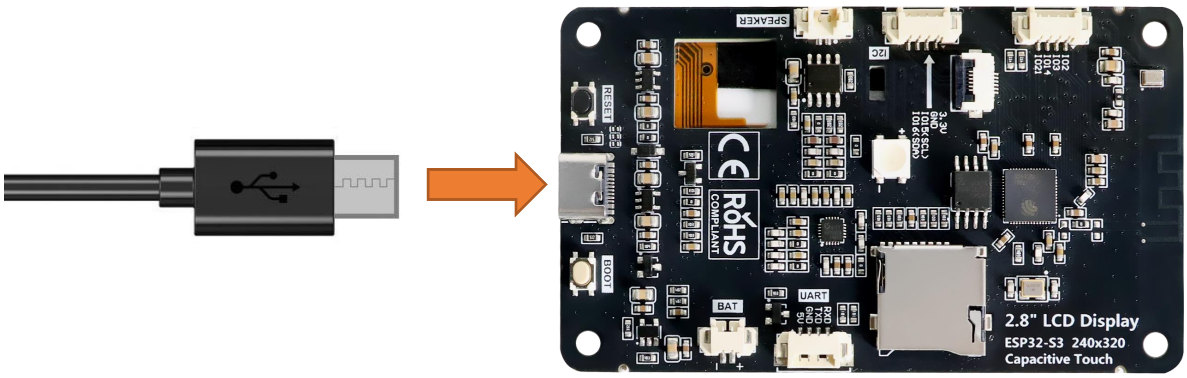
DMA

DMA, or Direct Memory Access, is a hardware feature that allows peripherals to transfer data to and from memory without needing the CPU to be directly involved, dramatically improving overall system efficiency while minimizing processor workload.

The core mechanism of DMA relies on a dedicated DMA controller taking over data transfer tasks. The CPU only needs to initialize the transfer parameters before offloading the operation, allowing computation and I/O operations to proceed in parallel.

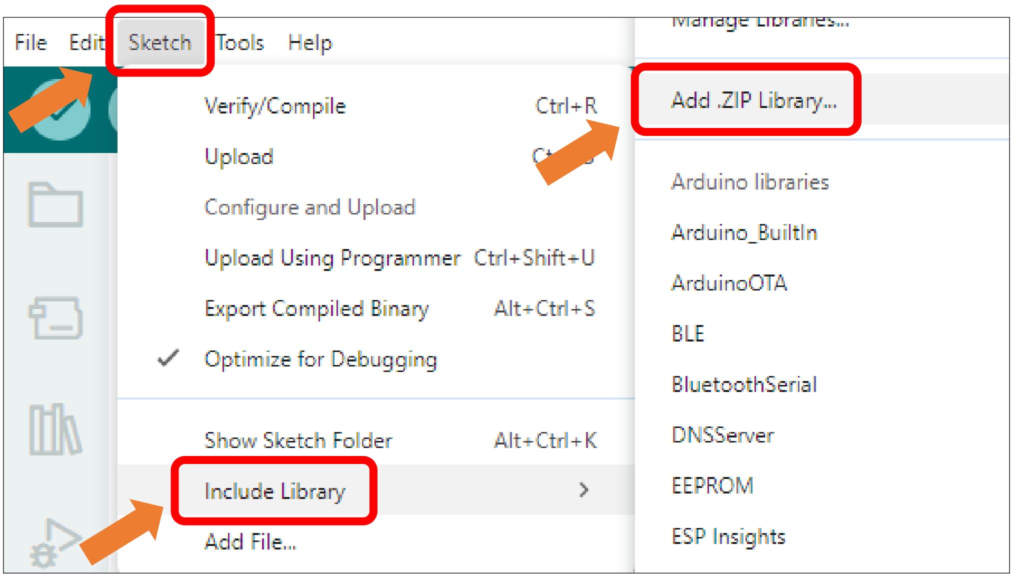
Circuit

Connect Freenove ESP32-S3 to the computer using the USB cable.

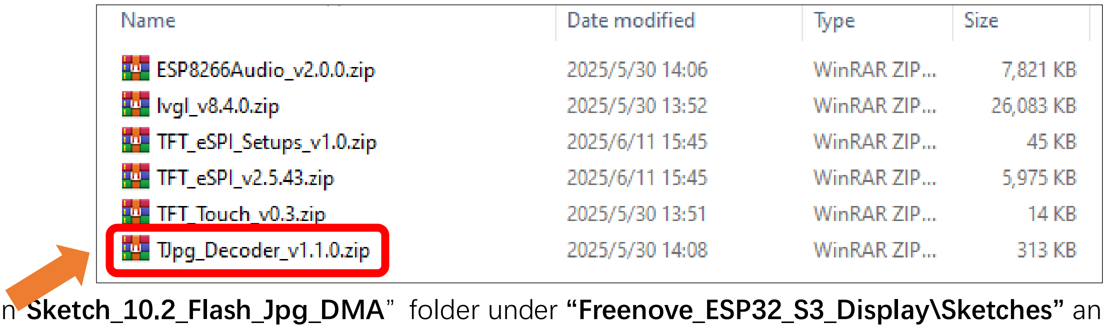


Sketch

Click **Sketch** -> **Include Library** -> **Add .ZIP Library...**



Install **TJpg_Decoder_v1.1.0.zip**



Open "**Sketch_10.2_Flash_Jpg_DMA**" folder under **"Freenove_ESP32_S3_Display\Sketches"** and double-click "**Sketch_10.2_Flash_Jpg_DMA.ino**".

Sketch_10.2_Flash_Jpg_DMA

The following is the program code:

```
#include <TFT_eSPI.h>  // TFT display library
#include <SPI.h>
#include "panda.h"          // Include raw JPEG image data array
#include <TJpg_Decoder.h>  // Include JPEG decoder library

#ifdef FNK0104N_3P5_320x480_ST77922
  TFT_eSPI tft_qspi = TFT_eSPI();
  TFT_eSprite tft = TFT_eSprite(&tft_qspi);
  ST77922 tft_st77922 = ST77922();
#else
  TFT_eSPI tft = TFT_eSPI();  // Create TFT object instance
#endif

// Callback function to draw decoded JPEG blocks on screen
```

```
bool tft_output(int16_t x, int16_t y, uint16_t w, uint16_t h, uint16_t* bitmap) {
  if (y >= tft.height()) return 0;  // Stop decoding if off-screen
  tft.pushImage(x, y, w, h, bitmap);  // Draw without DMA
  #ifdef FNK0104N_3P5_320x480_ST77922
    tft_st77922.Fill_Colors(0, 0, tft_st77922.Get_Width(), tft_st77922.Get_Height(), (uint16_t *)tft.getPointer());
  #endif
  return 1;  // Continue decoding next block
}

void setup() {
  Serial.begin(115200);
  Serial.println("\n\n Testing TJpg_Decoder library");
  #ifdef FNK0104N_3P5_320x480_ST77922
    tft_st77922.Init();
    tft_st77922.Set_Rotation(0);
    tft.createSprite(tft_st77922.Get_Width(), tft_st77922.Get_Height());
    tft.setSwapBytes(1);
    tft.fillSprite(TFT_BLACK);
    tft_st77922.Fill_Colors(0, 0, tft_st77922.Get_Width(), tft_st77922.Get_Height(), (uint16_t *)tft.getPointer());
  #else
    tft.begin();
    tft.fillScreen(TFT_BLACK);
  #endif
  TJpgDec.setJpgScale(1);           // Set scale factor (1=full size)
  tft.setSwapBytes(true);           // Match color byte order
  TJpgDec.setCallback(tft_output);  // Register drawing callback
}

void loop() {
  uint16_t w = 0, h = 0;
  TJpgDec.getJpgSize(&w, &h, img_asset, sizeof(img_asset));  // Get image dimensions
  Serial.print("Width = ");
  Serial.print(w);
  Serial.print(", height = ");
  Serial.print(h);

  uint32_t dt = millis();
  tft.startWrite();
  TJpgDec.drawJpg(0, 0, img_asset, sizeof(img_asset));  // Draw the JPEG image
  tft.endWrite();

  dt = millis() - dt;
```

```
  Serial.print(", dt = ");
  Serial.print(dt);
  Serial.println(" ms");

  delay(2000);  // Wait before redraw
}
```

Code Explanation

Include necessary header files.

```
#include <TFT_eSPI.h>  // TFT display library
#include <SPI.h>
#include "panda.h"          // Include raw JPEG image data array
#include <TJpg_Decoder.h>  // Include JPEG decoder library
```

JPEG decoding callback function

```
bool tft_output(int16_t x, int16_t y, uint16_t w, uint16_t h, uint16_t* bitmap) {
  if (y >= tft.height()) return 0;  // Stop decoding if off-screen
  tft.pushImage(x, y, w, h, bitmap);  // Draw without DMA
  #ifdef FNK0104N_3P5_320x480_ST77922
    tft_st77922.Fill_Colors(0, 0, tft_st77922.Get_Width(), tft_st77922.Get_Height(), (uint16_t
*)tft.getPointer());
  #endif
  return 1;  // Continue decoding next block
}
```

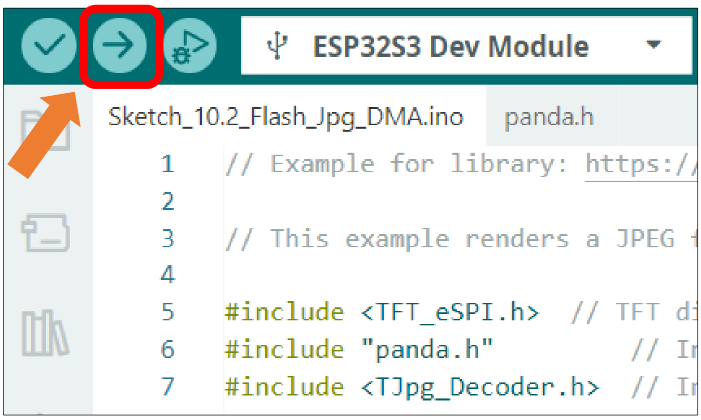
Get JPG size.

```
TJpgDec.getJpgSize(&w, &h, img_asset, sizeof(img_asset));  // Get image dimensions
```

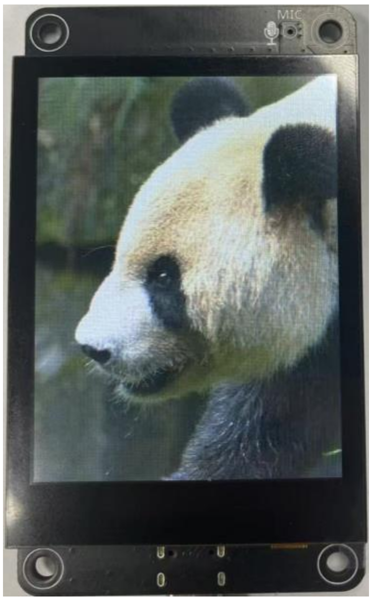
Draw images on the TFT screen.

```
tft.startWrite();
TJpgDec.drawJpg(0, 0, img_asset, sizeof(img_asset));  // Draw the JPEG image
tft.endWrite();
```

Click "**Upload**" to upload the code to Freenove ESP32 Display



After the code is uploaded, an image will be displayed on the TFT screen. The image is from https://github.com/Bodmer/TJpg_Decoder

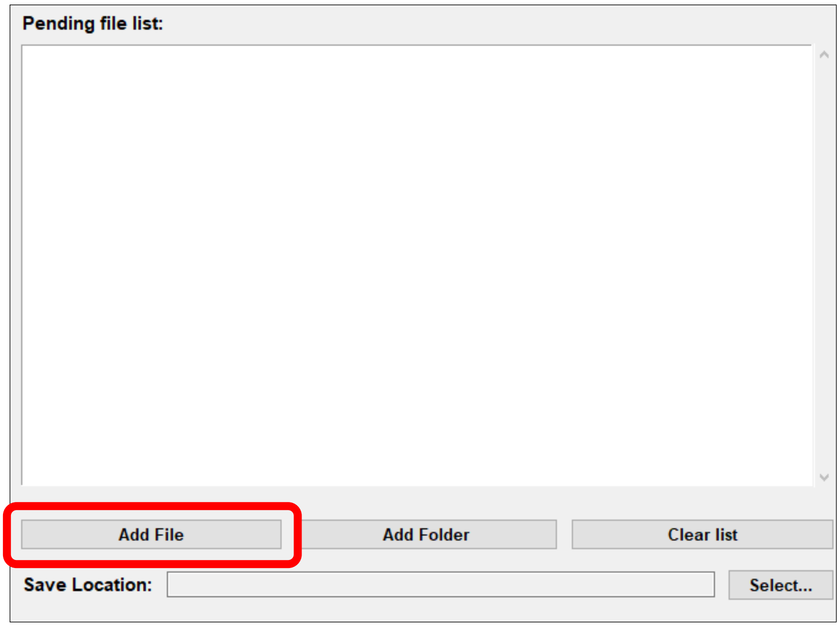


Custom image display

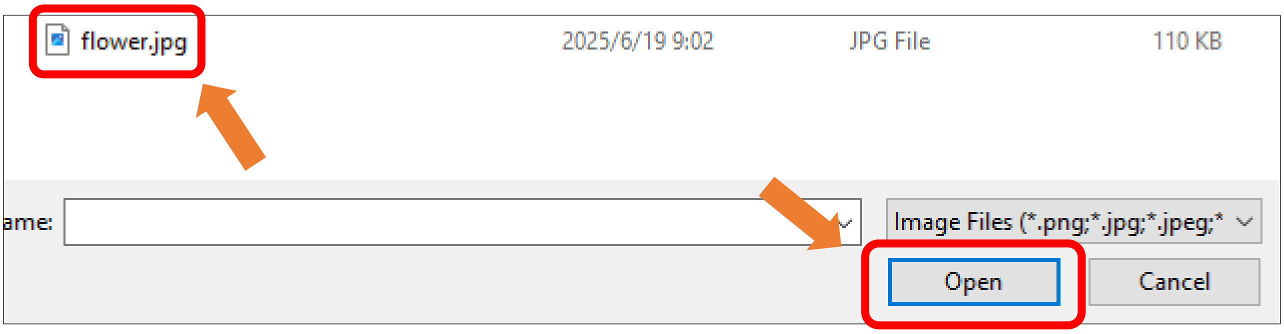
You can customize the image displayed on the display according to your personal preferences.

First, open **Freenove_ESP32_S3_Display\Sketches\Sketch_10.2_Flash_Jpg_DMA\Freenove Image Tool.exe**

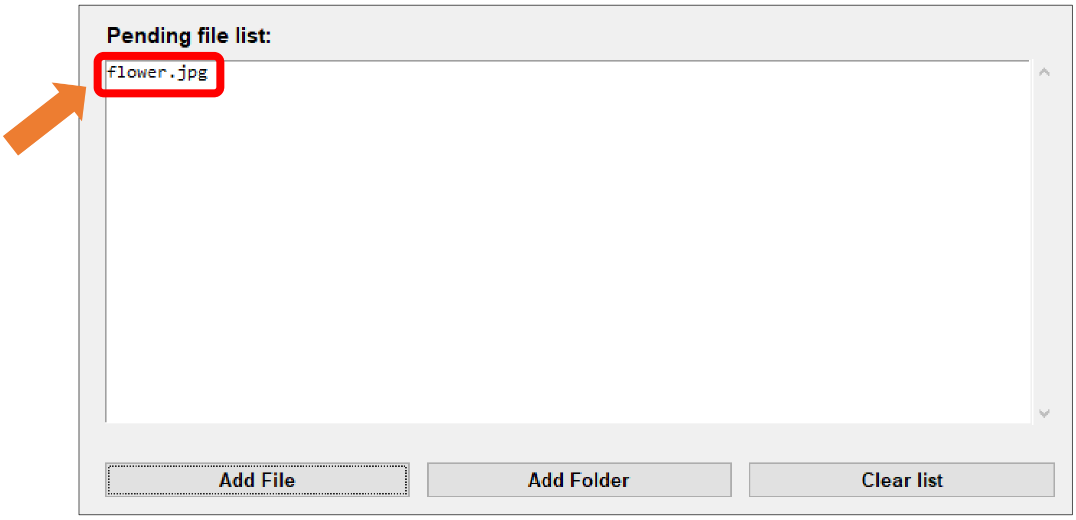
Click **"Add File"**



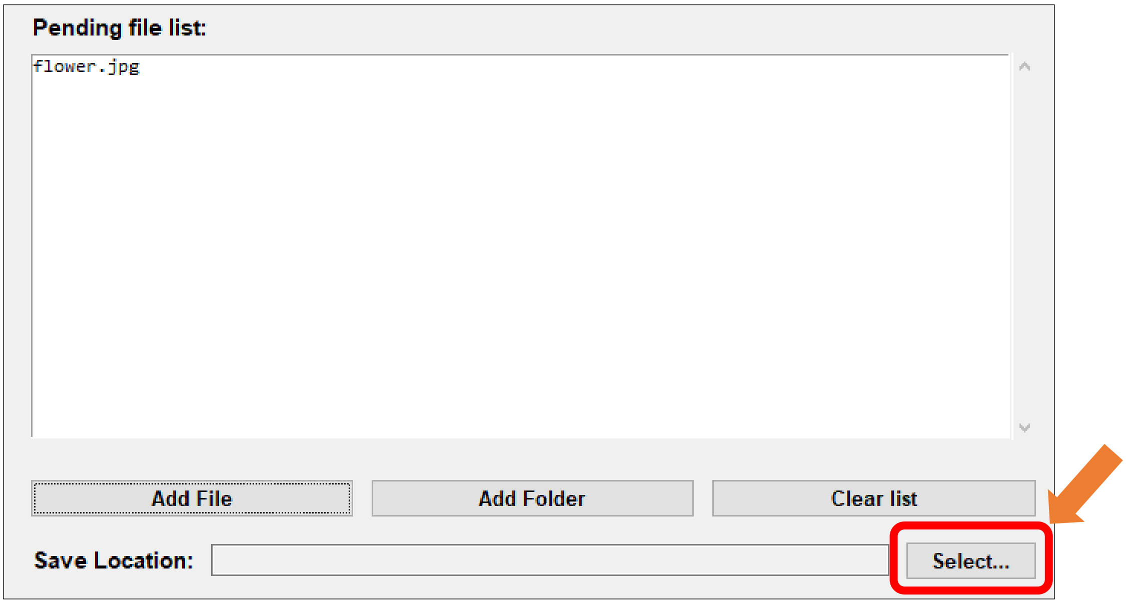
Select any image you like



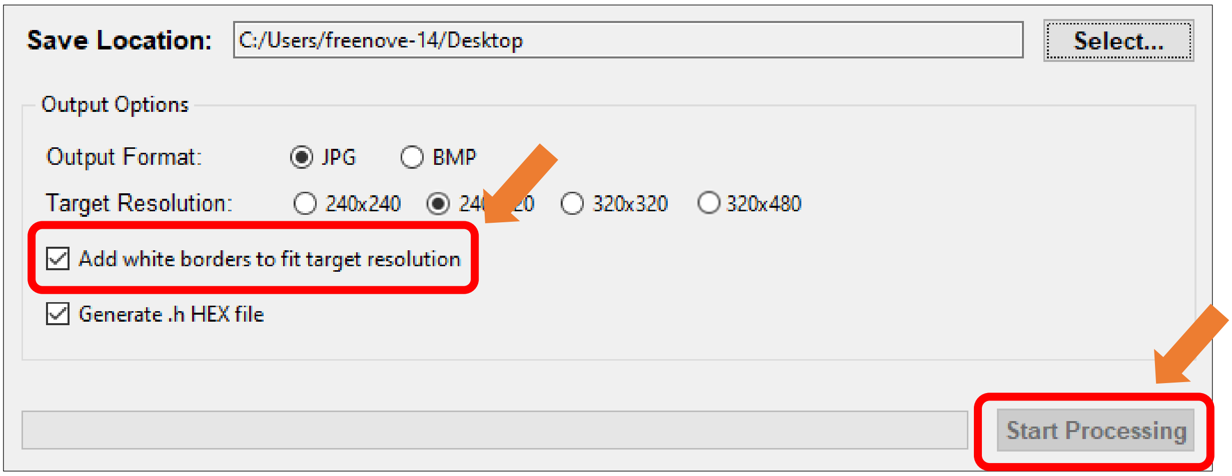
The image files from your folder will now appear in the **Pending File List**.



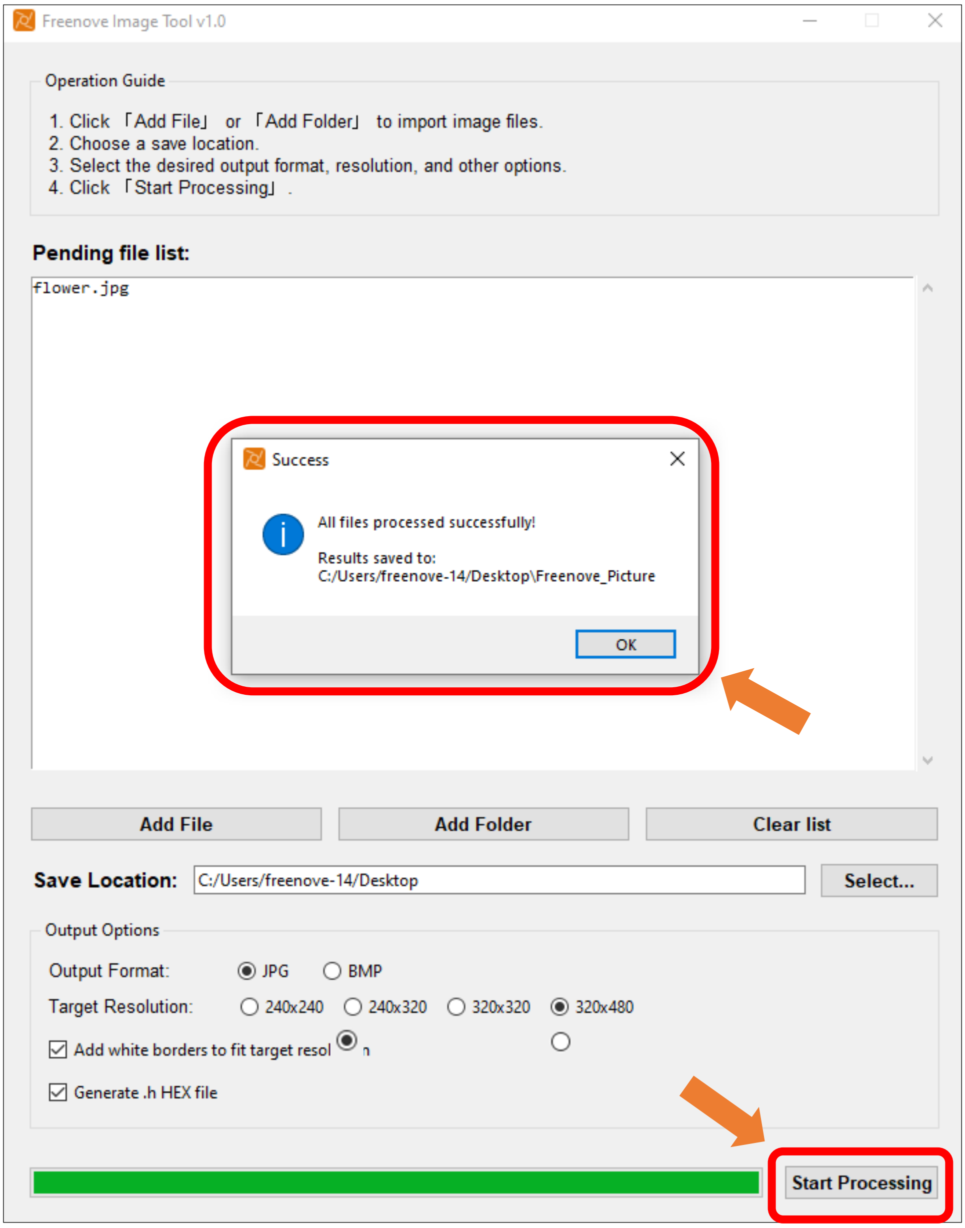
Click “Select…” to change the save location.



The resolution size is selected according to the screen resolution. Check the "**Add white borders to fit target resolution**" and "**Generate .h HEX file**" options.

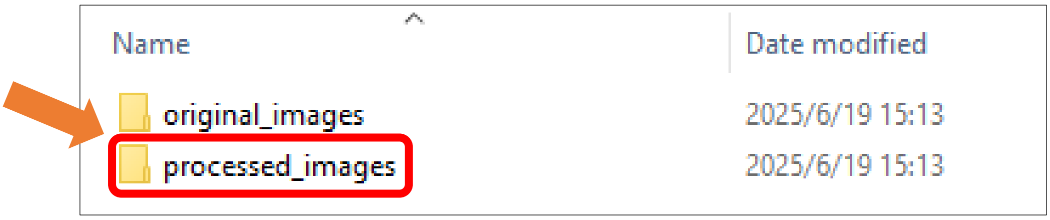


Click **Start Processing**. Wait for the progress bar to complete and the target folder will be generated.

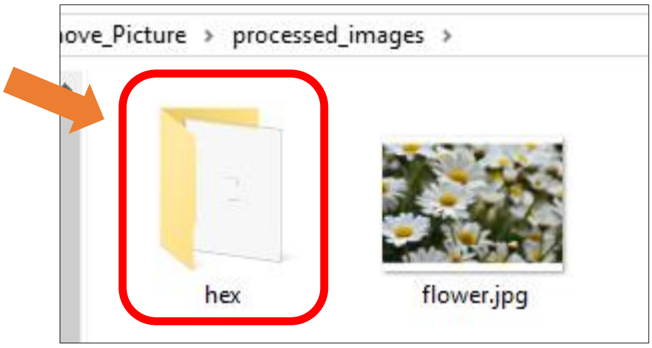


There will be three folders in the generated folder
original_images: backup of images before processing
processed_images: processed images
processed_images\hex: generated .h files corresponding to the images

Double click to open **processed_images**



Double click to open **hex**



Replace the entire content of panda.h with the copied data from the .h file, then upload the sketch again.

Sketch_10.2_Flash_Jpg_DMA.ino panda.h

```
/*
    If you need to customize the displayed image, first use the Freenove Image Tool
    to crop it to the appropriate width and height for your display.

    The tool will automatically adjust the image quality to ensure proper display.

    Paste the generated array into this file, ensuring the correct array format:

        const uint8_t name[] PROGMEM = {

    to start and and end with:

    };
*/

const unsigned char img_asset[] PROGMEM = {
    0xFF, 0xD8, 0xFF, 0xE0, 0x00, 0x10, 0x4A, 0x46, 0x49, 0x46, 0x00, 0x01,
    0x01, 0x00, 0x00, 0x01, 0x00, 0x01, 0x00, 0x00, 0xFF, 0xDB, 0x00, 0x43,
    0x00, 0x05, 0x03, 0x04, 0x04, 0x04, 0x03, 0x05, 0x04, 0x04, 0x04, 0x05,
    0x05, 0x05, 0x06, 0x07, 0x0C, 0x08, 0x07, 0x07, 0x07, 0x07, 0x0F, 0x0B,
    0x0B, 0x09, 0x0C, 0x11, 0x0F, 0x12, 0x12, 0x11, 0x0F, 0x11, 0x11, 0x13,
    0x16, 0x1C, 0x17, 0x13, 0x14, 0x1A, 0x15, 0x11, 0x11, 0x18, 0x21, 0x18,
    0x1A, 0x1D, 0x1D, 0x1F, 0x1F, 0x1F, 0x13, 0x17, 0x22, 0x24, 0x22, 0x1E,
    0x24, 0x1C, 0x1E, 0x1F, 0x1E, 0xFF, 0xDB, 0x00, 0x43, 0x01, 0x05, 0x05,
    0x05, 0x07, 0x06, 0x07, 0x0E, 0x08, 0x08, 0x0E, 0x1E, 0x14, 0x11, 0x14,
    0x1E, 0x1E, 0x1E, 0x1E, 0x1E, 0x1E, 0x1E, 0x1E, 0x1E, 0x1E, 0x1E, 0x1E,
    0x1E, 0x1E, 0x1E, 0x1E, 0x1E, 0x1E, 0x1E, 0x1E, 0x1E, 0x1E, 0x1E, 0x1E,
    0x1E, 0x1E, 0x1E, 0x1E, 0x1E, 0x1E, 0x1E, 0x1E, 0x1E, 0x1E, 0x1E, 0x1E,
    0x1E, 0x1E, 0x1E, 0x1E, 0x1E, 0x1E, 0x1E, 0x1E, 0x1E, 0x1E, 0x1E, 0x1E,
    0x1E, 0x1E, 0xFF, 0xC0, 0x00, 0x11, 0x08, 0x01, 0x40, 0x01, 0xE0, 0x03,
    0x01, 0x22, 0x00, 0x02, 0x11, 0x01, 0x03, 0x11, 0x01, 0xFF, 0xC4, 0x00,
    0x1C, 0x00, 0x00, 0x02, 0x02, 0x03, 0x01, 0x01, 0x00, 0x00, 0x00, 0x00,
```

The image will display on the screen.

Note:

To adjust the image display orientation, modify the following code in Sketch_10.2_Flash_Jpg_DMA.ino:

```
tft.setRotation(uint8_t rotation);
```

Value	Rotation Angle	Description
0	0°	Default orientation (no rotation)
1	90°	Clockwise 90-degree rotation
2	180°	Clockwise 180-degree rotation
3	270°	Clockwise 270-degree rotation

Chapter 11 TFT Touch

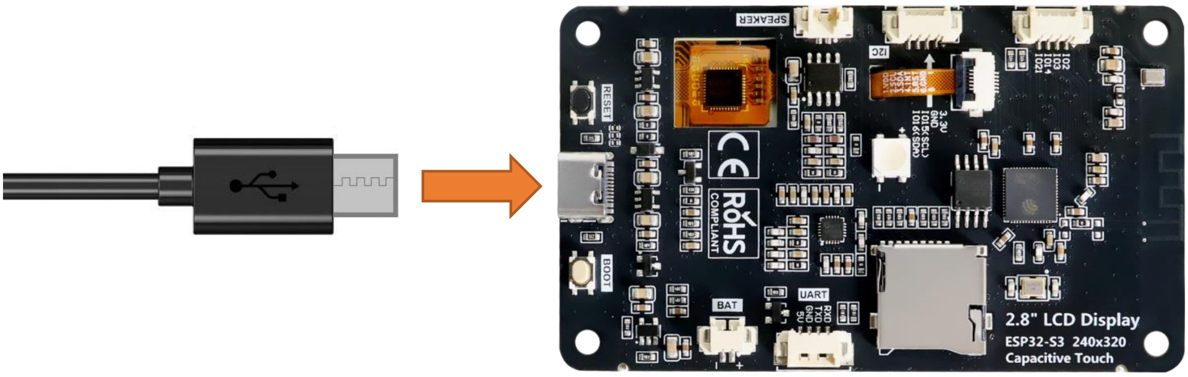
Project 11.1 TFT Touch

Component List

Freenove ESP32-S3 Display x 1	USB cable x1
	

Circuit

Connect Freenove ESP32 -S3 to the computer using the USB cable.

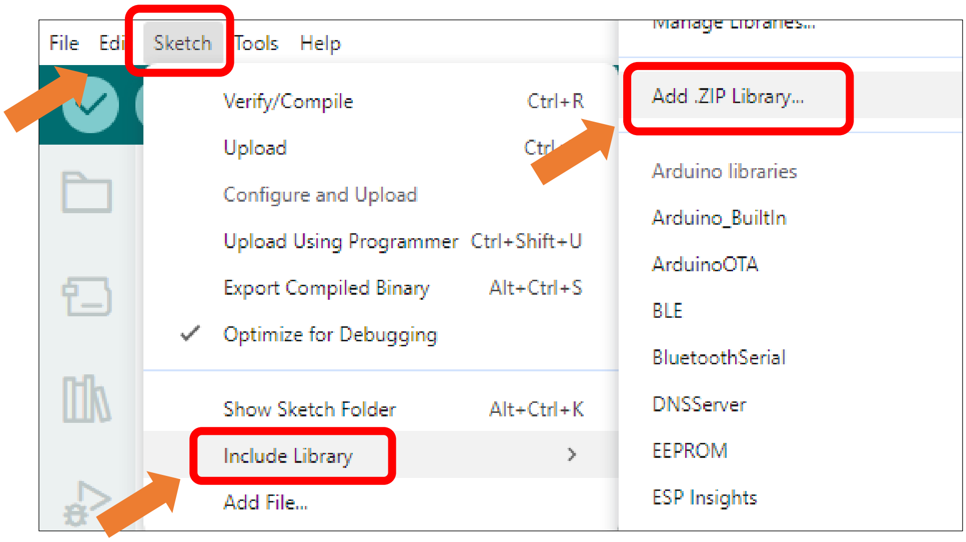


Sketch

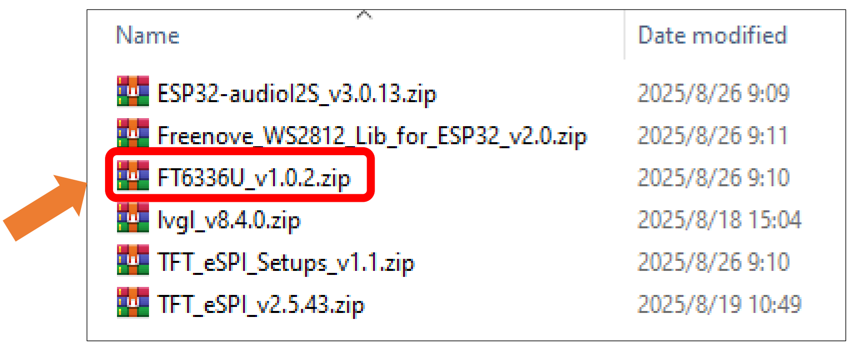
Open"**Sketch_11.1_Touch**" folder under **"Freenove_ESP32_S3_Display\Sketches"** and double-click "**Sketch_11.1_Touch.ino**".

Install the needed libraries.

Click **Sketch** -> **Include Library** -> **Add .ZIP Library...**



Select **ESP32-audioI2S_v3.0.13.zip**



Next, we download the code to Freenove_ESP32_S3_Display to test. Open"**Sketch_11_Touch**" folder under **"Freenove_ESP32_S3_Display\Sketcheses"** and double-click "**Sketch_11.1_Touch.ino**".

Sketch_11.1_Touch

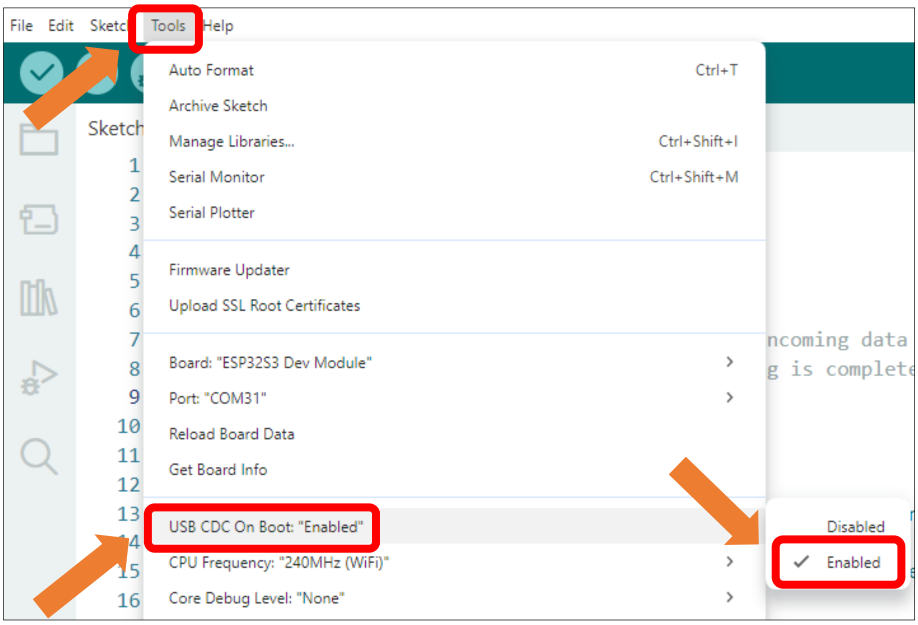
The following is the program code:

```
#include <Arduino.h>
#include <TFT_eSPI.h>

#ifdef FNK0104N_3P5_320x480_ST77922
  #include "ST77922.h"
  #include "ST77922_Touch.h"

  TFT_eSPI tft_qspi = TFT_eSPI();
  TFT_eSprite tft = TFT_eSprite(&tft_qspi);
  ST77922 tft_st77922 = ST77922();
```

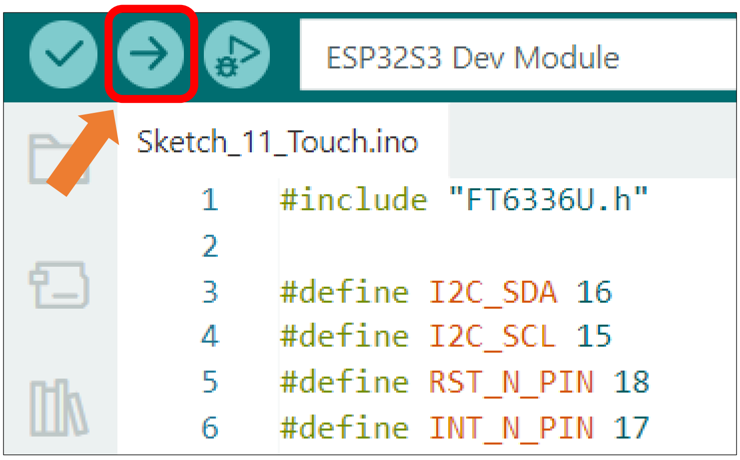
```
  ST77922_TOUCH touch_st77922;
#else
  #include "FT6336U.h"
  #define I2C_SDA 16
  #define I2C_SCL 15
  #define RST_N_PIN 18
  #define INT_N_PIN 17
  FT6336U ft6336u(I2C_SDA, I2C_SCL, RST_N_PIN, INT_N_PIN);
  FT6336U_TouchPointType tp;
#endif

void setup() {
    Serial.begin(115200);

    #ifdef FNK0104N_3P5_320x480_ST77922
      tft_st77922.Init();
      tft_st77922.Set_Rotation(1);
      tft.createSprite(tft_st77922.Get_Width(), tft_st77922.Get_Height());
      tft.setSwapBytes(true);

      tft.fillSprite(TFT_WHITE);  // Fill screen with solid red
      tft.setTextColor(TFT_BLACK);
      tft.setTextDatum(MC_DATUM);
      tft.drawString("ST77922 Touch", tft_st77922.Get_Width()/2, tft_st77922.Get_Height()/2,
4);
      tft_st77922.Fill_Colors(0, 0, tft_st77922.Get_Width(), tft_st77922.Get_Height(),
(uint16_t *)tft.getPointer());

      touch_st77922.init();
      touch_st77922.Set_Rotation(1);
    #else
      ft6336u.begin();
      Serial.print("FT6336U Firmware Version: ");
      Serial.println(ft6336u.read_firmware_id());
      Serial.print("FT6336U Device Mode: ");
      Serial.println(ft6336u.read_device_mode());
      Serial.println("ST77922 Touch Initialized.");
    #endif
}

void loop() {
    #ifdef FNK0104N_3P5_320x480_ST77922
      if (touch_st77922.Get_Touch()) {
          uint16_t x = touch_st77922.touch.x[0];
```

```
            uint16_t y = touch_st77922.touch.y[0];

            char tempString[128];
            sprintf(tempString, "ST77922 Touch: X: %4d | Y: %4d\r\n", x, y);

            Serial.print(tempString);
        }
        delay(50);
    #else
        tp = ft6336u.scan();
        char tempString[128];
        sprintf(tempString, "FT6336U TD Count %d / TD1 (%d, %4d, %4d) / TD2 (%d, %4d, %4d)\r\n",
tp.touch_count, tp.tp[0].status, tp.tp[0].x, tp.tp[0].y, tp.tp[1].status, tp.tp[1].x,
tp.tp[1].y);
        Serial.print(tempString);
        delay(100);
    #endif
}
```

Enable the "USB CDC On Boot" feature.



Click “**Upload**” to upload the code to Freenove ESP32 Display, set the baud rate to 115200.



When touching the screen, the serial monitor will print coordinates in real-time, as shown in the image below:

Please Note: Only the 3.5-inch Freenove ESP32-S3 Display supports 5-point multi-touch; other models support only single-point touch.

```
ST77922 Touch: [P1:  310,  114] [P2:  361,  160] [P3:  240,  214] [P4:  208,  110] [P5:  121,  189]
ST77922 Touch: [P1:  310,  114] [P2:  363,  165] [P3:  249,  203] [P4:  208,  110] [P5:  124,  191]
ST77922 Touch: [P1:  282,   84] [P2:  363,  165] [P3:  263,  186] [P4:  185,  111] [P5:  124,  191]
ST77922 Touch: [P1:  281,   83] [P2:  363,  165] [P3:  266,  187] [P4:  183,  111] [P5:  123,  197]
ST77922 Touch: [P1:  279,   80] [P2:  346,  124] [P3:  294,  198] [P4:  180,  111] [P5:  121,  202]
ST77922 Touch: [P1:  272,   56] [P2:  325,   81] [P3:  294,  198] [P4:  179,  111] [P5:  120,  203]
ST77922 Touch: [P1:  272,   56] [P2:  325,   79] [P3:  296,  200] [P4:  179,  111] [P5:  120,  203]
ST77922 Touch: [P1:  272,   56] [P2:  325,   79] [P3:  299,  206] [P4:  179,  111] [P5:  120,  203]
ST77922 Touch: [P1:  271,   54] [P2:  326,   80] [P3:  297,  224] [P4:  179,  111] [P5:  120,  203]
ST77922 Touch: [P1:  272,   54] [P2:  352,  111] [P3:  296,  226] [P4:  179,  111] [P5:  120,  203]
ST77922 Touch: [P1:  303,   80] [P2:  366,  140] [P3:  279,  233] [P4:  180,  107] [P5:  120,  203]
```

Chapter 12 TFT Touch Drawing

After learning this chapter, you will be able to draw freely on the screen.

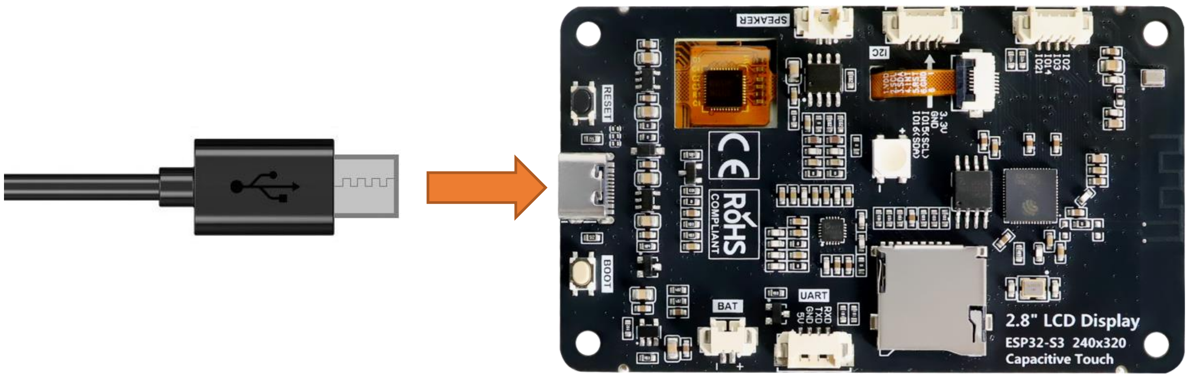
Project 12.1 TFT Touch Drawing

Component List

Freenove ESP32-S3 Display x 1	USB cable x1
	

Circuit

Connect Freenove ESP32 -S3 to the computer using the USB cable.



Sketch

Open "**Sketch_12.1_TFT_Touch_Draw**" folder under **"Freenove_ESP32_S3_Display\Sketches"** and double-click "**Sketch_12.1_TFT_Touch_Draw.ino**".

Sketch_12.1_TFT_Touch_Draw

The following is the program code:

```
/*
* @ File:   Sketch_12.1_TFT_Touch_Draw.ino
* @ Author: [Eason Shen]
```

```
* @ Date:   [2026-05-15]
*/

#include <Arduino.h>
#include <TFT_eSPI.h>           // TFT display library

#ifdef FNK0104N_3P5_320x480_ST77922
  #include "ST77922.h"
  #include "ST77922_Touch.h"

  TFT_eSPI tft_qspi = TFT_eSPI(); // Invoke custom library
  TFT_eSprite tft = TFT_eSprite(&tft_qspi);
  ST77922 tft_st77922 = ST77922();
  ST77922_TOUCH touch_st77922;

  void flushScreen() {
      tft_st77922.Fill_Colors(0, 0, tft_st77922.Get_Width(), tft_st77922.Get_Height(),
(uint16_t *)tft.getPointer());
  }
#else
  #include "FT6336U.h"

  // Invoke custom TFT driver library
  TFT_eSPI tft = TFT_eSPI();       // Invoke custom library

  #define I2C_SDA 16
  #define I2C_SCL 15
  #define RST_N_PIN 18
  #define INT_N_PIN 17

  /* Create an instance of the touch screen library */
  FT6336U ft6336u(I2C_SDA, I2C_SCL, RST_N_PIN, INT_N_PIN);
  FT6336U_TouchPointType tp;

#endif

#ifdef FNK0104N_3P5_320x480_ST77922
  #define SCREEN_WIDTH 320
  #define SCREEN_HEIGHT 480

  const uint8_t ColorPalette = 30;     // Height of the color palette area
  const uint8_t LineSize = 5;          // Line thickness (radius)
#elif defined FNK0104AB_2P8_240x320_ILI9341
  #define SCREEN_WIDTH 240
```

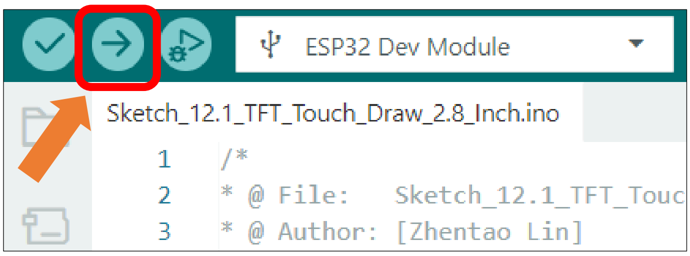
```
  #define SCREEN_HEIGHT 320

  const uint8_t ColorPalette = 21;      // Height of the color palette area
  const uint8_t BrushSize = 2;          // Brush thickness (radius)
  const uint8_t LineSize = 5;           // Line thickness (radius)
#elif defined FNK0104S_4P0_320x480_ST7796
  #define SCREEN_WIDTH 320
  #define SCREEN_HEIGHT 480

  const uint8_t ColorPalette = 30;      // Height of the color palette area
  const uint8_t BrushSize = 2;          // Brush thickness (radius)
  const uint8_t LineSize = 5;           // Line thickness (radius)
#endif

const uint8_t color_num = 12;
int current_color = TFT_WHITE;

unsigned int colors[color_num] = {
  TFT_RED, TFT_PINK, TFT_GREEN, TFT_BLUE, TFT_OLIVE,
  TFT_CYAN, TFT_YELLOW, TFT_WHITE, TFT_MAGENTA,
  TFT_ORANGE, TFT_BLACK, TFT_SILVER
};

// Define coordinate variables
uint16_t x, y;
uint16_t last_x = 0, last_y = 0;

void drawUI() {
    for (int i = 0; i < color_num; i++) {
        tft.fillRect(i * ColorPalette, 0, ColorPalette, ColorPalette, colors[i]);
    }
    #ifdef FNK0104N_3P5_320x480_ST77922
      tft.setTextColor(TFT_WHITE, TFT_BLACK);
      tft.drawString("CLEAR", (color_num * ColorPalette) + 10, 8, 2);
      tft.drawRect(tft.width() - 40, 5, 20, 20, TFT_WHITE);
      tft.fillRect(tft.width() - 39, 6, 18, 18, current_color);
      tft.drawFastHLine(0, ColorPalette, tft.width(), TFT_DARKGREY);
      flushScreen();
    #elif defined FNK0104AB_2P8_240x320_ILI9341
      // Draw clear screen button
      tft.setCursor(260 , 3, 2);
      tft.setTextColor(TFT_WHITE);
      tft.print("Clear");

```

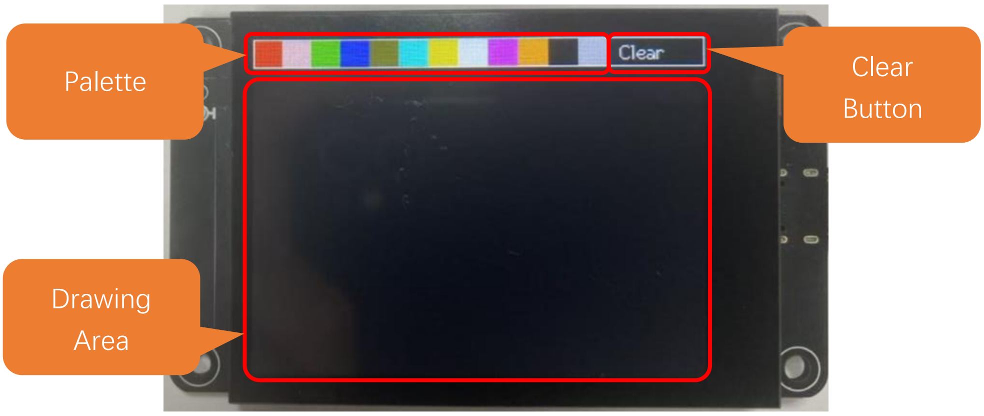
```
      // Display current color
      tft.fillRect(300, 5, 12, 12, current_color);

      // Add palette border/outline
      tft.drawRect(0, 0, SCREEN_HEIGHT, ColorPalette, TFT_WHITE);
    #elif defined FNK0104S_4P0_320x480_ST7796
      // Draw clear screen button
      tft.setCursor((color_num * ColorPalette) + 10 , 8, 2);
      tft.setTextColor(TFT_WHITE);
      tft.print("Clear");

      // Display current color
      tft.fillRect(tft.width() - 39, 6, 18, 18, current_color);
      tft.drawFastHLine(0, ColorPalette, tft.width(), TFT_DARKGREY);

      // Add palette border/outline
      tft.drawRect(tft.width() - 40, 5, 20, 20, TFT_WHITE);
    #endif
}

void setup() {
    Serial.begin(115200);
    #ifdef FNK0104N_3P5_320x480_ST77922
      tft_st77922.Init();
      // Set the TFT and touch screen to landscape orientation
      tft_st77922.Set_Rotation(1);

      tft.createSprite(tft_st77922.Get_Width(), tft_st77922.Get_Height());
      tft.setSwapBytes(true);
      tft.fillSprite(TFT_BLACK);

      touch_st77922.init();
      touch_st77922.Set_Rotation(1);
    #else
      ft6336u.begin();
      Serial.print("FT6336U Firmware Version: ");
      Serial.println(ft6336u.read_firmware_id());
      Serial.print("FT6336U Device Mode: ");
      Serial.println(ft6336u.read_device_mode());
      tft.init();

      // Set the TFT and touch screen to landscape orientation
      tft.setRotation(1);

```

```
      tft.fillScreen(TFT_BLACK);
    #endif
    drawUI();
}

void loop() {
    // Check if the touch screen is currently pressed
    // Raw and coordinate values are stored within library at this instant
    // for later retrieval by GetRaw and GetCoord functions.
    #ifdef FNK0104N_3P5_320x480_ST77922
      if (touch_st77922.Get_Touch())
      {
        // Read the current X and Y axis as co-ordinates at the last touch time
        x = touch_st77922.touch.x[0];
        y = touch_st77922.touch.y[0];
    #else
      tp = ft6336u.scan();
      if (tp.touch_count!=0) // Note this function updates coordinates stored within library variables
      {
        // Read the current X and Y axis as co-ordinates at the last touch time
        // The values were captured when Pressed() was called!
        x = tp.tp[0].y;
        #ifdef FNK0104S_4P0_320x480_ST7796
          y = 320-tp.tp[0].x;
        #elif defined FNK0104AB_2P8_240x320_ILI9341
          y = 240-tp.tp[0].x;
        #endif
        Serial.print(x); Serial.print(","); Serial.println(y);
    #endif
        /* Color palette area logic */
        if (y < ColorPalette + 3) {
            // Enter color selection area, stroke interrupted
            last_x = 0;
            last_y = 0;
            /* Determine stylus click area */
            // Clicked color palette
            if (x >= color_num * ColorPalette) {

              tft.fillRect(0, ColorPalette + 1, tft.width(), tft.height() - ColorPalette - 1, current_color);
              #ifdef FNK0104N_3P5_320x480_ST77922
                flushScreen();
              #endif
```

```
            }
            // Clicked Clear button
            else {
              current_color = colors[x / ColorPalette];

              tft.fillRect(tft.width() - 39, 6, 18, 18, current_color);
              #ifdef FNK0104N_3P5_320x480_ST77922
                flushScreen();
              #endif
            }
            return;
        }

        if (last_x == 0 && last_y == 0) {
            // Record current position
            last_x = x;
            last_y = y;
        }

        tft.drawWideLine(last_x, last_y, x, y, LineSize, current_color, current_color);
        #ifdef FNK0104N_3P5_320x480_ST77922
          flushScreen();
        #endif

        // Update coordinates
        last_x = x;
        last_y = y;
    }
    else {
        // Reset coordinates when stylus leaves the screen
        last_x = 0;
        last_y = 0;
    }
}
```

Click “**Upload**” to upload the code to Freenove ESP32 Display. Set the baud rate to115200





Tips: Aliasing & Anti-aliasing

In computer graphics, **aliasing** refers to the **jagged or stair-step appearance** of lines and curves in digital images, particularly noticeable on diagonal lines and edges. This occurs due to the discrete nature of pixel grids - screens compose images from tiny square pixels that cannot perfectly represent continuous geometry. **Anti-aliasing** mitigates these artifacts through technical means to **smooth edges**, achieving more natural-looking graphics. The core technique blends transitional colors at boundary pixels, simulating human visual perception of soft edges.

Chapter 13 LVGL

Project 13.1 LVGL

Component List

Freenove ESP32-S3 Display x 1	USB cable x1
	

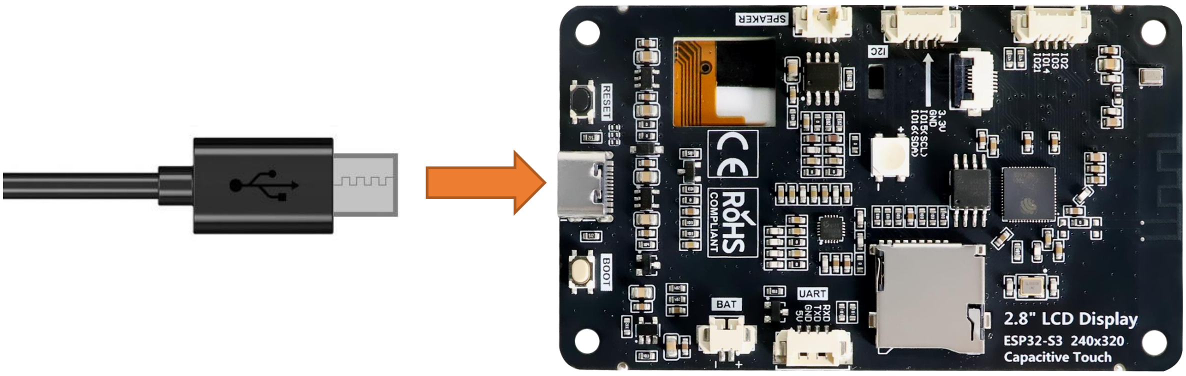
Component Knowledge

LVGL is a widely-used embedded GUI library that is implemented in pure C, making it highly portable and performant. It offers rich features and content, supporting both display and input devices such as touchscreens and keyboards.

	Features supported by LVGL
1	Powerful building blocks such as buttons, charts, lists, sliders, images, and more.
2	Advanced graphics with animation, anti-aliasing, opacity, and smooth scrolling.
3	Various input devices, such as touchpads, mice, keyboards, encoders, and more.
4	Multiple languages with UTF-8 encoding.
5	Multiple display types, including TFT and monochrome displays.
6	Fully customizable graphical elements.
7	LVGL can be used independently of any microcontroller or display hardware.
8	Highly extensible and can be configured to use very little memory (e.g. 64 kB of flash and 16 kB of RAM)
9	It can be used with or without an operating system, and supports external memory and GPUs as optional features.
10	Single-frame buffer operation, even with advanced graphics effects.
11	Written in C language to achieve maximum compatibility (compatible with C++ as well).
12	LVGL has a simulator that allows for embedded GUI design on a PC without any embedded hardware.
13	Resources to help developers quickly get started with the library, including tutorials, examples, and themes.
14	A wide range of resources.

Circuit

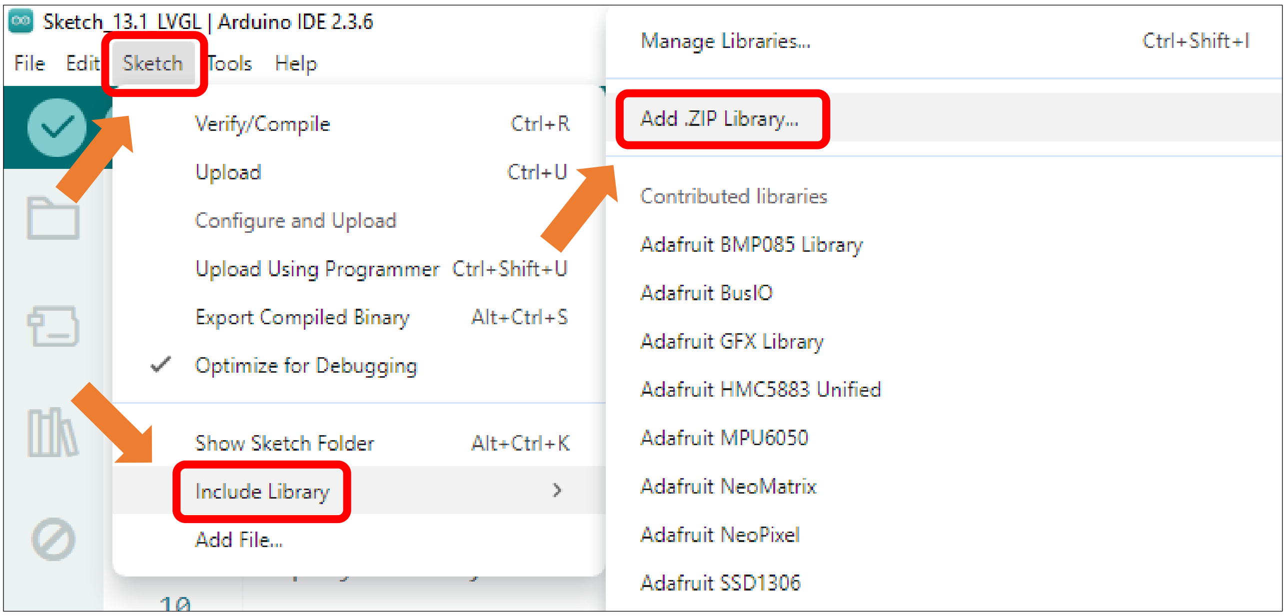
Connect Freenove ESP32 -S3 to the computer using the USB cable.



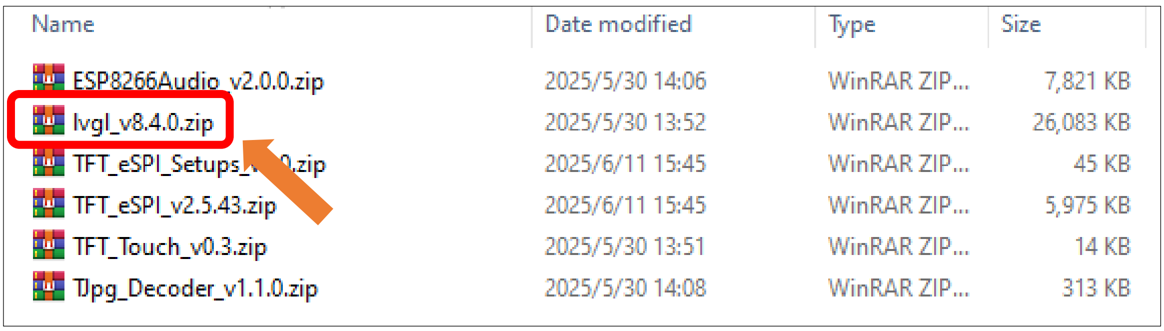
Sketch

Install Libraries

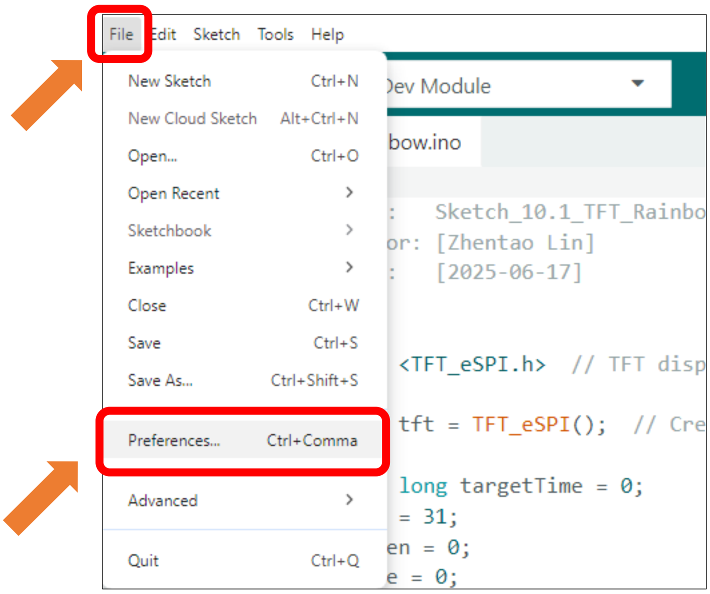
Click **Sketch** -> **Include Library** -> **Add .ZIP Library...**



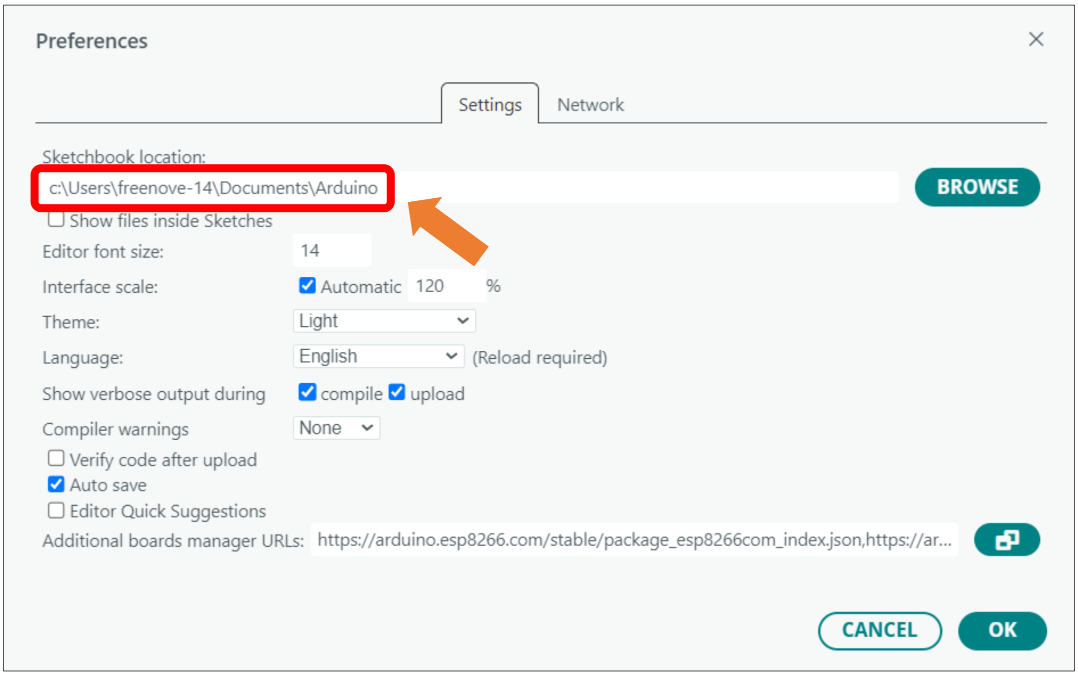
Install **lvgl_v8.4.0.zip**



Click **File** -> **Preferences...**

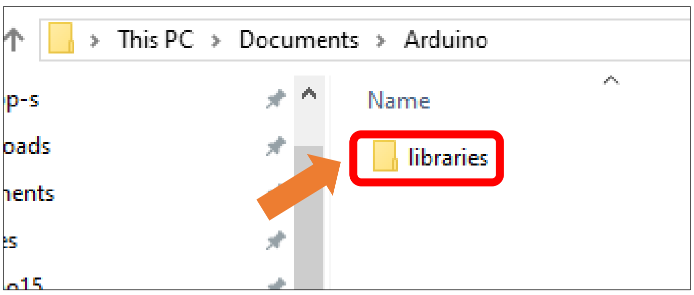


Linux and Mac users can use the **cd** command on Terminal to enter the sketchbook location, and Windows users can copy and paste it in the file explorer.

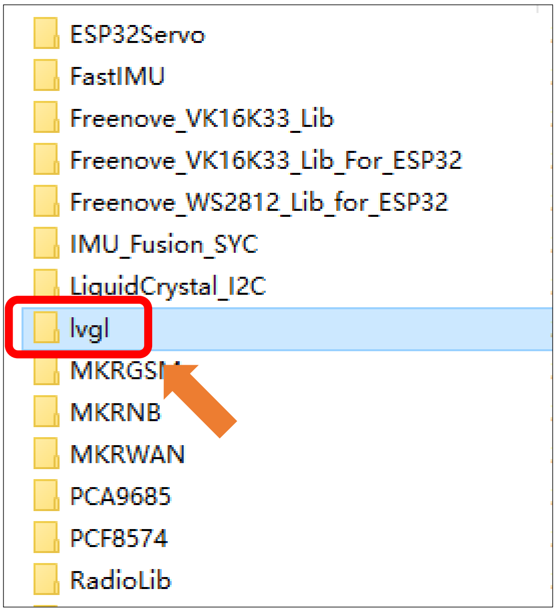


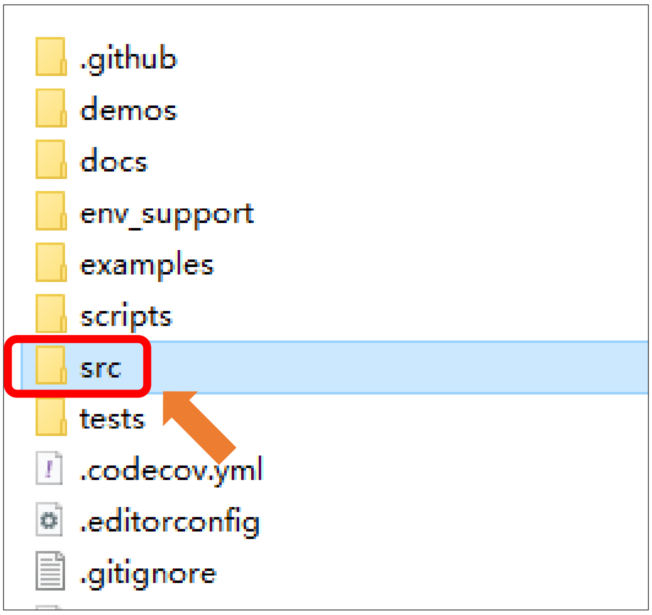
If your device is the FNK0104N (3.5-inch), please continue with the subsequent operations; otherwise, you can skip these steps.

Double click **libraries.**



Double click **lvgl->src.**





Open **lv_conf.h**，Define **LV_COLOR_16_SWAP** as 1.

```
/* clang-format off */
#if 1 /*Set it to "1" to enable content*/

#ifndef LV_CONF_H
#define LV_CONF_H

#include <stdint.h>
/*====================
   COLOR SETTINGS
 *====================*/

/*Color depth: 1 (1 byte per pixel), 8 (RGB332), 16 (RGB565), 32 (ARGB8888)*/
#define LV_COLOR_DEPTH 16

/*Swap the 2 bytes of RGB565 color. Useful if the display has an 8-bit interface (e.g. S
#define LV_COLOR_16_SWAP 1

/*Enable features to draw on transparent background.
 *It's required if opa, and transform_* style properties are used.
 *Can be also used if the UI is above another layer, e.g. an OSD menu or video player.*/
#define LV_COLOR_SCREEN_TRANSP 0

/* Adjust color mix functions rounding. GPUs might calculate color mix (blending) differ
 * 0: round down, 64: round up from x.75, 128: round up from half, 192: round up from x.
#define LV_COLOR_MIX_ROUND_OFS 0

/*Images pixels with this color will not be drawn if they are chroma keyed)*/
#define LV_COLOR_CHROMA_KEY lv_color_hex(0x00ff00)         /*pure green*/
```

Open "**Sketch_13.1_LVGL**" folder under **"Freenove_ESP32_S3_Display\Sketches"** and double-click "**Sketch_13.1_LVGL.ino**".

Sketch_13.1_LVGL

The following is the program code:

```
#include "display.h"

Display screen;

void setup()
{
    /* prepare for possible serial debug */
    Serial.begin( 115200 );

    /*** Init screen ***/
    screen.init();

    String LVGL_Arduino = "Hello Arduino! ";
    LVGL_Arduino += String('V') + lv_version_major() + "." + lv_version_minor() + "." +
lv_version_patch();

    Serial.println( LVGL_Arduino );
    Serial.println( "I am LVGL_Arduino" );

    /* Create simple label */
    lv_obj_t *label = lv_label_create( lv_scr_act() );
    lv_label_set_text( label, LVGL_Arduino.c_str() );
    lv_obj_align( label, LV_ALIGN_CENTER, 0, 0 );

    Serial.println( "Setup done" );
}

void loop()
{
    screen.routine(); /* let the GUI do its work */
    delay( 5 );
}
```

Code Explanation

Include the header file.

```
#include "display.h"
```

Set the baud rate to 115200

```
Serial.begin( 115200 );
```

Initialize the screen.

```
screen.init();
```

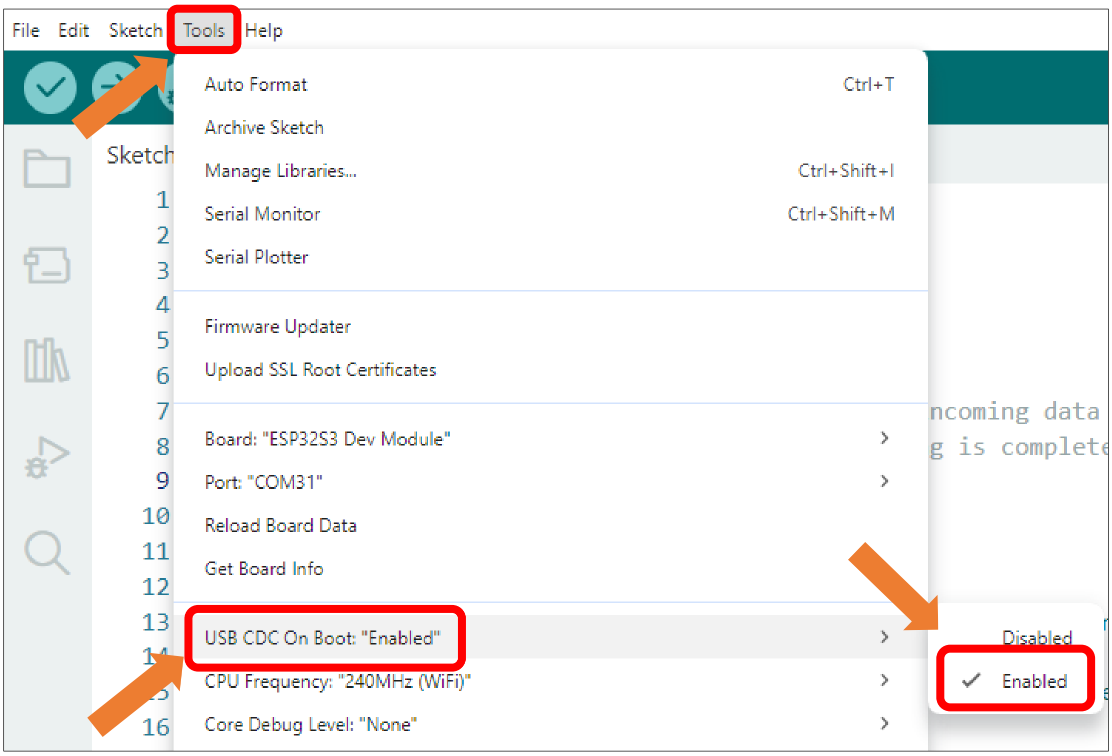
Configure the screen interface.

```
String LVGL_Arduino = "Hello Arduino! ";
LVGL_Arduino += String('V') + lv_version_major() + "." + lv_version_minor() + "." +
lv_version_patch();

Serial.println( LVGL_Arduino );
Serial.println( "I am LVGL_Arduino" );

/* Create simple label */
lv_obj_t *label = lv_label_create( lv_scr_act() );
lv_label_set_text( label, LVGL_Arduino.c_str() );
lv_obj_align( label, LV_ALIGN_CENTER, 0, 0 );
```

Let the LVGL handle the tasks.

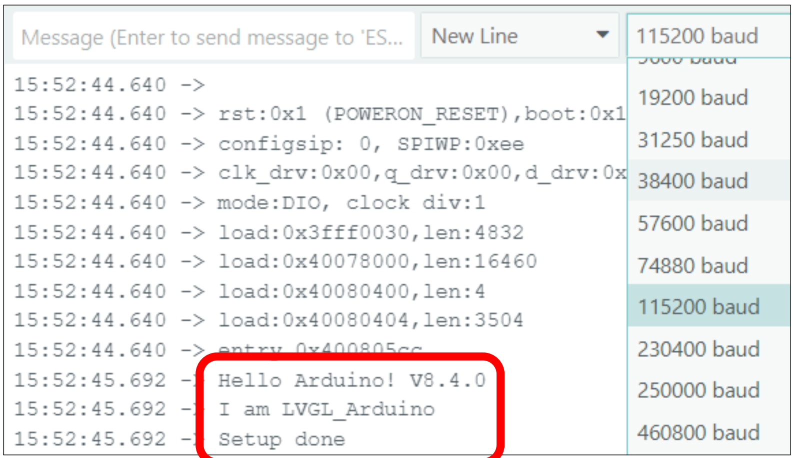
```
screen.routine(); /* let the GUI do its work */
```

Enable the "USB CDC On Boot" feature.

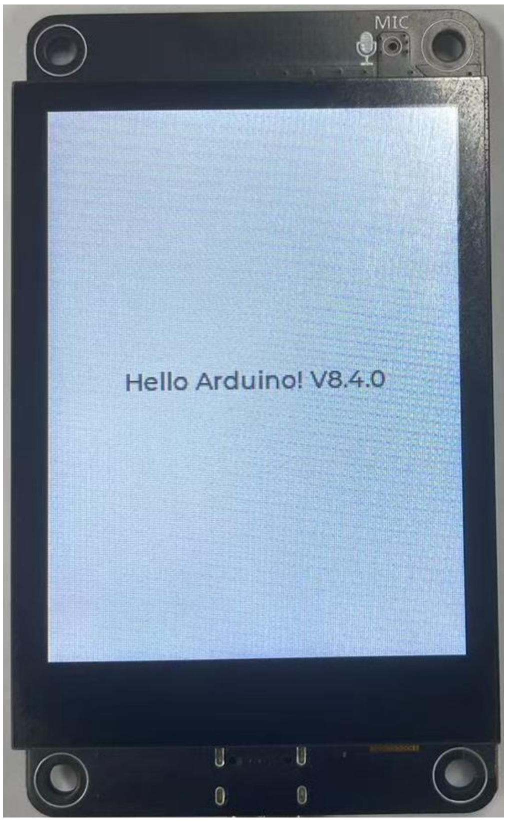


Click "**Upload**" to upload the code to Freenove ESP32 Display. Set the baud rate to 115200.

Data will be printed on the serial monitor.



The text "**Hello Arduino! V8.4.0**" will be displayed on the screen.



Chapter 14 LVGL Picture

Project 14.1 LVGL Picture

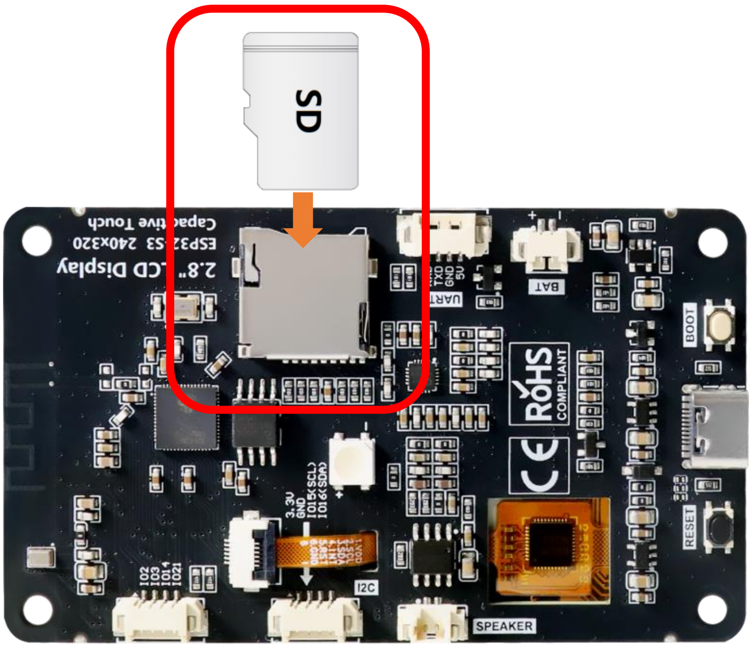
Component List

Freenove ESP32-S3 Display x 1	USB cable x1
	

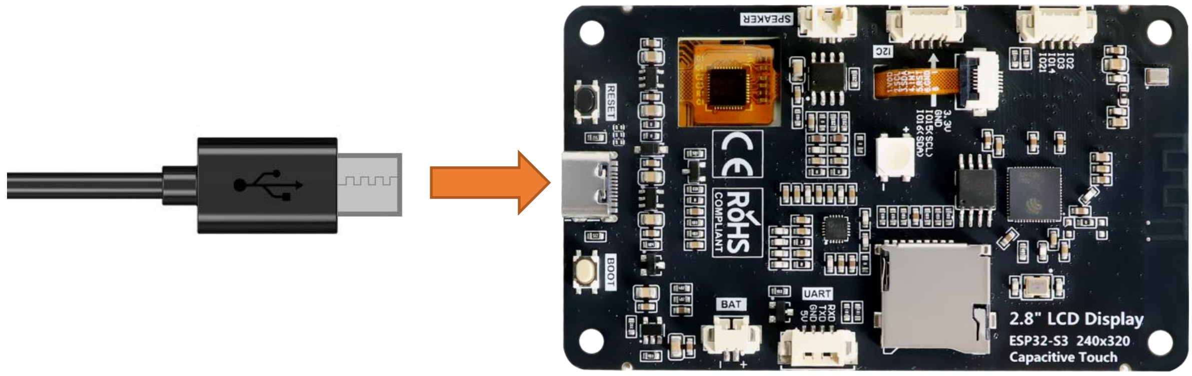
Circuit

Before connecting the USB cable, insert the SD card into the SD card slot on the back of the ESP32-S3.

Please note that this kit does not include SD card and card reader; please buy them yourself.



Connect Freenove ESP32-S3 to the computer using the USB cable.



If you have any concerns, please feel free to contact us via support@freenove.com

Sketch

Open "**Sketch_14.1_Lvgl_Picture**" folder under **"Freenove_ESP32_S3_Display\Sketches"** and double-click "**Sketch_14.1_Lvgl_Picture.ino**".

Sketch_14.1_Lvgl_Picture

The following is the program code:

```
#include "display.h"
#include "picture_ui.h"
#include "driver_sdmmc.h"

#ifdef FNK0104N_3P5_320x480_ST77922
  #define SD_MMC_CMD 4  // Please do not modify it.
  #define SD_MMC_CLK 5  // Please do not modify it.
  #define SD_MMC_D0  6  // Please do not modify it.
  #define SD_MMC_D1  7  // Please do not modify it.
  #define SD_MMC_D2  2  // Please do not modify it.
  #define SD_MMC_D3  3  // Please do not modify it.
#else
  #define SD_MMC_CMD 40  // Please do not modify it.
  #define SD_MMC_CLK 38  // Please do not modify it.
  #define SD_MMC_D0  39  // Please do not modify it.
  #define SD_MMC_D1  41  // Please do not modify it.
  #define SD_MMC_D2  48  // Please do not modify it.
  #define SD_MMC_D3  47  // Please do not modify it.
#endif

Display screen;  // Create an instance of the Display class

void setup() {
  /* Prepare for possible serial debug */
  Serial.begin(115200);  // Initialize serial communication at 115200 baud rate
```

```

  /* Initialize SD card */
  sdmmc_init(SD_MMC_CLK, SD_MMC_CMD, SD_MMC_D0, SD_MMC_D1, SD_MMC_D2, SD_MMC_D3);
  create_dir(PICTURE_FOLDER);

  /*** Initialize the screen ***/
  screen.init();  // Initialize the display

  // Create a string to display LVGL version information
  String LVGL_Arduino = "Hello Arduino! ";
  LVGL_Arduino += String('V') + lv_version_major() + "." + lv_version_minor() + "." +
lv_version_patch();

  // Print LVGL version information to the serial monitor
  Serial.println(LVGL_Arduino);
  Serial.println("I am LVGL_Arduino");

  setup_scr_picture(&guider_picture_ui);
  lv_scr_load(guider_picture_ui.picture);

  // Print setup completion message to the serial monitor
  Serial.println("Setup done");
}

void loop() {
  screen.routine(); /* Let the GUI do its work */  // Handle routine display tasks
  delay(5);         // Add a small delay to prevent the loop from running too fast
}
```

Code Explanation

Include the header files.

```
#include "display.h"
#include "picture_ui.h"
#include "driver_sdmmc.h"
```

Define the pins.

```
#ifdef FNK0104N_3P5_320x480_ST77922
  #define SD_MMC_CMD 4  // Please do not modify it.
  #define SD_MMC_CLK 5  // Please do not modify it.
  #define SD_MMC_D0  6  // Please do not modify it.
  #define SD_MMC_D1  7  // Please do not modify it.
  #define SD_MMC_D2  2  // Please do not modify it.
  #define SD_MMC_D3  3  // Please do not modify it.
#else
  #define SD_MMC_CMD 40  // Please do not modify it.
  #define SD_MMC_CLK 38  // Please do not modify it.
```

```
  #define SD_MMC_D0  39  // Please do not modify it.
  #define SD_MMC_D1  41  // Please do not modify it.
  #define SD_MMC_D2  48  // Please do not modify it.
  #define SD_MMC_D3  47  // Please do not modify it.
#endif
```

Set the baud rate to 115200

```
Serial.begin(115200);  // Initialize serial communication at 115200 baud rate
```

Initialize configuration.

```
sdmmc_init(SD_MMC_CLK, SD_MMC_CMD, SD_MMC_D0, SD_MMC_D1, SD_MMC_D2, SD_MMC_D3);
create_dir(PICTURE_FOLDER);
```

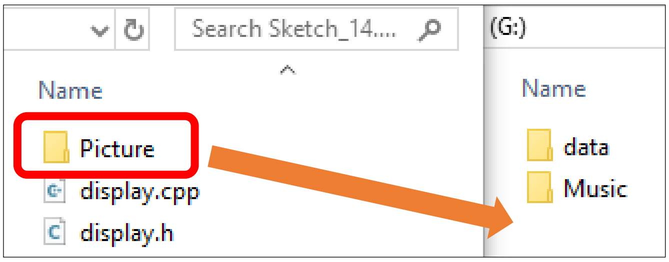
Create and load the interface.

```
setup_scr_picture(&guider_picture_ui);
lv_scr_load(guider_picture_ui.picture);
```

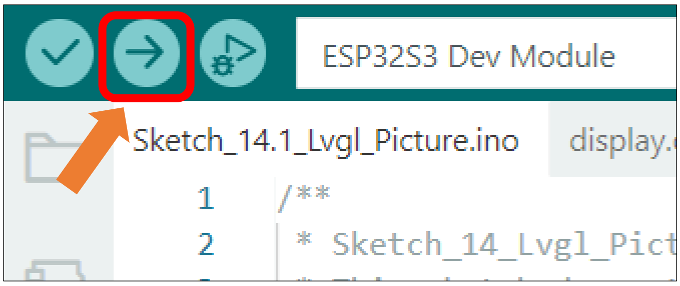
LVGL Task Processor

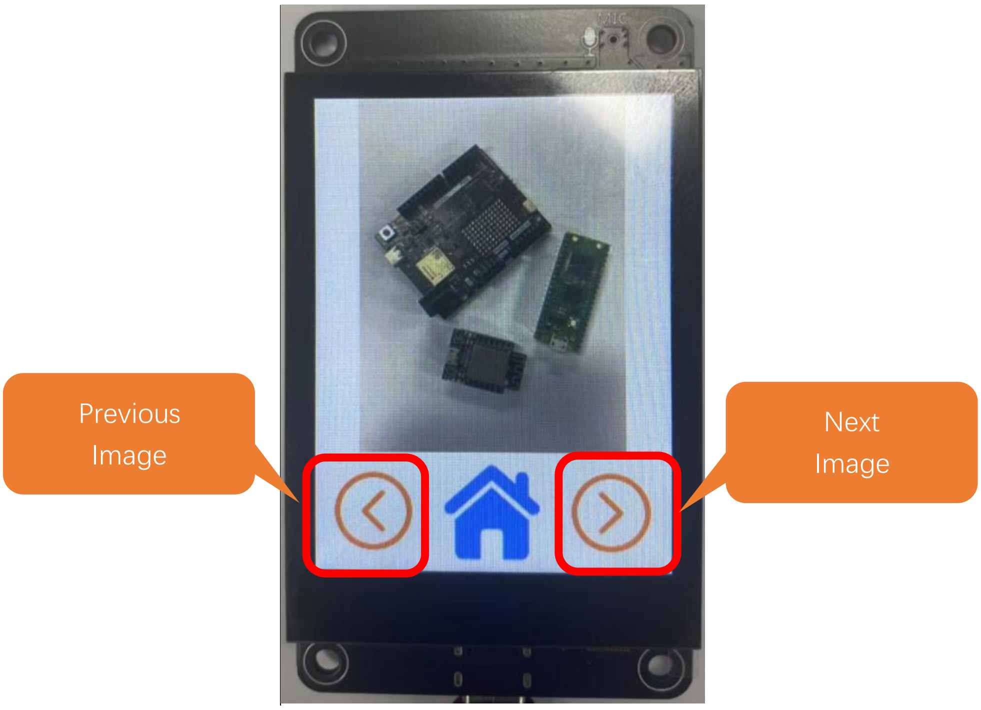
```
screen.routine(); /* Let the GUI do its work */  // Handle routine display tasks
```

Copy the Picture folder under the **Freenove_ESP32_S3_Display\Sketches\Sketch_14.1_Lvgl_Picture** directory to the SD card root directory.



Click “**Upload**” to upload the code to Freenove ESP32 Display. Set the baud rate to 115200

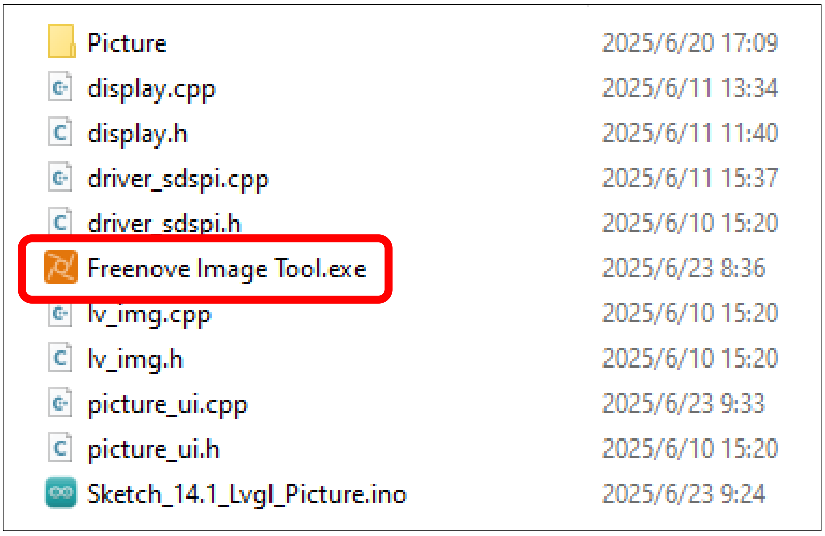




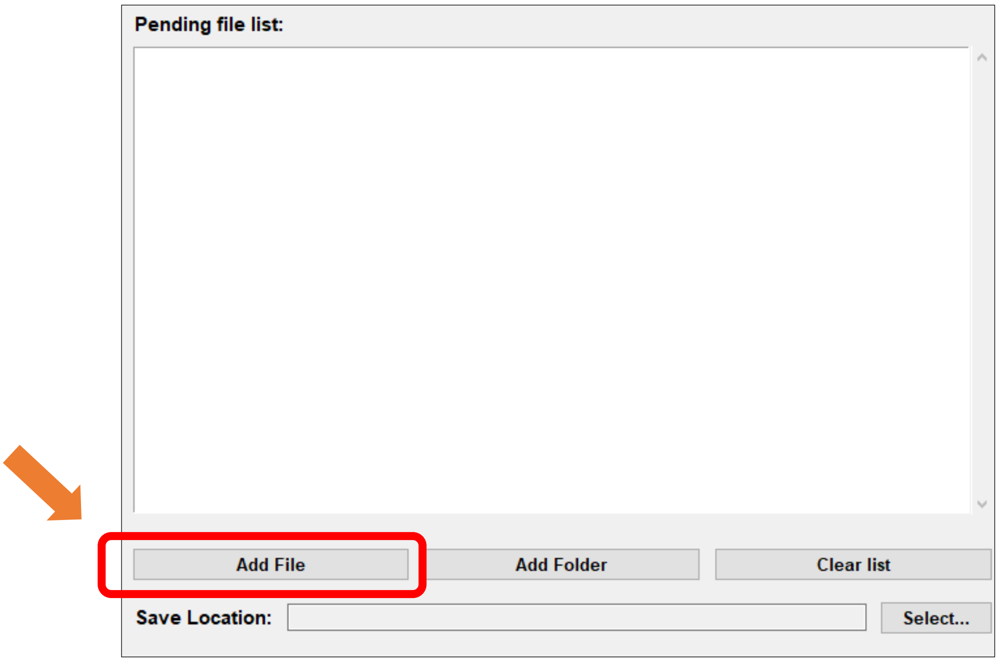
Custom image display

You can customize the image displayed on the display according to your personal preferences.

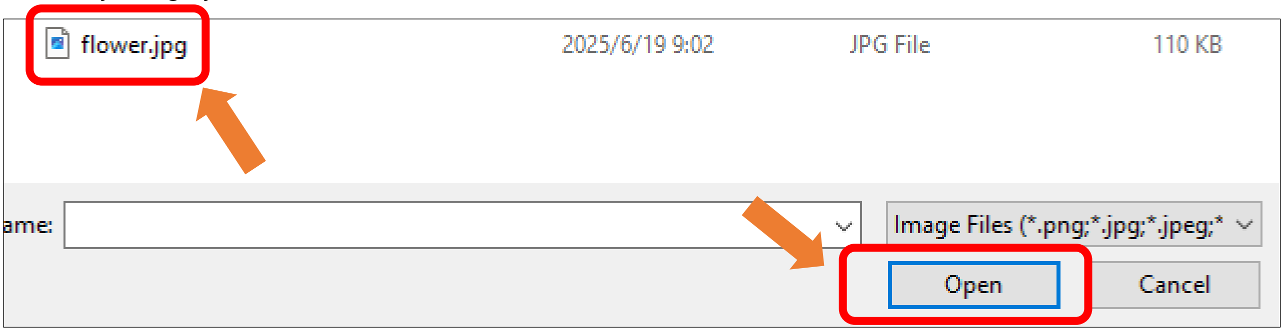
First, open **Freenove_ESP32_S3_Display\Sketches\Sketch_14.1_Lvgl_Picture\Freenove Image Tool.exe**



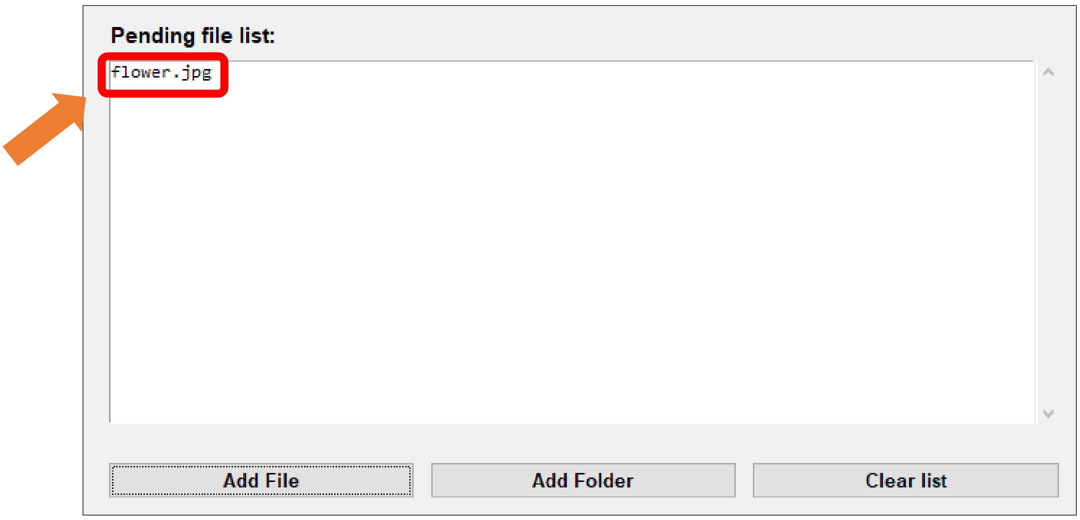
Click **"Add File"**



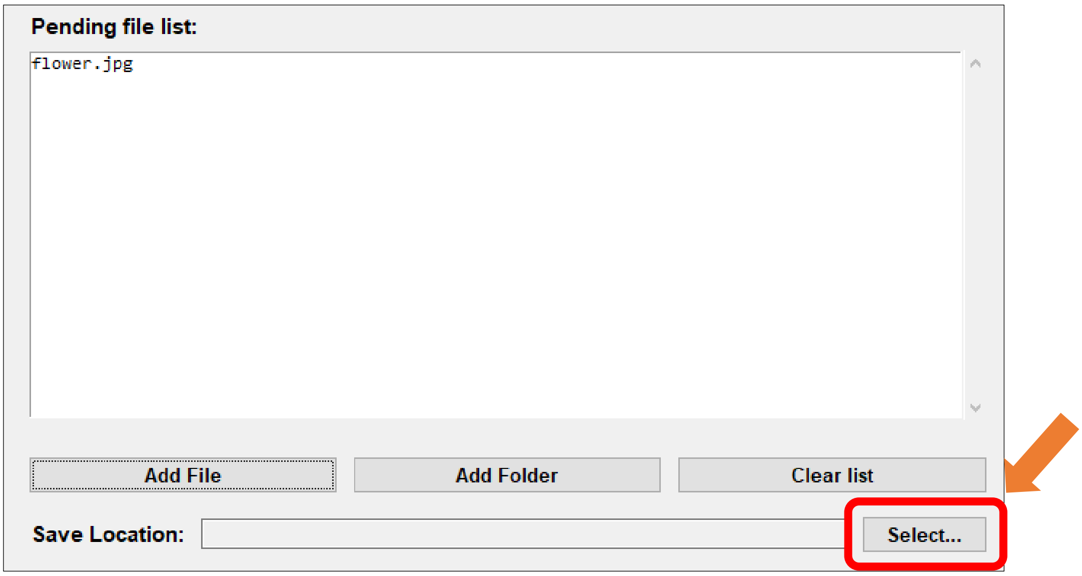
Select any image you like.



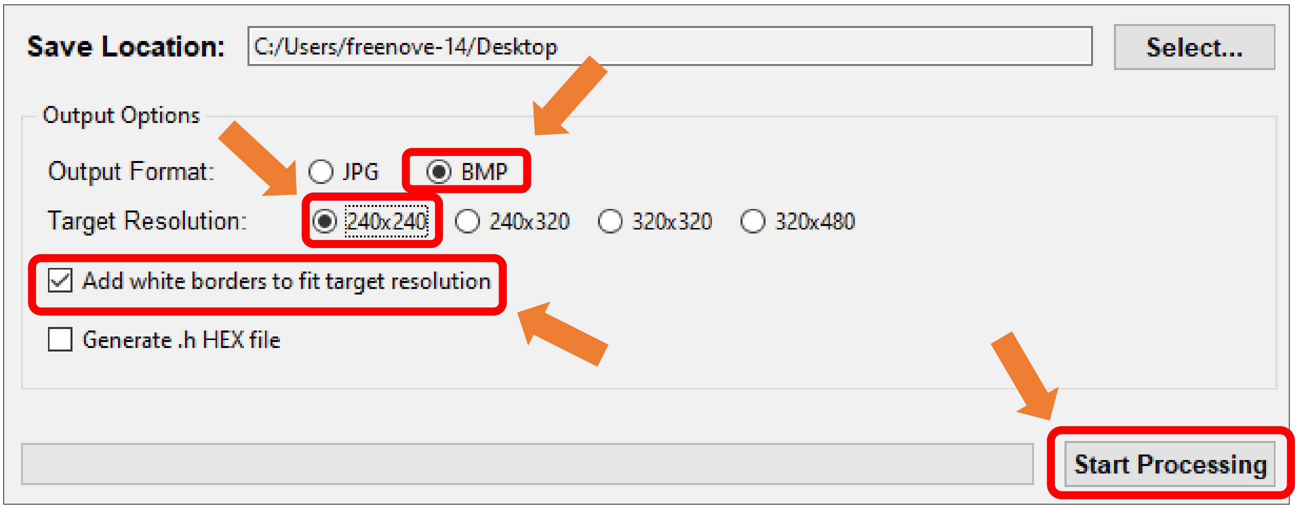
The image files from your folder will now appear in the **Pending File List**.



Click “Select···” to choose the image saving location.

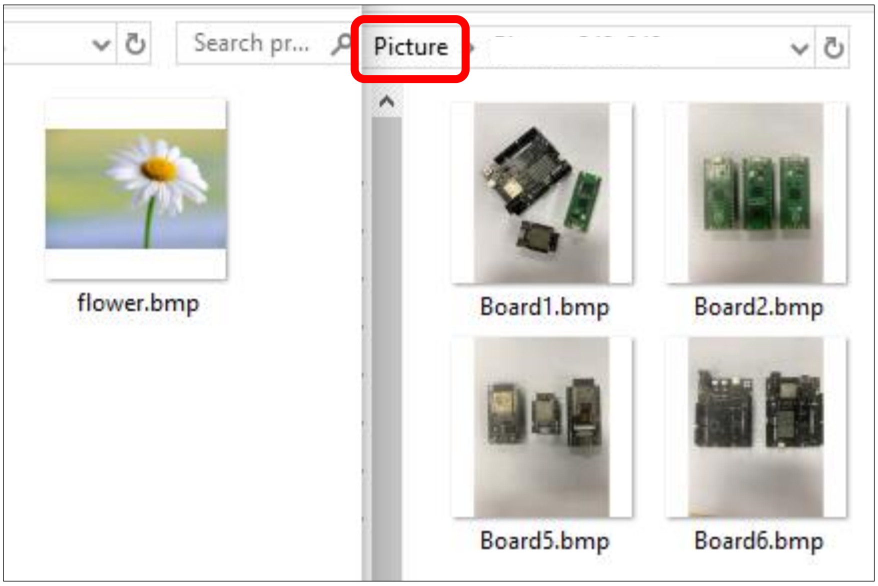


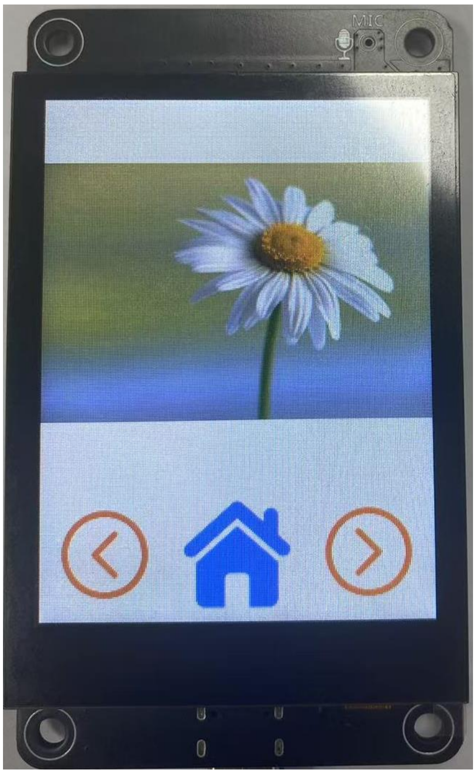
Select **BMP** for the format, set the resolution to **240x240**.



Wait for the progress bar to complete.

Copy or replace the generated image to the folder corresponding to the Picture folder in the root directory of the SD card.





Chapter 15 LVGL Timer

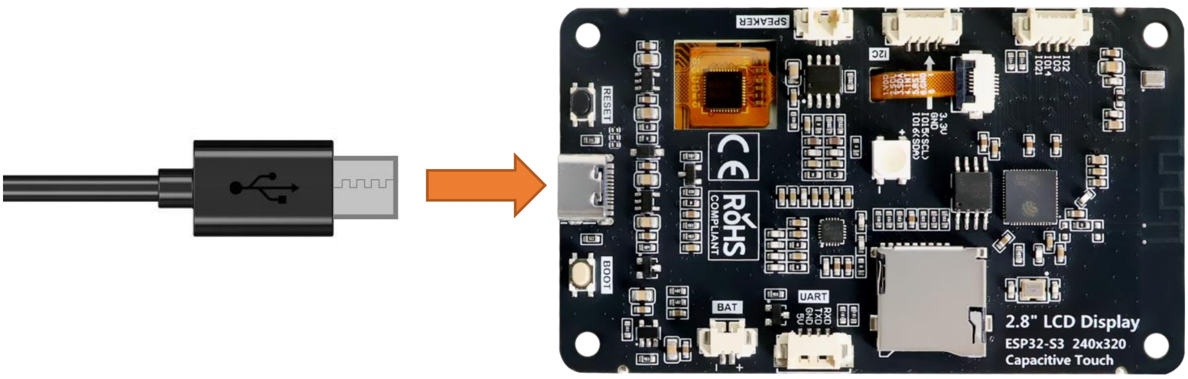
Project 15.1 LVGL Timer

Component List

Freenove ESP32-S3 Display x 1	USB cable x1
	

Circuit

Connect Freenove ESP32 -S3 to the computer using the USB cable.



Sketch

Open "**Sketch_15.1_Lvgl_Timer**" folder under **"Freenove_ESP32_S3_Display\Sketches"** and double-click "**Sketch_15.1_Lvgl_Timer.ino**".

Sketch_15.1_Lvgl_Timer

The following is the program code:

```
#include "display.h"
#include "chronograph_ui.h"

Display screen;
void setup() {
```

```
    Serial.begin(115200);

    /*** Init screen ***/
    screen.init();

    /***  Print lvgl version ***/
    String LVGL_Arduino = "Hello Arduino! ";
    LVGL_Arduino += String('V') + lv_version_major() + "." + lv_version_minor() + "." + lv_version_patch();
    Serial.println(LVGL_Arduino);
    Serial.println("I am LVGL_Arduino");
    Serial.println("Setup done");

    /*** The custom code ***/
    setup_scr_chronograph(&guider_chronograph_ui);
    lv_scr_load(guider_chronograph_ui.chronograph);
}

void loop() {
    screen.routine();
    delay(1);
}
```

Code Explanation

Include the header files.

```
#include "display.h"
#include "chronograph_ui.h"
```

Set the baud rate to 115200.

```
Serial.begin( 115200 );
```

Initialize configuration.

```
screen.init();
```

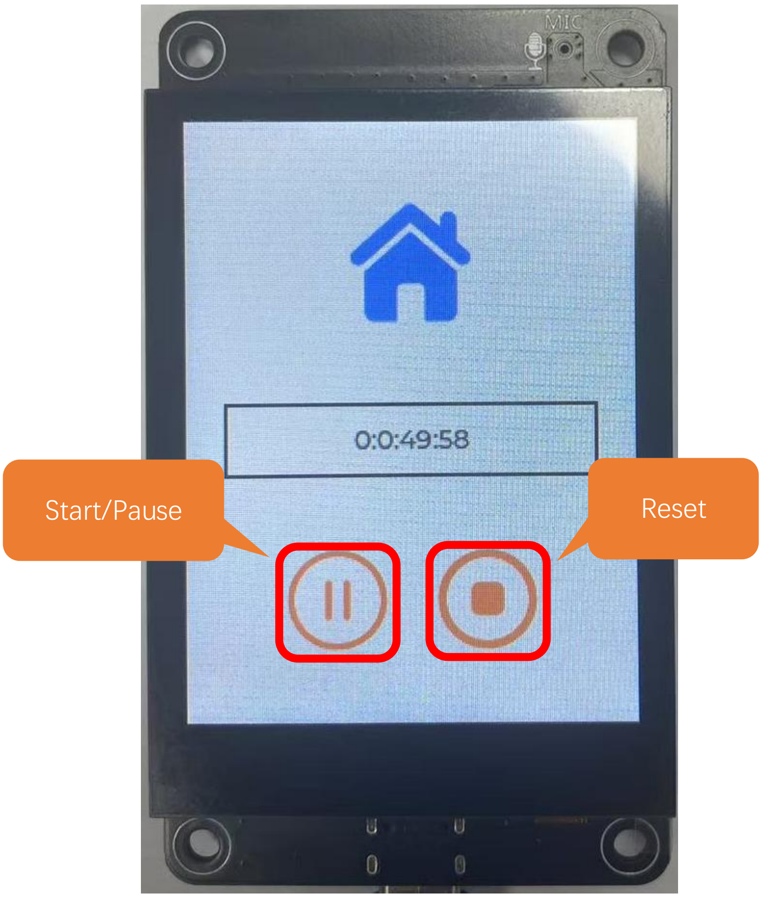
Create and load the interface.

```
setup_scr_chronograph(&guider_chronograph_ui);
lv_scr_load(guider_chronograph_ui.chronograph);
```

LVGL task processor.

```
screen.routine(); /* let the GUI do its work */
```

Click “**Upload**” to upload the code to Freenove ESP32-S3 Display. Set the baud rate to 115200.



Chapter 16 Lvgl RGB

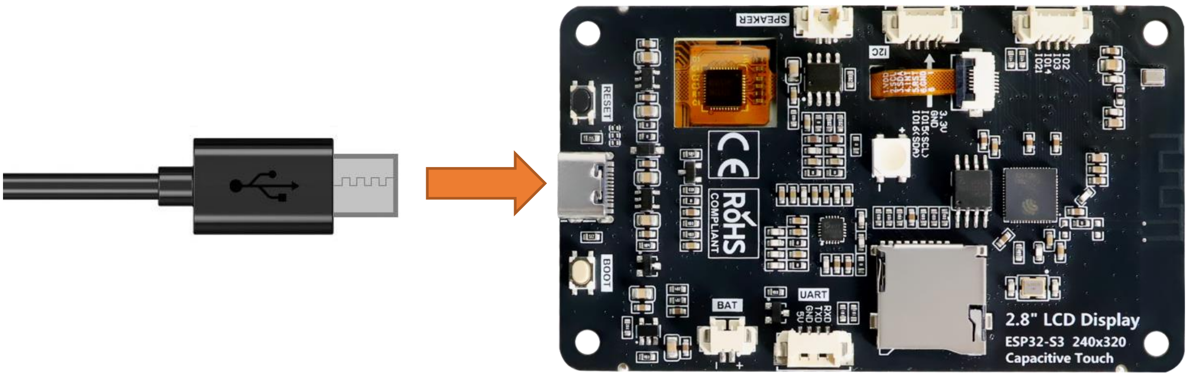
Project 16.1 LVGL RGB

Component List

Freenove ESP32-S3 Display x 1	USB cable x1
	

Circuit

Connect Freenove ESP32 -S3 to the computer using the USB cable.



Sketch

Open "**Sketch_16.1_Lvgl_RGB**" folder under **"Freenove_ESP32_S3_Display\Sketches"** and double-click "**Sketch_16.1_Lvgl_RGB.ino**".

Sketch_16.1_Lvgl_RGB

The following is the program code:

```
#include "display.h"
#include "rgb_ui.h"

#define RED_PIN 22
#define GREEN_PIN 16
```

```
#define BLUE_PIN 17

Display screen;
void setup() {
   Serial.begin(115200);

   /*** Init screen ***/
   screen.init();

   /***  Print lvgl version ***/
   String LVGL_Arduino = "Hello Arduino! ";
   LVGL_Arduino += String('V') + lv_version_major() + "." + lv_version_minor() + "." +
lv_version_patch();
   Serial.println(LVGL_Arduino);
   Serial.println("I am LVGL_Arduino");
   Serial.println("Setup done");

   /*** The custom code ***/
   rgb_init(RED_PIN, GREEN_PIN, BLUE_PIN);
   rgb_set_color(0, 0, 0);
   setup_scr_rgb(&guider_rgb_ui);
   lv_scr_load(guider_rgb_ui.rgb);
}

void loop() {
   screen.routine();
   delay(5);
}
```

Code Explanation

Include the header files.

```
#include <lvgl.h>
#include "Arduino.h"
#include "display.h"
#include "ws2812_ui.h"
```

Set the baud rate to 115200

```
Serial.begin( 115200 );
```

Initialize the RGB LED.

```
screen.init();
```

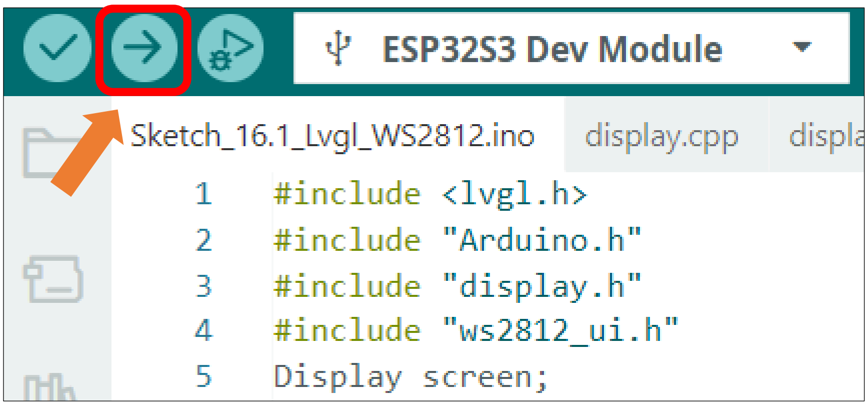
Create and load the interface.

```
setup_scr_rgb(&guider_rgb_ui);
lv_scr_load(guider_rgb_ui.rgb);
```

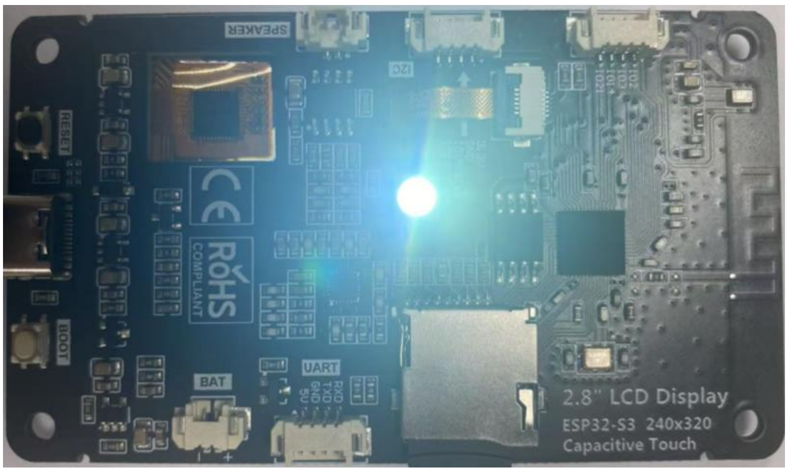
LVGL task processor.

```
screen.routine(); /* let the GUI do its work */
```

Click “**Upload**” to upload the code to Freenove ESP32 Display. Set the baud rate to 115200.



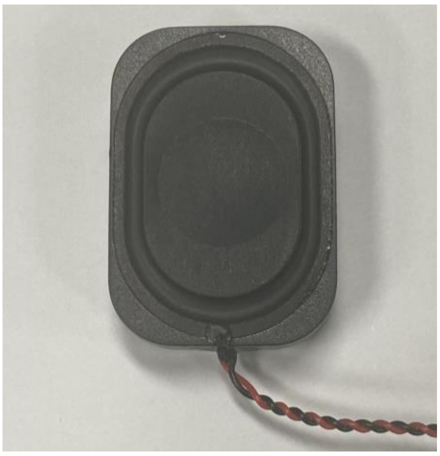
Drag to adjust the value of the red (R), green (G), and blue (B) color, and you will see the color of the LED change.



Chapter 17 LVGL Music

Project 17.1 LVGL Music

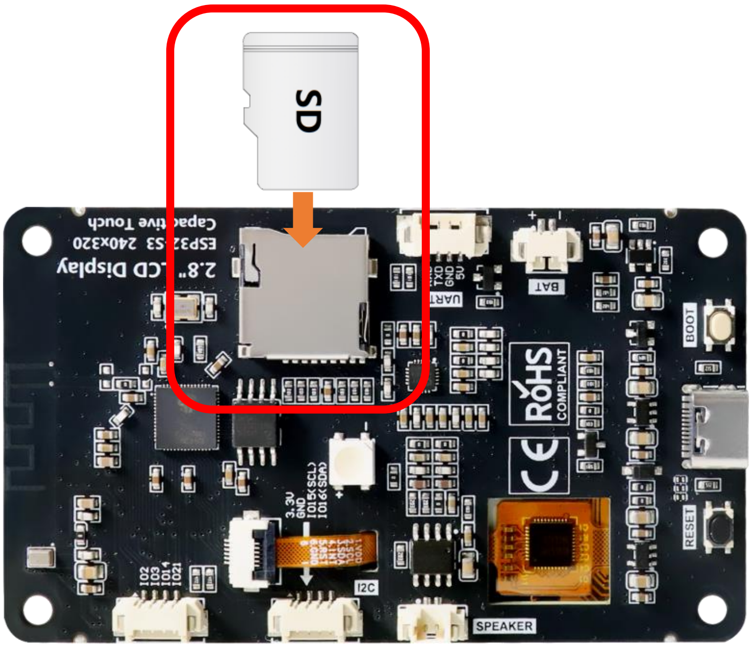
Component List

Freenove ESP32-S3 Display x 1	USB cable x1
	
Speaker x1 	

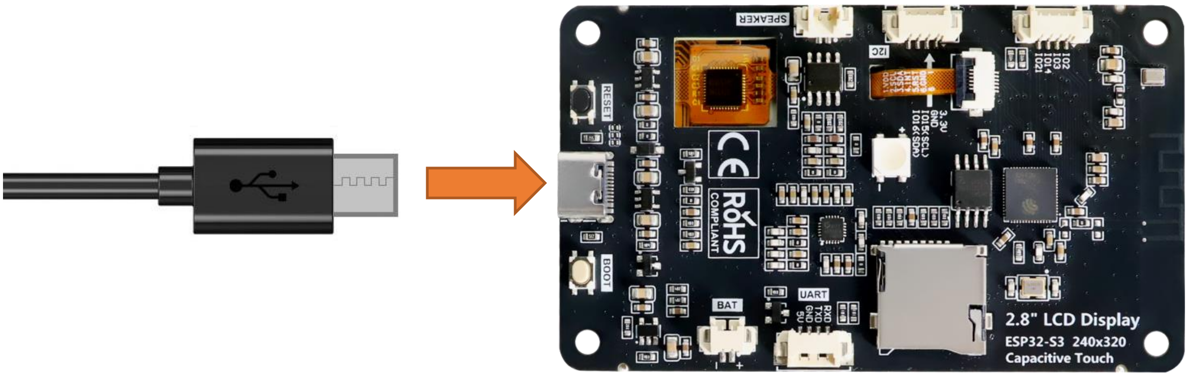
Note: This kit does not include SD card, or SD card reader. Please buy them yourself!

Circuit

Connect Freenove ESP32-S3 to the computer using the USB cable.



Connect Freenove ESP32-S3 to the computer using the USB cable.



Sketch

Open "**Sketch_17.1_Lvgl_Music**" folder under **"Freenove_ESP32_S3_Display\Sketches"** and double-click "**Sketch_17.1_Lvgl_Music.ino**".

Sketch_17.1_Lvgl_Music

The following is the program code:

```
#include <Arduino.h>
#include "Wire.h"
#include "display.h"
#include "driver_sdmmc.h"
#include "music_ui.h"
#include "es8311.h"
#include "ESP_I2S.h"
```

```

#ifdef FNK0104N_3P5_320x480_ST77922
#define SD_MMC_CLK 5
#define SD_MMC_CMD 4
#define SD_MMC_D0 6
#define SD_MMC_D1 7
#define SD_MMC_D2 2
#define SD_MMC_D3 3

//I2S IO Pin define
#define I2S_MCK 17
#define I2S_BCK 18
#define I2S_DINT 16
#define I2S_DOUT 15
#define I2S_WS 21
#define AP_ENABLE 1
#define I2C_SCL 39         /*!< GPIO number used for I2C master clock */
#define I2C_SDA 38         /*!< GPIO number used for I2C master data  */
#define I2C_SPEED 400000   /*!< I2C master clock frequency */

#else
#define SD_MMC_CMD 40  // Please do not modify it.
#define SD_MMC_CLK 38  // Please do not modify it.
#define SD_MMC_D0  39  // Please do not modify it.
#define SD_MMC_D1  41  // Please do not modify it.
#define SD_MMC_D2  48  // Please do not modify it.
#define SD_MMC_D3  47  // Please do not modify it.

//I2S IO Pin define
#define I2S_MCK 4
#define I2S_BCK 5
#define I2S_DINT 6
#define I2S_DOUT 8
#define I2S_WS 7
#define AP_ENABLE 1
#define I2C_SCL 15         /*!< GPIO number used for I2C master clock */
#define I2C_SDA 16         /*!< GPIO number used for I2C master data  */
#define I2C_SPEED 400000   /*!< I2C master clock frequency */
#endif

Display screen;
I2SClass es8311_i2s;

void driver_es8311_init(void) {
```

```
  pinMode(AP_ENABLE, OUTPUT);
  digitalWrite(AP_ENABLE, LOW);

  Wire.begin(I2C_SDA, I2C_SCL, I2C_SPEED);

  es8311_i2s.setPins(I2S_BCK, I2S_WS, I2S_DOUT, I2S_DINT, I2S_MCK);
  if (!es8311_i2s.begin(I2S_MODE_STD, 44100, I2S_DATA_BIT_WIDTH_16BIT, I2S_SLOT_MODE_STEREO,
I2S_STD_SLOT_LEFT)) {
    Serial.println("Failed to initialize I2S bus!");
  }
}

void setup(){
    /* prepare for possible serial debug */
    Serial.begin( 115200 );

   /*** Init drivers ***/
    sdmmc_init(SD_MMC_CLK, SD_MMC_CMD, SD_MMC_D0, SD_MMC_D1, SD_MMC_D2,
SD_MMC_D3);//Initialize the SD module
    driver_es8311_init();
    if (es8311_codec_init() != ESP_OK) {
      Serial.println("ES8311 init failed!");
      return;
    }
    es8311_set_mic_gain(ES8311_MIC_GAIN_MIN); // Close mic
    screen.init();

    String LVGL_Arduino = "Hello Arduino! ";
    LVGL_Arduino += String('V') + lv_version_major() + "." + lv_version_minor() + "." +
lv_version_patch();
    Serial.println( LVGL_Arduino );
    Serial.println( "I am LVGL_Arduino" );

    setup_scr_music(&guider_music_ui);
    lv_scr_load(guider_music_ui.music);

    Serial.println( "Setup done" );
}

void loop(){
    screen.routine(); /* let the GUI do its work */
    delay( 5 );
}
```

Code Explanation

Include the header files.

```
#include <Arduino.h>
#include "Wire.h"
#include "display.h"
#include "driver_sdmmc.h"
#include "music_ui.h"
#include "es8311.h"
#include "ESP_I2S.h"
```

Define the pins.

```
#ifdef FNK0104N_3P5_320x480_ST77922
#define SD_MMC_CLK 5
#define SD_MMC_CMD 4
#define SD_MMC_D0 6
#define SD_MMC_D1 7
#define SD_MMC_D2 2
#define SD_MMC_D3 3

//I2S IO Pin define
#define I2S_MCK 17
#define I2S_BCK 18
#define I2S_DINT 16
#define I2S_DOUT 15
#define I2S_WS 21
#define AP_ENABLE 1
#define I2C_SCL 39         /*!< GPIO number used for I2C master clock */
#define I2C_SDA 38         /*!< GPIO number used for I2C master data  */
#define I2C_SPEED 400000   /*!< I2C master clock frequency */

#else
#define SD_MMC_CMD 40  // Please do not modify it.
#define SD_MMC_CLK 38  // Please do not modify it.
#define SD_MMC_D0  39  // Please do not modify it.
#define SD_MMC_D1  41  // Please do not modify it.
#define SD_MMC_D2  48  // Please do not modify it.
#define SD_MMC_D3  47  // Please do not modify it.

//I2S IO Pin define
#define I2S_MCK 4
#define I2S_BCK 5
#define I2S_DINT 6
#define I2S_DOUT 8
#define I2S_WS 7
#define AP_ENABLE 1
```

```
#define I2C_SCL 15        /*!< GPIO number used for I2C master clock */
#define I2C_SDA 16        /*!< GPIO number used for I2C master data  */
#define I2C_SPEED 400000  /*!< I2C master clock frequency */
#endif
```

Set the baud rate to 115200

```
Serial.begin( 115200 );
```

Initialize configuration.

```
sdmmc_init(SD_MMC_CLK, SD_MMC_CMD, SD_MMC_D0, SD_MMC_D1, SD_MMC_D2, SD_MMC_D3
driver_es8311_init();
if (es8311_codec_init() != ESP_OK) {
  Serial.println("ES8311 init failed!");
  return;
}
es8311_set_mic_gain(ES8311_MIC_GAIN_MIN); // Close mic
screen.init();
```

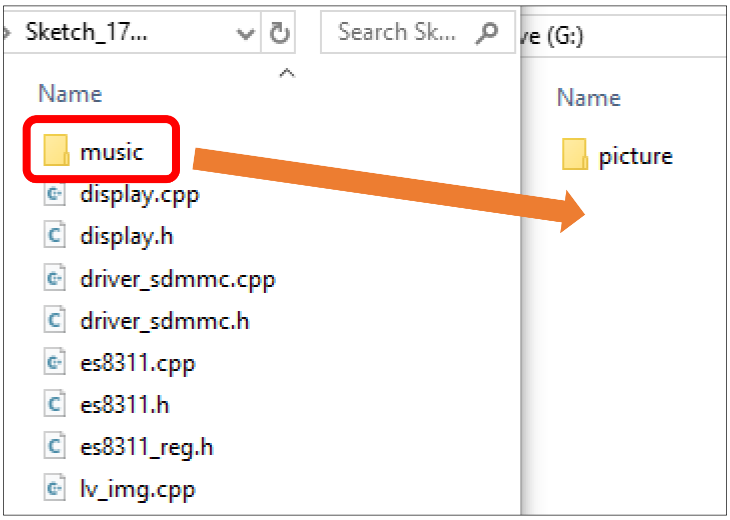
Create and load the interface.

```
setup_scr_music(&guider_music_ui);
lv_scr_load(guider_music_ui.music);
```

LVGL task processor.

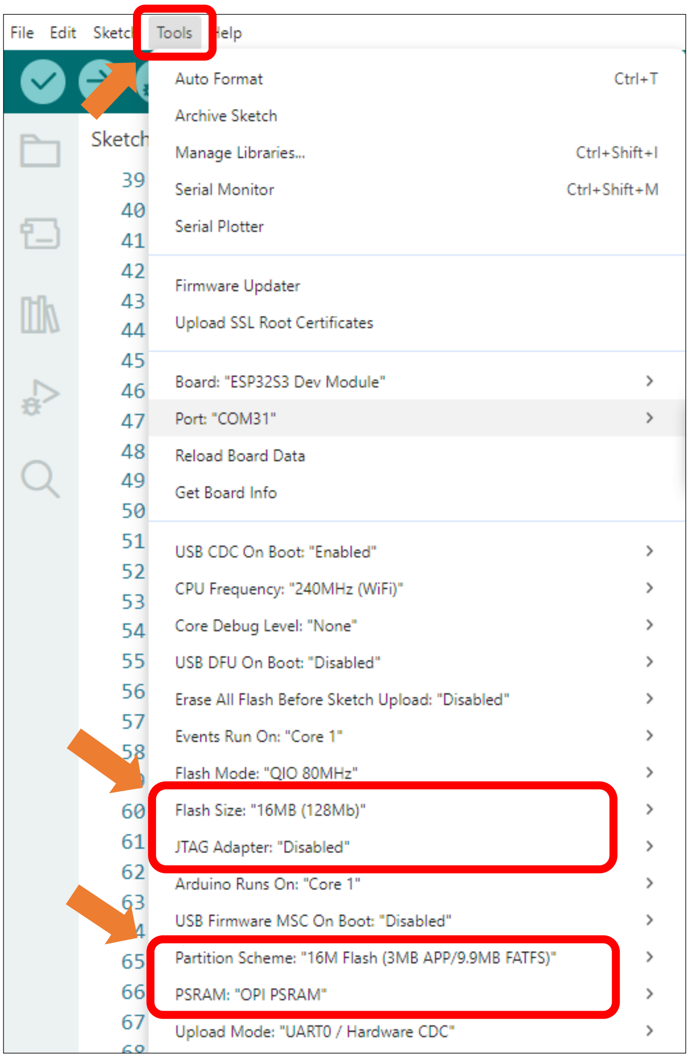
```
screen.routine();
```

Insert the SD card to the card reader and plug them to the computer. Copy the **Music** folder under the **Freenove_ESP32_S3_Display\Sketches\Sketch_17.1_Lvgl_Music** directory to the root directory of the SD card.

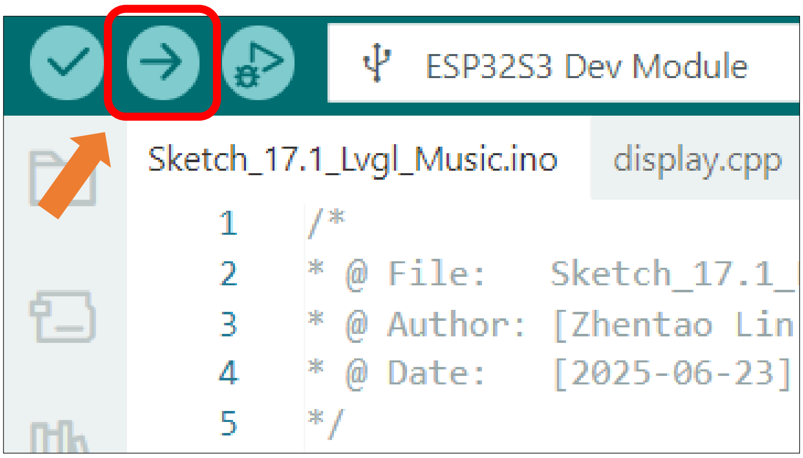


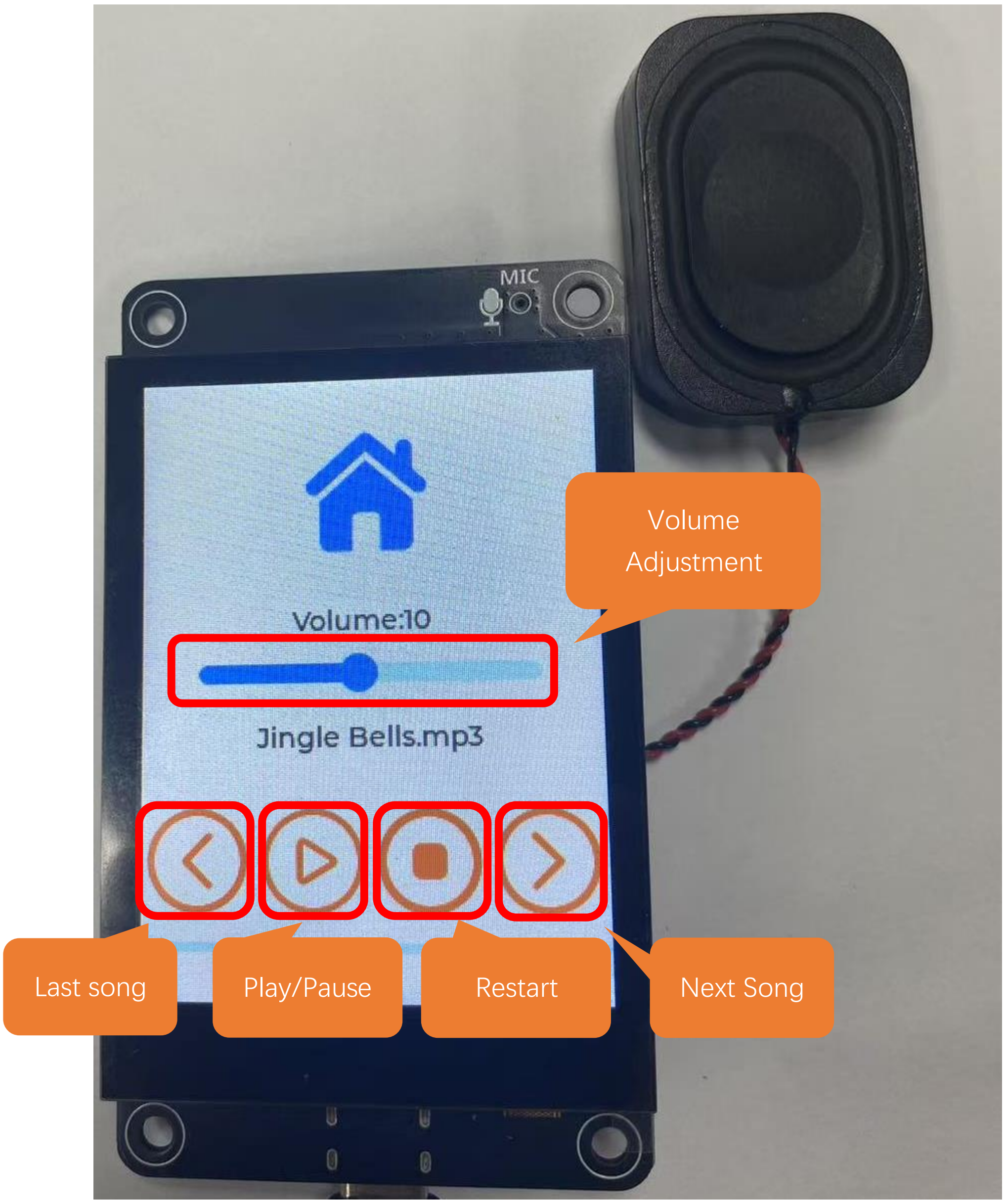
It is necessary to change the settings in Arduino IDE before clicking the Uploading button, as shown below.

Caution: Incorrect settings will result in compilation error or uploading failure. To achieve desired result, please configure exactly the same as below.



Click "**Upload**" to upload the code to Freenove ESP32 Display. Set the baud rate to 115200.



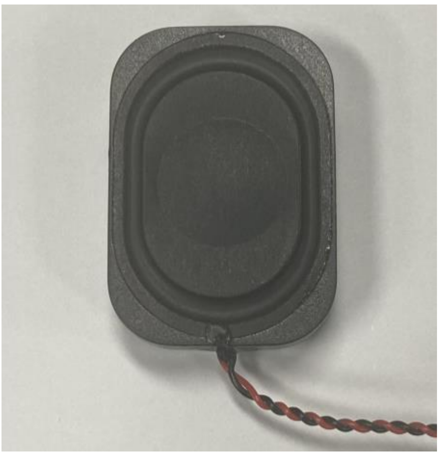


Note: If the screen flickers during playback, it may be due to insufficient power supply. You can try powering with a battery.

Chapter 18 LVGL Multifunctionality

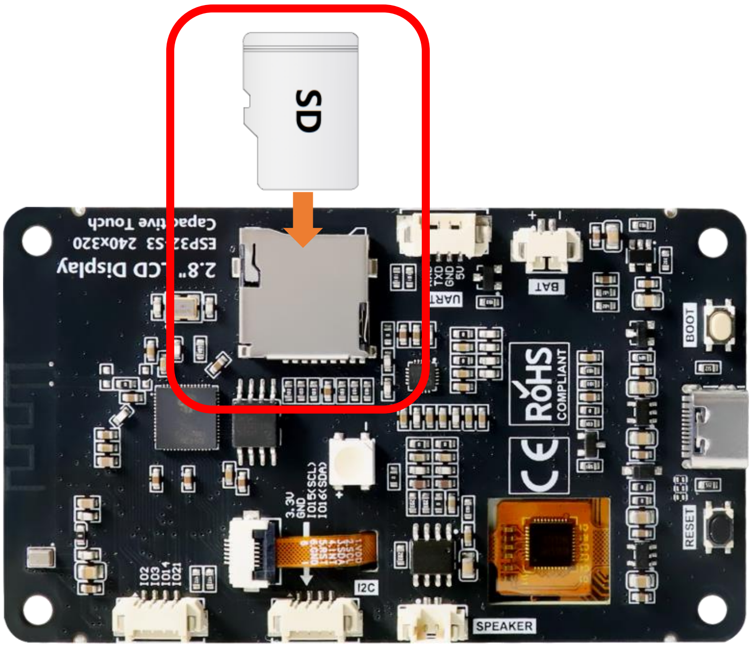
Project 18.1 LVGL Multifunctionality

Component List

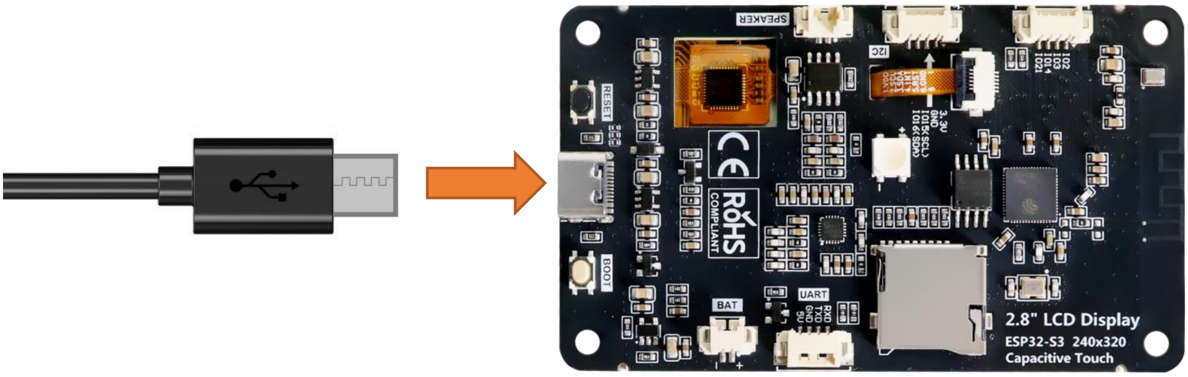
Freenove ESP32-S3 Display x 1 	USB cable x1 
Speaker x1 	

Circuit

Connect Freenove ESP32-S3 to the computer using the USB cable.



Connect Freenove ESP32-S3 to the computer using the USB cable.



Sketch

Open"**Sketch_18.1_Lvgl_Multifunctionality**" folder under **"Freenove_ESP32_S3_Display\Sketches"** and double-click "**Sketch_18.1_Lvgl_ Multifunctionality.ino**".

Sketch_18.1_Lvgl_Multifunctionality

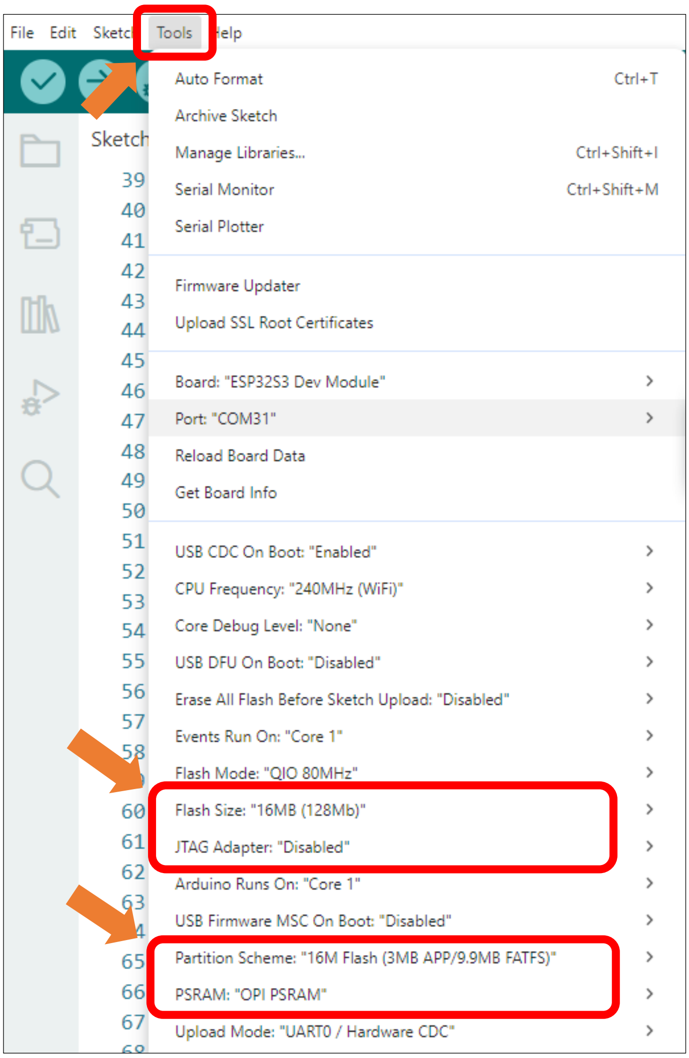
The following is the program code:

```
#include "public.h"

Display screen;
I2SClass es8311_i2s;

void driver_es8311_init(void) {
```

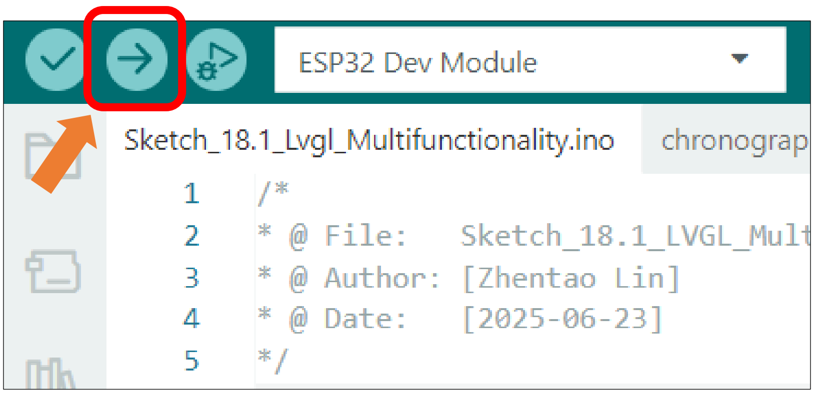
```
    pinMode(AP_ENABLE, OUTPUT);
    digitalWrite(AP_ENABLE, LOW);

    Wire.begin(I2C_SDA, I2C_SCL, I2C_SPEED);

    es8311_i2s.setPins(I2S_BCK, I2S_WS, I2S_DOUT, I2S_DINT, I2S_MCK);
    if (!es8311_i2s.begin(I2S_MODE_STD, 44100, I2S_DATA_BIT_WIDTH_16BIT, I2S_SLOT_MODE_STEREO,
I2S_STD_SLOT_LEFT)) {
      Serial.println("Failed to initialize I2S bus!");
    }
}

void setup(){
      /* prepare for possible serial debug */
      Serial.begin( 115200 );

     /*** Init drivers ***/
      sdmmc_init(SD_MMC_CLK, SD_MMC_CMD, SD_MMC_D0, SD_MMC_D1, SD_MMC_D2,
SD_MMC_D3);//Initialize the SD module
      driver_es8311_init();
      if (es8311_codec_init() != ESP_OK) {
        Serial.println("ES8311 init failed!");
        return;
      }
      screen.init();

      String LVGL_Arduino = "Hello Arduino! ";
      LVGL_Arduino += String('V') + lv_version_major() + "." + lv_version_minor() + "." +
lv_version_patch();
      Serial.println( LVGL_Arduino );
      Serial.println( "I am LVGL_Arduino" );

      setup_scr_main(&guider_main_ui);
      lv_scr_load(guider_main_ui.main);

      Serial.println( "Setup done" );
}

void loop(){
      screen.routine(); /* let the GUI do its work */
      delay( 5 );
}
```

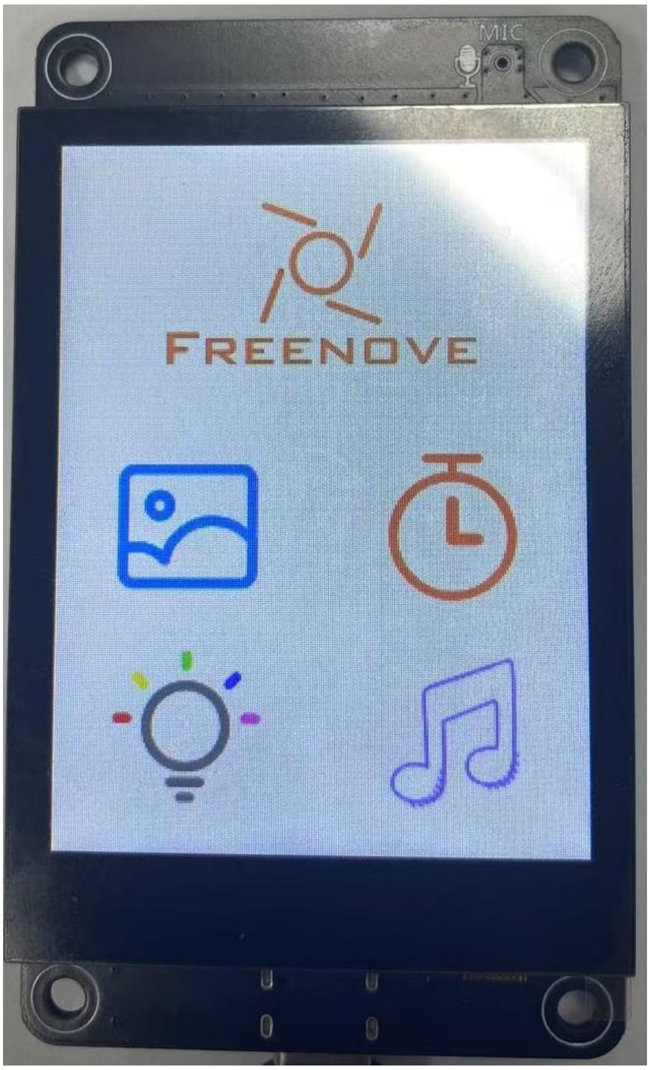
It is necessary to change the settings in Arduino IDE before clicking the Uploading button, as shown below.

Caution: Incorrect settings will result in compilation error or uploading failure. To achieve desired result, please configure exactly the same as below.



Click "**Upload**" to upload the code to Freenove ESP32 Display. Set the baud rate to 115200.




MIC
FREENOVE

Chapter 19 LVGL Arduino

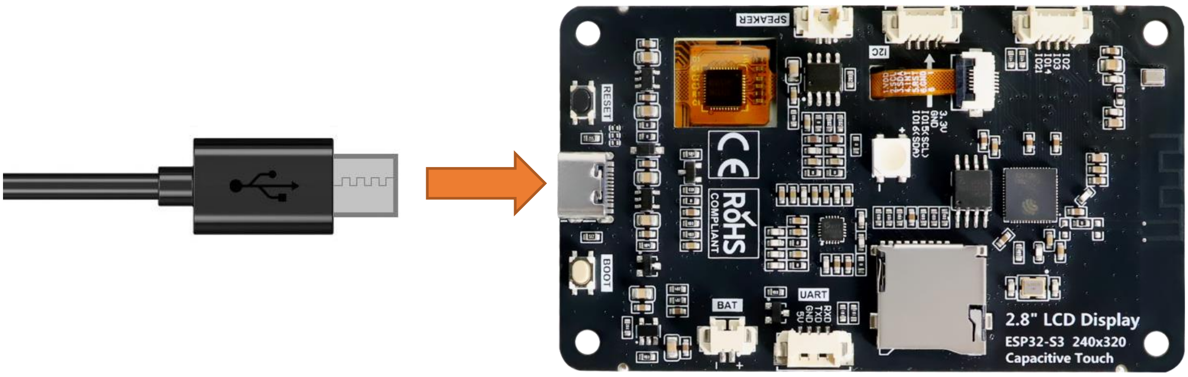
Project 19.1 LVGL Arduino

Component List

Freenove ESP32-S3 Display x 1	USB cable x1
	

Circuit

Connect Freenove ESP32-S3 to the computer using the USB cable.



Sketch

Open“**Sketch_19.1_Lvgl_Arduino**” folder under **“Freenove_ESP32_S3_Display\Sketches”** and double-click “**Sketch_19.1_Lvgl_Arduino.ino**”.

Sketch_19.1_Lvgl_Arduino

The following is the program code:

```
/*Using LVGL with Arduino requires some extra steps:
 *Be sure to read the docs here: https://docs.lvgl.io/master/get-
started/platforms/arduino.html  */

```

```
#include <lvgl.h>
#include "demos/lv_demos.h"
#include <TFT_eSPI.h>

/*To use the built-in examples and demos of LVGL uncomment the includes below respectively.
 *You also need to copy `lvgl/examples` to `lvgl/src/examples`. Similarly for the demos
`lvgl/demos` to `lvgl/src/demos`.
 Note that the `lv_examples` library is for LVGL v7 and you shouldn't install it for this
version (since LVGL v8)
 as the examples and demos are now part of the main LVGL library. */
#ifdef FNK0104N_3P5_320x480_ST77922
  #define TFT_SCREEN_WIDTH 320
  #define TFT_SCREEN_HEIGHT 480
#elif defined FNK0104AB_2P8_240x320_ILI9341
  #define TFT_SCREEN_WIDTH 240
  #define TFT_SCREEN_HEIGHT 320
#elif defined FNK0104S_4P0_320x480_ST7796
  #define TFT_SCREEN_WIDTH 320
  #define TFT_SCREEN_HEIGHT 480
#endif

// 3.5inch ST77922
#ifdef FNK0104N_3P5_320x480_ST77922
 #include <SPI.h>
 #include "ST77922.h"
 #include "ST77922_Touch.h"

 static lv_color_t buf[ TFT_SCREEN_WIDTH * 40 ];

 ST77922 tft_st77922 = ST77922();
 ST77922_TOUCH touch_st77922;

 void my_rounder_cb(lv_disp_drv_t * disp_drv, lv_area_t * area)
 {
   area->x1 = area->x1 & ~0x3;
   area->y1 = area->y1 & ~0x3;

   area->x2 = area->x2 | 0x3;
   area->y2 = area->y2 | 0x3;
 }

// 2.8inch ILI9341
#else
 #include "FT6336U.h"
```

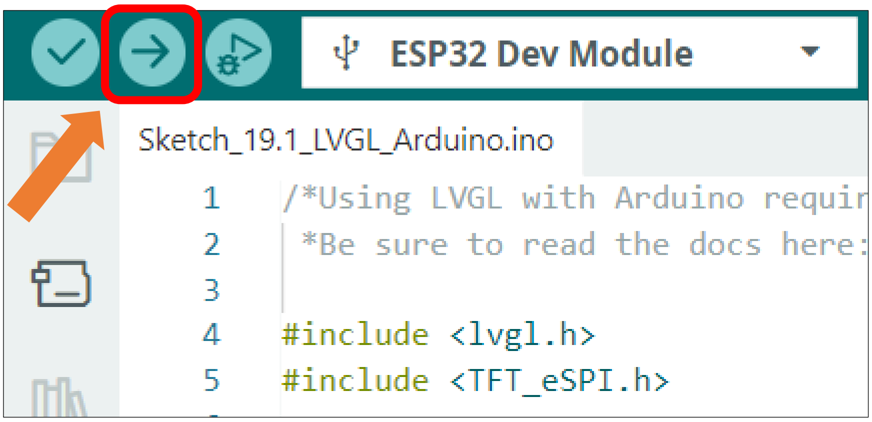
```

 #define I2C_SCL 15
 #define I2C_SDA 16
 #define INT_N_PIN 17
 #define RST_N_PIN 18

 static lv_color_t buf[ TFT_SCREEN_WIDTH * 40 ];

 TFT_eSPI tft = TFT_eSPI(TFT_SCREEN_WIDTH, TFT_SCREEN_HEIGHT); /* TFT instance */
 FT6336U ft6336u(I2C_SDA, I2C_SCL, RST_N_PIN, INT_N_PIN);
 FT6336U_TouchPointType tp;
#endif

static const uint16_t screenWidth = TFT_SCREEN_WIDTH;
static const uint16_t screenHeight = TFT_SCREEN_HEIGHT;
static const uint16_t screenHeightBuf = TFT_SCREEN_HEIGHT / 10;
static lv_disp_draw_buf_t draw_buf;
static lv_color_t draw_buf1[screenWidth * screenHeightBuf];

/*Change to your screen resolution*/
#define TFT_DIRECTION 0   //Select TFT Direction (0 - 3)

#if LV_USE_LOG != 0
/* Serial debugging */
void my_print(const char * buf)
{
    Serial.printf(buf);
    Serial.flush();
}
#endif

/* Display flushing */
void my_disp_flush( lv_disp_drv_t *disp, const lv_area_t *area, lv_color_t *color_p )
{
    uint32_t w = ( area->x2 - area->x1 + 1 );
    uint32_t h = ( area->y2 - area->y1 + 1 );

    #if defined FNK0104N_3P5_320x480_ST77922
      tft_st77922.Fill_Colors(area->x1, area->y1, w, h, (uint16_t *)color_p);
    #else
      tft.startWrite();
      tft.setAddrWindow( area->x1, area->y1, w, h );
      tft.pushColors((uint16_t*)&color_p->full, w * h, true );
      tft.endWrite();
```

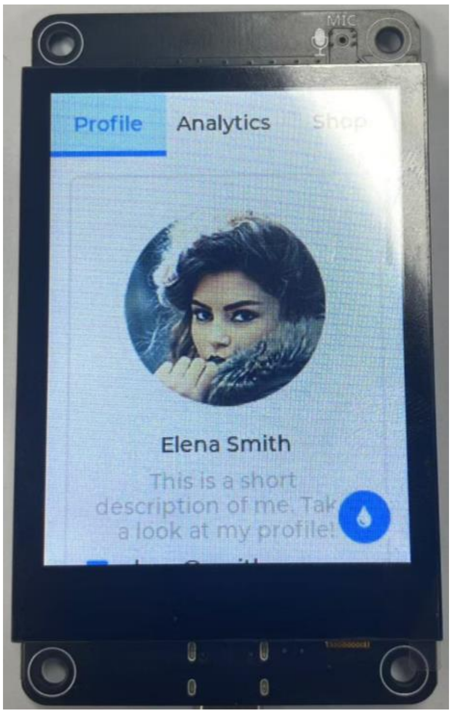
```
    #endif

    lv_disp_flush_ready( disp );
}

/*Read the touchpad*/
void my_touchpad_read( lv_indev_drv_t * indev_driver, lv_indev_data_t * data )
{
    #ifdef FNK0104N_3P5_320x480_ST77922
      uint16_t screenWidth = tft_st77922.Get_Width();
      uint16_t screenHeight = tft_st77922.Get_Height();
      if( touch_st77922.Get_Touch() )
      {
        uint16_t x = touch_st77922.touch.x[0];
        uint16_t y = touch_st77922.touch.y[0];
        if(x >= 0 && x < screenWidth && y >= 0 && y < screenHeight)
        {
            data->state = LV_INDEV_STATE_PR;
            data->point.x = x;
            data->point.y = y;
        }
      }
      else
      {
        data->state = LV_INDEV_STATE_REL;
      }
    #else
      uint16_t touchX, touchY;

      //bool touched = tft.getTouch( &touchX, &touchY, 600 );
      tp = ft6336u.scan();
      int touched = tp.touch_count;

      if( !touched )
      {
          data->state = LV_INDEV_STATE_REL;
      }
      else
      {
          int x = tp.tp[0].x;
          int y = tp.tp[0].y;
          if(x >= 0 && x < screenWidth && y >= 0 && y < screenHeight)
          {
              data->state = LV_INDEV_STATE_PR;
```

```
              data->point.x = tp.tp[0].x;
              data->point.y = tp.tp[0].y;
          }
      }
    #endif
}

void setup()
{
    Serial.begin( 115200 ); /* prepare for possible serial debug */

    String LVGL_Arduino = "Hello Arduino! ";
    LVGL_Arduino += String('V') + lv_version_major() + "." + lv_version_minor() + "." +
lv_version_patch();

    Serial.println( LVGL_Arduino );
    Serial.println( "I am LVGL_Arduino" );

    #ifdef FNK0104N_3P5_320x480_ST77922
      tft_st77922.Init();
      tft_st77922.Set_Rotation(TFT_DIRECTION);

      uint16_t screenWidth = tft_st77922.Get_Width();
      uint16_t screenHeight = tft_st77922.Get_Height();

      lv_init();
      lv_disp_draw_buf_init( &draw_buf, buf, NULL, screenWidth * 40 );
      touch_st77922.init();
      touch_st77922.Set_Rotation(TFT_DIRECTION);
    #else
      ft6336u.begin();
      lv_init();
      tft.begin();          /* TFT init */
      tft.setRotation( TFT_DIRECTION ); /* Landscape orientation, flipped */
      lv_disp_draw_buf_init( &draw_buf, buf, NULL, screenWidth * 10 );
    #endif

    /*Initialize the display*/
    static lv_disp_drv_t disp_drv;
    lv_disp_drv_init( &disp_drv );
    /*Change the following line to your display resolution*/
    disp_drv.hor_res = screenWidth;
    disp_drv.ver_res = screenHeight;
    disp_drv.flush_cb = my_disp_flush;
```

```
    disp_drv.draw_buf = &draw_buf;
    #ifdef FNK0104N_3P5_320x480_ST77922
      disp_drv.rounder_cb = my_rounder_cb;
    #endif
    lv_disp_drv_register( &disp_drv );

    /*Initialize the (dummy) input device driver*/
    static lv_indev_drv_t indev_drv;
    lv_indev_drv_init( &indev_drv );
    indev_drv.type = LV_INDEV_TYPE_POINTER;
    indev_drv.read_cb = my_touchpad_read;
    lv_indev_drv_register( &indev_drv );

    /* Create simple label */
    // lv_obj_t *label = lv_label_create( lv_scr_act() );
    // lv_label_set_text( label, "Hello Ardino and LVGL!");
    // lv_obj_align( label, LV_ALIGN_CENTER, 0, 0 );

    /* Try an example. See all the examples
     * online: https://docs.lvgl.io/master/examples.html
     * source codes:
https://github.com/lvgl/lvgl/tree/e7f88efa5853128bf871dde335c0ca8da9eb7731/examples */
     //lv_example_btn_1();

     /*Or try out a demo. Don't forget to enable the demos in lv_conf.h. E.g.
LV_USE_DEMOS_WIDGETS*/
    lv_demo_widgets();
    // lv_demo_benchmark();
    // lv_demo_keypad_encoder();
    // lv_demo_music();
    // lv_demo_printer();
    // lv_demo_stress();

    Serial.println( "Setup done" );
}

void loop()
{
    lv_timer_handler(); /* let the GUI do its work */
    delay( 5 );
}
```

Click “**Upload**” to upload the code to Freenove ESP32 Display





Note: The examples in this section require touch interaction. If you are using a non-touchscreen version of the Freenove ESP32-S3 Display, the interface will render correctly but will not respond to touch input. Nevertheless, You may still use these examples as a reference for exploring other LVGL features.

What's next?

Thank you again for choosing Freenove products.

THANK YOU for participating in this learning experience!

We have reached the end of this Tutorial. If you find errors, omissions, or you have suggestions and/or questions about the Tutorial or component contents of this Kit, please feel free to contact us:
support@freenove.com

We will make every effort to make changes and correct errors as soon as feasibly possible and publish a revised version.

If you want to learn more about Arduino, ESP32, Raspberry Pi Pico W, Raspberry Pi, Smart Cars, Robotics and other interesting products in science and technology, please continue to visit our website. We will continue to launch fun, cost effective, innovative and exciting products.

Thank you again for choosing Freenove products.